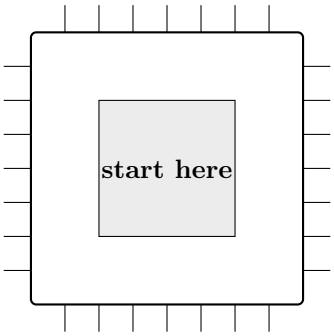
Hardware from the Ground Up

A beginner's book on circuits, chips and computer hardware

Written to prepare for the NVIDIA Ignite hardware assessment



First edition, October 2026

An unofficial study book. It is not produced or endorsed by NVIDIA.

Preface

Who this book is for

This book is for someone who wants to work on computer hardware and is starting from nothing. It assumes you know high-school algebra and can read a simple program. It does *not* assume you know what a voltage is, what a transistor does, or why a chip gets hot. Every idea is built from the one before it.

It was written for one specific goal: passing the hardware online assessment for NVIDIA's Ignite program, a 12-week summer internship for first- and second-year university students. But the content is the standard foundation every hardware engineer learns, so it will serve you well in your courses and in any hardware interview.

Why the assessment felt like a different subject

Many students prepare for hardware assessments by practicing Verilog on sites like HDLBits. That is useful, but the Ignite hardware assessment mostly tests *theory*: why circuits behave the way they do. Questions like these have appeared:

- What is one advantage of a flip-flop over a latch?
- How do you build an OR gate out of a 2:1 multiplexer?
- If you swap the transistors in an inverter so the NMOS connects to the supply and the PMOS to ground, how does the circuit behave?
- If the clock frequency doubles and the supply voltage doubles, what happens to dynamic power?
- When temperature rises, what happens to transistor carrier mobility and to wire resistance?
- True or false: for the antenna rule, as long as the metal wire is safe, a small connected transistor is good.

None of these needs code. Each one needs one idea, understood well enough to apply to a small twist. This book teaches those ideas from the beginning.

What the assessment looks like

NVIDIA does not publish the format, and it changes between years and teams. The picture below comes from candidate reports on Glassdoor and interview-preparation sites (see Appendix E). Treat it as a guide, not a promise.

- **Platform:** HackerRank, with webcam and screen recording. The link is usually open for about 48 hours; once started, the timer runs roughly 45 to 90 minutes.
- **Coding:** one or two easy coding problems. Reported examples include a linked-list problem, bit-manipulation problems (checking whether two addresses are on the same 4 KB page, building a bit mask), and an XOR-of-subarrays problem.
- **Hardware multiple choice and true/false:** reported topics include transistors, circuits, heat sinks, ground (GND) connections, FIFOs, timing diagrams, the output of short code snippets, and every question in the list above. One hardware intern reported 15 questions in 50 minutes. Some tracks use negative marking, so a wrong guess can cost points.
- **After the assessment:** reported interviews ask you to build gates and flip-flops out of multiplexers, list the stages of a five-stage processor pipeline, design clock dividers, and reason about setup and hold timing.

How the reported topics map to this book

Reported topic	Where it is taught
Voltage, current, resistance, ground connections	Chapter 1
Capacitors, decoupling, signal reflections	Chapter 2
Heat sinks and cooling	Chapter 3
Carrier mobility and temperature	Chapters 4 and 15
Transistors, strong and weak 0 and 1, sizing	Chapter 5
Inverters, the swapped inverter, NAND and NOR	Chapter 8
Gates built from multiplexers, XOR chains	Chapter 9
Latches versus flip-flops, flip-flops from multiplexers	Chapter 10
Setup, hold, timing diagrams, maximum frequency	Chapter 11
Clock dividers, counters, state machines	Chapter 12
Gate and wire delay	Chapter 13
Dynamic power scaling, power gating	Chapter 14
Wire resistance, crosstalk, bandwidth	Chapter 16
Antenna rule, electromigration, latch-up	Chapter 17
SRAM and DRAM	Chapter 18
FIFOs, clock-domain crossing, Verilog, verification	Chapter 19
Pipelines, caches, memory access time	Chapter 20
GPUs and number formats	Chapter 21
Computer arithmetic (named in the job posting)	Chapter 22
Linked lists, bit manipulation, C output questions	Chapter 23
Aptitude and puzzles	Chapter 24

How to read this book

Read the chapters in order. Each one builds on the last: you cannot understand why an inverter swapped upside down misbehaves (Chapter 8) until you understand why a transistor passes a “weak 1” (Chapter 5), and that rests on how a transistor’s channel forms (Chapter 4).

Throughout the book you will see these boxes:

The big idea

The one sentence to remember from a section. If you forget everything else, keep this.

An everyday picture

A comparison to something you already know, such as water in pipes, to build intuition before the math.

Worked example: a calculation

A problem solved step by step, in the style of the assessment.

Common trap

A twist that assessment questions like to use to catch people who memorized instead of understood.

Seen on the assessment

A topic or question that candidates have reported seeing on the NVIDIA assessment or in its interviews.

Every chapter ends with exercises and their answers. Try each exercise before reading the answer; the struggle is where the learning happens. Chapter 25 is a full practice assessment, Appendix C lays out a twelve-month plan to be ready for the next application cycle, and Appendix D lists what to learn beyond this book and the projects that make an application stand out.

A note on honesty

The reported questions come from anonymous candidate reports, and some come from other NVIDIA hardware roles rather than Ignite itself. Where a reported question depended on a diagram that was never published, this book teaches the method for that *kind* of question with its own example rather than pretending to know the original. The safest preparation is not memorizing past questions; it is understanding the ideas well enough that a new twist does not scare you.

Contents

Preface **ii**

I Electricity, from Zero **1**

1 Charge, Current, Voltage and Resistance **2**

1.1 Charge: the thing that moves . . . 2
1.2 Current: how fast charge flows . . . 3
1.3 Voltage: the push . . . 3
1.4 Ground: the reference point . . . 4
1.5 Resistance and Ohm's law . . . 5
1.6 Power: how fast energy is used . . . 6
1.7 Circuits: closed loops, opens and shorts . . . 7
1.8 Series and parallel . . . 7
1.9 Kirchhoff's laws . . . 8
1.10 The voltage divider . . . 9
1.11 Real sources and IR drop . . . 10
1.12 Measuring voltage and current . . . 10
1.13 DC, AC and frequency . . . 11
1.14 Summary . . . 11
1.15 Exercises . . . 12

2 Capacitors, Inductors and the Role of Time **14**

2.1 The capacitor . . . 14
2.2 Current into a capacitor . . . 15
2.3 Energy stored in a capacitor . . . 16
2.4 Charging through a resistor: the RC time constant . . . 16
2.5 Capacitors in series and parallel . . . 17

2.6 The inductor . . . 17
2.7 Decoupling capacitors . . . 18
2.8 How capacitors and inductors respond to frequency . . . 19
2.9 When wires become transmission lines . . . 19
2.10 Measuring fast signals: bandwidth . . . 21
2.11 Summary . . . 22
2.12 Exercises . . . 22

3 Heat: Where Every Watt Ends Up 24
3.1 Every watt becomes heat . . . 24
3.2 Why temperature matters . . . 24
3.3 The three ways heat moves . . . 25
3.4 Thermal resistance: Ohm's law for heat . . . 25
3.5 Conduction and materials . . . 26
3.6 Heat sinks . . . 26
3.7 Thermal interface materials . . . 27
3.8 Air versus liquid cooling . . . 27
3.9 Thermal throttling and runaway . . . 28
3.10 Summary . . . 28
3.11 Exercises . . . 28

II Semiconductors and Transistors 30

4 Semiconductors: Making Silicon Useful 31
4.1 Electrons that can and cannot move . . . 31
4.2 The silicon crystal . . . 31
4.3 Holes: the missing electron that moves . . . 32
4.4 Doping: controlling the carriers . . . 32
4.5 How carriers move: drift, diffusion and mobility . . . 33
4.6 What heat does to semiconductors and to metal . . . 34
4.7 The PN junction and the diode . . . 35
4.8 Silicon dioxide: silicon's perfect partner . . . 35
4.9 Summary . . . 36
4.10 Exercises . . . 36

5 The MOSFET: A Switch Controlled by Voltage 38

5.1 What we want . . . 38
5.2 Inside an NMOS transistor . . . 38
5.3 How the channel forms . . . 39
5.4 The PMOS transistor . . . 40
5.5 The switch model . . . 40
5.6 How much current: regions of operation . . . 41
5.7 Strong and weak logic levels . . . 42
5.8 Chains of pass transistors . . . 43
5.9 The body effect . . . 45
5.10 Sizing transistors . . . 45
5.11 Transistor capacitance . . . 45
5.12 Threshold voltage as a design knob . . . 45
5.13 Modern transistor shapes . . . 46
5.14 Summary . . . 46
5.15 Exercises . . . 46

6 A Short Tour of Analog Circuits 48
6.1 The transistor as an amplifier . . . 48
6.2 The operational amplifier . . . 49
6.3 The bipolar junction transistor, briefly . . . 50
6.4 Filters . . . 51
6.5 Converting between analog and digital . . . 51
6.6 Power supplies . . . 52
6.7 Summary . . . 53
6.8 Exercises . . . 53

III From Transistors to Logic 55

7 Binary Numbers, Bits and Boolean Algebra 56
7.1 Why two values? . . . 56
7.2 Binary numbers . . . 56
7.3 Hexadecimal . . . 57
7.4 How many values fit in n bits? . . . 57
7.5 Negative numbers: two's complement . . . 58
7.6 Bitwise operations . . . 59
7.7 Gray code . . . 60

7.8 Boolean algebra . . . 61
7.9 From a truth table to an expression . . . 62
7.10 Karnaugh maps . . . 63
7.11 Summary . . . 64
7.12 Exercises . . . 64

8 Building Logic Gates from Transistors 66
8.1 The rule of complementary logic . . . 66
8.2 The CMOS inverter . . . 66
8.3 The voltage transfer characteristic . . . 67
8.4 The swapped inverter . . . 68
8.5 NAND and NOR gates . . . 70
8.6 Designing any gate . . . 70
8.7 Why NAND is preferred over NOR . . . 72
8.8 Pass transistors and transmission gates . . . 72
8.9 Tristate buffers . . . 73
8.10 Other logic styles . . . 73
8.11 Summary . . . 74
8.12 Exercises . . . 75

9 Combinational Building Blocks 77
9.1 Universal gates . . . 77
9.2 The multiplexer . . . 77
9.3 Building any gate from a 2:1 multiplexer . . . 78
9.4 Decoders, encoders and demultiplexers . . . 80
9.5 Adding binary numbers . . . 80
9.6 XOR chains and parity . . . 81
9.7 Delay through combinational logic . . . 82
9.8 Summary . . . 83
9.9 Exercises . . . 84

IV Memory and Time 86

10 Latches and Flip-Flops 87
10.1 Memory from feedback . . . 87
10.2 The SR latch . . . 88

10.3 The D latch . . . 88
10.4 The edge-triggered D flip-flop . . . 89
10.5 A D flip-flop from two 2:1 multiplexers . . . 90
10.6 Latch versus flip-flop . . . 91
10.7 Other kinds of flip-flop . . . 91
10.8 Set, reset and enable . . . 92
10.9 Metastability . . . 93
10.10 Summary . . . 93
10.11 Exercises . . . 94

11 Timing: Setup, Hold and Clock Speed 95
11.1 Synchronous design . . . 95
11.2 The clock . . . 95
11.3 Timing parameters . . . 96
11.4 The setup constraint: how fast can the clock go? . . . 96
11.5 The hold constraint: data arriving too early . . . 97
11.6 Clock skew and jitter . . . 98
11.7 Slack . . . 99
11.8 Reading timing diagrams . . . 99
11.9 Shift registers and hold time . . . 100
11.10 Pipelining . . . 101
11.11 Long wires and retiming flip-flops . . . 102
11.12 Time borrowing with latches . . . 102
11.13 Special paths and sign-off corners . . . 102
11.14 Making and distributing the clock . . . 103
11.15 Summary . . . 103
11.16 Exercises . . . 103

12 Counters, Clock Dividers and State Machines 105
12.1 Counters . . . 105
12.2 Clock dividers . . . 107
12.3 Finite state machines . . . 109
12.4 Example: how many flip-flops for a traffic light? . . . 110
12.5 Example: a sequence detector . . . 110
12.6 Other useful counters . . . 111
12.7 Summary . . . 112

12.8 Exercises . . . 112

V Speed, Power and the Real World 114

13 Delay: Why Circuits Take Time 115

13.1 The RC model of a gate . . . 115
13.2 Rise, fall and propagation . . . 116
13.3 How sizing changes delay . . . 117
13.4 Driving a big load: buffer chains . . . 117
13.5 Logical effort . . . 117
13.6 Elmore delay: delays through networks . . . 118
13.7 Wires . . . 119
13.8 Other effects on delay . . . 119
13.9 Summary . . . 120
13.10 Exercises . . . 120

14 Power: Where the Energy Goes 122

14.1 Three kinds of power . . . 122
14.2 Dynamic power . . . 122
14.3 Scaling questions . . . 123
14.4 Power versus energy . . . 124
14.5 Short-circuit power . . . 124
14.6 Leakage power . . . 125
14.7 Saving power . . . 125
14.8 The bigger picture: Dennard scaling and dark silicon . . . 127
14.9 Summary . . . 127
14.10 Exercises . . . 128

15 Process, Voltage and Temperature 130

15.1 Process variation . . . 130
15.2 Voltage variation . . . 131
15.3 Temperature . . . 131
15.4 Temperature inversion . . . 132
15.5 Corners for sign-off . . . 133
15.6 Aging . . . 134
15.7 Living with variation . . . 134

15.8 Summary . . . 134
15.9 Exercises . . . 134

VI Building Real Chips **136**

16 Wires, Crosstalk and Signals Between Chips **137**
16.1 The metal stack . . . 137
16.2 Wire resistance . . . 138
16.3 Wire capacitance and crosstalk . . . 138
16.4 Signals between chips . . . 139
16.5 Bandwidth calculations . . . 140
16.6 Packets and address alignment . . . 141
16.7 Signal integrity on the board . . . 141
16.8 Networking: Ethernet, IP and UDP . . . 142
16.9 Summary . . . 144
16.10 Exercises . . . 144

17 Manufacturing and Reliability **146**
17.1 From sand to wafer . . . 146
17.2 Photolithography: printing the patterns . . . 146
17.3 Other process steps . . . 147
17.4 Test, dicing and packaging . . . 147
17.5 Yield . . . 147
17.6 The design flow . . . 148
17.7 The antenna effect . . . 149
17.8 Electromigration . . . 150
17.9 Latch-up . . . 151
17.10 Electrostatic discharge . . . 151
17.11 Aging and soft errors . . . 151
17.12 Testing manufactured chips . . . 152
17.13 Summary . . . 152
17.14 Exercises . . . 153

18 Memory Circuits **155**
18.1 Kinds of memory . . . 155
18.2 How a memory array is organized . . . 155

18.3 The 6T SRAM cell . . . 156
18.4 Sense amplifiers . . . 157
18.5 DRAM . . . 157
18.6 Flash memory . . . 159
18.7 Register files, CAMs and ROMs . . . 159
18.8 Protecting stored data: ECC . . . 159
18.9 Summary . . . 160
18.10 Exercises . . . 160

VII Digital Systems 162

19 Describing and Verifying Hardware: RTL, Verilog, Clock Domains and FIFOs 163
19.1 An HDL describes hardware, not steps . . . 163
19.2 Verilog in one page . . . 163
19.3 Blocking and nonblocking assignments . . . 164
19.4 Accidental latches . . . 166
19.5 Resets . . . 166
19.6 Clock domain crossing . . . 167
19.7 FIFOs . . . 169
19.8 How deep must a FIFO be? . . . 169
19.9 Handshakes and arbiters . . . 170
19.10 Verification basics . . . 171
19.11 SystemVerilog for design . . . 174
19.12 Reading VHDL . . . 177
19.13 FPGAs: where your RTL lands . . . 178
19.14 Summary . . . 180
19.15 Exercises . . . 181

20 Computer Architecture 183
20.1 What a processor does . . . 183
20.2 Measuring performance . . . 184
20.3 The five-stage pipeline . . . 184
20.4 Hazards . . . 185
20.5 Amdahl's law . . . 185
20.6 The memory hierarchy . . . 186
20.7 How a cache works . . . 186

20.8 Average memory access time . . . 188
20.9 Virtual memory . . . 188
20.10 Endianness . . . 189
20.11 Multiple cores . . . 189
20.12 Out-of-order execution . . . 190
20.13 Inside a modern core: prediction, prefetching, SIMD and memory ordering . . . 191
20.14 On-chip buses: AXI . . . 192
20.15 Summary . . . 192
20.16 Exercises . . . 193

21 GPUs and Parallel Computing **195**
21.1 Two philosophies: latency and throughput . . . 195
21.2 How a GPU is organized . . . 196
21.3 Hiding latency with many threads . . . 196
21.4 Two performance pitfalls . . . 197
21.5 The CUDA programming model . . . 197
21.6 Matrix multiplication and tensor cores . . . 198
21.7 Number formats . . . 198
21.8 Compute-bound or memory-bound? . . . 199
21.9 Summary . . . 200
21.10 Exercises . . . 200

22 Computer Arithmetic: How Hardware Does Math **202**
22.1 When a result does not fit: carry, overflow and saturation . . . 202
22.2 Shifting . . . 203
22.3 Adding faster . . . 204
22.4 Multiplication . . . 205
22.5 Division . . . 207
22.6 Fixed point: fractions on integer hardware . . . 207
22.7 Floating-point arithmetic . . . 208
22.8 How arithmetic hardware is checked . . . 211
22.9 Summary . . . 211
22.10 Exercises . . . 212

VIII The Assessment **214**

23 Programming for Hardware Engineers **215**

23.1 What the coding section looks like . . . 215
23.2 C essentials . . . 215
23.3 Output-prediction classics . . . 216
23.4 Structs, padding and alignment . . . 216
23.5 Linked lists . . . 217
23.6 Bit manipulation problems . . . 219
23.7 Complexity in one paragraph . . . 220
23.8 Hardware-flavored C . . . 220
23.9 Summary . . . 220
23.10 Exercises . . . 221

24 Aptitude and Puzzles **222**

24.1 Work and rates . . . 222
24.2 Speed, distance and time . . . 222
24.3 Percentages and compound growth . . . 223
24.4 Factorials and trailing zeros . . . 223
24.5 Probability . . . 223
24.6 Counting . . . 224
24.7 Classic puzzles . . . 224
24.8 Binary search . . . 224
24.9 Estimation . . . 224
24.10 Exercises . . . 225

25 A Full Practice Assessment **226**

25.1 Part A: Hardware (multiple choice and true/false) . . . 226
25.2 Part B: What does this C code print? . . . 229
25.3 Part C: Aptitude . . . 230
25.4 Part D: Coding . . . 230
25.5 Answer key . . . 230

A Formula Sheet **234**

B Glossary **238**

C A Twelve-Month Plan and Resources **243**

D Beyond This Book: Gaps and Projects **246**

D.1 What the four books cover together 246
D.2 What is still missing 247
D.3 Check your eligibility 247
D.4 What the job description asks for 248
D.5 The plan: from transistor to software 248
D.6 Project 1: A pipelined RISC-V processor on an FPGA 249
D.7 Project 2: A systolic-array matrix multiplier (a mini tensor core) 250
D.8 Project 3: Put a design on a real chip with Tiny Tapeout 250
D.9 Project 4: Optimizing matrix multiplication in CUDA 251
D.10 Project 5: Characterizing standard cells in SPICE 252
D.11 Track-specific options 252
D.12 When to do what 253
D.13 Making the projects count 253

E Sources **254**

Part I

Electricity, from Zero

Chapter 1

Charge, Current, Voltage and Resistance

Every chip, from a calculator to a GPU, is a machine for moving tiny amounts of electric charge around. Before we can talk about transistors or logic gates, we need four words that every later chapter uses: *charge*, *current*, *voltage* and *resistance*. This chapter builds each one from scratch, then uses them to analyze simple circuits.

1.1 Charge: the thing that moves

All matter is made of atoms. An atom has a small, heavy center (the **nucleus**, made of protons and neutrons) surrounded by much lighter **electrons**. Protons carry a positive electric **charge**; electrons carry an equal amount of negative charge. Neutrons carry none.

Charges push and pull on each other:

- Opposite charges attract (a positive and a negative pull together).
- Like charges repel (two negatives push apart, as do two positives).

Charge is measured in **coulombs** (C). One electron carries 1.6×10^{-19} C, so one coulomb is about 6.2×10^{18} electrons. That is a huge number of electrons, which is why everyday electricity feels continuous rather than like individual particles.

In some materials, the outermost electrons are only loosely held by their atoms and can wander from atom to atom. These materials are **conductors**: metals such as copper, aluminum, gold and silver. In other materials every electron is tightly bound, so charge cannot move; these are **insulators**: glass, plastic, rubber, air, and silicon dioxide (which matters a lot later). Between the two are **semiconductors**, like silicon, whose ability to conduct can be controlled. Semiconductors get their own chapter (Chapter 4).

1.2 Current: how fast charge flows

Current is the rate at which charge flows past a point. If 1 coulomb passes a point every second, the current is 1 **ampere** (A), usually shortened to "amp."

$$I = \frac{\Delta Q}{\Delta t} \qquad \text{(current = charge per unit time)}$$

Typical sizes:

- A single logic gate switching: microamps (10^{-6} A) or less.
- A phone charging: about 1–3 A.
- A large GPU at full load: hundreds of amps, delivered at around 1 V.

An everyday picture

Think of a wire as a pipe full of water. Current is how much water flows past a point each second. A wide river moving slowly and a narrow jet moving fast can carry the same flow.

Which direction does current flow? By an old convention (chosen before electrons were discovered), current is drawn flowing from the positive terminal, through the circuit, to the negative terminal. Electrons, being negative, actually drift the opposite way. Engineers always use the convention, **conventional current**, and so does this book. The direction almost never matters for understanding a circuit, as long as you are consistent.

1.3 Voltage: the push

Charge does not move on its own; something has to push it. **Voltage** is that push. More precisely, voltage is the *energy per unit charge* available between two points:

$$V = \frac{\text{energy}}{\text{charge}}, \qquad 1 \text{ volt} = 1 \text{ joule per coulomb.}$$

An everyday picture

Voltage is like water pressure, or like height. Water flows from high pressure to low pressure, and a ball rolls from high ground to low ground. Charge flows from high voltage to low voltage. A battery is a pump that keeps one end at high pressure and the other at low pressure.

The most important fact about voltage is this:

The big idea

Voltage is always measured *between two points*. "This wire is at 5 volts" really means "this wire is 5 volts higher than some reference point."

A height of "100 meters" only means something if you know where you measured from (sea level? the floor?). Voltage is the same.

1.4 Ground: the reference point

Because voltage is always relative, every circuit picks one node as the reference and calls it 0 V. That node is called **ground**, written GND, and drawn with this symbol:



When someone says a chip's supply is "1.0 V", they mean the supply wire is 1.0 V above that chip's ground.

Ground is not magic. It is just a wire (or, on a circuit board, a large copper plane) that everything agrees to measure from. Several practical rules follow from this.

Every connected device must share ground. Suppose board A sends a "1" to board B as 3.3 V (Figure 1.1). That 3.3 V is measured relative to A's ground. If B's ground is not connected to A's ground, B has no idea what A's 3.3 V means: the two boards have no common reference, and the signal is meaningless. This is why every cable between two devices carries a ground wire as well as the signal wires, and why a missing ground connection is one of the most common reasons a circuit "does nothing."

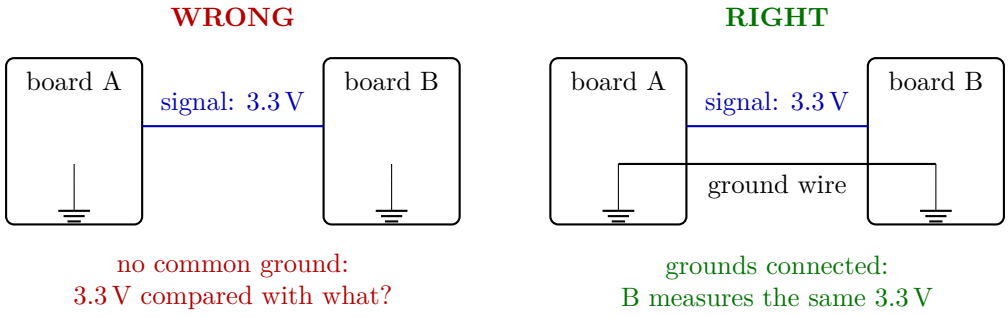


Figure 1.1: Two boards must share ground. Without a ground wire (left), board A's 3.3 V has no meaning to board B. With the grounds connected (right), both measure from the same reference, and the signal current has a path back.

Current needs a return path. Charge flows in loops. Current that leaves a power supply through the supply wire must come back through ground. If the ground path is broken, no current flows at all, just as water cannot circulate in a pipe loop with a gap in it.

Floating inputs. An input that is connected to nothing (neither a voltage nor ground) is called **floating**. Its voltage is undefined: it drifts with stray charge and electrical noise, so a digital input reads randomly as 0 or 1. Never leave a logic input floating (Figure 1.2). Tie it to the supply or to ground, directly or through a resistor.

Pull-up and pull-down resistors. A common fix for a floating input is a resistor to the supply (a **pull-up**) or to ground (a **pull-down**). In Figure 1.3, when the button is open, no current flows

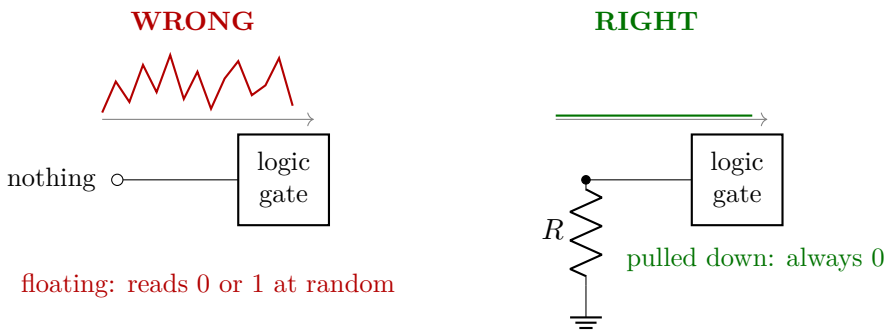


Figure 1.2: A floating input (left) is connected to nothing, so stray charge and noise move it and it reads randomly. A pull-down resistor (right) ties it to a definite 0; a pull-up would tie it to 1.

through the resistor, so the input sits at V_{DD} and reads 1. When the button is pressed, the input is connected straight to ground and reads 0. The resistor limits the current that flows from the supply to ground while the button is pressed.

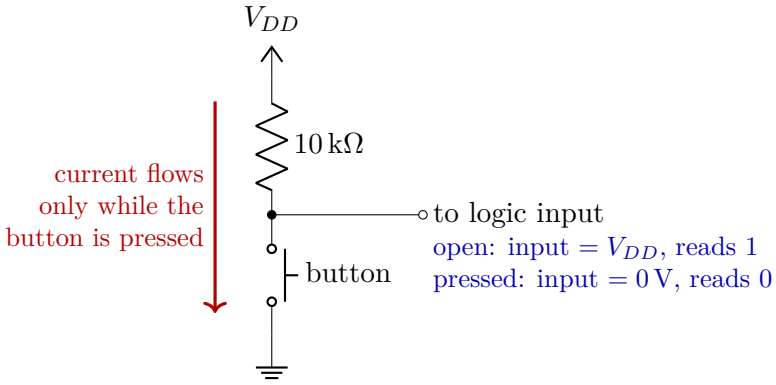


Figure 1.3: A pull-up resistor keeps the input at a defined 1 until the button connects it to ground. Red: the current, which flows only while the button is pressed. Blue: what the input sees.

Ground is not perfectly 0 V everywhere. Real ground wires have a little resistance. When large currents flow through them, different points on "ground" end up at slightly different voltages. This effect, called **ground bounce** when it happens quickly, returns in Chapters 2 and 15.

Seen on the assessment

Candidates have reported questions about GND connections. Whatever the exact wording, the ideas above are the ones being tested: ground is the shared reference, current needs a return path through it, devices must share ground to communicate, and inputs must never float.

1.5 Resistance and Ohm's law

Even in a good conductor, moving charge bumps into atoms and loses energy as heat. **Resistance** measures how strongly a component opposes current. It is measured in **ohms** (Ω).

For many materials, current is proportional to the voltage across them. This is **Ohm's law**:

$$\boxed{V = IR} \qquad \text{equivalently} \qquad I = \frac{V}{R}, \quad R = \frac{V}{I}.$$

An everyday picture

Resistance is how narrow the pipe is. For the same pressure (voltage), a narrow pipe (high resistance) lets less water (current) through.

Worked example: Ohm's law

A 1 kΩ resistor (1000 ohms) is connected across a 5 V battery. What current flows?

$$I = \frac{V}{R} = \frac{5\ \text{V}}{1000\ \Omega} = 0.005\ \text{A} = 5\ \text{mA}.$$

What sets a wire's resistance? A wire's resistance depends on its material and shape:

$$R = \rho \frac{L}{A}$$

where L is the length, A is the cross-sectional area, and ρ (rho) is the material's **resistivity**. Copper's resistivity is about 1.7×10^{-8} Ω·m; aluminum's is about 2.7×10^{-8}. So:

- A wire twice as long has twice the resistance (charge must travel further).
- A wire with twice the cross-section has half the resistance (more room to flow).
- Copper conducts better than aluminum, which is one reason modern chips use copper wiring.

Resistivity also changes with temperature; for metals it rises as they heat up. That fact is central to Chapter 15.

1.6 Power: how fast energy is used

When current flows through a resistance, electrical energy turns into heat. **Power** is the rate of energy use, measured in **watts** (W): one watt is one joule per second.

$$\boxed{P = VI}$$

Combining with Ohm's law gives two more forms that are useful when you know only two of the three quantities:

$$P = I^2 R = \frac{V^2}{R}.$$

Worked example: power in a GPU

A GPU runs at a supply of 0.9 V and draws 400 A. How much power does it use?

$$P = VI = 0.9 \times 400 = 360 \text{ W}.$$

All 360 W ends up as heat that the cooling system must remove (Chapter 3).

Energy versus power. Energy is power multiplied by time: a 360 W GPU running for one hour uses $360 \times 3600 = 1.3$ million joules. Power is how fast you spend energy; energy is how much you spent in total. Batteries are rated in energy; chips are usually rated in power. This distinction matters in Chapter 14.

1.7 Circuits: closed loops, opens and shorts

A **circuit** is a closed loop through which current can flow: from a source, through components, and back. Two failure modes appear constantly (Figure 1.4):

- An **open circuit** is a break in the loop. No current flows, no matter how high the voltage. A switch that is "off" is an open circuit.
- A **short circuit** is a path with (nearly) zero resistance. By Ohm's law, $I = V/R$ becomes very large. A short across a power supply can melt wires or destroy the supply.

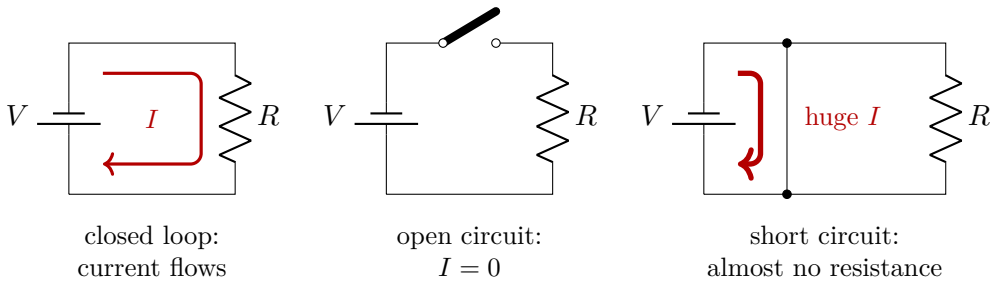


Figure 1.4: A closed loop carries current (left). An open circuit, here a switch that is off, breaks the loop, so no current flows (middle). A short circuit is a path with almost no resistance: it bypasses R, and the current becomes very large (right). Red arrows show the current.

1.8 Series and parallel

Components can be connected in two basic ways (Figure 1.5).

Series. Components in series sit one after another on a single path. The *same current* flows through each, and their voltages add up. Resistances in series add:

$$R_{\text{series}} = R_1 + R_2 + \cdots$$

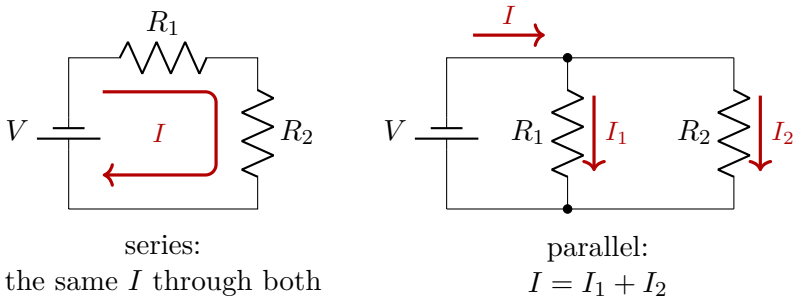


Figure 1.5: Series (left): one path, so the same current flows through both. Parallel (right): two paths, so the current splits, and both resistors have the same voltage across them. Red arrows show the current.

Parallel. Components in parallel connect across the same two nodes. Each has the *same voltage*, and their currents add. Resistances in parallel combine as

$$\frac{1}{R_{\text{parallel}}} = \frac{1}{R_1} + \frac{1}{R_2} + \cdots, \qquad \text{and for two:} \quad R_{\text{parallel}} = \frac{R_1 R_2}{R_1 + R_2}.$$

Parallel resistance is always *smaller* than the smallest resistor, because each new path gives current another way through. Two equal resistors R in parallel give $R/2$.

The big idea

Series: same current, resistances add. Parallel: same voltage, the result is smaller than either. This exact pattern returns when we build logic gates: transistors in series make it *harder* for current to flow, and transistors in parallel make it *easier*.

Worked example: series and parallel together

A $100\,\Omega$ resistor is in series with two $200\,\Omega$ resistors that are in parallel with each other. A $12\,\text{V}$ source drives the combination. Find the total current.
The parallel pair: $200 \parallel 200 = 100\ \Omega$. Add the series resistor: $100 + 100 = 200\ \Omega$. Current: $I = 12/200 = 0.06\,\text{A} = 60\,\text{mA}$. Each parallel resistor carries half of it, $30\,\text{mA}$.

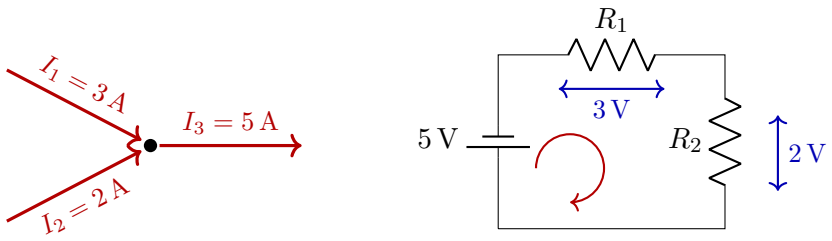
1.9 Kirchhoff's laws

Two rules, both forms of "nothing is created or lost," let you solve any circuit (Figure 1.6).

Kirchhoff's current law (KCL). The total current flowing into a node equals the total current flowing out. Charge does not pile up in a wire junction.

$$\sum I_{\text{in}} = \sum I_{\text{out}}.$$

Kirchhoff's voltage law (KVL). Around any closed loop, the voltage rises (from sources) equal the voltage drops (across components). If you walk around a loop and return to your start, you



current law: in = out, $3 + 2 = 5$ voltage law: rise = drops, $5 = 3 + 2$

Figure 1.6: Kirchhoff's laws. Current law: the current flowing into a node equals the current flowing out (left). Voltage law: around a loop, the rise from the source equals the drops across the components (right). Red: currents and the direction of the walk around the loop; blue: voltages.

are back at the same "height."

$$\sum V_{\text{around a loop}} = 0.$$

1.10 The voltage divider

The single most useful small circuit is the **voltage divider**: two resistors in series across a voltage, with the output taken from the middle (Figure 1.7).

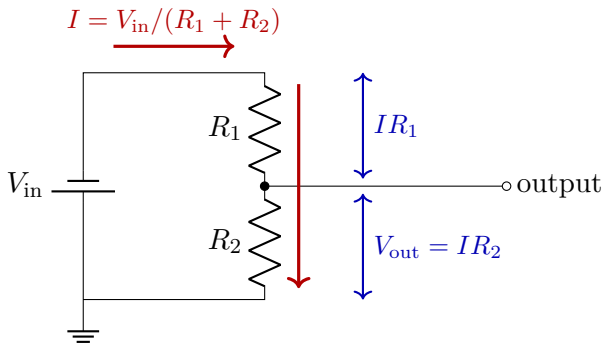


Figure 1.7: A voltage divider. The same current I (red) flows through both resistors, so the input voltage splits between them in proportion to their resistances (blue). The output is the part across R_2.

The same current $I = V_{\text{in}}/(R_1 + R_2)$ flows through both resistors (they are in series). The output is the voltage across R_2:

$$\boxed{V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2}}$$

Worked example: divider

$V_{\text{in}} = 5\,\text{V}$, $R_1 = 3\,\text{k}\Omega$, $R_2 = 2\,\text{k}\Omega$. Then $V_{\text{out}} = 5 \times 2/(3 + 2) = 2\,\text{V}$.

You will meet the divider idea again and again: whenever two things "fight" over a node, one

pulling up and one pulling down, the node settles where their strengths balance. This is exactly what happens inside a logic gate when both transistors are partly on (Chapter 8) and inside an SRAM cell during a read (Chapter 18).

1.11 Real sources and IR drop

A real power supply is not a perfect voltage source. The supply itself, and every wire, connector, board trace and package pin between the supply and the chip, has some resistance. Current flowing through that resistance causes a voltage drop, $V = IR$, called **IR drop** (I times R).

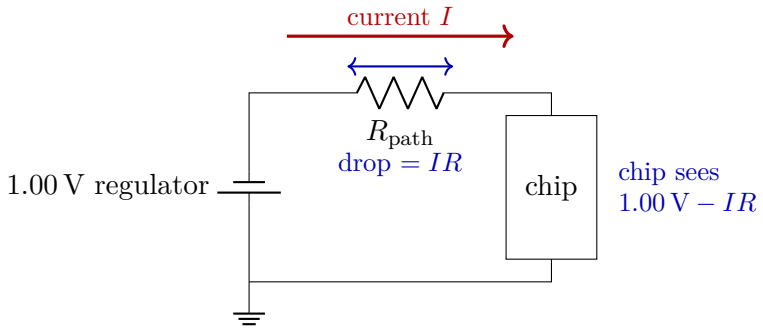


Figure 1.8: Every bit of resistance between the regulator and the chip steals some voltage. The current I (red) drops IR across the path (blue), so the chip gets less than the regulator's 1.00 V.

Worked example: voltage at the chip's power pins

A voltage regulator outputs 1.00 V. The path to the chip has a total resistance of 0.25 mΩ (0.25×10^{-3} Ω). The chip draws 200 A. What voltage reaches the chip?

$$V_{\text{drop}} = IR = 200 \times 0.00025 = 0.05 \text{ V}, \qquad V_{\text{chip}} = 1.00 - 0.05 = 0.95 \text{ V}.$$

A quarter of a milliohm sounds like nothing, but at hundreds of amps it costs 5% of the supply. Lower voltage makes transistors slower (Chapter 15), so power delivery design matters.

Notice why high-power chips are hard: power is $V \times I$, and chip voltages are low (around 1 V), so a few hundred watts means a few hundred amps. Large currents make even tiny resistances matter.

1.12 Measuring voltage and current

A **voltmeter** measures the voltage between two points, so it connects *across* (in parallel with) the thing being measured (Figure 1.9). A good voltmeter has very high resistance so it draws almost no current and does not disturb the circuit.

An **ammeter** measures the current through a wire, so the wire must be broken and the meter placed *in series*. A good ammeter has very low resistance so it does not add a voltage drop. Connecting an ammeter across a power supply (in parallel) creates a short circuit, a classic lab mistake (Figure 1.10).

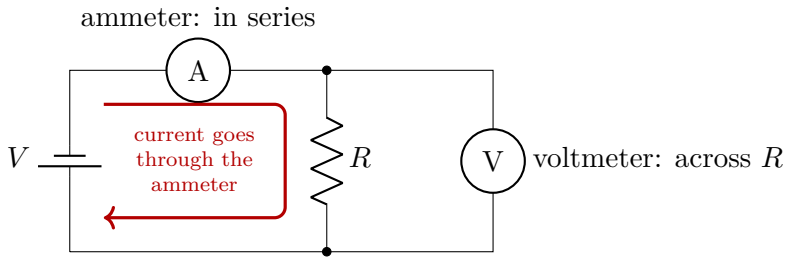


Figure 1.9: Connecting meters. The ammeter goes in series, so the current it measures (red) flows through it. The voltmeter goes across the part whose voltage it measures.

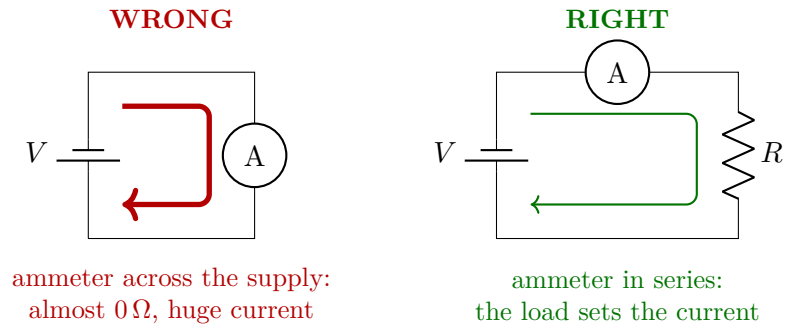


Figure 1.10: The classic mistake. An ammeter has almost no resistance, so connecting it straight across the supply (left) is a short circuit. It belongs in series with the load (right), which limits the current.

1.13 DC, AC and frequency

A **DC** (direct current) signal stays at a constant value, like a battery. An **AC** (alternating current) signal changes over time and repeats, like the mains power in a wall outlet or a clock signal in a chip (Figure 1.11).

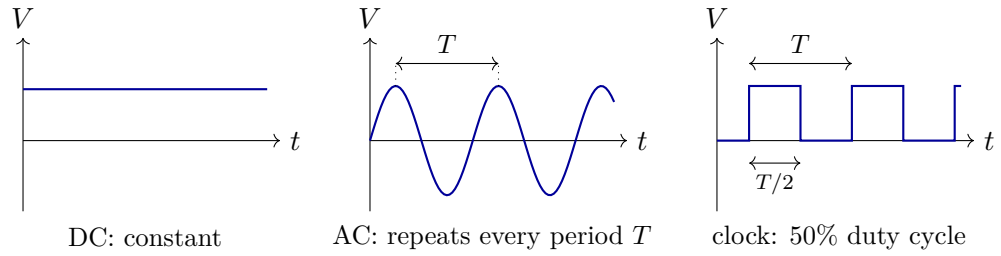
A repeating signal has a **period** T, the time for one full repetition, and a **frequency** f, the number of repetitions per second, measured in **hertz** (Hz):

$$f = \frac{1}{T}.$$

A 2 GHz clock (2×10^9 cycles per second) has a period of $1/(2 \times 10^9) = 0.5$ ns. Square-wave signals that switch between a low and a high level also have a **duty cycle**: the fraction of each period spent high. A clock that is high for half its period has a 50% duty cycle.

1.14 Summary

- Charge (coulombs) is what moves. Current (amps) is how fast it flows. Voltage (volts) is the push, always measured between two points.
- Ground is the shared 0 V reference. Connected devices must share ground, current returns through it, and inputs must never float.



DC: constant AC: repeats every period T clock: 50% duty cycle

Figure 1.11: A DC signal stays constant (left). An AC signal repeats every period T (middle). A clock is a square wave; with a 50% duty cycle it is high for half of each period (right).

- Ohm's law: $V = IR$. Power: $P = VI = I^2R = V^2/R$.
- Series: same current, resistances add. Parallel: same voltage, resistance falls.
- KCL: current in equals current out at a node. KVL: voltages around a loop sum to zero.
- Voltage divider: $V_{\text{out}} = V_{\text{in}} R_2/(R_1 + R_2)$.
- IR drop: big currents through small resistances still lose voltage.

1.15 Exercises

1. A $470\,\Omega$ resistor has $2.35\,\text{V}$ across it. What current flows through it?
2. Three $300\,\Omega$ resistors are connected in parallel. What is the total resistance?
3. A $10\,\Omega$ resistor carries $2\,\text{A}$. How much power does it dissipate?
4. In a voltage divider, $V_{\text{in}} = 3.3\,\text{V}$, $R_1 = 10\,\text{k}\Omega$, $R_2 = 20\,\text{k}\Omega$. Find V_{out}.
5. True or false: if a wire is made twice as long and twice as wide (in cross-sectional area), its resistance is unchanged.
6. A microcontroller on board A sends signals to board B over a single wire, the signal wire, and nothing else. Board B reads random values. What is the most likely problem?
7. A chip draws $150\,\text{A}$ from a $0.80\,\text{V}$ regulator through $0.4\,\text{m}\Omega$ of path resistance. What voltage reaches the chip, and how much power is wasted in the path?
8. Why is an unconnected (floating) digital input a problem, and what are two ways to fix it?

Answers

1. $I = V/R = 2.35/470 = 0.005\,\text{A} = 5\,\text{mA}$.
2. Three equal resistors in parallel: $300/3 = 100\ \Omega$.
3. $P = I^2R = 2^2 \times 10 = 40\,\text{W}$.
4. $V_{\text{out}} = 3.3 \times 20/(10 + 20) = 2.2\,\text{V}$.
5. True. $R = \rho L/A$: doubling L doubles R, doubling A halves it; the effects cancel.

6. There is no shared ground. Board A's signal voltage is relative to A's ground, so without a ground wire B has no reference (and the current has no return path). Add a ground connection between the boards.

7. Drop $= 150 \times 0.0004 = 0.06\,\text{V}$, so the chip sees $0.74\,\text{V}$. Power lost in the path $= I^2R = 150^2 \times 0.0004 = 9\,\text{W}$.

8. Its voltage is undefined, so noise makes it read randomly as 0 or 1 (and it can also make the input circuit waste power). Fix it with a pull-up resistor to the supply or a pull-down resistor to ground, or by tying it directly to a defined level.

Chapter 2

Capacitors, Inductors and the Role of Time

Resistors respond instantly: change the voltage and the current changes at the same moment. Real circuits also contain components that *store energy*, and stored energy takes time to fill and to drain. Those components, capacitors and inductors, are the reason every signal in a chip takes time to switch, the reason chips need dozens of small capacitors around them, and the reason fast signals on a circuit board can bounce like waves. This chapter explains all three.

2.1 The capacitor

A **capacitor** is two conductors ("plates") separated by a thin insulator (Figure 2.1). Connect it to a voltage and charge flows onto the plates: positive charge piles up on one plate and an equal negative charge on the other. The insulator stops charge from crossing, so the charge stays there, held by the attraction between the plates.

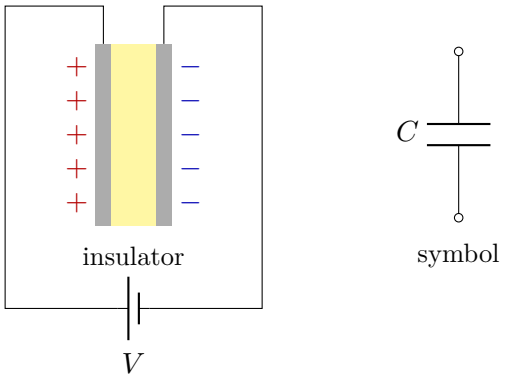


Figure 2.1: A capacitor is two conducting plates separated by an insulator (left). The source pushes positive charge onto one plate and an equal negative charge onto the other, and the charge stays there. Its circuit symbol (right).

The amount of charge stored is proportional to the voltage across the capacitor:

$$\boxed{Q = CV}$$

The constant C is the **capacitance**, measured in **farads** (F). A farad is enormous; real values are written with prefixes:

Prefix	Value	Typical example
femtofarad (fF)	10^{-15} F	the gate of one small transistor
picofarad (pF)	10^{-12} F	a long on-chip wire, an I/O pin
nanofarad (nF)	10^{-9} F	a small ceramic capacitor on a board
microfarad (μF)	10^{-6} F	a typical board-level decoupling capacitor

For two flat plates of area A separated by a gap d filled with an insulator of permittivity ε,

$$C = \varepsilon \frac{A}{d}.$$

Bigger plates hold more charge; a thinner gap holds more charge because the plates attract each other more strongly.

The big idea

Every wire and every transistor gate in a chip *is* a capacitor, whether we want it or not. A wire running next to another wire, or above the silicon, forms plates separated by insulation. A transistor's gate sits on a very thin insulator above the silicon. So every time a signal switches, some capacitance must be charged or discharged. That takes time (delay) and energy (power). Most of Chapters 13 and 14 follow from this sentence.

An everyday picture

A capacitor is like a water tank. The water level is the voltage; the amount of water is the charge. A wide tank (large capacitance) needs a lot of water to raise its level a little. Filling a tank through a pipe takes time, and the narrower the pipe (larger resistance), the longer it takes.

2.2 Current into a capacitor

Since $Q = CV$, changing the voltage requires moving charge, which is a current:

$$\boxed{I = C\,\frac{dV}{dt}}$$

In words: the current into a capacitor equals its capacitance times how fast its voltage is changing. Two consequences:

- A capacitor's voltage *cannot change instantly.* An instant change would need infinite current. Voltage across a capacitor always changes smoothly.
- When the voltage is steady, no current flows into the capacitor. At DC, a capacitor acts like an open circuit.

2.3 Energy stored in a capacitor

A charged capacitor holds energy:

$$\boxed{E = \tfrac{1}{2}CV^2}$$

The square on V is important. Doubling the voltage stores four times the energy. This is why the power a chip burns when it switches grows with the *square* of the supply voltage (Chapter 14).

2.4 Charging through a resistor: the RC time constant

Connect a capacitor, initially at 0 V, through a resistor R to a voltage V_s (Figure 2.2).

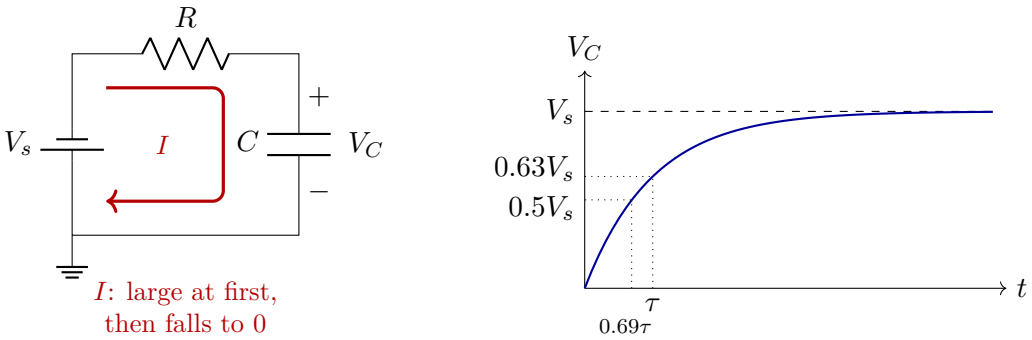


Figure 2.2: Charging a capacitor through a resistor. The current (red) is largest at the start, when the whole of V_s is across R; as V_C rises, the current falls, so the voltage rises quickly at first, then slows as it approaches V_s.

At the first instant the capacitor is at 0 V, so the full V_s appears across the resistor and current is large: $I = V_s/R$. As the capacitor charges, its voltage rises, the voltage left across the resistor shrinks, and the current falls. Charging slows down as it goes. The result is an exponential curve:

$$V_C(t) = V_s\left(1 - e^{-t/RC}\right).$$

The product $\tau = RC$ is the **time constant** (ohms times farads gives seconds). These checkpoints are worth memorizing:

Time	Capacitor voltage
0.69 RC	50% of final value
1 RC	63%
2.2 RC	time to go from 10% to 90% (the “rise time”)
3 RC	95%
5 RC	over 99% (“fully” charged)

Discharging works the same way in reverse: a capacitor at V_0 discharging through R follows $V_C(t) = V_0 e^{-t/RC}$ and reaches half its starting voltage after 0.69 RC.

Worked example: an on-chip RC delay

A transistor that behaves like a 2 kΩ resistor drives a wire with 50 fF of capacitance. How long until the wire reaches half the supply voltage?
$\tau = RC = 2000 \times 50 \times 10^{-15} = 10^{-10}\,\text{s} = 100\,\text{ps}$. Half-voltage takes $0.69\,\tau \approx 69\,\text{ps}$.

This little calculation is the heart of all delay estimates in chip design: a transistor is a resistor, the thing it drives is a capacitor, and the delay is about $0.69\,RC$.

2.5 Capacitors in series and parallel

Capacitors combine the *opposite* way from resistors:

- **Parallel capacitors add**: $C = C_1 + C_2$. Side by side, they act like one capacitor with more plate area.
- **Series capacitors combine like parallel resistors**: $1/C = 1/C_1 + 1/C_2$. In series, the gap effectively gets wider, so capacitance falls.

2.6 The inductor

An **inductor** is usually a coil of wire. Current through it creates a magnetic field, and the field stores energy. Changing the current means changing the field, and the inductor fights that change by producing a voltage:

$$\boxed{V = L\,\frac{dI}{dt}}$$

L is the **inductance**, measured in **henries** (H). Practical values are nanohenries (nH) to microhenries (μH). The stored energy is $E = \frac{1}{2}LI^2$.

An everyday picture

An inductor is like a heavy water wheel in a pipe. Once water is moving, the wheel's momentum keeps it moving; to speed the flow up or slow it down quickly, you have to push hard. A capacitor resists sudden changes in *voltage*; an inductor resists sudden changes in *current*.

Every wire has inductance. Coils are not the only inductors. Any loop of current, including the path from a power supply to a chip and back through ground, has some inductance. A rule of thumb is roughly 1 nH per millimeter of a thin wire, such as the bond wires inside an older chip package. Small numbers, but chips change their current very fast.

Worked example: why fast current changes are dangerous

The power path into a chip package has an effective inductance of 10 pH (10×10^{-12} H). The chip's current jumps by 50 A in 10 ns when a big block starts working. What voltage appears

across that inductance?

$$V = L\frac{dI}{dt} = 10 \times 10^{-12} \times \frac{50}{10 \times 10^{-9}} = 0.05 \text{ V}.$$

On a 0.8 V supply, a 50 mV dip is over 6%. Faster current changes, or more inductance, make it worse. This dip is called **supply droop** or $L\,di/dt$ noise.

2.7 Decoupling capacitors

Look at any graphics card or motherboard near the processor and you will see dozens or hundreds of tiny capacitors. They are **decoupling capacitors** ("decaps"), and they solve the problem in the last example.

The problem. The voltage regulator that powers a chip is centimeters away. Between them sit board traces, connectors, package pins and solder balls, all with resistance and inductance. When the chip's current demand suddenly jumps, the inductance prevents the regulator's current from arriving quickly ($V = L\,dI/dt$). For a short time the chip is starved and its supply voltage dips.

The solution. Put a capacitor right next to the chip's power pins, between the supply and ground. It is a small reservoir of charge, already charged to the supply voltage. When the chip suddenly needs current, the capacitor supplies it from just millimeters away, through very little inductance, while the far-away regulator catches up.

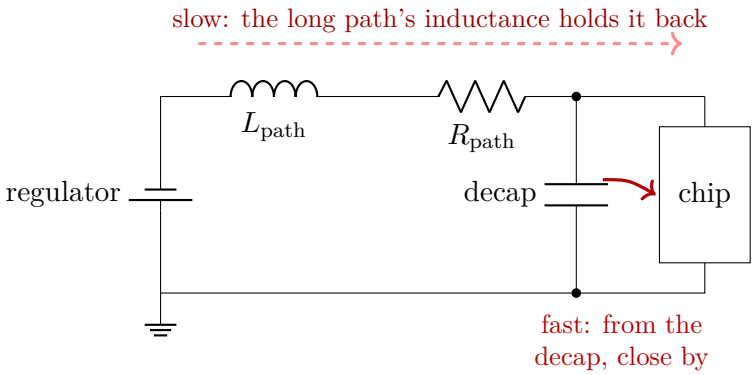


Figure 2.3: A decoupling capacitor next to the chip supplies sudden bursts of current (solid red) that cannot get through the inductance of the long path from the regulator (dashed) quickly enough.

Why it must be close. The capacitor only helps if *its own* path to the chip has little inductance. Every millimeter of trace between the capacitor and the pin adds inductance and undoes the benefit. Real capacitors also have a little built-in inductance, called **ESL** (equivalent series inductance), and resistance, **ESR** (equivalent series resistance). Smaller capacitor packages have lower ESL.

Why several sizes. Different capacitors handle different speeds of change, so designs use a ladder of them:

- **Bulk capacitors** (tens to hundreds of μF) store lots of charge but have higher inductance. They cover slow changes over microseconds.
- **Ceramic capacitors near the chip** (roughly 0.1 to 10 μF, in tiny packages) cover faster changes.
- **Capacitors inside the package and on the die itself** (on-die decap, made from transistor gates or metal layers) cover the fastest changes, within nanoseconds, because they are closest of all.

Seen on the assessment

A reported style of question asks which decoupling arrangement "better supports rapid changes in current, and why." The answer to reason toward: for *fast* changes, what matters is low inductance in the path, so a small, low-ESL capacitor placed as close as possible to the power pins beats a large capacitor far away. A large capacitor stores more total charge, but it cannot deliver it quickly through the inductance of a long path. Large capacitors help with slow, sustained changes.

2.8 How capacitors and inductors respond to frequency

Because a capacitor's current depends on how fast voltage changes, its effect depends on frequency. For a sine wave of frequency f, the capacitor's opposition to current, its **reactance**, is

$$X_C = \frac{1}{2\pi f C}.$$

At DC ($f = 0$) this is infinite: a capacitor blocks steady current. At high frequency it becomes small: a capacitor passes fast changes. An inductor does the opposite, $X_L = 2\pi f L$: it passes DC and blocks fast changes.

A resistor followed by a capacitor to ground is a **low-pass filter**. Slow signals pass through unchanged; fast signals are smoothed away because the capacitor cannot charge and discharge fast enough to follow them. The boundary is the **cutoff frequency**

$$f_c = \frac{1}{2\pi RC}.$$

Every wire on a chip is a little low-pass filter, which is why signals sent over long wires get slow and rounded.

2.9 When wires become transmission lines

So far we assumed a wire's voltage is the same everywhere along it. That is true when signals change slowly compared with the time a signal takes to travel down the wire. Signals do not travel instantly: on a circuit board they move at roughly half the speed of light, about 15 cm per

nanosecond. A signal that switches in 100 ps has its edge spread over only about 1.5 cm of board trace.

When a wire is long compared with the distance a signal edge covers, the signal travels as a wave, and the wire must be treated as a **transmission line**. A transmission line has a **characteristic impedance** Z_0, set by its geometry and materials. Board traces are usually designed for 50 Ω (or 100 Ω for a differential pair).

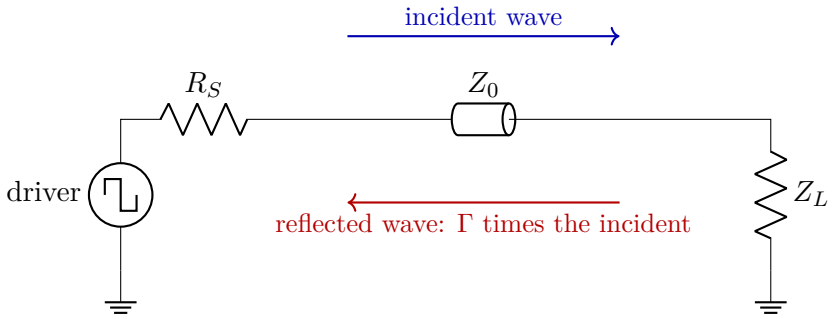


Figure 2.4: A driver sends a wave down a line of impedance Z_0. If the load Z_L does not match Z_0, part of the wave reflects back toward the driver.

Reflections. When a wave reaches the end of the line, some of it bounces back unless the load at the end matches Z_0. The fraction of the voltage that reflects is the **reflection coefficient**:

$$\boxed{\Gamma = \frac{Z_L - Z_0}{Z_L + Z_0}}$$

Load at the end	Γ	What happens
Matched, $Z_L = Z_0$	0	no reflection; the ideal
Open circuit, $Z_L = \infty$	+1	everything reflects, same polarity (voltage doubles at the end)
Short circuit, $Z_L = 0$	−1	everything reflects, inverted
$Z_L > Z_0$	positive	partial reflection, same polarity
$Z_L < Z_0$	negative	partial reflection, inverted polarity

Worked example: reflection from a mismatched load

A 50 Ω trace ends in a 25 Ω load. What fraction of an incoming voltage step reflects, and is it inverted?

$$\Gamma = \frac{25 - 50}{25 + 50} = \frac{-25}{75} = -\frac{1}{3}.$$

One third of the step reflects, and since Γ is negative, the reflection is inverted. With a 150 Ω load instead, $\Gamma = (150 - 50)/(150 + 50) = +0.5$: half reflects, same polarity.

Reflections cause **ringing** (Figure 2.5): the signal overshoots and wobbles before settling, which can make a receiver see extra transitions. The cure is **termination**: adding resistors so the line

sees a matched load at the end (parallel termination) or a matched source at the start (series termination).

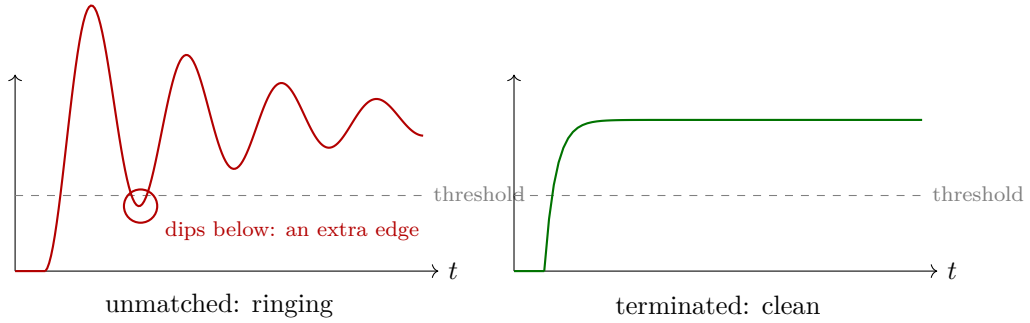


Figure 2.5: The same step at the far end of a line. Unmatched (left), reflections make it overshoot and ring; a dip back below the threshold looks like an extra transition. With termination (right) it settles cleanly.

2.10 Measuring fast signals: bandwidth

A square wave is not as simple as it looks. Mathematically, a square wave of frequency f is a sum of sine waves: one at f (the **fundamental**), plus smaller ones at $3f$, $5f$, $7f$ and so on (the **odd harmonics**). The sharp corners come from those high harmonics (Figure 2.6).

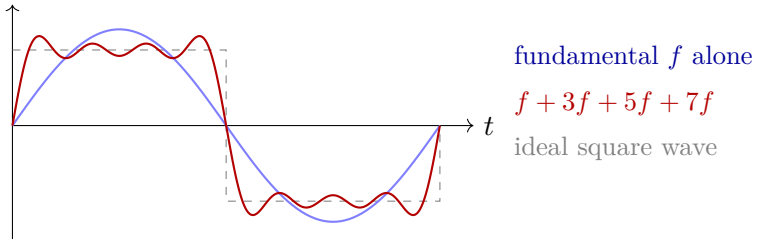


Figure 2.6: A square wave is a sum of sine waves at odd multiples of its frequency. The fundamental alone (blue) is a smooth sine; adding the 3rd, 5th and 7th harmonics (red) sharpens the edges toward the ideal square wave (dashed).

An oscilloscope and its probe can only follow signals up to their **bandwidth**; above it, they act like a low-pass filter. A useful rule links bandwidth to the fastest rise time an instrument can show:

$$t_{\text{rise}} \approx \frac{0.35}{\text{bandwidth}}.$$

Worked example: why a square wave looks like a sine wave

A 100 MHz square wave is viewed with a 150 MHz probe and scope. The fundamental (100 MHz) passes, but the third harmonic (300 MHz) and everything above it are filtered out. With only the fundamental left, the display shows something close to a 100 MHz sine wave. The signal is fine; the measurement is too slow. To see the true edges you need roughly five times the signal frequency in bandwidth, or more.

2.11 Summary

- Capacitor: $Q = CV$, $I = C\,dV/dt$, energy $\frac{1}{2}CV^2$. Its voltage cannot jump; it blocks DC.
- Every wire and transistor gate is a capacitor, so switching costs time and energy.
- RC charging: 50% at $0.69RC$, 63% at RC, essentially done at $5RC$.
- Inductor: $V = L\,dI/dt$. It resists sudden current changes. Every current loop has some inductance.
- Decoupling capacitors are local charge reservoirs. For fast current changes, low inductance (small and close) matters most; large capacitors handle slow changes.
- Long, fast wires are transmission lines. Reflection coefficient $\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$; negative means inverted.
- Too little measurement bandwidth makes a square wave look like a sine wave.

2.12 Exercises

1. A 10 pF capacitor is charged to 1.2 V. How much charge does it hold, and how much energy?
2. A 2 kΩ resistance drives a 50 fF load. Find the time constant and the time to reach 50%.
3. Two 1 μF capacitors are connected (a) in parallel and (b) in series. What is the total capacitance in each case?
4. A current through a 1 nH inductance changes by 2 A in 1 ns. What voltage appears across it?
5. True or false: the voltage across a capacitor can change instantly if the current is large enough.
6. You can place either a 100 μF capacitor 10 cm from a chip or a 0.1 μF capacitor right next to its power pins. Which better handles a sudden current step lasting a few nanoseconds? Why?
7. A 50 Ω line drives a 150 Ω load. Find the reflection coefficient. Is the reflection inverted?
8. A 200 MHz clock looks like a sine wave on a 250 MHz oscilloscope. Is the clock broken?

Answers

1. $Q = CV = 10 \times 10^{-12} \times 1.2 = 12$ pC. $E = \frac{1}{2}CV^2 = 0.5 \times 10 \times 10^{-12} \times 1.44 = 7.2$ pJ.
2. $\tau = 2000 \times 50 \times 10^{-15} = 100$ ps. 50% at $0.69\tau \approx 69$ ps.
3. (a) 2 μF. (b) 0.5 μF.
4. $V = L\,dI/dt = 10^{-9} \times 2/10^{-9} = 2$ V.
5. False. In any real circuit current is finite, so the voltage changes at a finite rate; it changes quickly with large current, but never instantly.
6. The small capacitor next to the pins. A fast current step is limited by inductance, and the capacitor that is 10 cm away sits behind a lot of path inductance. The big capacitor helps only with slower, longer changes.

7. $\Gamma = (150 - 50)/(150 + 50) = 0.5$. Positive, so not inverted.
8. Probably not. The scope's bandwidth removes the third harmonic (600 MHz) and above, so only the 200 MHz fundamental is displayed, which is a sine wave. Use a faster scope and probe to judge the real edges.

Chapter 3

Heat: Where Every Watt Ends Up

A chip does useful work, but physically, all the electrical energy it consumes turns into heat. A GPU drawing 700 W is, from the room's point of view, a 700 W heater. Getting that heat out without letting the chip overheat is a major part of hardware engineering, and candidates have reported assessment questions on heat sinks. This chapter explains how heat moves, how to calculate a chip's temperature, and why heat sinks are built the way they are.

3.1 Every watt becomes heat

Energy is never destroyed, only converted. When current flows through the resistance of wires and transistors, electrical energy becomes heat (Chapter 1). Charging and discharging capacitance also turns energy into heat in the transistors that do the charging (Chapter 14). The chip stores almost none of this energy, so in steady operation:

$$\text{heat produced per second} = \text{electrical power consumed}.$$

What makes chips hard to cool is not just total power but **power density**: power per unit area. A large GPU die is roughly $8\,\text{cm}^2$. At 700 W that is close to $90\,\text{W/cm}^2$, many times the power density of a kitchen stove burner, concentrated in a sliver of silicon smaller than a postage stamp.

3.2 Why temperature matters

Silicon chips are typically allowed to run up to a maximum **junction temperature** (the temperature of the transistors themselves) of around 100 to 110 °C, depending on the product. Staying below that limit matters because temperature affects nearly everything:

- **Leakage power rises steeply with temperature** (Chapter 14). Hotter chips waste more power even when idle, which makes them hotter still.
- **Speed changes.** In most conditions transistors and wires get slower when hot (Chapter 15).
- **Aging speeds up.** Wear-out mechanisms such as electromigration (Chapter 17) accelerate strongly with temperature. A common rule of thumb is that many failure rates roughly double for every 10 °C rise.

3.3 The three ways heat moves

- **Conduction**: heat flows through a solid (or still fluid) from hot to cold, passed along by vibrating atoms and moving electrons. This is how heat gets from the transistors, through the silicon and metal, to the heat sink.
- **Convection**: a moving fluid (air or liquid) picks up heat from a surface and carries it away. This is how a heat sink hands its heat to the air. A fan makes it *forced* convection, which is far more effective than air drifting on its own (*natural* convection).
- **Radiation**: hot objects emit infrared light. For chips at normal temperatures this is a small effect, and we will ignore it.

3.4 Thermal resistance: Ohm's law for heat

Heat flow behaves just like current in a circuit, which gives us a powerful shortcut:

Electrical	Thermal
Current I (A)	Heat flow P (W)
Voltage difference V (V)	Temperature difference ΔT (°C)
Resistance R (Ω)	Thermal resistance θ (°C/W)
$V = IR$	$\Delta T = P\,\theta$

The big idea

Temperature rise equals power times thermal resistance: $\Delta T = P\,\theta$. A thermal resistance of 0.2 °C/W means every watt flowing through it raises the temperature difference across it by 0.2 °C.

Heat leaving a chip passes through a chain of layers, one after another, so their thermal resistances add in series exactly like resistors (Figure 3.1).

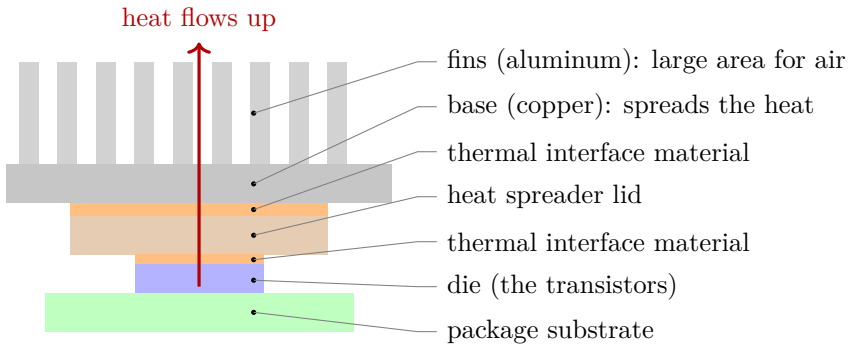


Figure 3.1: The path heat takes from the transistors to the air. Each layer adds thermal resistance.

The standard names are:

- θ_{jc}: junction to case (from the transistors, through the die and lid).
- θ_{cs}: case to sink (through the thermal interface material).
- θ_{sa}: sink to ambient (from the heat sink into the surrounding air).

$$T_{\text{junction}} = T_{\text{ambient}} + P\,(\theta_{jc} + \theta_{cs} + \theta_{sa}).$$

Worked example: junction temperature

A 300 W chip has $\theta_{jc} = 0.05$, $\theta_{cs} = 0.03$ and $\theta_{sa} = 0.10\,°\text{C/W}$. The air entering the heat sink is at 35 °C. What is the junction temperature?
Total thermal resistance: $0.05 + 0.03 + 0.10 = 0.18\,°\text{C/W}$. Temperature rise: $300 \times 0.18 = 54\,°\text{C}$. Junction temperature: $35 + 54 = 89\,°\text{C}$.

Worked example: sizing a heat sink

The same chip must stay below 95 °C with 40 °C air. What is the largest θ_{sa} the heat sink may have?
Allowed rise: $95 - 40 = 55\,°\text{C}$. Allowed total resistance: $55/300 \approx 0.183\,°\text{C/W}$. Subtract the fixed parts: $0.183 - 0.05 - 0.03 = 0.103\,°\text{C/W}$. The heat sink and fan must achieve about $0.10\,°\text{C/W}$ or better.

3.5 Conduction and materials

How well a material conducts heat is its **thermal conductivity** k, in W/(m·K). A slab of thickness L and area A has thermal resistance

$$\theta = \frac{L}{kA},$$

the same form as electrical resistance $R = \rho L/A$. Thin, wide layers of high-k material are best.

Material	k (W/(m·K)), approx.	Notes
Copper	400	excellent conductor; heavy, costly
Aluminum	205–237	about half of copper; light, cheap, easy to shape
Silicon	150	the die itself conducts well
Thermal paste	1–10	far worse than metal, but far better than air
Water (still)	0.6	poor conductor when not moving
Air (still)	0.026	a very good insulator

3.6 Heat sinks

A heat sink has one job: take heat that is concentrated in a small area and spread it over a very large surface that moving air can touch. Its design follows from the two steps it performs.

Spreading (conduction). The base sits on the chip and must carry heat sideways and upward quickly, without large temperature differences across it. This calls for high thermal conductivity, which is why bases are often copper. Many heat sinks also use **heat pipes** or **vapor chambers**: sealed copper tubes or plates containing a little liquid. The liquid boils at the hot end, the vapor rushes to the cold end and condenses, and the liquid wicks back. Moving heat by evaporation and condensation gives an effective conductivity far higher than solid copper.

Dumping (convection). The **fins** give a large surface area to the air. Convection is roughly proportional to surface area, so more fins (or taller, thinner ones) remove more heat, up to the point where fins are so close that air cannot flow between them. A fan forces air through the fins and greatly lowers θ_{sa}.

Why combine copper and aluminum? Copper conducts heat about twice as well as aluminum, but it is about 3.3 times as dense (8.96 versus 2.70 $\mathrm{g/cm^3}$), costs more, and is harder to form into thin fins. A heat sink made entirely of copper would be very heavy and expensive. So a common design uses copper where heat is concentrated and conduction matters most (the base and heat pipes), and aluminum where large area matters most (the fins). Each metal goes where its strength counts.

Seen on the assessment

Candidates have reported multiple-choice questions on heat sinks. The ideas to have ready: fins increase surface area for convection; fans lower the sink-to-air thermal resistance; copper conducts better but is heavier and costlier than aluminum, so the two are combined; heat pipes move heat by phase change; and thermal paste fills air gaps.

3.7 Thermal interface materials

Two metal surfaces that look flat are rough at the microscopic scale. Pressed together, they touch only at high points, leaving tiny gaps filled with air, and air is an excellent insulator. A **thermal interface material** (TIM), such as thermal paste, a pad, or a soldered metal layer, fills those gaps.

TIM conducts heat much worse than metal, so it should be as thin as possible: just enough to fill the gaps. Too much paste adds thermal resistance instead of removing it.

3.8 Air versus liquid cooling

The heat a moving fluid carries away is

$$P = \dot{m}\, c\, \Delta T,$$

where $\dot{m}$ is the mass flow rate (kg/s), c the **specific heat** (energy to warm 1 kg by 1 °C), and ΔT how much the fluid warms up. Water's specific heat is about 4.18 J/(g·K), about four times air's (about 1.0), and water is roughly 800 times denser. Per unit volume, water carries several thousand times more heat per degree than air.

Worked example: removing 1000 W

How much water, and how much air, must flow to carry away 1000 W while warming by 10 °C?
Water: $\dot{m} = 1000/(4180 \times 10) \approx 0.024\,\text{kg/s}$, about 1.4 liters per minute: a gentle trickle.
Air: $\dot{m} = 1000/(1005 \times 10) \approx 0.10\,\text{kg/s}$. At about $1.2\,\text{kg/m}^3$, that is about $0.083\,\text{m}^3/\text{s}$, or 83 liters every second: a powerful fan.

This is why the highest-power data-center GPU systems are increasingly liquid cooled: air simply cannot move enough heat through a small space.

3.9 Thermal throttling and runaway

Chips contain on-die temperature sensors. When the temperature approaches the limit, power-management firmware lowers the clock frequency and supply voltage. Because dynamic power is proportional to $V^2 f$ (Chapter 14), even a modest reduction cuts power, and therefore heat, sharply. This is **thermal throttling**.

Thermal runaway is the dangerous loop in the other direction: higher temperature raises leakage power, which raises temperature, which raises leakage further. A cooling system must have enough margin that this loop settles instead of running away.

3.10 Summary

- All the power a chip consumes becomes heat; power density is what makes cooling hard.
- Heat moves by conduction, convection and radiation (the last is minor for chips).
- $\Delta T = P\,\theta$; thermal resistances in the heat path add in series.
- Heat sinks spread heat (conduction: copper base, heat pipes) and dump it (convection: large fin area, fans).
- Copper conducts better but is heavy and costly; aluminum is light and cheap; good designs combine them.
- TIM replaces insulating air gaps; it should be thin.
- Liquid carries far more heat per volume than air.

3.11 Exercises

1. A 250 W chip has a total junction-to-ambient thermal resistance of 0.2 °C/W in 25 °C air. Find its junction temperature.
2. A 400 W chip must stay below 95 °C in 40 °C air. What is the maximum total thermal resistance allowed?
3. Why do many heat sinks use a copper base with aluminum fins rather than one metal throughout?

4. Why is thermal paste needed between a chip and its heat sink, and why should the layer be thin?
5. True or false: adding fins to a heat sink increases its thermal resistance.
6. A chip is overheating. Its firmware lowers the supply voltage by 10% and the clock frequency by 10%. By roughly what percentage does dynamic power fall?
7. Explain thermal runaway in one or two sentences.

Answers

1. $T_j = 25 + 250 \times 0.2 = 75\,^{\circ}\mathrm{C}$.
2. $(95 - 40)/400 = 0.1375\,^{\circ}\mathrm{C/W}$.
3. Copper conducts heat about twice as well, which matters most where heat is concentrated (the base). But copper is about 3.3 times denser and more expensive, so the large fin area is made of light, cheap aluminum. Each material is used where it helps most.
4. Microscopic roughness leaves air gaps, and air insulates. Paste fills the gaps. Paste conducts much worse than metal, so a thick layer adds resistance; it should be just thick enough to fill the gaps.
5. False. Fins add surface area for convection, which lowers the sink-to-air thermal resistance.
6. Power scales with $V^2 f$: $0.9^2 \times 0.9 = 0.729$, so power falls by about 27%.
7. Higher temperature increases leakage power, which produces more heat and raises temperature further. If cooling cannot keep up, the loop keeps escalating.

Part II

Semiconductors and Transistors

Chapter 4

Semiconductors: Making Silicon Useful

A computer needs billions of switches that can be flipped by an electrical signal, with no moving parts, billions of times per second. Metals cannot do this: they always conduct. Insulators cannot either: they never conduct. We need a material whose ability to conduct can be *controlled*. That material is a semiconductor, almost always silicon. This chapter explains what makes silicon special, how "doping" turns it into a building material, and how temperature changes its behavior, which is one of the questions from the assessment.

4.1 Electrons that can and cannot move

Recall from Chapter 1 that a material conducts when some of its electrons are free to move. Whether they are free depends on how tightly atoms hold their outermost electrons, called **valence electrons**.

- In a **metal**, valence electrons are not tied to any one atom. They form a "sea" of free electrons that drifts easily when a voltage is applied. Metals conduct very well.
- In an **insulator**, every electron is locked in a chemical bond. Breaking one free takes a lot of energy, so almost none are free.
- In a **semiconductor**, electrons are bound, but not very tightly. A small amount of energy, such as heat at room temperature, frees a few of them.

Physicists describe this with an energy gap, the **band gap**: the minimum energy needed to free a bound electron. Silicon's band gap is about 1.12 electron-volts (eV). Silicon dioxide, a good insulator, has a gap of about 9 eV. Metals have no gap at all.

4.2 The silicon crystal

A silicon atom has four valence electrons. In a crystal, each silicon atom shares one electron with each of four neighbors, forming four bonds (Figure 4.1). Every electron is used in a bond, so at very low temperature pure silicon is an insulator.

At room temperature, atoms vibrate. Occasionally the vibration shakes an electron out of its bond. Two things result:

1. A **free electron** that can wander through the crystal and carry current.
2. A **hole**: the empty spot where the electron used to be.

In pure silicon at room temperature, fewer than one atom in a trillion has lost an electron this way, so pure silicon conducts very poorly.

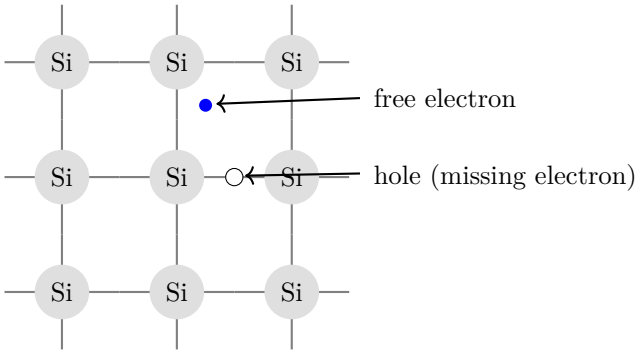


Figure 4.1: Silicon atoms each share four bonds. Heat occasionally frees an electron, leaving a hole behind.

4.3 Holes: the missing electron that moves

A hole is not a particle; it is the absence of an electron. But it behaves like a positive charge that can move. Here is why. If a neighboring bonded electron hops into the hole, the hole has effectively moved to where that electron came from. Under a voltage, electrons hop one way and the hole travels the other way, exactly as a positive charge would.

An everyday picture

Picture a row of full seats in a theater with one empty seat. When the person next to the empty seat slides over, the empty seat moves one place the other way. Keep sliding and the empty seat "travels" down the row, even though only people moved. The hole is the empty seat.

So a semiconductor has *two* kinds of charge carriers: negative electrons and positive holes. This is the key to building two kinds of transistors.

4.4 Doping: controlling the carriers

Pure silicon has too few carriers to be useful. **Doping** adds a tiny, carefully controlled amount of another element, typically a few atoms per million or less, and changes conductivity enormously (Figure 4.2).

n-type silicon. Add atoms with *five* valence electrons, such as phosphorus or arsenic. Four electrons bond with silicon neighbors; the fifth has nothing to bond with and is freed easily. The silicon now has many free electrons, which carry negative charge, so it is called **n-type**. Electrons are its **majority carriers**; holes are **minority carriers**.

p-type silicon. Add atoms with *three* valence electrons, such as boron. One bond is left incomplete, creating a hole. The silicon now has many holes, which act as positive carriers, so it is called **p-type**. Holes are its majority carriers.

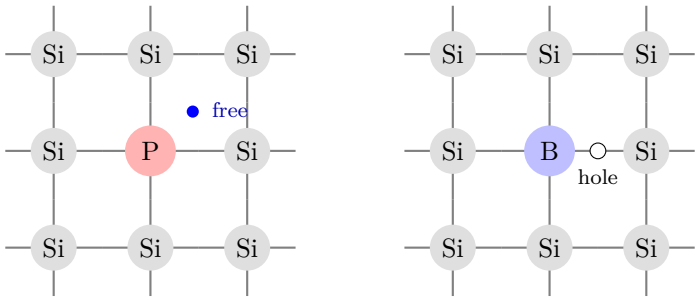


Figure 4.2: Doping. A phosphorus atom (left) bonds with four silicon neighbors and has a fifth electron left over, free to move: n-type. A boron atom (right) has only three, so one bond is missing an electron, a hole: p-type.

Remember

n-type: extra **n**egative carriers (electrons), from 5-electron dopants like phosphorus.
p-type: extra **p**ositive carriers (holes), from 3-electron dopants like boron.
A doped crystal is still electrically neutral overall: every extra free electron is balanced by a positive dopant nucleus, and every hole by a negative one.

Chip designers write **n+** or **p+** for heavily doped regions (which conduct well and form transistor terminals), and speak of an **n-well**: a region of n-type silicon built into a p-type wafer so that both kinds of transistor can live on the same chip (Chapter 5).

4.5 How carriers move: drift, diffusion and mobility

Carriers move for two reasons:

- **Drift**: an electric field (a voltage across a distance) pushes them. Electrons drift toward the higher voltage; holes toward the lower.
- **Diffusion**: carriers spread from where they are crowded to where they are sparse, like a drop of ink spreading in water. No voltage is needed.

How fast a carrier drifts in a given field is its **mobility**, written μ (mu):

$$v_{\text{drift}} = \mu E,$$

where E is the electric field (volts per meter). Higher mobility means faster carriers, more current, and faster transistors.

Carrier in silicon	Mobility (lightly doped, room temperature)
Electrons	about $1400\,\text{cm}^2/(\text{V}\cdot\text{s})$
Holes	about $450\,\text{cm}^2/(\text{V}\cdot\text{s})$

The big idea

Electrons are about two to three times more mobile than holes. Holes move only by electrons hopping from bond to bond, which is slower than a free electron's travel. This single fact is why PMOS transistors (which use holes) are weaker than NMOS transistors (which use electrons), and why PMOS transistors are usually drawn about twice as wide (Chapter 5).

At very high fields, carriers stop speeding up; their velocity levels off at roughly $10^7\,\text{cm/s}$. This **velocity saturation** matters in modern, very short transistors.

4.6 What heat does to semiconductors and to metal

Temperature changes conduction in two competing ways, and semiconductors and metals respond differently.

Effect 1: more scattering, so lower mobility. A hotter crystal vibrates more. Carriers moving through it collide with the vibrating atoms more often (these vibrations are called **phonons**, and the collisions **phonon scattering**). More collisions mean carriers cannot build up speed, so **mobility decreases as temperature increases**. Transistors therefore carry less current and get slower.

Effect 2: more carriers. Heat also frees more electrons from bonds. In pure silicon the number of free carriers rises very steeply (exponentially) with temperature. In a transistor that is supposed to be off, those extra carriers show up as **leakage current**, which grows rapidly when the chip is hot.

Metals. A metal's free electrons are already all free; heat does not add more. Only effect 1 applies: more vibration means more scattering. So **metal resistance increases with temperature**. For copper, resistance rises by about 0.4% per °C, so a wire 100 °C hotter has roughly 40% more resistance.

Seen on the assessment

A reported question asked how carrier mobility and wire resistance change when temperature increases. Answer: **carrier mobility decreases** (more phonon scattering), so transistors drive less current; **wire resistance increases** (also more scattering), so wires get slower. Both effects make the circuit slower at high temperature. Chapter 15 adds one twist: the

transistor's threshold voltage also drops when hot, which pushes the other way and, at low supply voltages, can win.

4.7 The PN junction and the diode

Something important happens where p-type and n-type silicon meet, a **PN junction**.

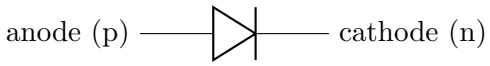
Near the boundary, free electrons from the n side diffuse into the p side and fill holes; holes from the p side diffuse into the n side. Carriers near the boundary cancel out, leaving a thin **depletion region** with no free carriers. The dopant atoms left behind are charged (positive on the n side, negative on the p side), creating a built-in electric field that stops further diffusion. The built-in voltage is a few tenths of a volt.

This structure is a **diode**, a one-way valve for current:

- **Forward bias** (p side more positive than n side): the applied voltage fights the built-in barrier, the depletion region shrinks, and current flows easily once the voltage exceeds about 0.6–0.7 V for silicon. Current grows exponentially with voltage:

$$I = I_S \left(e^{V/V_T} - 1 \right), \qquad V_T = \frac{kT}{q} \approx 26 \text{ mV at room temperature.}$$

- **Reverse bias** (n side more positive): the barrier grows, the depletion region widens, and only a tiny leakage current flows. At a large enough reverse voltage the junction **breaks down** and conducts heavily.



current flows easily from anode to cathode

Diodes matter to a digital designer in three ways:

- Every transistor's source and drain form PN junctions with the silicon beneath them. These junctions must always stay reverse biased, which is why the p-type body under NMOS transistors is tied to ground and the n-well under PMOS transistors is tied to the supply (Chapter 5).
- Reverse-biased junctions leak a little, adding to leakage power.
- Diodes are deliberately added to protect circuits: from static electricity (ESD protection) and from charge collected during manufacturing (antenna diodes, Chapter 17).

4.8 Silicon dioxide: silicon's perfect partner

Silicon has a lucky property: heat it in oxygen and it grows a layer of **silicon dioxide** (SiO_2), essentially glass, which is an excellent insulator and bonds cleanly to the silicon. A thin layer of insulator on silicon is exactly what a transistor's gate needs (next chapter). In the most advanced processes, the gate insulator is so thin that designers switched to materials with higher permittivity ("high-k" dielectrics such as hafnium oxide), which give the same capacitance with a physically thicker, less leaky layer.

4.9 Summary

- Semiconductors sit between conductors and insulators: a little energy frees their electrons.
- Freeing an electron leaves a hole, which behaves like a moving positive charge.
- Doping controls carriers: phosphorus or arsenic makes n-type (electrons); boron makes p-type (holes).
- Mobility measures how fast carriers drift. Electrons are 2–3 times more mobile than holes, so NMOS is stronger than PMOS.
- Heat lowers mobility (transistors slow down), raises metal resistance (wires slow down), and sharply raises leakage.
- A PN junction is a diode: it conducts in forward bias and blocks (with slight leakage) in reverse bias.

4.10 Exercises

1. Why is pure silicon a poor conductor at room temperature?
2. Which dopant makes n-type silicon, boron or phosphorus? Explain using valence electrons.
3. What are the majority carriers in p-type silicon?
4. Electron mobility is about three times hole mobility. Name one consequence for chip design.
5. As temperature rises, what happens to (a) carrier mobility, (b) copper wire resistance, (c) transistor leakage current?
6. True or false: a hole is a positively charged particle, like a proton, that moves through the crystal.
7. A silicon diode has 0.7 V across it with the p side positive. Is it conducting? What if the polarity is reversed?
8. Why is the p-type body under NMOS transistors connected to ground?

Answers

1. All its valence electrons are locked in bonds; at room temperature heat frees only a tiny number, so there are very few carriers.
2. Phosphorus. It has five valence electrons; four bond with silicon neighbors and the fifth is easily freed, adding free electrons.
3. Holes.
4. PMOS transistors are weaker for the same size, so they are made wider (about 2–3 times) to match NMOS strength; also, gate designs prefer to put PMOS in parallel rather than in series (Chapter 8).
5. (a) Decreases. (b) Increases. (c) Increases, steeply.

6. False. A hole is the absence of an electron in a bond. It behaves like a positive charge because neighboring electrons hop into it, making the vacancy move.
7. Yes, it is forward biased and conducting. Reversed, it blocks current except for a tiny leakage.
8. To keep the PN junctions between the n+ source/drain regions and the p-type body reverse biased (the body is at the lowest voltage), so they do not conduct.

Chapter 5

The MOSFET: A Switch Controlled by Voltage

Every digital chip is built from one device repeated billions of times: the **MOSFET**, short for metal–oxide–semiconductor field-effect transistor. If you understand this chapter, a surprising number of assessment questions become easy, including the swapped inverter and every question about "strong" and "weak" logic levels.

5.1 What we want

We want a switch with three wires. Two wires are the ends of the switch, and current flows between them when the switch is closed. The third wire controls the switch: a high voltage on it closes the switch, a low voltage opens it. Crucially, the control wire should not draw any steady current, so one switch can control many others without being drained.

The MOSFET does exactly this. Its terminals are:

- the **gate** (G): the control input;
- the **source** (S) and **drain** (D): the two ends of the switch;
- the **body** (B), also called the bulk or substrate: the silicon underneath, usually tied to a fixed voltage and often not drawn.

5.2 Inside an NMOS transistor

Figure 5.1 shows a slice through an NMOS transistor. Starting from p-type silicon (the body), two heavily doped n+ regions are formed: the source and the drain. Between them, on the surface, sits a very thin layer of insulating oxide, and on top of that the gate electrode. The distance from source to drain is the channel **length** L; the distance into the page is the channel **width** W.

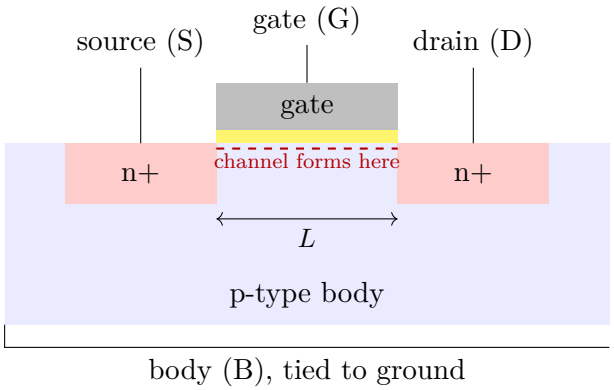


Figure 5.1: Cross-section of an NMOS transistor. The yellow strip under the gate is the thin insulating oxide.

5.3 How the channel forms

With the gate at 0 V, look at the path from source to drain: n+ silicon, then p-type body, then n+ silicon. That is two PN junctions back to back (Chapter 4), and whichever way you push current, one of them is reverse biased. No current flows. The switch is *open*.

Now raise the gate voltage. The gate, the oxide and the silicon form a capacitor. A positive gate attracts negative charge to the silicon surface just under the oxide: it pushes holes away and pulls in electrons. Once the gate voltage is high enough, electrons outnumber holes in a thin surface layer. That layer has effectively become n-type: an **inversion layer**, or **channel**. Now there is a continuous n-type path from source to drain, and current can flow. The switch is *closed*.

The gate-to-source voltage needed to form the channel is the **threshold voltage**, V_t. In modern chips it is typically 0.2 to 0.5 V, with supply voltages around 0.7 to 1.2 V.

The big idea

An NMOS transistor is on when its gate is higher than its source by more than the threshold voltage: $V_{GS} > V_{tn}$. The higher the gate goes above that, the more electrons fill the channel, the lower its resistance, and the more current it carries.

The gate draws no steady current. The gate sits on an insulator, so no DC current flows into it. The only current is the brief flow needed to charge or discharge the gate capacitance when its voltage changes. This is why a CMOS logic gate uses almost no power when nothing is switching, and why one gate's output can drive many other gates' inputs.

Which end is the source? An NMOS transistor is physically symmetric; nothing in the silicon marks one n+ region as "source." By definition, the **source of an NMOS is whichever end is at the lower voltage** (electrons flow out of the source). This sounds like a technicality, but it is the key to the "weak 1" later in this chapter.

5.4 The PMOS transistor

A **PMOS** transistor is the mirror image. It is built in n-type silicon (an **n-well**), with p+ source and drain regions (Figure 5.2). A channel of *holes* forms when the gate is pulled sufficiently *below* the source:

$$V_{SG} > |V_{tp}| \quad \text{(gate lower than source by more than the threshold magnitude).}$$

The source of a PMOS is whichever end is at the *higher* voltage, usually the supply V_{DD}. The n-well is tied to V_{DD}, the highest voltage, to keep its junctions reverse biased.

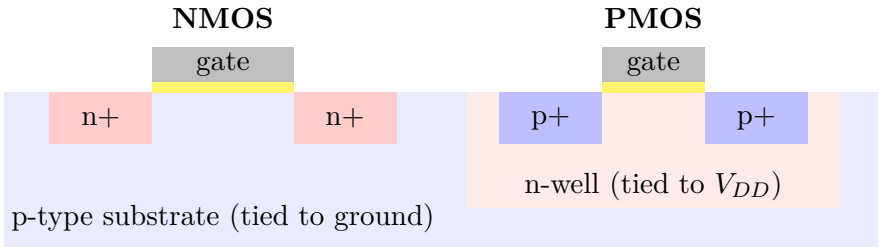


Figure 5.2: NMOS and PMOS on one chip. The NMOS is built directly in the p-type substrate; the PMOS sits in an n-well, a region of n-type silicon, so both kinds of transistor share one wafer.

Because holes are about two to three times less mobile than electrons (Chapter 4), a PMOS carries less current than an NMOS of the same size.

5.5 The switch model

For most logic design, this simple model is enough. Here V_{DD} (logic 1) is the supply and 0 V (logic 0) is ground.

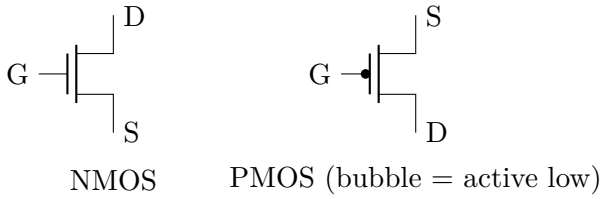


Figure 5.3: Circuit symbols. The PMOS symbol's bubble on the gate means "on when the gate is low."

	Gate = 1 (V_{DD})	Gate = 0 (ground)
NMOS	ON (closed)	OFF (open)
PMOS	OFF (open)	ON (closed)

Remember

NMOS: on when the gate is high. PMOS: on when the gate is low. The bubble on the PMOS symbol is a reminder: the bubble means "inverted."

5.6 How much current: regions of operation

The switch model says only on or off. To understand speed and power we need to know how *much* current flows. Define the **overdrive** $V_{ov} = V_{GS} - V_t$: how far the gate is above threshold. An NMOS operates in one of three regions:

- **Cutoff** ($V_{GS} < V_t$): no channel. Ideally zero current; really a small **subthreshold leakage** (below).
- **Linear** or **triode** ($V_{GS} > V_t$ and $V_{DS} < V_{ov}$): the channel stretches all the way from source to drain. The transistor acts like a resistor whose value the gate controls:

$$I_D = \mu C_{ox} \frac{W}{L} \left(V_{ov} V_{DS} - \tfrac{1}{2} V_{DS}^2 \right).$$

- **Saturation** ($V_{GS} > V_t$ and $V_{DS} \geq V_{ov}$): near the drain, the voltage between gate and channel is too small to hold the channel open, so the channel "pinches off." Raising the drain voltage further barely changes the current, which is now set by the gate:

$$\boxed{I_D = \tfrac{1}{2} \mu C_{ox} \frac{W}{L} (V_{GS} - V_t)^2}$$

Here μ is the carrier mobility and C_{ox} the gate-oxide capacitance per unit area. The product μC_{ox} is a constant for a given manufacturing process.

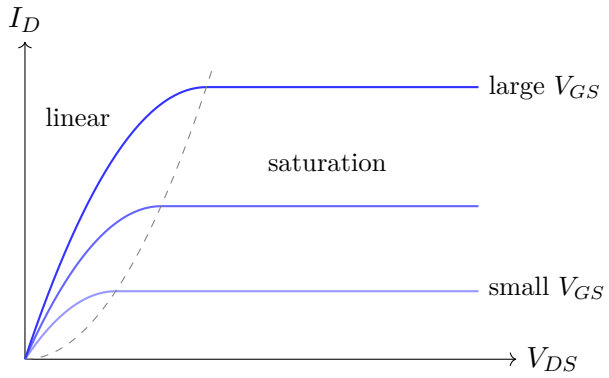


Figure 5.4: Drain current versus drain voltage for three gate voltages. The dashed curve separates the linear region (left) from saturation (right).

What the saturation formula tells you:

- **Wider is stronger**: current is proportional to W. Doubling the width doubles the current.

- **Shorter is stronger**: current is proportional to $1/L$. This is why each new manufacturing generation shrinks L.
- **More overdrive is stronger**: current grows with $(V_{GS} - V_t)^2$. Raising the supply voltage speeds up circuits; raising the threshold slows them down.
- **Higher mobility is stronger**: NMOS beats PMOS of the same size.

Worked example: saturation current

An NMOS has $\mu C_{ox} = 200\ \mu\text{A/V}^2$, $W/L = 4$, $V_t = 0.4\,\text{V}$ and $V_{GS} = 1.0\,\text{V}$, in saturation. Find I_D.

$$I_D = \tfrac{1}{2} \times 200 \times 10^{-6} \times 4 \times (1.0 - 0.4)^2 = 400 \times 10^{-6} \times 0.36 = 144\ \mu\text{A}.$$

Real modern transistors. The square law is a teaching model from older, longer transistors. In today's very short transistors, carriers hit their maximum velocity (velocity saturation, Chapter 4), and current grows more like $(V_{GS} - V_t)^1$ to $(V_{GS} - V_t)^{1.3}$ than the square. Saturation current also creeps up slightly with V_{DS} (**channel-length modulation**). The trends above still hold, and assessment questions use the simple model.

Subthreshold leakage. Below threshold the current is small but not zero; it falls off *exponentially* as V_{GS} drops below V_t, by roughly a factor of 10 for every 60 to 100 mV at room temperature. So a transistor with a lower threshold voltage leaks exponentially more when it is "off." This is the main source of leakage power (Chapter 14).

5.7 Strong and weak logic levels

This section is the single most important idea for transistor-level assessment questions.

Use an NMOS as a **pass transistor**: its gate is held at V_{DD} (fully on), one end is the input, and the other end drives an output node that has some capacitance (Figure 5.5).

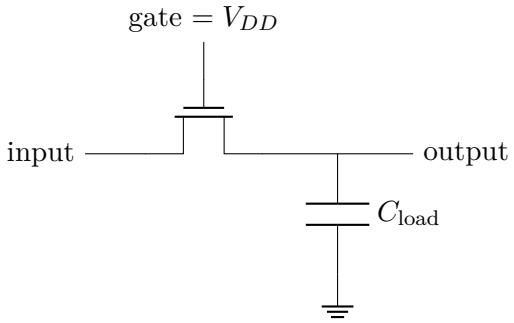


Figure 5.5: An NMOS pass transistor charging or discharging a load.

Passing a 0. The input is 0 V and the output starts high. The input end is the lower voltage, so it is the source, fixed at 0 V. Then $V_{GS} = V_{DD} - 0 = V_{DD}$, comfortably above V_t, for the whole

discharge. The transistor stays fully on and pulls the output all the way to 0 V. This is a **strong 0**.

Passing a 1. The input is V_{DD} and the output starts at 0 V. Now the *output* is the lower-voltage end, so the output is the source. As the output charges up, $V_{GS} = V_{DD} - V_{\text{out}}$ shrinks. When V_{out} reaches $V_{DD} - V_{tn}$, the gate is only V_{tn} above the source, and the transistor turns off. The output gets stuck one threshold below the supply:

$$V_{\text{out, max}} = V_{DD} - V_{tn}.$$

This is a **weak 1** (also called a degraded 1).

Worked example: a weak 1

$V_{DD} = 1.0\,\text{V}$, $V_{tn} = 0.3\,\text{V}$. An NMOS pass transistor with its gate at V_{DD} passes a 1. The output rises only to $1.0 - 0.3 = 0.7\,\text{V}$.

The PMOS is the mirror image. Its source is its higher end. Passing a 1, the input (at V_{DD}) is the source, the gate is at 0, and the transistor stays fully on: a **strong 1**. Passing a 0, the output becomes the source; the PMOS turns off when the output falls to $|V_{tp}|$, leaving a **weak 0**.

	Passes 0	Passes 1
NMOS	strong (reaches 0 V)	weak (stops at $V_{DD} - V_{tn}$)
PMOS	weak (stops at $\lvert V_{tp}\rvert$)	strong (reaches V_{DD})

The big idea

NMOS is good at pulling *down*; PMOS is good at pulling *up*. That is why every CMOS logic gate uses NMOS transistors to connect the output to ground and PMOS transistors to connect it to the supply, and never the other way around. Chapter 8 shows what goes wrong when you swap them.

The general rule. An NMOS whose gate is at V_G can pass a voltage up to $V_G - V_{tn}$, and no higher:

$$V_{\text{out}} = \min\left(V_{\text{in}},\ V_G - V_{tn}\right).$$

(This ignores the body effect, described below, which makes the drop slightly larger.)

5.8 Chains of pass transistors

Questions often chain several pass transistors together. Two cases behave very differently.

Case 1: transistors in series, all gates at V_{DD}. The threshold drop happens only *once*. The first transistor passes at most $V_{DD} - V_t$. The second transistor also has its gate at V_{DD}, so it too can pass up to $V_{DD} - V_t$; it has no trouble passing a voltage that is already that low. The output of the chain is $V_{DD} - V_t$, not $V_{DD} - 2V_t$.

Case 2: one transistor's output drives the next transistor's gate. Now the drops *accumulate*. If node N1 is only at $V_{DD} - V_t$ and it drives the gate of the next transistor, that transistor can pass only up to $(V_{DD} - V_t) - V_t$.

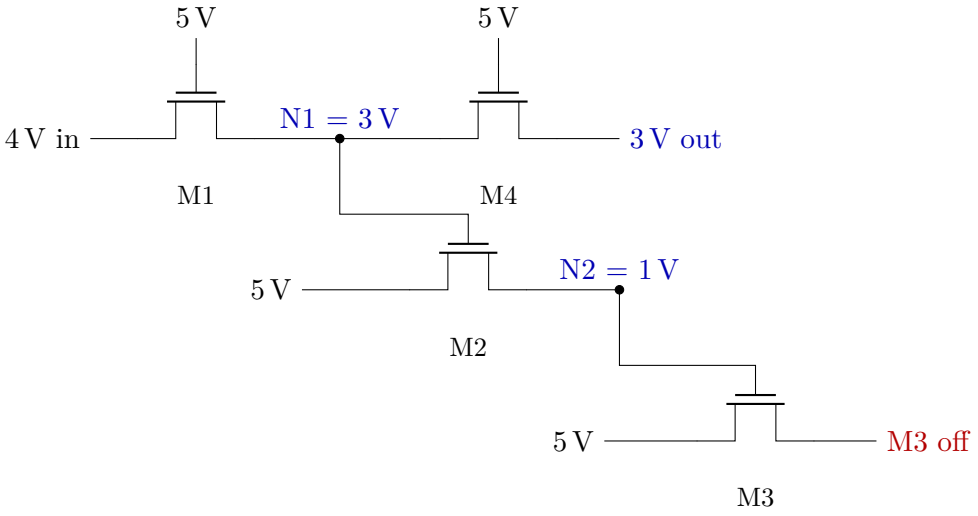


Figure 5.6: The network in the worked example below ($V_{DD} = 5\,\text{V}$, $V_t = 2\,\text{V}$). M1 and M4 are in series with their gates at 5 V, so the threshold drop happens once. N1 drives M2's gate and N2 drives M3's gate, so there the drops accumulate.

Worked example: node voltages in a pass-transistor network

All NMOS transistors are identical with $V_t = 2\,\text{V}$; the supply is $V_{DD} = 5\,\text{V}$ (an older, high-voltage process, used here to make the numbers easy to follow). Ignore the body effect.

1. Transistor M1 has its gate at 5 V and passes an input of 4 V to node N1.
   $V_{N1} = \min(4,\ 5 - 2) = 3\,\text{V}$. The input is limited by the gate.
2. Node N1 (3 V) drives the gate of transistor M2, which passes 5 V from the supply to node N2.
   $V_{N2} = \min(5,\ 3 - 2) = 1\,\text{V}$.
3. Node N2 (1 V) drives the gate of transistor M3. Since $1\ \text{V} < V_t = 2\,\text{V}$, M3 is off and passes nothing.
4. Instead, suppose M1 is followed by M4, in series, also with its gate at 5 V. The output of M4 is $\min(3,\ 5 - 2) = 3\,\text{V}$: no extra drop.

Seen on the assessment

A reported question asked for the voltages at several nodes of a network of identical NMOS transistors with $V_{DD} = 5\,\text{V}$, $V_t = 2\,\text{V}$ and an input of 4 V. Without the original diagram, the method above is what such questions test: at each node apply $V_{\text{out}} = \min(V_{\text{in}}, V_G - V_t)$, notice whether a gate is driven by the full supply or by an already-degraded node, and check whether any transistor's gate falls below threshold.

5.9 The body effect

So far we assumed the threshold voltage is fixed. In fact, an NMOS's threshold *increases* when its source is above its body. In a pass transistor passing a 1, the source rises toward V_{DD} while the body stays at ground, so V_t grows as the output rises, making the weak 1 even weaker. Transistors stacked in series (where the upper ones have sources above ground) are also slightly weakened. For assessment questions, it is enough to know the direction: **a source above the body raises V_t.**

5.10 Sizing transistors

Because current is proportional to $\mu W/L$, a designer chooses W (and sometimes L) to set each transistor's strength. To make a PMOS deliver the same current as an NMOS, it must make up for its lower mobility with extra width:

$$\frac{(W/L)_p}{(W/L)_n} = \frac{\mu_n}{\mu_p} \approx 2 \text{ to } 3.$$

Worked example: equal saturation currents

In a process where $\mu_n = 2.5\,\mu_p$, the NMOS in an inverter is $1\,\mu$m wide. Both have the same length. How wide should the PMOS be so both deliver the same current?
$W_p = 2.5 \times 1 = 2.5$ μm.

The trade-off: a wider transistor is stronger, but its gate has more capacitance, so whatever drives *it* has a bigger load and becomes slower. Sizing is a balance (Chapter 13).

5.11 Transistor capacitance

A transistor's gate is a capacitor: gate on one side, channel on the other, oxide between. Its capacitance is roughly

$$C_g = C_{ox}\, W\, L.$$

The source and drain regions also have capacitance to the body, called **diffusion capacitance**. When a gate's output switches, it must charge the input capacitance of every gate it drives, the wires in between, and its own diffusion capacitance.

5.12 Threshold voltage as a design knob

Chip processes usually offer several transistor flavors with different thresholds:

- **Low-V_t** transistors: more overdrive, so faster, but they leak exponentially more when off.
- **High-V_t** transistors: slower but with far less leakage.

Designers use fast low-V_t transistors only on the few paths that limit speed, and high-V_t transistors everywhere else to save leakage power. Temperature also changes V_t: it *decreases* as the chip heats up (Chapter 15).

5.13 Modern transistor shapes

As L shrank to tens of nanometers, a gate sitting only on top of the channel could no longer fully control it, and leakage grew. Two newer structures fix this by wrapping the gate around more of the channel (Figure 5.7):

- **FinFET**: the channel is a thin vertical fin, and the gate wraps over three sides.
- **Gate-all-around** (GAA) or **nanosheet**: the channel is a stack of thin horizontal sheets, each surrounded by the gate on all four sides.

More gate control means the transistor turns off more completely, which reduces leakage. The basic switch behavior in this chapter is unchanged.

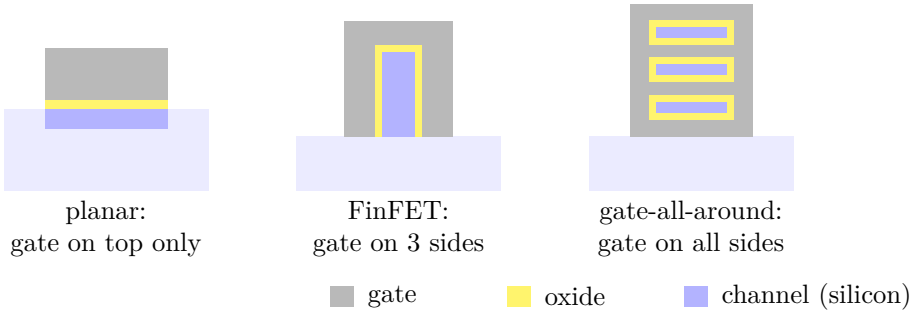


Figure 5.7: Three transistor shapes, seen end-on with current flowing into the page. The more of the channel the gate surrounds, the more completely it can turn the transistor off.

5.14 Summary

- A MOSFET is a voltage-controlled switch: gate controls whether current flows between source and drain. The gate draws no DC current.
- NMOS: on when $V_{GS} > V_{tn}$ (gate high). PMOS: on when $V_{SG} > |V_{tp}|$ (gate low).
- NMOS source is its lower-voltage end; PMOS source is its higher-voltage end.
- Saturation current: $I_D = \frac{1}{2}\mu C_{ox}(W/L)(V_{GS}-V_t)^2$. Wider, shorter, more overdrive and higher mobility all mean more current.
- NMOS passes a strong 0 and a weak 1 ($V_{DD} - V_{tn}$). PMOS passes a strong 1 and a weak 0 ($|V_{tp}|$).
- Pass-transistor rule: $V_{\text{out}} = \min(V_{\text{in}}, V_G - V_t)$. Drops do not add in a series chain with full gates; they do add when degraded nodes drive gates.
- PMOS is made about 2–3 times wider than NMOS for equal current.
- Lower V_t means faster but exponentially leakier.

5.15 Exercises

1. An NMOS has $V_{GS} = 0.2\,\text{V}$ and $V_{tn} = 0.35\,\text{V}$. Is it on or off?

2. A PMOS has its source at 1.0 V, its gate at 0 V, and $|V_{tp}| = 0.3$ V. Is it on or off?
3. An NMOS pass transistor with its gate at $V_{DD} = 1.2$ V passes a 1. If $V_{tn} = 0.4$ V, what is the highest voltage the output reaches?
4. A PMOS pass transistor with its gate at 0 V passes a 0. If $|V_{tp}| = 0.35$ V, what is the lowest voltage the output reaches?
5. An NMOS in saturation has $\mu C_{ox}(W/L) = 1\,\text{mA/V}^2$ and $V_{GS} - V_t = 0.5$ V. Find I_D. What happens to I_D if W doubles?
6. Two NMOS transistors are in series, both gates at $V_{DD} = 1$ V, $V_t = 0.3$ V. The input is 1 V. What is the output: 0.7 V or 0.4 V?
7. An NMOS has its gate driven by a node at 0.7 V and passes a 1.0 V input. With $V_t = 0.3$ V, what is the output?
8. In a process with $\mu_n/\mu_p = 2$, an NMOS is 0.2 μm wide. How wide should a same-length PMOS be for equal current?
9. Why does a MOSFET's gate draw no DC current?
10. What is the trade-off between low-V_t and high-V_t transistors?

Answers

1. Off (below threshold), apart from a small subthreshold leakage.
2. On: $V_{SG} = 1.0 - 0 = 1.0\ \text{V} > 0.3$ V.
3. $1.2 - 0.4 = 0.8$ V.
4. 0.35 V.
5. $I_D = \frac{1}{2} \times 1 \times 10^{-3} \times 0.25 = 125\ \mu$A. Doubling W doubles it to 250 μA.
6. 0.7 V. Both gates are at the full supply, so only one threshold drop occurs.
7. $\min(1.0,\ 0.7 - 0.3) = 0.4$ V.
8. 0.4 μm.
9. The gate is separated from the channel by an insulating oxide, so the gate is a capacitor: current flows only briefly while its voltage changes.
10. Low-V_t is faster (more overdrive) but leaks exponentially more; high-V_t is slower but leaks far less.

Chapter 6

A Short Tour of Analog Circuits

Digital circuits treat voltages as just 0 or 1. Underneath, every voltage is a continuous, **analog** quantity, and some parts of every chip system must deal with it directly: the power supplies, the sensors, the converters between analog and digital, and the high-speed links between chips. Hardware assessments sometimes include a basic analog question, and the ideas here also deepen your understanding of transistors. This chapter is a short tour, not a full course.

6.1 The transistor as an amplifier

In Chapter 5 we used the transistor as a switch. Used differently, it is an **amplifier**: a small change in input voltage produces a large change in output voltage.

In saturation, the drain current depends on the gate voltage. A small wiggle in V_{GS} produces a wiggle in I_D. How much current change you get per volt of gate change is the **transconductance**:

$$g_m = \frac{\Delta I_D}{\Delta V_{GS}} = \mu C_{ox} \frac{W}{L}(V_{GS} - V_t) = \frac{2I_D}{V_{GS} - V_t}.$$

The common-source amplifier. Connect a resistor R_D from the supply to the drain, ground the source, apply the input to the gate, and take the output from the drain (Figure 6.1). The drain voltage is $V_{\text{out}} = V_{DD} - I_D R_D$. A small input change ΔV_{in} changes the current by $g_m \, \Delta V_{\text{in}}$, which changes the output by $-g_m R_D \, \Delta V_{\text{in}}$. The **voltage gain** is

$$\boxed{A_v = \frac{\Delta V_{\text{out}}}{\Delta V_{\text{in}}} = -g_m R_D}$$

The minus sign means the output moves the opposite way from the input: raising the gate pulls more current, which drops more voltage across R_D, which lowers the output. (A CMOS inverter is a common-source amplifier with a PMOS in place of the resistor. That is why an inverter has high gain in the middle of its range, Chapter 8.)

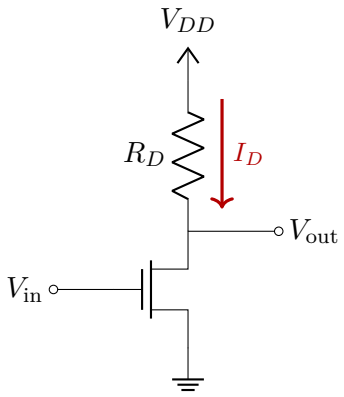


Figure 6.1: A common-source amplifier. Raising V_{in} increases the current I_D (red), which drops more voltage across R_D and lowers V_{out}.

Worked example: amplifier gain

An NMOS is biased at $I_D = 100\ \mu\text{A}$ with overdrive $V_{GS} - V_t = 0.2\,\text{V}$, and $R_D = 10\,\text{k}\Omega$. Find the gain.

$$g_m = \frac{2 \times 100\,\mu\text{A}}{0.2\,\text{V}} = 1\ \text{mA/V}, \qquad A_v = -g_m R_D = -10^{-3} \times 10^4 = -10.$$

A 10 mV wiggle at the input becomes a 100 mV wiggle at the output, inverted.

6.2 The operational amplifier

An **operational amplifier**, or op-amp, is a packaged amplifier with two inputs, V_+ (non-inverting) and V_- (inverting), and one output. Its output is the difference between the inputs multiplied by an enormous gain A, often 10^5 or more:

$$V_{\text{out}} = A\,(V_+ - V_-).$$

With such huge gain, an op-amp is almost always used with **negative feedback**: part of the output is fed back to the V_- input. Then two "golden rules" let you analyze circuits in seconds:

1. **No current flows into the inputs.**
2. **With negative feedback, the output adjusts itself until $V_+ = V_-$.** If they differ even slightly, the huge gain drives the output in the direction that removes the difference. (This is called a **virtual short**.)

Inverting amplifier. V_+ is grounded; the input drives V_- through R_{in}; a feedback resistor R_f connects the output back to V_- (Figure 6.2, left). By rule 2, $V_- = V_+ = 0$. The current through R_{in} is $V_{\text{in}}/R_{\text{in}}$; by rule 1, it all continues through R_f, so $V_{\text{out}} = 0 - R_f\,(V_{\text{in}}/R_{\text{in}})$:

$$\boxed{V_{\text{out}} = -\frac{R_f}{R_{\text{in}}}\,V_{\text{in}}}$$

Non-inverting amplifier. The input drives V_+ directly; R_f and R_g form a divider from the output to V_- (Figure 6.2, right). By rule 2, $V_- = V_{\text{in}}$, and the divider gives $V_- = V_{\text{out}}\, R_g/(R_f + R_g)$:

$$\boxed{V_{\text{out}} = \left(1 + \frac{R_f}{R_g}\right) V_{\text{in}}}$$

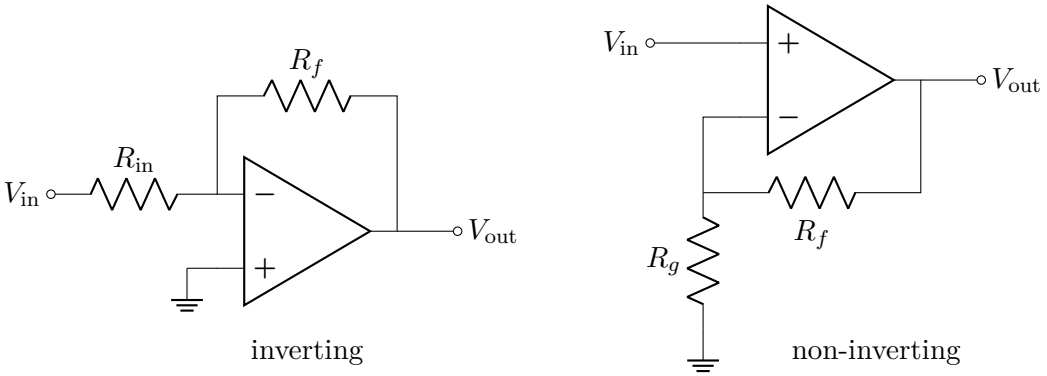


Figure 6.2: The two basic op-amp amplifiers.

Voltage follower. Connect the output straight to V_- and drive V_+ with the input. The gain is exactly 1, so it seems useless, but it is not: the op-amp's input draws no current while its output can supply plenty. A follower lets a weak signal drive a heavy load without being dragged down. It is a **buffer**.

Comparator. With no feedback at all, the huge gain slams the output to its maximum if $V_+ > V_-$ and to its minimum if $V_+ < V_-$. The output is effectively a digital 1 or 0 answering "which input is bigger?" A comparator is a one-bit analog-to-digital converter.

6.3 The bipolar junction transistor, briefly

Before MOSFETs dominated, the main transistor was the **bipolar junction transistor** (BJT), made of three layers: n-p-n or p-n-p. Its terminals are the **base**, **collector** and **emitter**. In an NPN transistor:

- it turns on when the base–emitter junction is forward biased, about $V_{BE} \approx 0.7\,\text{V}$;
- a small base current controls a large collector current, $I_C = \beta I_B$, with β typically around 100.

The key contrast: a MOSFET is *voltage-controlled* and its gate draws no DC current; a BJT is *current-controlled* and its base needs a steady current. BJTs are still used in some analog and radio circuits. For a CMOS designer they matter mostly because CMOS chips contain unwanted *parasitic* BJTs that can cause latch-up (Chapter 17).

6.4 Filters

A resistor and capacitor make a simple filter. With the capacitor to ground at the output, it is a **low-pass filter** (Figure 6.3): slow signals pass, fast ones are smoothed away. Swap the two components and it becomes a **high-pass filter**: fast changes pass, steady levels are blocked. Either way the boundary is the cutoff frequency

$$f_c = \frac{1}{2\pi RC}.$$

At f_c the output is $1/\sqrt{2}$ of the input, about 71%.

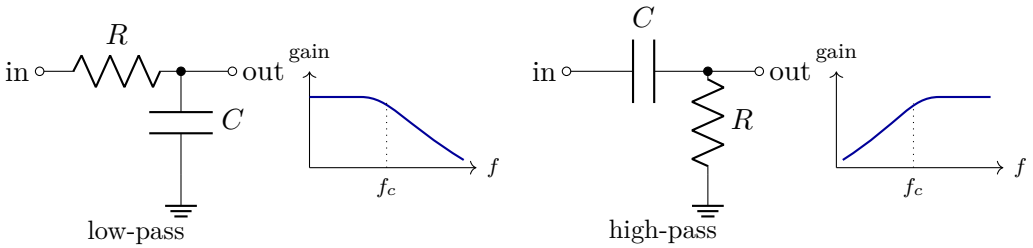


Figure 6.3: RC filters. Low-pass (left): the capacitor to ground shorts fast signals away, so only frequencies below f_c reach the output. High-pass (right): the series capacitor blocks steady levels, so only fast changes pass.

6.5 Converting between analog and digital

An **analog-to-digital converter** (ADC) measures a voltage and outputs a number. An N-bit ADC divides its input range into 2^N steps (Figure 6.4). The voltage of one step, the **least significant bit** (LSB) or **resolution**, is

$$\boxed{\text{LSB} = \frac{V_{\text{ref}}}{2^N}}$$

where V_{ref} is the full-scale range. (Some texts divide by $2^N - 1$ instead; for large N the difference is tiny. Use whichever convention a question implies.)

Worked example: ADC resolution

A 12-bit ADC has a 3.3 V range. What voltage change corresponds to one code step?

$$\frac{3.3}{2^{12}} = \frac{3.3}{4096} \approx 0.806 \text{ mV}.$$

A 10-bit ADC with a 1.024 V range has exactly $1.024/1024 = 1$ mV per step.

Sampling. An ADC measures at regular intervals, the **sampling rate**. To capture a signal faithfully, you must sample at more than twice its highest frequency, the **Nyquist rate**. Sampling too slowly causes **aliasing**: fast signals masquerade as slow ones (Figure 6.5). Audio CDs sample at 44.1 kHz to capture sound up to about 20 kHz.

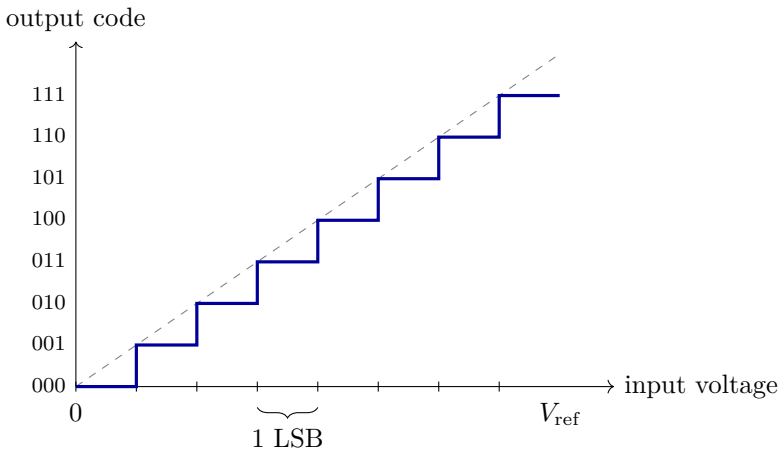


Figure 6.4: A 3-bit ADC divides its input range into $2^3 = 8$ steps. Every input within one step gives the same code; the width of one step is one LSB, $V_{\text{ref}}/2^N$.

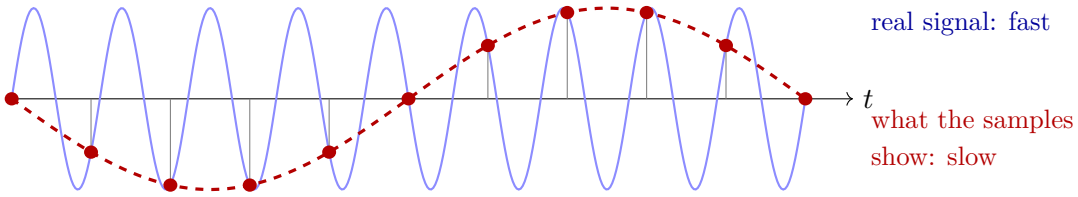


Figure 6.5: Aliasing. The signal (blue) completes 9 cycles in the time the ADC takes 10 samples, far below the Nyquist rate. The samples (dots) trace a slow wave (dashed) that is not really there.

A **digital-to-analog converter** (DAC) does the reverse, turning numbers into voltages.

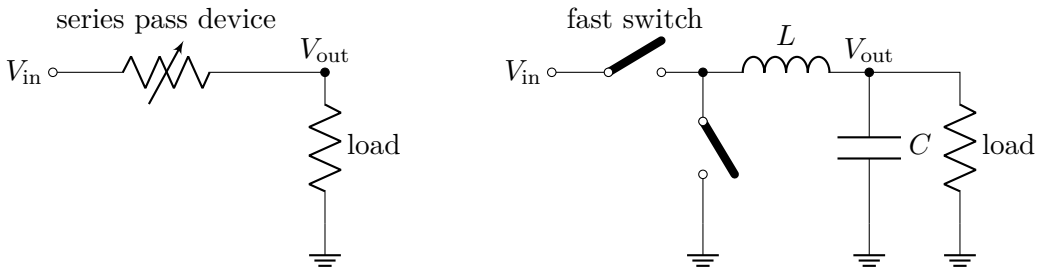
6.6 Power supplies

Chips need a precise, low supply voltage (around 1 V), but power arrives at higher voltages (12 V or more in a computer). Two kinds of circuit convert it (Figure 6.6).

- A **linear regulator** (often an LDO, "low-dropout" regulator) acts like an automatically adjusted resistor in series with the load. It is simple and quiet, but it burns the voltage difference as heat. Its efficiency is about $V_{\text{out}}/V_{\text{in}}$.
- A **switching regulator** (such as a **buck converter**) rapidly switches the input on and off and smooths the result with an inductor and capacitor. It wastes little power, typically 85–95% efficient. GPUs are powered by multi-phase buck converters, called voltage regulator modules (VRMs), placed close to the chip.

Worked example: why linear regulators waste power

An LDO converts 3.3 V to 1.0 V for a load drawing 2 A. The same 2 A flows through the regulator, which drops $3.3 - 1.0 = 2.3$ V. Power wasted: $2.3 \times 2 = 4.6$ W. Power delivered: $1.0 \times 2 = 2$ W. Efficiency: $2/(2 + 4.6) \approx 30\%$, matching $V_{\text{out}}/V_{\text{in}} = 1.0/3.3$.



linear regulator (LDO):
burns $(V_{\text{in}} - V_{\text{out}})\, I$ as heat

switching (buck) regulator:
L and C smooth the switched input

Figure 6.6: Two ways to make a low supply voltage. A linear regulator (left) is an automatically adjusted resistor in series with the load. A buck converter (right) switches the input on and off quickly and smooths the result.

6.7 Summary

- A transistor in saturation amplifies: $g_m = 2I_D/(V_{GS} - V_t)$; common-source gain $A_v = -g_m R_D$.
- Op-amp golden rules (with negative feedback): no input current; $V_+ = V_-$.
- Inverting gain $-R_f/R_{\text{in}}$; non-inverting gain $1 + R_f/R_g$; a follower is a unity-gain buffer; with no feedback an op-amp is a comparator.
- BJTs are current-controlled ($I_C = \beta I_B$, $V_{BE} \approx 0.7\,\text{V}$); MOSFETs are voltage-controlled.
- ADC step size $= V_{\text{ref}}/2^N$. Sample at more than twice the highest frequency.
- Linear regulators are simple but waste the voltage difference; switching regulators are efficient.

6.8 Exercises

1. An NMOS is biased at $I_D = 200\ \mu\text{A}$ with overdrive 0.25 V. Find g_m, and the gain of a common-source amplifier with $R_D = 5\,\text{k}\Omega$.
2. An inverting amplifier has $R_{\text{in}} = 2\,\text{k}\Omega$ and $R_f = 10\,\text{k}\Omega$. If $V_{\text{in}} = 0.3\,\text{V}$, what is V_{out}?
3. A non-inverting amplifier has $R_f = 9\,\text{k}\Omega$ and $R_g = 1\,\text{k}\Omega$. What is its gain?
4. An op-amp with no feedback has $V_+ = 1.2\,\text{V}$ and $V_- = 1.0\,\text{V}$. What does the output do?
5. A 10-bit ADC has a 1.024 V range. What is one step?
6. A signal contains frequencies up to 20 kHz. What is the minimum sampling rate?
7. A linear regulator supplies 3.3 V at 1 A from a 5 V input. How much power does it waste, and what is its efficiency?

Answers

1. $g_m = 2 \times 200\,\mu\text{A}/0.25\,\text{V} = 1.6\,\text{mA/V}$. Gain $= -1.6 \times 10^{-3} \times 5000 = -8$.
2. $V_{\text{out}} = -(10/2) \times 0.3 = -1.5\,\text{V}$.

3. $1 + 9/1 = 10$.
4. It swings to its maximum (positive) output, because $V_+ > V_-$. It is acting as a comparator.
5. $1.024/1024 = 1\,\text{mV}$.
6. More than 40 kHz (twice 20 kHz).
7. Wasted: $(5 - 3.3) \times 1 = 1.7\,\text{W}$. Efficiency: $3.3/5 = 66\%$.

Part III

From Transistors to Logic

Chapter 7

Binary Numbers, Bits and Boolean Algebra

We now have switches (transistors). Before wiring them into logic, we need the language that logic speaks: binary numbers and Boolean algebra. This chapter covers how computers represent numbers, how to manipulate individual bits (a favorite topic of the coding part of the assessment), and the algebra used to design and simplify logic circuits.

7.1 Why two values?

A wire's voltage can take any value, so why do computers use only two, 0 and 1? Because of *noise*. Every real signal picks up small unwanted disturbances: from neighboring wires, from the power supply, from temperature. If we tried to represent ten different values with ten voltage levels between 0 and 1 V, levels would be only 0.1 V apart, and a little noise would turn a 6 into a 7.

With only two values, we can treat *any* voltage near 0 as a 0 and *any* voltage near the supply as a 1, with a wide forbidden zone between. Noise has to be enormous to flip a value. Better still, each logic gate restores its output to a clean 0 or 1, so noise does not accumulate as signals pass through many gates (Chapter 8). Two values are the most robust choice.

One binary digit is a **bit**. Groups of bits have names: 4 bits are a **nibble**, 8 bits a **byte**, and a **word** is the natural size a processor works with (often 32 or 64 bits).

7.2 Binary numbers

In decimal, each digit position is worth ten times the one to its right: $347 = 3\times100 + 4\times10 + 7\times1$. In binary, each position is worth *two* times the one to its right:

$$1011_2 = 1\times8 + 0\times4 + 1\times2 + 1\times1 = 11_{10}.$$

The rightmost bit is the **least significant bit** (LSB); the leftmost is the **most significant bit** (MSB).

Decimal to binary. Repeatedly divide by 2 and record the remainders; read them from last to first. For 45: $45/2 = 22$ r1, $22/2 = 11$ r0, $11/2 = 5$ r1, $5/2 = 2$ r1, $2/2 = 1$ r0, $1/2 = 0$ r1. Reading upward: $45 = 101101_2$. Check: $32 + 8 + 4 + 1 = 45$.

Powers of two to know by heart.

2^0	2^1	2^2	2^3	2^4	2^5	2^6
1	2	4	8	16	32	64
2^7	2^8	2^{10}	2^{12}	2^{16}	2^{20}	2^{32}
128	256	1024	4096	65 536	1 048 576	$\approx 4.3 \times 10^9$

In computing, a "kilobyte" of memory usually means $2^{10} = 1024$ bytes (strictly a kibibyte, KiB), a "megabyte" 2^{20} bytes, and a "4 KB page" is $2^{12} = 4096$ bytes.

7.3 Hexadecimal

Long binary strings are hard to read, so engineers write them in **hexadecimal** (base 16). Each hex digit stands for exactly 4 bits, using 0–9 then A–F for 10–15:

0	1	2	3	4	5	6	7
0000	0001	0010	0011	0100	0101	0110	0111
8	9	A	B	C	D	E	F
1000	1001	1010	1011	1100	1101	1110	1111

To convert binary to hex, split into groups of four from the right: $1011\,0111_2 = \mathrm{B7}_{16}$. In code, hex is written with a `0x` prefix: `0xB7` $= 11 \times 16 + 7 = 183$. Memory addresses are almost always written in hex.

7.4 How many values fit in n bits?

Each bit doubles the number of possible patterns, so n bits can represent 2^n different values. Going the other way, to represent N different things you need

$$n = \lceil \log_2 N \rceil \text{ bits}$$

(the ceiling $\lceil \cdot \rceil$ means round up). For 1000 values: $2^9 = 512$ is too few, $2^{10} = 1024$ is enough, so 10 bits. This calculation appears constantly: how many flip-flops a state machine needs (Chapter 12), how many address bits a memory needs, how many select lines a multiplexer needs.

7.5 Negative numbers: two's complement

Computers almost always store signed integers in **two's complement**. The idea is simple: the MSB gets a *negative* weight (Figure 7.1). For 8 bits, the place values are

$$-128,\ 64,\ 32,\ 16,\ 8,\ 4,\ 2,\ 1.$$

So $1111\,1111_2 = -128 + 127 = -1$, and $1000\,0000_2 = -128$.

- Range of n-bit two's complement: -2^{n-1} to $2^{n-1} - 1$. For 8 bits: -128 to 127.
- The MSB tells the sign: 1 means negative.
- **To negate a number: invert every bit, then add 1.**

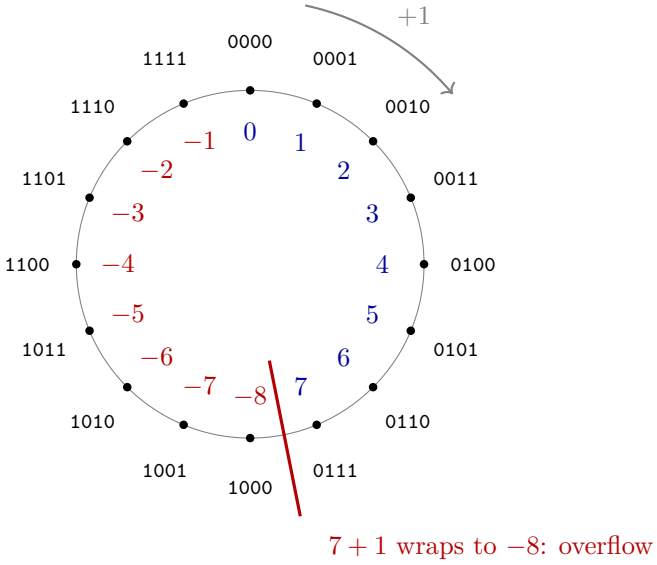


Figure 7.1: Four-bit two's complement around a circle. Adding 1 moves one step clockwise. Patterns that start with 1 are negative (red), and counting up past 0111 = 7 wraps to 1000 = −8: overflow.

Worked example: representing −20 in 8 bits

+20 = 0001 0100. Invert: 1110 1011. Add 1: 1110 1100. Check with weights: $-128 + 64 + 32 + 8 + 4 = -20$. Correct.

The beauty of two's complement is that ordinary binary addition works for signed numbers too: the same adder hardware handles both, and subtraction is just adding the negation, $A - B = A + \overline{B} + 1$ (Chapter 9).

Overflow. If a result does not fit in n bits, it wraps around. In 8 bits, $127 + 1$ gives $1000\,0000 = -128$. Signed overflow happens exactly when two numbers of the same sign are added and the result has the opposite sign.

Sign extension. To widen a two's complement number, copy its sign bit into the new upper bits: −3 in 4 bits is 1101; in 8 bits it is 1111 1101.

7.6 Bitwise operations

Programming languages (and hardware) can operate on every bit of a word at once:

Operation	C syntax	Rule per bit	Example (`1100` op `1010`)
AND	`a & b`	1 only if both are 1	`1000`
OR	`a | b`	1 if either is 1	`1110`
XOR	`a ^ b`	1 if they differ	`0110`
NOT	`~a`	flip every bit	`~1100 = 0011`
Left shift	`a << k`	move bits left, fill with 0	`0011 << 1 = 0110`
Right shift	`a >> k`	move bits right	`1100 >> 2 = 0011`

Shifting left by k multiplies by 2^k; shifting right by k divides by 2^k (rounding down) for unsigned numbers. So with `i = 8` and `j = 2`, `i << j` is 32 and `i >> j` is 2. For *signed* numbers, a right shift usually copies the sign bit in from the left (an **arithmetic shift**) so negative numbers stay negative; unsigned values get zeros (a **logical shift**).

Bit tricks worth knowing

Using a **mask** (a word with 1s only where you care), you can work on individual bits:

Goal	Expression
Read bit k	`(x >> k) & 1`
Set bit k to 1	`x | (1 << k)`
Clear bit k to 0	`x & ~(1 << k)`
Toggle bit k	`x ^ (1 << k)`
Keep only the lowest set bit	`x & (-x)`
Clear the lowest set bit	`x & (x - 1)`
Is x a power of two?	`x != 0 && (x & (x - 1)) == 0`
Mask of bits `lo` to `hi`	`((1 << (hi - lo + 1)) - 1) << lo`

Why does `x & (x-1)` clear the lowest set bit? Subtracting 1 flips the lowest 1 to 0 and all the 0s below it to 1s, and leaves everything above unchanged. ANDing with the original keeps the upper bits and zeros the rest. A power of two has exactly one 1 bit, so clearing it leaves zero.

Worked example: building a bit mask

Build a mask with 1s in bits 4 through 7. Width $= 7 - 4 + 1 = 4$. `(1 << 4) - 1` $= 1111_2$. Shift left by 4: $1111\,0000_2 =$ `0xF0`. (Careful: if the width is the full 32 bits, `1 << 32` overflows a 32-bit integer in C; use a 64-bit type for the intermediate value.)

Worked example: are two addresses on the same 4 KB page?

Memory is divided into pages of $4096 = 2^{12}$ bytes. The low 12 bits of an address are the offset within a page; the remaining upper bits are the *page number*. Two addresses are on the same page exactly when their page numbers match:

`(a >> 12) == (b >> 12)` or equivalently `((a ^ b) & ~0xFFF) == 0`

`0x12345FF0` and `0x12345010`: both have page number `0x12345`, so same page. `0x12345FFF` and `0x12346000`: page numbers `0x12345` and `0x12346`, so different pages, even though the addresses differ by only 1.

Seen on the assessment

Both of the worked examples above match coding problems reported on NVIDIA's Ignite assessment: building a bit mask from bit `lo` to bit `hi`, and checking whether two 32-bit addresses fall on the same 4 KB page. Chapter 23 has full code.

XOR, the most useful operator

XOR has algebraic properties that make it a favorite in puzzles and in hardware:

- $a \oplus 0 = a$ and $a \oplus a = 0$: XORing with a value twice cancels it.
- $a \oplus 1 = \overline{a}$: XOR with 1 inverts. So XOR is a *controlled inverter*.
- XOR is commutative and associative, so order does not matter.
- The XOR of all bits of a word is its **parity**: 1 if it has an odd number of 1s.

A consequence used in coding problems: if P_i is the XOR of the first i elements of an array (a **prefix XOR**), then the XOR of elements l through r is $P_{r+1} \oplus P_l$, because everything before l appears twice and cancels.

7.7 Gray code

In ordinary binary, counting from 3 (011) to 4 (100) changes three bits at once. If those bits are read at the wrong moment, while some have changed and some have not, you can get any value at all. **Gray code** orders the numbers so that *exactly one bit changes* between neighbors:

Decimal	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111
Gray	000	001	011	010	110	111	101	100

Conversion: $g = b \oplus (b \gg 1)$. Gray code is used for counters that are read by a different clock (Chapter 19) and to label Karnaugh maps (below).

7.8 Boolean algebra

Boolean algebra, named after George Boole, is the mathematics of variables that are only 0 or 1. It has three basic operations (their symbols are in Figure 7.2):

- AND, written $A \cdot B$ or just AB: 1 only if both are 1.
- OR, written $A + B$: 1 if at least one is 1. (Here $1 + 1 = 1$.)
- NOT, written $\overline{A}$ (or A'): the opposite value.

Precedence, as in ordinary algebra: NOT first, then AND, then OR. So $A + BC$ means $A + (B \cdot C)$.

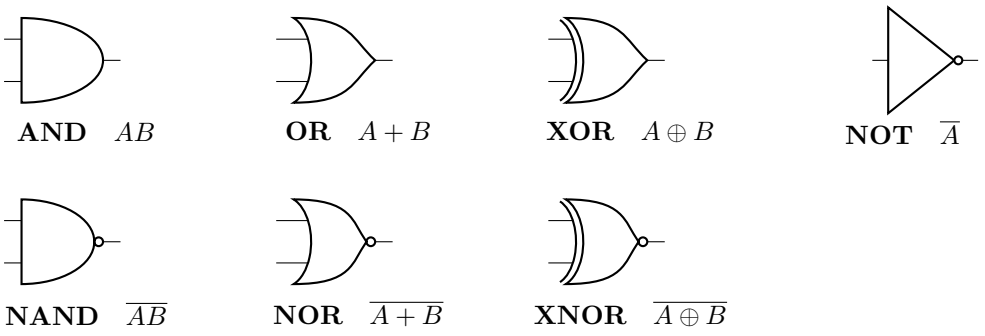


Figure 7.2: Logic gate symbols. A small circle (a bubble) on an output means NOT: NAND is AND followed by NOT, NOR is OR followed by NOT, and XNOR is XOR followed by NOT.

Combined truth table of the common two-input functions:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

The laws

Law	OR form	AND form
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Complement	$A + \overline{A} = 1$	$A \cdot \overline{A} = 0$
Involution	$\overline{\overline{A}} = A$	
Commutative	$A + B = B + A$	$AB = BA$
Associative	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
Distributive	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
Also useful	$A + \overline{A}B = A + B$	$A(\overline{A} + B) = AB$
De Morgan	$\overline{A + B} = \overline{A}\,\overline{B}$	$\overline{AB} = \overline{A} + \overline{B}$
Consensus	$AB + \overline{A}C + BC = AB + \overline{A}C$	

Two of these surprise people because they do not hold in ordinary algebra: $A + 1 = 1$, and the first distributive law $A + BC = (A + B)(A + C)$. Check any law yourself with a truth table; that is always a valid proof.

The big idea

De Morgan's laws: to complement an expression, swap AND with OR and complement each variable. "Not (A or B)" is "not A and not B." These laws are why a NAND gate is the same as an OR gate with inverted inputs, and they are the key to building logic from transistors (Chapter 8).

Worked example: simplifying

Simplify $F = AB + A\overline{B} + \overline{A}B$.
$AB + A\overline{B} = A(B + \overline{B}) = A$. So $F = A + \overline{A}B = A + B$.

7.9 From a truth table to an expression

Any truth table can be turned into a formula:

- **Sum of products (SOP)**: for each row where the output is 1, write an AND term (a **minterm**) that is 1 only for that row, then OR them all. For $A = 0, B = 1$, the minterm is $\overline{A}B$.
- **Product of sums (POS)**: for each row where the output is 0, write an OR term (a **maxterm**) that is 0 only for that row, then AND them.

Minterms are numbered by the row's binary value; $F = \sum m(1, 2)$ means "1 in rows 1 and 2." For

two inputs, that is $\overline{A}B + A\overline{B} = A \oplus B$.

How many functions are there? An n-input truth table has 2^n rows, and each row's output can be 0 or 1 independently, so there are 2^{2^n} different functions. For 2 inputs: $2^4 = 16$. For 3 inputs: $2^8 = 256$.

7.10 Karnaugh maps

A **Karnaugh map** (K-map) is a truth table drawn as a grid so that simplifications become visible. Rows and columns are labeled in Gray code, so any two neighboring cells differ in exactly one variable. When two neighboring cells are both 1, that variable does not matter, and the two minterms merge into one simpler term.

Rules for grouping the 1s:

1. Groups are rectangles of 1, 2, 4, 8 or 16 cells (powers of two).
2. Groups may wrap around the edges: the left column neighbors the right column, and the top row neighbors the bottom row (the four corners are neighbors too).
3. Make each group as large as possible, and use as few groups as possible. Groups may overlap.
4. Each group becomes one product term containing only the variables that stay constant across the group.
5. "Don't care" cells (inputs that never occur, written X) may be included in groups if that helps, or ignored.

$AB\backslash CD$	00	01	11	10
00	(1)	0	0	(1)
01	0	1	1	0
11	0	1	1	0
10	(1)	0	0	(1)

BD (center four)

$\overline{B}\,\overline{D}$ (four corners, wrapping)

Figure 7.3: K-map for $F = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$. Two groups of four give $F = BD + \overline{B}\,\overline{D}$.

Worked example: a four-variable K-map

Simplify $F(A, B, C, D) = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$ (Figure 7.3).
The center four cells have $B = 1$ and $D = 1$ while A and C vary: the group is BD. The four corner cells are neighbors through wrap-around; they have $B = 0$ and $D = 0$: the group is $\overline{B}\,\overline{D}$. So $F = BD + \overline{B}\,\overline{D}$, which is B XNOR D. Eight minterms collapsed into one two-input gate.

7.11 Summary

- Binary uses two values because they are the most robust against noise.
- n bits hold 2^n values; N values need $\lceil \log_2 N \rceil$ bits. Hex digits are 4 bits each.
- Two's complement: MSB has weight -2^{n-1}; negate by inverting and adding 1; range -2^{n-1} to $2^{n-1} - 1$.
- Shifts multiply and divide by powers of two; masks select bits; `x & (x-1)` clears the lowest set bit.
- Same 4 KB page: compare `a >> 12` with `b >> 12`.
- XOR: $a \oplus a = 0$, $a \oplus 1 = \overline{a}$; prefix XOR gives any subarray's XOR.
- Gray code changes one bit at a time.
- Boolean laws, especially De Morgan; SOP from the 1-rows; K-maps group neighboring 1s in powers of two.

7.12 Exercises

1. Convert 45 to binary and to hex.
2. Convert `0xB7` to decimal and to binary.
3. What is the range of an 8-bit two's complement number? Write -20 and -1 in 8-bit two's complement.
4. How many bits are needed to give 1000 items unique IDs?
5. For `x = 0b10110110`, compute `x & (x - 1)` and `x & (-x)`.
6. Are `0x7FFFF000` and `0x7FFFFFFF` on the same 4 KB page? What about `0x00001FFF` and `0x00002000`?
7. Write a mask with 1s in bits 8 through 11 (bit 0 is the LSB), in hex.
8. Simplify: (a) $A + \overline{A}B$; (b) $(A + B)(A + C)$; (c) apply De Morgan to $\overline{AB + C}$.
9. Use a K-map (or algebra) to simplify $F(A, B, C) = \sum m(1, 3, 5, 7)$.
10. How many different Boolean functions of 3 inputs exist?
11. What is the 3-bit Gray code for the number 6?
12. In C, with `int i = 8, j = 2;`, what do `i << j` and `i >> j` evaluate to?

Answers

1. $45 = 101101_2 = $ `0x2D`.
2. `0xB7` $= 183 = 1011\,0111_2$.
3. -128 to 127. $-20 = 1110\,1100$. $-1 = 1111\,1111$.
4. 10 bits ($2^{10} = 1024 \geq 1000$).

5. `x & (x-1)` = 1011 0100 (lowest set bit, bit 1, cleared). `x & (-x)` = 0000 0010 (only the lowest set bit kept).
6. First pair: both have page number `0x7FFFF`, so yes. Second pair: `0x00001` versus `0x00002`, so no.
7. `0xF00`.
8. (a) $A + B$. (b) $A + BC$. (c) $\overline{AB} \cdot \overline{C} = (\overline{A} + \overline{B})\,\overline{C}$.
9. The 1s are exactly the rows where $C = 1$, so $F = C$.
10. $2^{2^3} = 256$.
11. $6 = 110_2$; $g = 110 \oplus 011 = 101$.
12. 32 and 2.

Chapter 8

Building Logic Gates from Transistors

We have switches (Chapter 5) and the algebra of 0s and 1s (Chapter 7). This chapter joins them: it shows how NMOS and PMOS transistors are wired into logic gates, why the standard arrangement works so well, and what goes wrong when it is changed, including the swapped-inverter question from the assessment.

8.1 The rule of complementary logic

Every standard logic gate in a modern chip uses **static CMOS** (complementary MOS) design. Its output node is connected to two networks of transistors (Figure 8.1):

- a **pull-up network** (PUN) of PMOS transistors, between the supply V_{DD} and the output, which connects the output to 1;
- a **pull-down network** (PDN) of NMOS transistors, between the output and ground, which connects the output to 0.

The two networks are designed so that, for every combination of inputs, *exactly one* of them conducts. The output is therefore always firmly connected to either the supply or ground, never to both and never to neither.

Why PMOS on top and NMOS on the bottom? Because of the strong and weak levels from Section 5.7: PMOS passes a strong 1 and NMOS passes a strong 0. Each transistor is placed where it is good.

8.2 The CMOS inverter

The simplest gate is the **inverter** (NOT gate): one PMOS and one NMOS, gates tied together as the input, drains tied together as the output (Figure 8.2).

- **Input = 0:** the PMOS is on (gate low) and the NMOS is off. The output connects to V_{DD}: output = 1.
- **Input = 1:** the NMOS is on and the PMOS is off. The output connects to ground: output = 0.

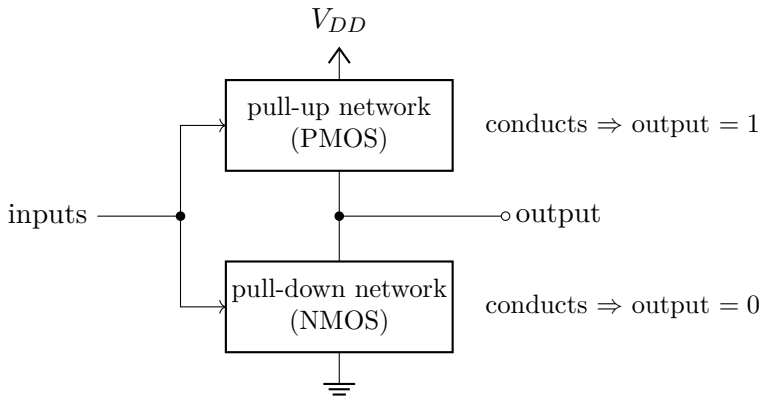


Figure 8.1: Every static CMOS gate: a pull-up network of PMOS transistors between V_{DD} and the output, and a pull-down network of NMOS transistors between the output and ground. The same inputs drive both, and for every input combination exactly one of them conducts.

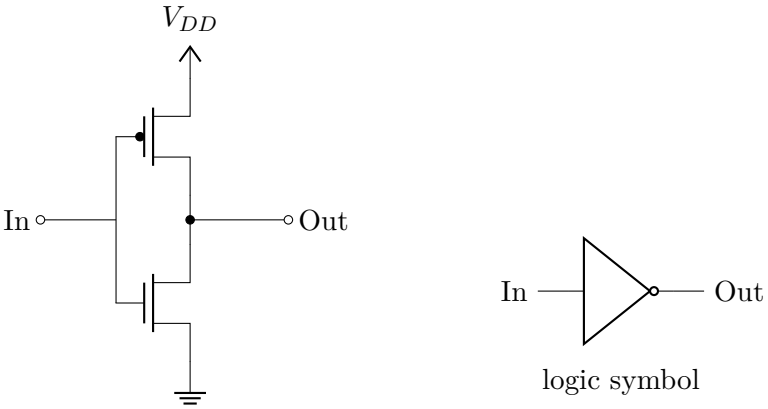


Figure 8.2: The CMOS inverter and its logic symbol.

Three properties make this the ideal gate:

1. **Full swing.** The output reaches exactly V_{DD} and exactly 0 V, because each transistor passes its strong level.
2. **No static current.** In either steady state one transistor is off, so there is no path from supply to ground. An idle inverter burns only tiny leakage power.
3. **Restoring.** The output is driven by the supply rails, not by the input, so a slightly noisy input still produces a clean output.

8.3 The voltage transfer characteristic

What happens for inputs between 0 and V_{DD}? Plotting the output voltage against the input voltage gives the **voltage transfer characteristic** (VTC), Figure 8.3.

- For low inputs the output stays near V_{DD}; for high inputs it stays near 0. In between, it drops steeply.

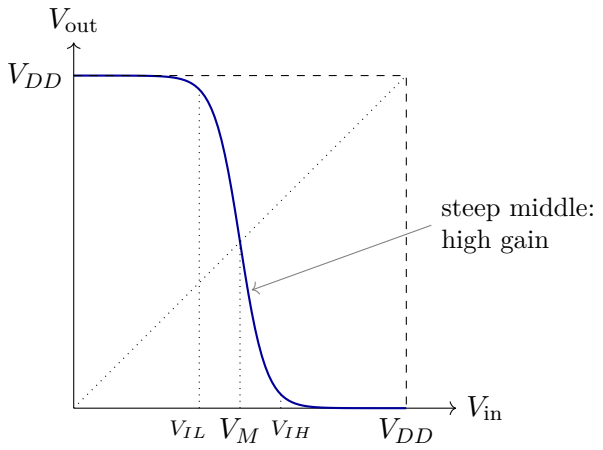


Figure 8.3: Voltage transfer characteristic of an inverter. Where the curve crosses the diagonal, $V_{\text{in}} = V_{\text{out}} = V_M$.

- In the steep middle both transistors are partly on. A current flows straight from supply to ground for the short time the input spends there (the short-circuit current, Chapter 14).
- The **switching threshold** V_M is the input voltage at which $V_{\text{out}} = V_{\text{in}}$. With equally strong pull-up and pull-down (PMOS sized up to match the NMOS), $V_M = V_{DD}/2$. A stronger PMOS shifts V_M higher, a stronger NMOS shifts it lower: the stronger side wins the tug-of-war, just like a voltage divider (Chapter 1).
- The steep middle means *high gain*: the inverter acts as an amplifier there (Chapter 6). This gain is what lets gates clean up noisy signals.

Noise margins. Define four voltages:

- V_{OH}, V_{OL}: the output levels a gate actually produces for 1 and 0;
- V_{IH}: the lowest input voltage guaranteed to be read as 1;
- V_{IL}: the highest input voltage guaranteed to be read as 0.

(V_{IL} and V_{IH} are usually taken where the VTC's slope is -1.) The **noise margins** are how much noise a signal can pick up between one gate and the next and still be read correctly:

$$NM_H = V_{OH} - V_{IH}, \qquad NM_L = V_{IL} - V_{OL}.$$

Larger is better. A CMOS inverter has $V_{OH} = V_{DD}$ and $V_{OL} = 0$, which gives it large noise margins.

8.4 The swapped inverter

Now the assessment question. Take an inverter and swap the transistors: the NMOS now connects the output to V_{DD}, and the PMOS connects the output to ground (Figure 8.4). What does the circuit do?

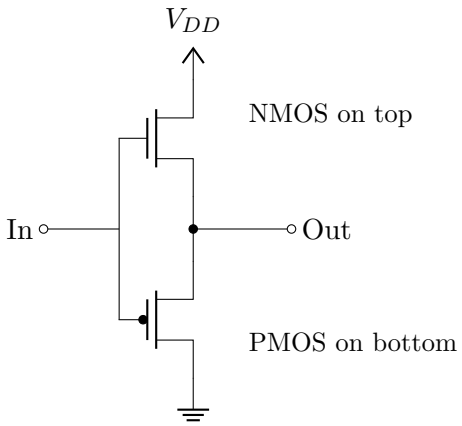


Figure 8.4: The "swapped" inverter: NMOS to the supply, PMOS to ground.

Work through both inputs using the rules from Chapter 5. Let $V_{DD} = 1.0\,\text{V}$, $V_{tn} = 0.3\,\text{V}$ and $|V_{tp}| = 0.3\,\text{V}$.

Input = 1 (V_{DD}). The top NMOS has its gate high, so it turns on and starts pulling the output *up* toward V_{DD}. The bottom PMOS has its gate high, so it is off. But the NMOS is passing a 1, and NMOS passes a weak 1: the output is the NMOS's source, and the transistor turns off when the output reaches $V_{DD} - V_{tn} = 0.7\,\text{V}$. The output settles at about 0.7 V.

Input = 0. The top NMOS is off. The bottom PMOS has its gate at 0, so it turns on and pulls the output *down*. But PMOS passes a weak 0: the output is the PMOS's source, and the transistor turns off when the output falls to $|V_{tp}| = 0.3\,\text{V}$. The output settles at about 0.3 V.

Input	Normal inverter output	Swapped circuit output
0	V_{DD} (strong 1)	$\lvert V_{tp}\rvert = 0.3\,\text{V}$ (weak 0)
1	0 (strong 0)	$V_{DD} - V_{tn} = 0.7\,\text{V}$ (weak 1)

Seen on the assessment

The swapped circuit **no longer inverts**. It is a **non-inverting buffer with degraded output levels**: a high input gives a weak high ($V_{DD} - V_{tn}$), and a low input gives a weak low ($|V_{tp}|$). Each transistor acts as a *source follower*: its output follows its gate, one threshold behind. The output swing is reduced, drive strength is weak (the transistors turn off as the output approaches its final value, so it gets there slowly), noise margins shrink, and the gain is less than 1, so the circuit cannot clean up noise. Chaining such stages makes it worse: in the simple model, a second stage driven by 0.7 V can only reach $0.7 - 0.3 = 0.4\,\text{V}$, and the logic levels collapse.

The lesson generalizes: whenever a question puts NMOS in a pull-up position or PMOS in a pull-down position, expect a threshold-voltage drop on that level.

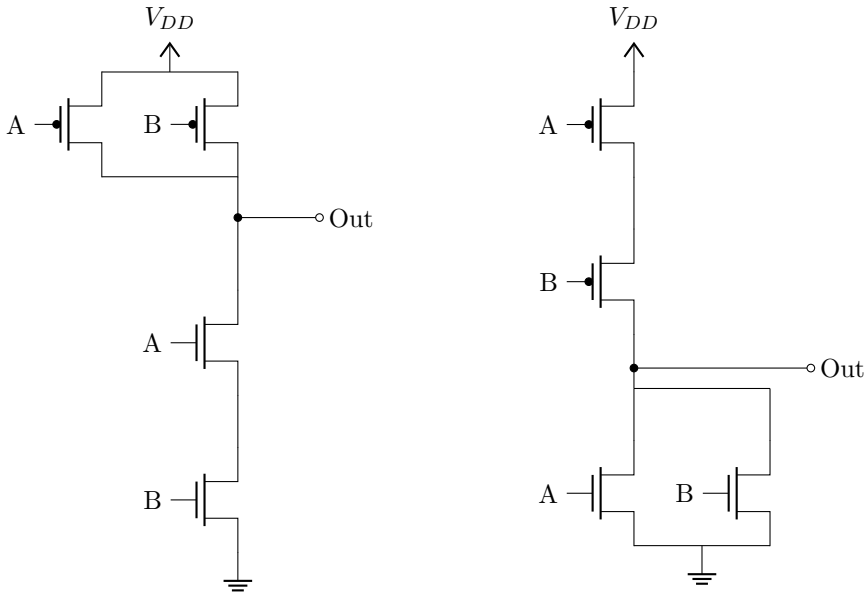
8.5 NAND and NOR gates

The two-input NAND

A NAND gate's output is 0 only when both inputs are 1. So the pull-down network must conduct only when $A = 1$ *and* $B = 1$: two NMOS transistors **in series** (current must pass through both, so both must be on). The pull-up must conduct whenever $A = 0$ *or* $B = 0$: two PMOS transistors **in parallel** (either one can connect the output to V_{DD}). See Figure 8.5, left.

The two-input NOR

A NOR gate's output is 0 when $A = 1$ *or* $B = 1$: two NMOS **in parallel**. Its output is 1 only when both inputs are 0: two PMOS **in series** (Figure 8.5, right).



NAND: PMOS parallel, NMOS series NOR: PMOS series, NMOS parallel

Figure 8.5: Two-input NAND (left) and NOR (right). In each, the pull-up network is the *dual* of the pull-down: series becomes parallel and parallel becomes series.

The big idea

In the pull-down network, **series NMOS means AND** (all must be on) and **parallel NMOS means OR** (any can be on). The pull-up network is the **dual**: swap series and parallel, and use PMOS. This guarantees exactly one network conducts for every input combination.

8.6 Designing any gate

The same recipe builds any inverting function:

1. Write the function as $F = \overline{\text{something}}$, where "something" uses only ANDs and ORs of the inputs. That "something" is when the output should be 0.
2. Build the pull-down network from it: AND becomes series NMOS, OR becomes parallel NMOS.
3. Build the pull-up network as the dual: swap series and parallel, use PMOS, same inputs.

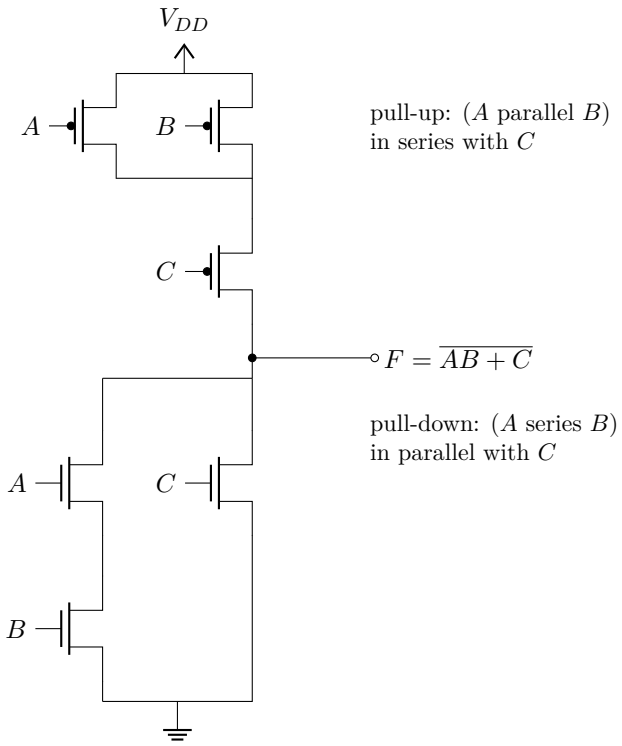


Figure 8.6: The AOI21 gate $F = \overline{AB + C}$ in six transistors. The pull-down network is built straight from $AB + C$; the pull-up network is its dual, with series and parallel swapped.

Worked example: an AND-OR-INVERT gate

Build $F = \overline{AB + C}$ (called AOI21: AND-OR-INVERT with a 2-input AND and a 1-input term).
Pull-down conducts when $AB + C = 1$: NMOS A in series with NMOS B, and that pair in parallel with NMOS C.
Pull-up is the dual: PMOS A in parallel with PMOS B, and that pair in series with PMOS C.
Total: 6 transistors (Figure 8.6). Building the same function from separate AND, OR and NOT gates would take 12 or more, and be slower. Compound gates like AOI and OAI (OR-AND-INVERT) are common for this reason.

Why static CMOS gates always invert. In a single stage, turning an input *on* (to 1) can only turn NMOS transistors on, which can only pull the output *down*. So a rising input can only cause a falling output. A non-inverting function needs two stages: AND is a NAND followed by an inverter, OR is a NOR followed by an inverter.

Gate	Transistors (static CMOS)
Inverter	2
NAND2, NOR2	4
NAND3, NOR3	6
AND2, OR2 (gate + inverter)	6
AOI21, OAI21	6
XOR2 (straightforward static design)	12

An n-input static CMOS gate uses $2n$ transistors: one NMOS and one PMOS per input.

8.7 Why NAND is preferred over NOR

NAND and NOR are both universal (Chapter 9), but designers strongly prefer NAND. The reason is mobility. Transistors in series are slower, because their resistances add (Chapter 1). A NAND puts its series stack in the NMOS, which is the fast device. A NOR puts its series stack in the PMOS, which is already about two to three times slower. To compensate, the series PMOS in a NOR must be made very wide, which costs area and adds input capacitance.

Sizing for the same strength as a reference inverter (NMOS width 1, PMOS width 2, assuming $\mu_n = 2\mu_p$):

Gate	Each NMOS	Each PMOS
Inverter	1	2
NAND2	2 (two in series)	2 (in parallel)
NOR2	1 (in parallel)	4 (two in series)

Two transistors in series act like one with twice the resistance, so each must be twice as wide to keep the same total strength. The NOR's PMOS pair ends up four times the NMOS width.

Fan-in limits. The more inputs a gate has, the taller its series stack. Tall stacks are slow, both because resistances add and because the upper transistors suffer the body effect (Chapter 5). Gates are usually limited to about four inputs; wider functions are built as trees of smaller gates.

8.8 Pass transistors and transmission gates

Instead of connecting outputs to the supply rails, a **pass transistor** simply passes an input signal through. A single NMOS pass transistor is small, but it passes a weak 1. The fix is the **transmission gate** (TG): an NMOS and a PMOS in parallel, controlled by opposite signals (Figure 8.7). When the control is on, both conduct: the NMOS carries the 0s well and the PMOS carries the 1s well, so the TG passes *both* levels at full strength.

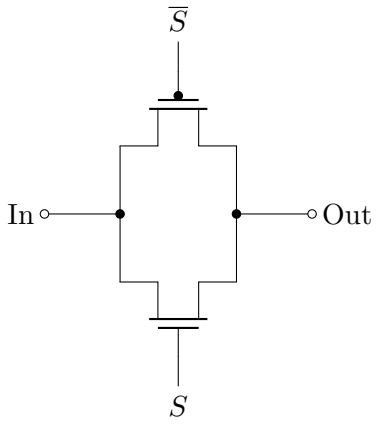


Figure 8.7: A transmission gate: when $S = 1$, both transistors conduct and the input passes to the output at full strength.

Transmission gates make compact multiplexers (two TGs and an inverter make a 2:1 mux), XOR gates, and the latches inside flip-flops (Chapter 10). Their drawback is that they do not restore signal levels or add drive; a long chain of TGs gets slow and must be broken up with buffers.

8.9 Tristate buffers

Sometimes several circuits must share one wire, such as a bus. Only one may drive it at a time; the others must disconnect. A **tristate buffer** has an enable input (Figure 8.8). When enabled, it drives its input value (0 or 1) onto the output. When disabled, both its pull-up and pull-down are off and the output is disconnected, a third state called **high impedance** or **Z**. If two enabled drivers fight over one wire (one driving 1, the other 0), the result is **contention** (Figure 8.9): a large current flows between them and the wire sits at an undefined middle voltage.

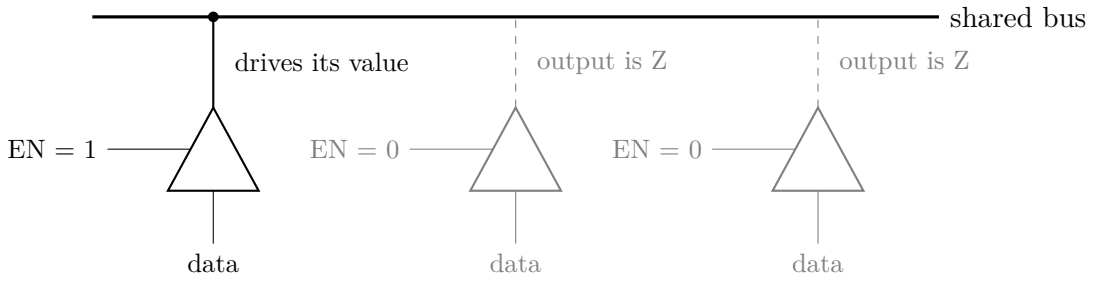


Figure 8.8: Tristate buffers sharing a bus. Only the enabled buffer (EN = 1) drives the wire; the disabled ones are in high impedance (Z), as if unplugged.

8.10 Other logic styles

Dynamic logic. A dynamic gate replaces the pull-up network with a single clocked PMOS. In the *precharge* phase (clock low), it charges the output high. In the *evaluate* phase (clock high),

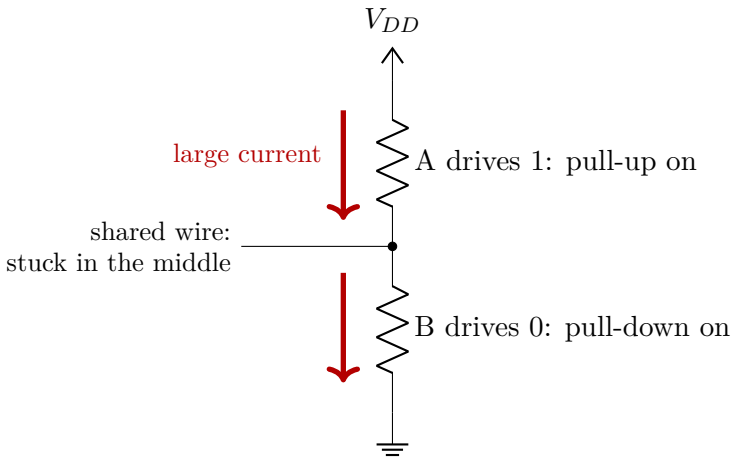


Figure 8.9: Bus contention. When two enabled drivers disagree, one connects the wire to the supply and the other to ground. Current flows straight through both, and the wire sits at a divider voltage that is neither 0 nor 1.

a clocked NMOS "foot" turns on and the NMOS pull-down network discharges the output if the logic function says so. Dynamic gates use fewer transistors and are fast, but the precharged output is held only by capacitance, so it can leak away, be disturbed by noise, or lose charge to internal nodes (**charge sharing**). They also need a clock and burn more power. **Domino logic** adds an inverter after each dynamic gate so that cascaded gates work correctly.

Ratioed logic (pseudo-NMOS). Replace the pull-up network with a single PMOS that is always on, acting like a resistor. The pull-down network must overpower it to make a 0. It saves transistors, but the output low is set by a tug-of-war (a voltage divider), so it never reaches exactly 0 V, and current flows from supply to ground continuously whenever the output is low: static power.

8.11 Summary

- Static CMOS: PMOS pull-up network to V_{DD}, NMOS pull-down network to ground, exactly one conducting.
- The inverter has full swing, no static current, high gain and restores clean levels.
- VTC: $V_M = V_{DD}/2$ when balanced; the stronger device pulls V_M toward its side. Noise margins $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.
- Swapped inverter (NMOS up, PMOS down): non-inverting buffer with weak levels, $V_{DD} - V_{tn}$ and $|V_{tp}|$; gain below 1; degrades when chained.
- Pull-down: series NMOS = AND, parallel = OR. Pull-up is the dual.
- Single-stage static CMOS gates are inverting. n inputs need $2n$ transistors.
- NAND beats NOR because NOR stacks slow PMOS in series.
- Transmission gates pass both levels strongly. Tristate outputs can disconnect (Z).
- Dynamic logic is fast but fragile; pseudo-NMOS burns static power.

8.12 Exercises

1. In a CMOS inverter with input 0, which transistor is on, and what is the output?
2. How much static current does an ideal CMOS inverter draw when its input is steady?
3. The PMOS in an inverter is made much stronger than the NMOS. Does V_M move up or down?
4. A swapped inverter has $V_{DD} = 1.2\,\text{V}$, $V_{tn} = 0.4\,\text{V}$, $|V_{tp}| = 0.35\,\text{V}$. What are the outputs for input 1 and input 0? Is it an inverter?
5. How many transistors are in a static CMOS NOR3? In an AND2?
6. Design the pull-down and pull-up networks for $F = \overline{(A + B)\,C}$ (an OAI21 gate).
7. Why are NAND gates generally preferred over NOR gates?
8. Assuming $\mu_n = 2\mu_p$ and a reference inverter with NMOS width 1, size a NOR2 for equal pull-up and pull-down strength.
9. Why does a transmission gate pass both 0 and 1 at full strength?
10. Give one advantage and one disadvantage of dynamic logic.
11. A gate has $V_{OH} = 1.0\,\text{V}$, $V_{OL} = 0.05\,\text{V}$, $V_{IH} = 0.6\,\text{V}$ and $V_{IL} = 0.4\,\text{V}$. Find both noise margins.
12. True or false: a single static CMOS stage can implement AND.

Answers

1. The PMOS is on (its gate is low), connecting the output to V_{DD}: output = 1.
2. Essentially none; only leakage, because one transistor is always off.
3. Up: the stronger pull-up wins the tug-of-war until the input is higher.
4. Input 1: $1.2 - 0.4 = 0.8\,\text{V}$. Input 0: $0.35\,\text{V}$. It does not invert; it is a weak, non-inverting buffer.
5. NOR3: 6. AND2: 6 (a NAND2 plus an inverter).
6. Output is 0 when $(A + B)C = 1$. Pull-down: NMOS A parallel with NMOS B, that pair in series with NMOS C. Pull-up (dual): PMOS A in series with PMOS B, that pair in parallel with PMOS C.
7. NOR puts the slow PMOS transistors in series, making the pull-up slow or requiring very wide PMOS devices. NAND puts the fast NMOS in series and the PMOS in parallel.
8. Each NMOS width 1 (parallel), each PMOS width 4 (two in series, each needs twice the reference PMOS width of 2).
9. Its NMOS passes a strong 0 and its PMOS passes a strong 1; with both on in parallel, whichever level is being passed has a strong path.
10. Advantage: fewer transistors and faster. Disadvantage: the output is held only by charge, so it is sensitive to leakage, noise and charge sharing; it also needs a clock and uses more power.

11. $NM_H = 1.0 - 0.6 = 0.4\,\text{V}$. $NM_L = 0.4 - 0.05 = 0.35\,\text{V}$.
12. False. A single static CMOS stage is always inverting; AND needs NAND plus an inverter.

Chapter 9

Combinational Building Blocks

A **combinational** circuit is one whose output depends only on its *current* inputs, with no memory of the past. Gates are combinational; so are the larger blocks built from them: multiplexers, decoders, adders, comparators. This chapter covers those blocks, with special attention to the multiplexer, because "build gate X out of 2:1 multiplexers" is one of the most frequently reported assessment and interview questions. It ends with timing: how long a combinational circuit takes to settle, and the glitches it can produce on the way.

9.1 Universal gates

A gate type is **universal** if any Boolean function can be built using only that gate. NAND is universal, and so is NOR. With NAND alone (Figure 9.1):

- **NOT**: tie both inputs together. $\overline{A \cdot A} = \overline{A}$. (1 gate)
- **AND**: a NAND followed by a NAND-inverter. (2 gates)
- **OR**: invert each input, then NAND them: $\overline{\overline{A} \cdot \overline{B}} = A + B$ by De Morgan. (3 gates)
- **XOR**: the classic 4-NAND circuit: let $M = \overline{AB}$; then $A \oplus B = \overline{\overline{AM} \cdot \overline{BM}}$. (4 gates)

Because NOT, AND and OR can express any function (any truth table can be written as a sum of products), a gate that can build all three is universal.

9.2 The multiplexer

A **multiplexer** (mux) is a selector switch. A 2:1 mux has two data inputs I_0 and I_1, one select input S, and one output Y. When $S = 0$, the output copies I_0; when $S = 1$, it copies I_1:

$$\boxed{Y = \overline{S}\, I_0 + S\, I_1} \qquad \text{(in C notation: } \texttt{Y = S ? I1 : I0}\text{)}$$

Bigger muxes choose among more inputs: a 4:1 mux has 2 select bits, an 8:1 mux has 3, and in general an N:1 mux needs $\log_2 N$ select bits. Muxes are everywhere in hardware: choosing which register feeds an adder, which input a pipeline stage takes, which data a cache returns.

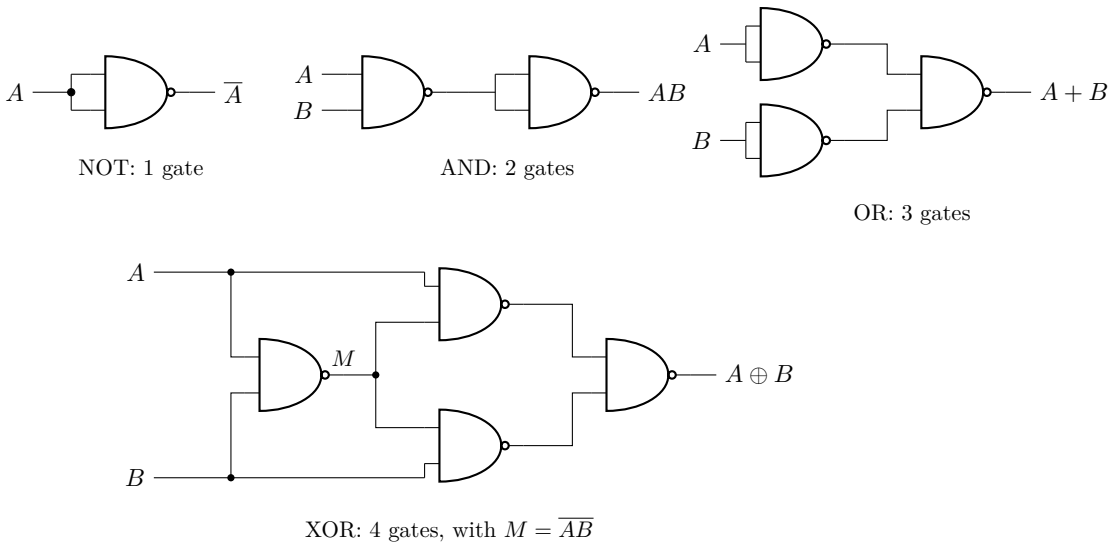
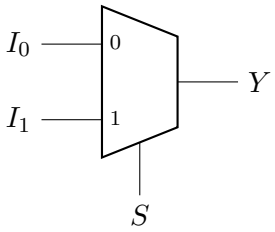


Figure 9.1: NOT, AND, OR and XOR built from NAND gates alone, so NAND is universal.



S	Y
0	I_0
1	I_1

Figure 9.2: A 2:1 multiplexer. The input marked 0 is chosen when $S = 0$.

9.3 Building any gate from a 2:1 multiplexer

Here is the method that answers every "build X from muxes" question. Any function F can be split on any one of its variables, say S:

$$F = \overline{S} \cdot F\big|_{S=0} \; + \; S \cdot F\big|_{S=1}$$

This is **Shannon expansion**. Compare it with the mux equation $Y = \overline{S}\, I_0 + S\, I_1$: they have the same shape. So:

The big idea

To build F from a 2:1 mux, pick one variable for the select line. Feed I_0 with what F equals when that variable is 0, and feed I_1 with what F equals when it is 1.

Worked example: an OR gate from one 2:1 mux

Build $F = A + B$. Choose B as the select.

- When $B = 0$: $F = A + 0 = A$. So $I_0 = A$.
- When $B = 1$: $F = A + 1 = 1$. So $I_1 = 1$.

Result: $S = B$, $I_0 = A$, $I_1 = 1$. Check: $Y = \overline{B}A + B \cdot 1 = \overline{B}A + B = A + B$ (by the rule $X + \overline{X}Y = X + Y$).
Choosing A as the select instead gives $S = A$, $I_0 = B$, $I_1 = 1$, which is equally correct, because OR treats its two inputs the same way.

Seen on the assessment

The question "build an OR gate from a 2:1 mux" was on the Ignite hardware assessment, with the two answers in the worked example above (both are valid). Interviews have also asked for AND and OR from 2:1 muxes, and for XOR from 2:1 muxes, covered next.

Applying the method to the other gates (assuming the constants 0 and 1 are available):

Function	S	I_0	I_1	2:1 muxes needed
Buffer (A)	A	0	1	1
NOT ($\overline{A}$)	A	1	0	1
AND (AB)	A	0	B	1
OR ($A + B$)	A	B	1	1
NAND ($\overline{AB}$)	A	1	$\overline{B}$	2 (one to make $\overline{B}$)
NOR ($\overline{A + B}$)	A	$\overline{B}$	0	2
XOR ($A \oplus B$)	A	B	$\overline{B}$	2
XNOR	A	$\overline{B}$	B	2

Worked example: XOR from 2:1 multiplexers

$A \oplus B$ split on A: when $A = 0$ it equals B; when $A = 1$ it equals $\overline{B}$. So the main mux has $S = A$, $I_0 = B$, $I_1 = \overline{B}$. If $\overline{B}$ is not available as an input, make it with a second mux wired as an inverter ($S = B$, $I_0 = 1$, $I_1 = 0$). Total: two 2:1 muxes (Figure 9.3).

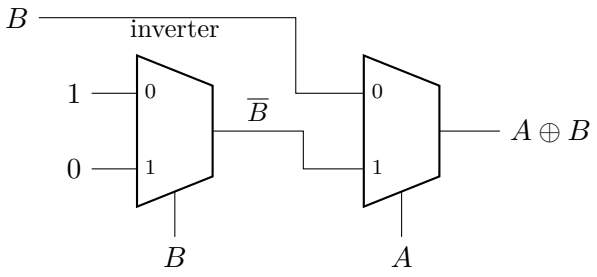


Figure 9.3: XOR from two 2:1 muxes: the first makes $\overline{B}$, the second selects B or $\overline{B}$ based on A.

Larger muxes from smaller ones. A 4:1 mux is three 2:1 muxes: two pick within pairs using the low select bit, and a third picks between their outputs using the high select bit. An 8:1 mux takes seven 2:1 muxes. In general, an N:1 mux takes $N - 1$ two-input muxes arranged as a tree.

Any function from a bigger mux. A 2^n:1 mux with the n inputs on its selects can implement *any* n-input function: just put the function's truth-table output values (0s and 1s) on the data inputs. That is exactly a **lookup table** (LUT), the basic element of an FPGA. With one more trick, a 4:1 mux implements any 3-input function: put two inputs on the selects and feed each data input with 0, 1, C or $\overline{C}$.

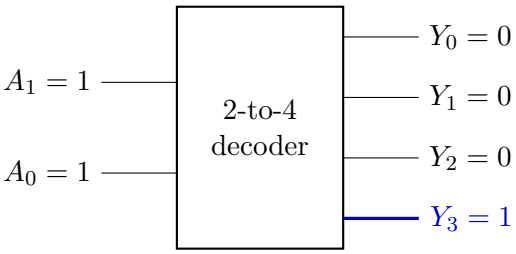
Worked example: majority function from a 4:1 mux

The 3-input majority function is 1 when at least two inputs are 1. Use A and B as selects: $AB = 00$: majority needs both others, impossible, so feed 0. $AB = 01$ or 10: majority iff $C = 1$, so feed C. $AB = 11$: already a majority, so feed 1.

Muxes can also build *memory*: a mux whose output is fed back to one of its inputs is a latch. That leads to building flip-flops from muxes in Chapter 10.

9.4 Decoders, encoders and demultiplexers

A **decoder** takes an n-bit binary number and raises exactly one of its 2^n outputs: a 2-to-4 decoder with input 11 raises output 3. Decoders select memory rows: a memory with 1024 rows uses a 10-to-1024 decoder on its address bits (Chapter 18).



A_1	A_0	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Figure 9.4: A 2-to-4 decoder raises exactly one output: the one whose number is on the inputs. Input 11 raises Y_3.

An **encoder** does the reverse: given which one of 2^n inputs is active, it outputs that input's number. A **priority encoder** handles several active inputs by reporting the highest-priority one, which is useful for interrupt controllers and arbiters.

A **demultiplexer** routes one input to one of several outputs, chosen by select bits: the opposite of a mux.

9.5 Adding binary numbers

Half adder. Adding two bits gives a sum bit and a carry bit:

$$S = A \oplus B, \qquad C = AB.$$

Full adder. To add multi-bit numbers, each position must also add the carry from the position to its right. A **full adder** adds three bits, A, B and carry-in C_{in}:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + C_{in}(A \oplus B).$$

C_{out} is the **majority** function: it is 1 when at least two of the three inputs are 1. The sum is 1 when an odd number of inputs are 1.

Ripple-carry adder. Chain n full adders, each passing its carry to the next (Figure 9.5). It is simple and small, but slow: in the worst case (for example, adding 1 to $0111\ldots1$) a carry must ripple through every stage, so delay grows in proportion to n.

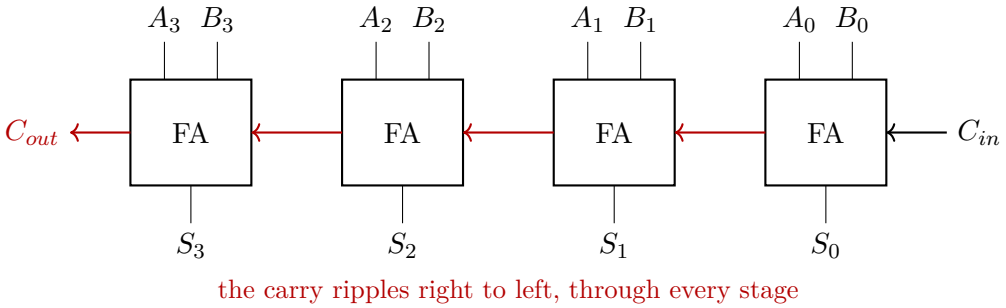


Figure 9.5: A 4-bit ripple-carry adder: four full adders (FA), each passing its carry to the next. In the worst case the carry travels through every stage.

Faster adders. Each position can be described by whether it **generates** a carry ($G = AB$: both bits are 1) or **propagates** an incoming carry ($P = A \oplus B$: exactly one bit is 1). A **carry-lookahead** adder computes the carries for many positions at once from G and P instead of waiting for the ripple. Tree-shaped **prefix adders** (such as Kogge–Stone) bring the delay down to grow with $\log_2 n$ instead of n, at the cost of more area and wiring. A **carry-select** adder computes the upper half twice, once assuming carry-in 0 and once assuming 1, then uses a mux to pick the right one when the real carry arrives.

Subtraction. Using two's complement (Chapter 7), $A - B = A + \overline{B} + 1$. An adder becomes a subtractor by inverting B (XOR gates controlled by a "subtract" signal act as optional inverters) and setting the carry-in to 1.

Comparators. Two numbers are equal when every bit pair is equal: XNOR each pair, then AND all the results. To find which is larger, subtract and look at the sign or the carry.

Shifters and multipliers. A **barrel shifter** shifts by any amount in one step using layers of muxes (shift by 1, 2, 4, 8, …, each layer enabled or not). A **multiplier** adds up shifted copies of one operand (one for each 1 bit in the other); fast multipliers add the partial products with trees of adders. Multipliers are among the largest arithmetic blocks, and a GPU contains thousands of them.

9.6 XOR chains and parity

Because XOR with 1 inverts and XOR with 0 passes, a chain of XOR gates is easy to analyze if you track what each stage does.

Worked example: a chain of 50 XOR gates

Gate 1 has inputs 1 and X. Every later gate k has inputs (output of gate $k-1$) and X. What does each gate output?

- Gate 1: $1 \oplus X = \overline{X}$.
- Gate 2: $\overline{X} \oplus X = 1$.
- Gate 3: $1 \oplus X = \overline{X}$.

The pattern repeats: odd-numbered gates output $\overline{X}$ and even-numbered gates output 1. Gate 50 outputs 1. Gate 49 outputs $\overline{X}$, so gate 49 followed by an inverter gives X.

Seen on the assessment

A reported question gave a chain of 50 XOR gates with input X and asked for the output of the Nth gate and the inverted output of the $(N+1)$th gate. The exact wiring may differ from this example, but the method is the same: compute the first two or three stages, spot the period-2 pattern, and use whether N is odd or even.

The XOR of many bits is their **parity**. Parity bits detect single-bit errors in memory and communication. A parity tree of XOR gates computes the parity of n bits in about $\log_2 n$ gate delays.

9.7 Delay through combinational logic

Gates are not instant: each takes time to charge its output capacitance (Chapter 2). Two numbers describe a gate's timing:

- **Propagation delay** t_{pd}: the *longest* time from an input change until the output has settled to its final value.
- **Contamination delay** t_{cd}: the *shortest* time from an input change until the output *starts* to change.

A gate may have different delays for a rising output and a falling output (t_{pLH} and t_{pHL}), because the PMOS pull-up and NMOS pull-down have different strengths. The delay of a whole circuit is found by adding delays along each path from input to output. The slowest path is the **critical path**; it sets the circuit's propagation delay. The fastest path sets its contamination delay. Both matter for timing (Chapter 11).

Worked example: how long does the output stay at 0?

$Z = A \cdot \overline{B}$, where $\overline{B}$ comes from an inverter. Delays: inverter rise 2 ns and fall 1 ns; AND gate rise 3 ns and fall 2 ns. Initially $A = 0$, $B = 1$, so $\overline{B} = 0$ and $Z = 0$. At $t = 0$ both inputs flip: A becomes 1 and B becomes 0. When does Z become 1?
A is high from $t = 0$. The inverter output rises (0 to 1), which takes its *rise* delay: $\overline{B}$ is high at $t = 2$ ns. The AND gate's inputs are both high only from $t = 2$, and its output rises after its rise delay: Z becomes 1 at $t = 2 + 3 = 5$ ns. So Z stays at 0 for 5 ns.

Seen on the assessment

Reported questions give a small gate circuit with rise and fall delays, flip the inputs at $t = 0$, and ask how long the output stays at a value. The method: walk forward from the inputs, at each gate note which input arrives last for the transition that matters, and add the gate's rise or fall delay according to the direction its output moves.

Glitches and hazards

Unequal path delays can produce a brief wrong pulse, a **glitch**, even when the output should not change at all (Figure 9.6).

Worked example: a glitch

$Z = A + \overline{A}$ should always be 1. Build $\overline{A}$ with an inverter of delay 1 ns, and suppose A falls from 1 to 0 at $t = 0$. From $t = 0$ to $t = 1$ ns, A is already 0 but $\overline{A}$ has not yet risen. Both OR inputs are 0, so after the OR gate's delay, Z dips to 0 for about 1 ns before returning to 1.

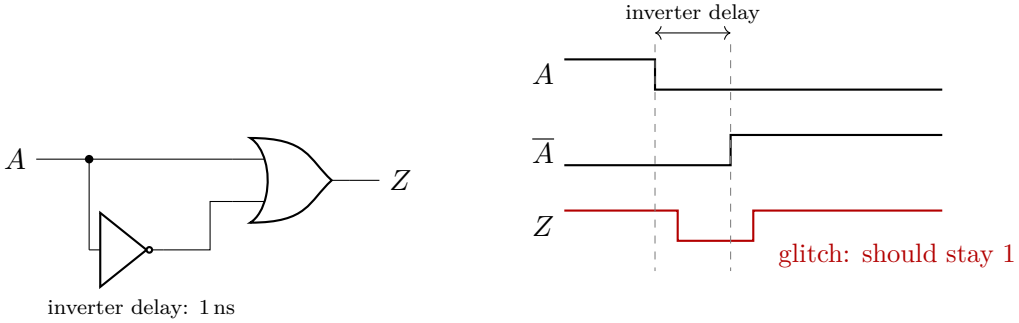


Figure 9.6: A static-1 hazard. $Z = A + \overline{A}$ should always be 1, but when A falls, $\overline{A}$ rises only after the inverter's delay. For that moment both OR inputs are 0, and Z glitches low.

This is a **static-1 hazard**: an output that should stay 1 briefly drops to 0. In a K-map it appears when two groups of 1s touch but do not overlap; adding the redundant *consensus* term that covers the boundary removes the hazard. In synchronous designs (Chapter 11), glitches are usually harmless as long as everything settles before the next clock edge, but they waste power, and a glitch on a clock or an asynchronous reset is a serious bug.

9.8 Summary

- NAND and NOR are each universal. OR from NANDs takes 3 gates; XOR takes 4.
- A 2:1 mux computes $Y = \overline{S}I_0 + SI_1$. Shannon expansion builds any function from muxes: select on a variable, feed each data input with the function's value for that select value.
- OR: $S = B$, $I_0 = A$, $I_1 = 1$. AND: $S = A$, $I_0 = 0$, $I_1 = B$. NOT: $S = A$, $I_0 = 1$, $I_1 = 0$. XOR: $S = A$, $I_0 = B$, $I_1 = \overline{B}$ (two muxes).
- N:1 mux $= N - 1$ two-input muxes. A 2^n:1 mux is a lookup table for any n-input function.

- Full adder: $S = A \oplus B \oplus C_{in}$, C_{out} = majority. Ripple carry is $O(n)$; lookahead and prefix adders are $O(\log n)$. Subtract with $A + \overline{B} + 1$.
- XOR chains alternate; track the pattern and the parity of N.
- Propagation delay is the slowest path; contamination delay the fastest. Unequal paths cause glitches.

9.9 Exercises

1. Give the S, I_0 and I_1 connections that make a 2:1 mux act as NOT, AND and OR.
2. How many 2:1 muxes are needed for XOR if $\overline{B}$ is not available? For NAND?
3. How many 2:1 muxes make a 4:1 mux? An 8:1 mux?
4. Implement the 3-input majority function with a 4:1 mux using A and B as selects.
5. A 32-bit ripple-carry adder's full adders each have a 50 ps carry delay. Estimate the worst-case carry delay.
6. Compute $5 - 3$ in 4-bit two's complement using addition.
7. In the 50-gate XOR chain of Section 9.6, what is the output of gate 37? Of gate 37 followed by an inverter?
8. $Z = A + \overline{A}$, inverter delay 1 ns. Does Z glitch when A *rises* from 0 to 1? Explain.
9. In the worked example $Z = A\overline{B}$ (inverter rise 2, fall 1; AND rise 3, fall 2), the inputs now flip back: at $t = 0$, A falls to 0 and B rises to 1. When does Z fall to 0?
10. How many outputs does a 3-to-8 decoder have? What size decoder selects one of 1024 memory rows?

Answers

1. NOT: $S = A$, $I_0 = 1$, $I_1 = 0$. AND: $S = A$, $I_0 = 0$, $I_1 = B$. OR: $S = A$, $I_0 = B$, $I_1 = 1$ (or $S = B$, $I_0 = A$, $I_1 = 1$).
2. XOR: 2 (one makes $\overline{B}$). NAND: 2 ($S = A$, $I_0 = 1$, $I_1 = \overline{B}$, with a second mux making $\overline{B}$).
3. 3; 7.
4. Data inputs for $AB = 00, 01, 10, 11$: $0, C, C, 1$.
5. $32 \times 50 = 1600$ ps $= 1.6$ ns.
6. $0101 + \overline{0011} + 1 = 0101 + 1100 + 1 = 1\,0010$. Drop the carry out: $0010 = 2$.
7. Gate 37 is odd, so it outputs $\overline{X}$; inverted, X.
8. No. When A rises, the OR gate sees $A = 1$ immediately, which already makes $Z = 1$; $\overline{A}$ falling later does not matter. The glitch happens only when A falls.
9. Z falls as soon as either AND input falls. A falls at $t = 0$; $\overline{B}$ falls at $t = 1$ ns (inverter fall delay). The earliest falling input is A at $t = 0$, so Z falls after the AND gate's fall delay: $t = 2$ ns.

10. 8 outputs. A 10-to-1024 decoder.

Part IV

Memory and Time

Chapter 10

Latches and Flip-Flops

Combinational circuits forget: the moment their inputs change, their old outputs are gone. But a computer must remember things: the value in a register, the count in a counter, which step a machine is on. This chapter builds memory out of gates. It explains latches and flip-flops, the difference between them (an assessment question), how to build a flip-flop from multiplexers (an interview question), and the strange behavior called metastability.

10.1 Memory from feedback

Connect two inverters in a loop: the output of the first drives the second, and the output of the second drives the first (Figure 10.1).

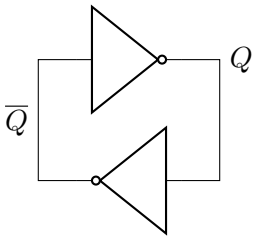


Figure 10.1: Two cross-coupled inverters: a circuit with two stable states.

If $Q = 1$, the lower inverter makes $\overline{Q} = 0$, and the upper inverter turns that back into $Q = 1$. The loop holds itself. If $Q = 0$, the same reasoning shows the loop holds that too. The circuit is **bistable**: it has two stable states and stays in whichever one it is put in, as long as power is on. That is one bit of memory.

An everyday picture

A light switch is bistable: push it up and it stays up, push it down and it stays down. A doorbell button is not: it only stays pressed while you hold it. Memory circuits are the light-switch kind.

The remaining problem is how to *write* a new value into the loop. Every latch and flip-flop is a bistable loop plus some way to control when it accepts new data.

10.2 The SR latch

Replace the two inverters with two NOR gates, giving each a spare input: S (set) and R (reset). This is the **SR latch** (Figure 10.2).

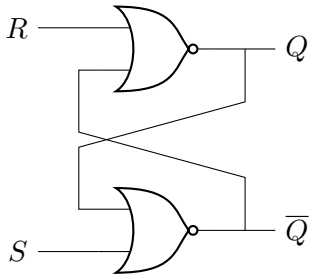


Figure 10.2: An SR latch made of two cross-coupled NOR gates.

Recall that a NOR outputs 0 whenever any input is 1.

S	R	Result
0	0	hold: Q keeps its previous value
1	0	set: $Q = 1$
0	1	reset: $Q = 0$
1	1	forbidden: both outputs forced to 0

$S = R = 1$ is forbidden for two reasons: Q and $\overline{Q}$ are both 0, so they are no longer opposites; and if S and R then drop to 0 at the same moment, the latch races to an unpredictable state. An SR latch built from NAND gates works the same way with active-low inputs ($\overline{S}$ and $\overline{R}$, where 0 means "do it").

10.3 The D latch

The SR latch has two awkward inputs and a forbidden case. The **D latch** fixes both. It has a data input D and an enable EN (often the clock):

- While $EN = 1$, the latch is **transparent**: Q follows D, changing whenever D changes.
- While $EN = 0$, the latch is **opaque**: Q holds whatever value D had at the moment EN fell.

The simplest way to build one is a 2:1 mux whose output feeds back to one of its own inputs:

$$Q = EN \; ? \; D : Q.$$

When $EN = 1$ the mux passes D; when $EN = 0$ it passes its own output back to itself, a loop that holds the value (Figure 10.3). In transistor-level designs, the mux is made of two transmission gates, one letting D in and one closing the feedback loop of two inverters.

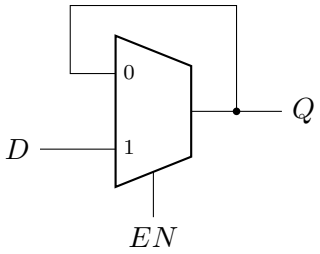


Figure 10.3: A D latch from a 2:1 mux. The 1 input takes new data; the 0 input feeds the output back to hold it.

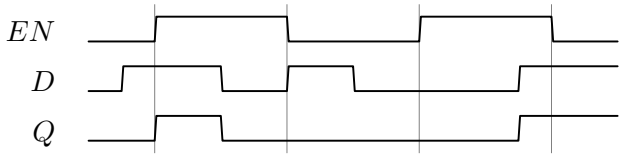


Figure 10.4: A D latch: while EN is high, Q follows D; while EN is low, Q holds (the pulse on D during the low phase never reaches Q).

10.4 The edge-triggered D flip-flop

A latch is transparent for the whole time its enable is high, which makes timing hard to control (we will see why shortly). Usually we want a storage element that captures its input at one precise instant, the moment the clock rises, and ignores the input at all other times. That is the **D flip-flop**.

Master–slave construction. Put two D latches in a row with opposite enables (Figure 10.5):

- the **master** latch is transparent while $CLK = 0$;
- the **slave** latch is transparent while $CLK = 1$.

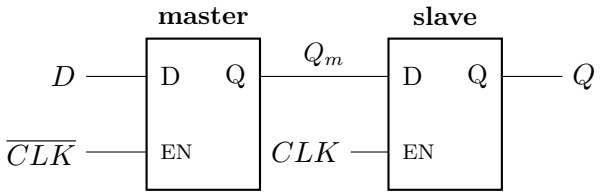


Figure 10.5: A positive-edge-triggered D flip-flop: master latch enabled by $\overline{CLK}$, slave latch enabled by CLK.

Step through one clock cycle:

1. **While $CLK = 0$:** the master is transparent, so Q_m follows D. The slave is opaque, so Q holds its old value.
2. **At the rising edge:** the master closes, freezing Q_m at the value D had at that instant. At the same moment the slave opens and passes that frozen value to Q.

3. **While $CLK = 1$:** the master is closed, so changes on D cannot get in. The slave is transparent, but its input Q_m is frozen, so Q stays put.

The net effect: Q takes the value of D at each rising clock edge and holds it for the rest of the cycle. At no time is there a transparent path all the way from D to Q.

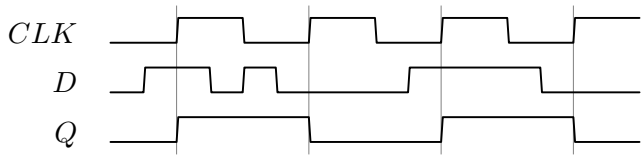


Figure 10.6: A D flip-flop: Q changes only at rising clock edges (gridlines), taking the value D had at that instant. The brief dip in D between edges is ignored.

10.5 A D flip-flop from two 2:1 multiplexers

Since a D latch is one mux with feedback, a flip-flop is two of them:

- **Master mux:** $S = CLK$, $I_0 = D$, $I_1 = Q_m$ (its own output fed back). It is transparent when $CLK = 0$ and holds when $CLK = 1$.
- **Slave mux:** $S = CLK$, $I_0 = Q$ (its own output fed back), $I_1 = Q_m$. It holds when $CLK = 0$ and is transparent when $CLK = 1$.

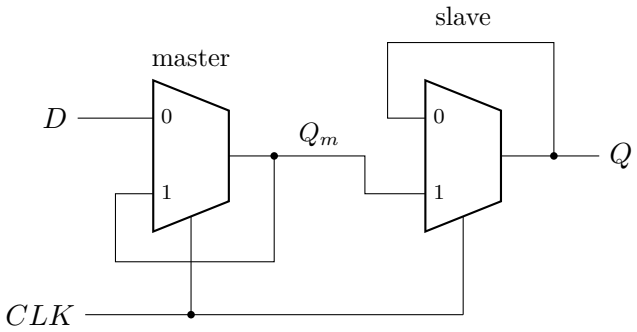


Figure 10.7: A positive-edge D flip-flop from two 2:1 muxes. Both selects are driven by CLK; the feedback goes to opposite data inputs, so the two latches are transparent in opposite clock phases.

Seen on the assessment

Interview reports include "make a D flip-flop using 2:1 muxes." The answer is Figure 10.7: two mux-latches in series, enabled in opposite clock phases. (Moving the feedback to the other input of each mux gives a negative-edge flip-flop.) In real silicon, care is needed so that both latches are never transparent at the same time during the clock transition, but the logical structure is exactly this.

10.6 Latch versus flip-flop

	Latch	Flip-flop
Triggered by	clock **level**	clock **edge**
While the clock is active	transparent: output follows input	opaque: input only sampled at the edge
Size	one latch (smaller)	two latches (about twice the size)
Power	lower	higher (more transistors, more clock load)
Glitches on the input	passed through while transparent	ignored except near the edge
Timing	harder: data can race through; but allows time borrowing	simpler: one capture per cycle

Seen on the assessment

"What is one advantage of a flip-flop over a latch?" was an Ignite assessment question. The key answer: **a flip-flop is edge-triggered, so it samples its input at one instant per cycle and is never transparent.** Data cannot "race through" several stages in one clock phase, glitches on the input are ignored, and timing analysis is simple and predictable. Advantages of a latch in return: it is smaller, uses less power, and allows *time borrowing* (a slow stage can use some of the next stage's time while the latch is transparent, Chapter 11).

Why race-through is a problem. Imagine a pipeline of latches all enabled by the same clock. While the clock is high, every latch is transparent at once, and data entering the first latch can rush through the logic and into the second, third and fourth before the clock falls. The circuit no longer advances one step per cycle; it advances an unpredictable number of steps depending on gate delays. Flip-flops cannot do this, because no flip-flop is ever transparent from input to output. (Latch-based designs avoid the problem by alternating latches on opposite clock phases, which is exactly what a master–slave flip-flop does internally.)

10.7 Other kinds of flip-flop

Older textbooks and some interview questions use other flip-flop types. Each can be built from a D flip-flop plus a little logic.

Type	Behavior at the clock edge	Next state Q^+
D	copy D	D
T (toggle)	flip if $T = 1$, hold if $T = 0$	$T \oplus Q$
JK	J sets, K resets, both 1 toggles, both 0 holds	$J\overline{Q} + \overline{K}Q$
SR	S sets, R resets, both 1 not allowed	$S + \overline{R}Q$

- A **T flip-flop** is a D flip-flop with $D = T \oplus Q$.
- A **T flip-flop with** $T = 1$, or a D flip-flop with $\overline{Q}$ fed back to D, toggles on every clock edge. Its output is a square wave at *half* the clock frequency: a divide-by-two circuit (Chapter 12).

10.8 Set, reset and enable

Asynchronous set and clear. Many flip-flops have extra inputs that force the output immediately, without waiting for a clock edge: **preset** (or set) forces $Q = 1$, and **clear** (or reset) forces $Q = 0$. These are often **active low**, written $\overline{PRE}$ and $\overline{CLR}$: they act when held at 0.

Common trap

"Set and clear tied to zero" means opposite things depending on the polarity. For *active-high* set and clear, tying them to 0 disables them, and the flip-flop works normally. For *active-low* inputs, tying them to 0 asserts them, and the output is stuck (and with both asserted, the behavior is invalid). Always check whether the input has a bubble on the symbol (Figure 10.8).

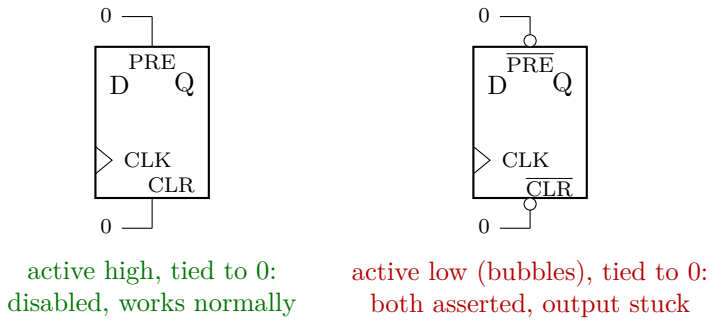


Figure 10.8: The same wiring, opposite results. Tying set and clear to 0 turns them off when they are active high (left), but turns them both on when they are active low (right), shown by the bubbles.

Synchronous versus asynchronous reset. An **asynchronous** reset acts immediately, even with no clock running. A **synchronous** reset is just a condition in the logic in front of D, so it takes effect only at the next clock edge. Both are common; Chapter 19 discusses the trade-offs.

Enable. To make a flip-flop update only when an enable signal is 1, put a mux in front of it: $D_{\text{ff}} = EN \ ? \ D : Q$. When disabled, the flip-flop reloads its own value each cycle. (An alternative is to stop its clock, called clock gating, which saves power, Chapter 14.)

Scan flip-flops. Chips are tested after manufacturing by loading known patterns into every flip-flop. A **scan flip-flop** has a mux in front that, in test mode, connects all flip-flops into one long shift register (a **scan chain**) so test patterns can be shifted in and results shifted out.

10.9 Metastability

A flip-flop needs its input to be stable for a short window around the clock edge (the setup and hold times, Chapter 11). What if the input changes inside that window? The bistable loop can be caught balanced exactly between its two states, like a ball balanced on top of a hill. Eventually noise tips it one way or the other, but *how long* that takes is unpredictable. While it hangs there, the output sits at an in-between voltage that is neither a valid 0 nor a valid 1. This is **metastability**.

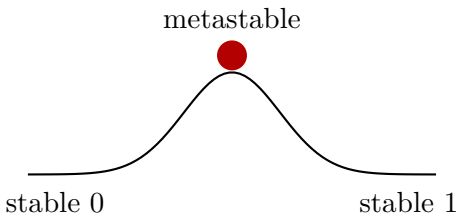


Figure 10.9: Metastability: a ball balanced on a hill eventually rolls to one side, but nobody can say exactly when.

Key facts:

- Metastability happens whenever the input is **asynchronous**: it comes from outside the flip-flop's clock domain (a button, another chip, a different clock), so it can change at any time.
- It *cannot be eliminated*, only made very unlikely.
- The probability that the flip-flop is still unresolved after waiting time t falls *exponentially* with t. So the mean time between failures (MTBF) grows exponentially with the time you allow it to settle.
- The standard defense is a **synchronizer**: two flip-flops in a row on the receiving clock. The first may go metastable, but it has a whole clock cycle to resolve before the second samples it. This reduces the failure rate to something like once in thousands of years (Chapter 19).

10.10 Summary

- Memory comes from feedback: two cross-coupled inverters are bistable.
- SR latch: set, reset, hold; $S = R = 1$ forbidden.
- D latch: transparent while enabled, holds while disabled. A 2:1 mux with feedback is a D latch.
- D flip-flop: master latch (transparent on $CLK = 0$) plus slave latch (transparent on $CLK = 1$) captures D at the rising edge. Two muxes make one.
- Flip-flop advantages: edge-triggered, never transparent, no race-through, glitches ignored, easy timing. Latch advantages: smaller, lower power, time borrowing.
- T flip-flop: $Q^+ = T \oplus Q$; with $T = 1$ it divides the clock by 2. JK: $Q^+ = J\overline{Q} + \overline{K}Q$.
- Check set/clear polarity before deciding what "tied to 0" means.
- Metastability cannot be eliminated; synchronizers make it exponentially unlikely.

10.11 Exercises

1. A NOR SR latch has $S = 1$, $R = 0$. Then S returns to 0. What is Q now?
2. Why is $S = R = 1$ not allowed in a NOR SR latch?
3. A D latch has $EN = 1$ and D toggles three times. How many times does Q change? What if $EN = 0$?
4. Draw (or describe) a D latch made from one 2:1 mux. Which input gets the feedback?
5. Describe the connections of a positive-edge D flip-flop built from two 2:1 muxes.
6. State one advantage of a flip-flop over a latch, and one advantage of a latch over a flip-flop.
7. How do you make a T flip-flop from a D flip-flop?
8. A JK flip-flop has $J = K = 1$. What does it do on each clock edge?
9. A flip-flop has active-low asynchronous clear, and the clear input is tied to 0. What is Q?
10. True or false: with careful design, metastability can be completely eliminated.
11. A D flip-flop is clocked at 100 MHz with $\overline{Q}$ connected to D. What is the frequency of Q?

Answers

1. $Q = 1$: the latch was set, and with $S = R = 0$ it holds.
2. It forces both Q and $\overline{Q}$ to 0 (no longer complements), and if both inputs then return to 0 together, the final state depends on a race and is unpredictable.
3. Three times (it is transparent; Q follows each change of D). With $EN = 0$, zero times.
4. The mux's select is EN; D goes to the 1 input; the output is fed back to the 0 input.
5. Master mux: select CLK, $I_0 = D$, $I_1 =$ its own output Q_m. Slave mux: select CLK, $I_1 = Q_m$, $I_0 =$ its own output Q.
6. Flip-flop: edge-triggered, never transparent, so no race-through and simpler timing (also ignores glitches). Latch: smaller and lower power, and allows time borrowing.
7. Feed $D = T \oplus Q$.
8. It toggles ($Q^+ = \overline{Q}$).
9. The clear is permanently asserted, so Q is stuck at 0.
10. False. Its probability can only be made extremely small.
11. 50 MHz (the output toggles once per clock period, so its period is two clock periods).

Chapter 11

Timing: Setup, Hold and Clock Speed

How fast can a chip run? What limits it? Why can one mistake be fixed by slowing the clock while another makes the chip useless at any speed? This chapter answers those questions. Timing is the single most important topic in digital hardware interviews, and it is the foundation for the clock divider, pipeline and FIFO questions later in the book.

11.1 Synchronous design

Almost every digital chip follows one pattern, called **synchronous design**: banks of flip-flops (registers) hold the current state, blocks of combinational logic compute the next state from it, and one shared clock tells every flip-flop to capture its new value at the same moment (Figure 11.1).

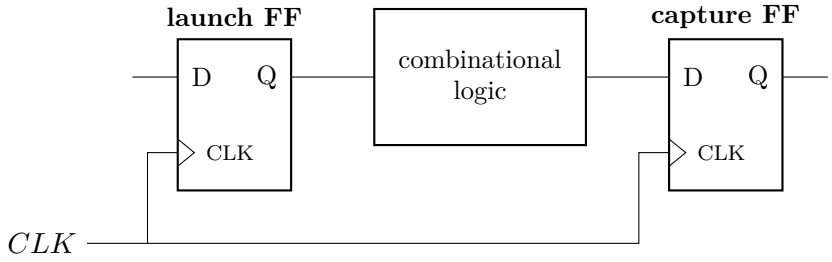


Figure 11.1: The basic timing path: a launch flip-flop, logic, and a capture flip-flop on the same clock.

At each rising clock edge, the launch flip-flop sends out a new value. It ripples through the logic. At the *next* rising edge, the capture flip-flop must grab the result. The whole chip advances one step per clock cycle, and as long as every path obeys two timing rules, gate delays never cause errors. Those two rules are the subject of this chapter.

11.2 The clock

A clock is a square wave. Its **period** T is the time between rising edges; its **frequency** is $f = 1/T$. A 2 GHz clock has a 500 ps period. The **duty cycle** is the fraction of the period the clock spends high, normally 50%.

11.3 Timing parameters

Every flip-flop and logic block has a few timing numbers:

- **Setup time** t_{su}: how long *before* the clock edge the input D must be stable.
- **Hold time** t_h: how long *after* the clock edge D must stay stable.
- **Clock-to-Q delay** t_{cq}: how long after the clock edge the output Q changes.
- For the logic: the **propagation delay** t_{pd} (slowest path, time until the output is final) and the **contamination delay** t_{cd} (fastest path, time until the output first starts changing), as in Chapter 9.

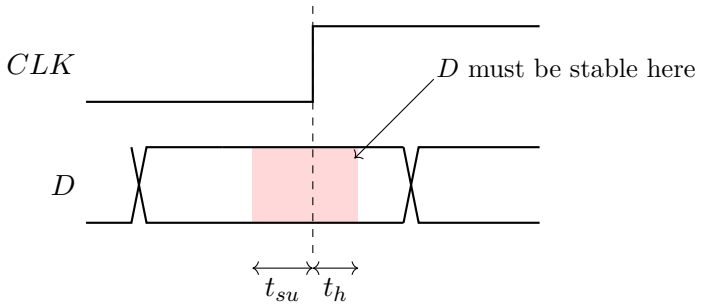


Figure 11.2: D must not change inside the shaded window around the clock edge: from t_{su} before it to t_h after it.

If D changes inside that window (Figure 11.2), the flip-flop may capture the wrong value or go metastable (Chapter 10).

11.4 The setup constraint: how fast can the clock go?

Follow the data launched at one clock edge (time 0) to the capture flip-flop.

1. The launch flip-flop's output changes at t_{cq}.
2. The change ripples through the logic and settles by $t_{cq} + t_{pd}$.
3. The capture flip-flop samples at the next edge, time T, and needs the data stable t_{su} before that.

So the data must arrive in time (Figure 11.3):

$$\boxed{t_{cq} + t_{pd} + t_{su} \leq T}$$

The minimum period is the sum on the left, and the maximum frequency is its reciprocal:

$$f_{max} = \frac{1}{t_{cq} + t_{pd} + t_{su}}.$$

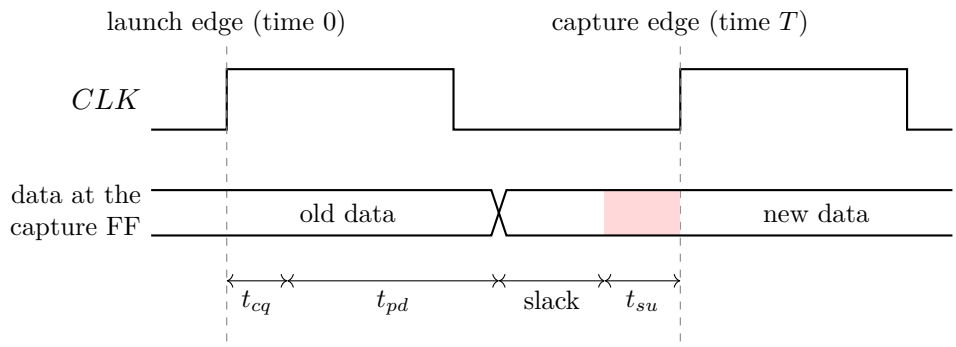


Figure 11.3: The setup check. Data launched at time 0 leaves the launch flip-flop after t_{cq} and settles t_{pd} later. It must be stable for t_{su} before the next edge (shaded). Any time left over is slack.

The big idea

The setup constraint is about data arriving *too late*. It is set by the slowest path (the **critical path**), and it depends on the clock period. If it fails, you can always fix it by slowing the clock down.

Worked example: maximum clock frequency

$t_{cq} = 50\,\text{ps}$, the slowest logic path $t_{pd} = 400\,\text{ps}$, $t_{su} = 30\,\text{ps}$.

$$T_{min} = 50 + 400 + 30 = 480\ \text{ps}, \qquad f_{max} = \frac{1}{480\ \text{ps}} \approx 2.08\ \text{GHz}.$$

Worked example: the logic budget

A design must run at 500 MHz ($T = 2\,\text{ns}$). Its flip-flops have $t_{cq} = 0.3\,\text{ns}$ and $t_{su} = 0.2\,\text{ns}$. What is the longest the logic may take?

$$t_{pd} \le T - t_{cq} - t_{su} = 2 - 0.3 - 0.2 = 1.5\ \text{ns}.$$

11.5 The hold constraint: data arriving too early

There is a second, sneakier danger. At the same clock edge at which the capture flip-flop samples the *old* data, the launch flip-flop is sending *new* data. If that new data races through a very short path and arrives before the capture flip-flop's hold time is over, it corrupts the value being captured.

The earliest the new data can arrive is $t_{cq} + t_{cd}$ (using the fastest clock-to-Q and the fastest logic path). It must not arrive before the hold window closes (Figure 11.4):

$$\boxed{t_{cq} + t_{cd} \ge t_h}$$

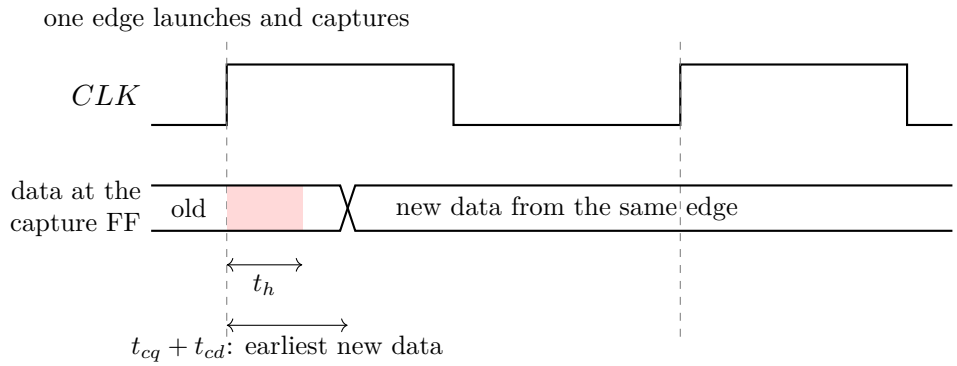


Figure 11.4: The hold check. The capture flip-flop is still holding the old value for t_h after the edge (shaded), while the same edge has already launched new data. The new data must not arrive before the hold window closes. The clock period does not appear.

The big idea

The hold constraint involves only *one* clock edge, the same edge launching and capturing, so **the clock period does not appear in it**. A hold violation cannot be fixed by slowing the clock. A chip with a hold violation fails at every frequency. It is fixed by *adding delay* (buffers) to the too-fast path.

Worked example: a hold violation

$t_{cq} = 40\,\text{ps}$, the shortest logic path is $10\,\text{ps}$, $t_h = 60\,\text{ps}$. Arrival $= 40 + 10 = 50\,\text{ps} < 60\,\text{ps}$: a hold violation of $10\,\text{ps}$. Lowering the frequency changes nothing. Add at least $10\,\text{ps}$ of delay on the short path (in practice, a little more for margin).

Common trap

"The chip fails a timing check; can we fix it by lowering the clock frequency?" Yes for setup, *no* for hold. This is one of the most common true/false questions. A related trap: adding logic delay *helps* hold and *hurts* setup.

11.6 Clock skew and jitter

So far we assumed the clock reaches every flip-flop at the same instant. In reality the clock travels through wires and buffers, and different flip-flops receive it at slightly different times (Figure 11.5). The difference is **clock skew**. Define

$$t_{skew} = (\text{clock arrival at capture FF}) - (\text{clock arrival at launch FF}).$$

Positive skew means the capture flip-flop's clock arrives later.

With skew, the constraints become:

$$\text{setup: } t_{cq} + t_{pd} + t_{su} \le T + t_{skew}, \qquad \text{hold: } t_{cq} + t_{cd} \ge t_h + t_{skew}.$$

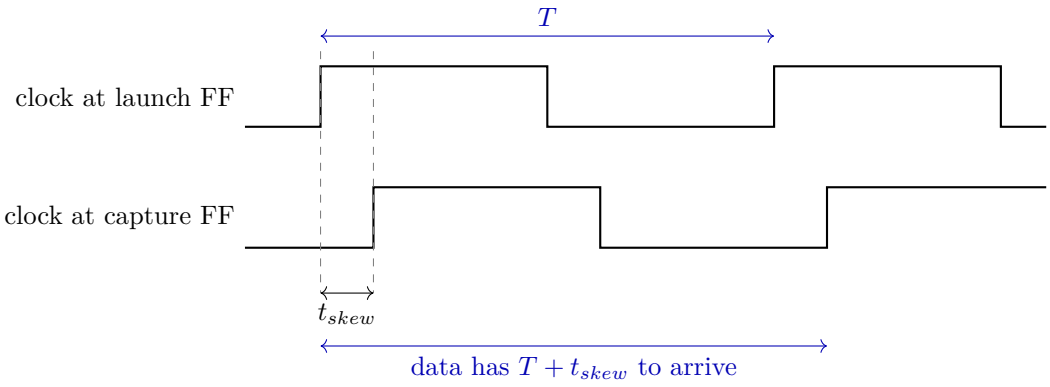


Figure 11.5: Positive clock skew: the clock reaches the capture flip-flop t_{skew} later than the launch flip-flop. Data now has $T + t_{skew}$ to arrive, which helps setup, but the capture flip-flop also holds later, which hurts hold.

Positive skew gives the data extra time to arrive before the capture edge, which **helps setup**. But it also keeps the capture flip-flop's hold window open later, which **hurts hold**. Negative skew does the opposite. Skew is a trade: whatever it gives one constraint, it takes from the other.

Jitter is random variation in the clock edge timing from cycle to cycle. Because nobody can predict it, it always eats into the margin; for setup, it effectively shortens the period.

Worked example: skew in a clock buffer

At 500 MHz, $t_{cq} = 0.3$ ns, $t_{su} = 0.2$ ns, and the capture flip-flop's clock passes through one extra buffer of 0.1 ns, so $t_{skew} = +0.1$ ns. The logic budget becomes

$$t_{pd} \le T + t_{skew} - t_{cq} - t_{su} = 2 + 0.1 - 0.3 - 0.2 = 1.6 \text{ ns}.$$

If the hold time is 0.15 ns, the shortest path must satisfy $0.3 + t_{cd} \ge 0.15 + 0.1$: true for any $t_{cd} \ge 0$, so hold is safe.

11.7 Slack

Timing tools report **slack**: how much margin a path has.

$$\text{setup slack} = (T + t_{skew}) - (t_{cq} + t_{pd} + t_{su}), \qquad \text{hold slack} = (t_{cq} + t_{cd}) - (t_h + t_{skew}).$$

Positive slack means the path passes with that much to spare; negative slack is a violation. **Static timing analysis** (STA) tools compute the slack of every path in a chip, using worst-case delays, without simulating any data.

11.8 Reading timing diagrams

Assessment questions often show a waveform and ask for the output. The method:

1. Mark every active clock edge (usually rising).

2. At each edge, read the value of D just *before* the edge (that is what is captured).
3. Draw Q changing to that value a little after the edge (t_{cq}), and holding until the next edge.
4. For a chain of flip-flops, repeat stage by stage: each stage's input is the previous stage's output, which changed only just after the edge, so each stage sees the *old* value at that edge.

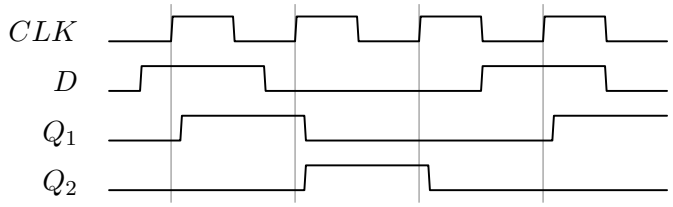


Figure 11.6: Two flip-flops in a row ($D \to Q_1 \to Q_2$). Each output changes slightly after a rising edge (gridlines). Q_2 copies Q_1 one cycle later.

11.9 Shift registers and hold time

A **shift register** is a chain of flip-flops with each Q wired directly to the next D, no logic in between (Figure 11.7). On each clock edge every bit moves one place along. Because there is no logic, $t_{cd} = 0$, and the hold check is simply $t_{cq} \geq t_h + t_{skew}$.

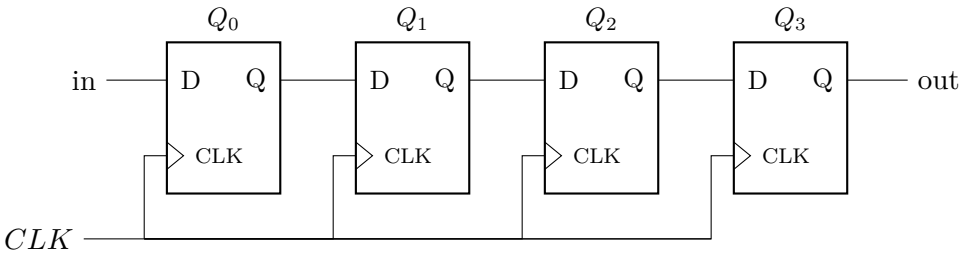


Figure 11.7: A 4-bit shift register: each Q wired straight to the next D, all on one clock. Every edge moves each bit one place to the right.

Worked example: a 4-bit shift register

Four D flip-flops form a shift register. Each has $t_{cq} = 50\,\text{ps}$ (positive) and $t_h = 0$.

- With zero skew: $50 \geq 0$. Hold is met. Each stage captures its neighbor's old value before the neighbor changes, and the register shifts exactly one place per edge.
- Now suppose the clock reaches each flip-flop 60 ps later than the one before it ($t_{skew} = +60\,\text{ps}$ along the shift direction). Hold requires $50 \geq 0 + 60$: violated. The new value from one flip-flop arrives at the next one before that one's (late) clock edge, so data can skip through two stages in one cycle and bits are lost (Figure 11.8).
- Fix: route the clock the other way (so skew is negative along the data direction), or add delay between stages.

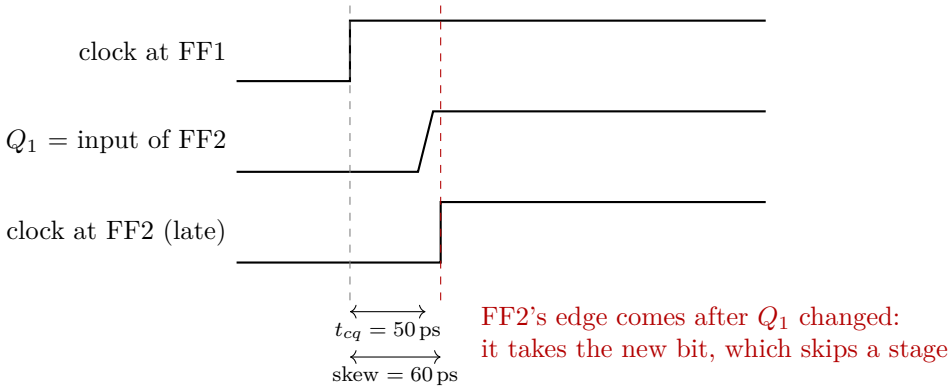


Figure 11.8: The shift-register hold failure from the worked example. The clock reaches FF2 60 ps late, but Q_1 changes only 50 ps after FF1's edge, so FF2 captures the new value instead of the old one.

Seen on the assessment

A reported question describes a 4-bit shift register of D flip-flops with positive clock-to-Q delay and zero hold time. The point being tested: with no logic between stages, correct shifting depends on t_{cq} being larger than the hold time plus any skew. With $t_h = 0$ and no skew, any positive t_{cq} works.

11.10 Pipelining

If the critical path is too slow for the clock you want, you can split the logic into stages with registers between them. This is **pipelining**.

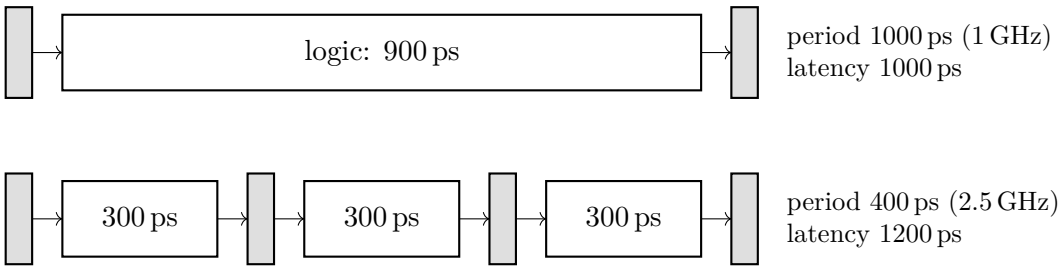


Figure 11.9: Pipelining the worked example below; the gray bars are registers. Top: one 900 ps block between two registers. Bottom: the same logic in three 300 ps stages. A new input can enter every 400 ps, but each one takes 1200 ps to come out.

Worked example: pipelining a slow block

A 900 ps block of logic sits between two registers whose overhead is $t_{cq} + t_{su} = 100$ ps. The clock period must be at least 1000 ps (1 GHz). Split the logic into three 300 ps stages with registers between: each stage needs $300 + 100 = 400$ ps, so the clock can run at 2.5 GHz.

- **Throughput** (results per second) rises from 1 to 2.5 billion: a new input enters every 400 ps.
- **Latency** (time for one input to come out) rises from 1000 ps to $3 \times 400 = 1200$ ps.

Pipelining trades latency for throughput. Its returns diminish: each extra stage adds another $t_{cq} + t_{su}$ of overhead, and stages can never be perfectly balanced. Processor pipelines (Chapter 20) are the most famous example.

11.11 Long wires and retiming flip-flops

On a large chip, just sending a signal across the die can take longer than a clock period. The fix is the same idea as pipelining: insert flip-flops along the wire so that each segment fits in one cycle. These are sometimes called **pipeline** or **retiming flops**.

Worked example: flip-flops on a long wire

A signal must travel 60 mm between two flip-flops. The wire (with repeaters) has a delay of 0.1 ns per mm. The clock period is 5 ns, and each flip-flop's overhead is $t_{cq} + t_{su} = 0.2$ ns. Assume the clock arrives everywhere at the same time.
Total wire delay: $60 \times 0.1 = 6$ ns, longer than one period. Each segment can use at most $5 - 0.2 = 4.8$ ns, which covers $4.8/0.1 = 48$ mm. Segments needed: $\lceil 60/48 \rceil = 2$. Two segments need **one** extra flip-flop in the middle. The signal now takes two cycles to arrive.

Seen on the assessment

A reported question gives two flip-flops 60 mm apart, data and clock wire delays, and a 5 ns period, and asks for the minimum number of retiming flip-flops. If the clock is routed along with the data, its wire delay creates skew: add the skew to the time each segment may use (positive skew in the data direction helps setup). Then divide the total data delay by the time per segment, round up to get segments, and subtract one to get flip-flops.

11.12 Time borrowing with latches

Latch-based designs can do something flip-flop designs cannot. Because a latch is transparent for a whole clock phase, data that arrives late at a latch is not lost; it passes through while the latch is open, and the next stage simply starts late. A slow stage can **borrow time** from a fast one that follows it. This is one reason high-performance custom designs sometimes use latches instead of flip-flops, despite their harder timing analysis.

11.13 Special paths and sign-off corners

- A **false path** is a path that exists in the wiring but can never actually carry a meaningful transition (for example, through two muxes whose selects are never set that way together). It is excluded from timing checks.

- A **multicycle path** is allowed more than one clock cycle by design (for example, a slow divider whose result is only read every other cycle).
- Timing must be met across manufacturing and operating variation (Chapter 15). **Setup** is checked at the **slow** corner (slow transistors, low voltage, worst temperature), where paths are longest. **Hold** is checked at the **fast** corner, where short paths are shortest.

11.14 Making and distributing the clock

- A **phase-locked loop** (PLL) generates a fast, stable clock from a slower reference crystal, by multiplying its frequency.
- A **clock tree** distributes the clock to millions of flip-flops through a balanced network of buffers (H-trees, meshes, spines) designed to keep skew small.
- **Clock gating** turns off the clock to idle flip-flops to save power (Chapter 14). It must be done with a special cell (a latch plus an AND gate) so the gated clock never glitches.
- A group of flip-flops driven by one clock forms a **clock domain**. Signals that cross between domains need special handling (Chapter 19).

11.15 Summary

- Setup: $t_{cq} + t_{pd} + t_{su} \le T + t_{skew}$. It limits frequency; fix it by slowing the clock, shortening logic, or pipelining.
- Hold: $t_{cq} + t_{cd} \ge t_h + t_{skew}$. No T in it; fix it by adding delay, never by slowing the clock.
- Positive skew (capture clock late) helps setup and hurts hold.
- Slack is the margin; negative slack is a violation.
- In a timing diagram, each flip-flop captures the value just before the edge and changes t_{cq} after it.
- Shift registers depend on $t_{cq} > t_h + t_{skew}$.
- Pipelining raises throughput and frequency but adds latency and register overhead.
- Long wires need retiming flip-flops: segments $= \lceil$wire delay$/(T-$overhead$)\rceil$, flops $=$ segments -1.
- Setup is signed off at the slow corner, hold at the fast corner.

11.16 Exercises

1. $t_{cq} = 80\,\text{ps}$, $t_{pd} = 600\,\text{ps}$, $t_{su} = 40\,\text{ps}$, no skew. What is the maximum clock frequency?
2. $t_{cq} = 30\,\text{ps}$, $t_{cd} = 5\,\text{ps}$, $t_h = 50\,\text{ps}$. Is there a hold problem? How would you fix it?
3. The capture clock arrives 40 ps after the launch clock. Does this help or hurt setup? Hold?
4. A design targets 1 GHz with $t_{cq} = 100\,\text{ps}$ and $t_{su} = 50\,\text{ps}$. What is the maximum logic delay?

5. A 1200 ps logic block with 100 ps of register overhead is split into four equal pipeline stages. Find the new clock frequency, and compare the latency before and after.
6. A shift register has $t_{cq} = 50$ ps and $t_h = 20$ ps. Does it work with zero skew? With the capture clock 40 ps late at every stage?
7. True or false: a hold violation can be fixed by lowering the clock frequency.
8. At which process corner is setup timing checked? Hold timing?
9. A 50 mm wire has 0.08 ns/mm delay. The clock period is 1 ns and flip-flop overhead is 0.2 ns. How many retiming flip-flops are needed?
10. In Figure 11.6, what is the value of Q_2 just after the third rising edge?

Answers

1. $T = 80 + 600 + 40 = 720$ ps, so $f_{max} \approx 1.39$ GHz.
2. Yes: $30 + 5 = 35 < 50$, a 15 ps violation. Add at least 15 ps of delay to the short path. Changing the frequency will not help.
3. That is +40 ps skew: it helps setup (40 ps more time) and hurts hold (the hold requirement grows by 40 ps).
4. $1000 - 100 - 50 = 850$ ps.
5. Each stage: $300 + 100 = 400$ ps, so 2.5 GHz. Latency before: $1200 + 100 = 1300$ ps. After: $4 \times 400 = 1600$ ps.
6. Zero skew: $50 \geq 20$, works. With +40 ps skew: $50 \geq 20 + 40 = 60$ fails, a 10 ps hold violation.
7. False.
8. Setup: slow corner. Hold: fast corner.
9. Each segment may use $1 - 0.2 = 0.8$ ns, covering 10 mm. Total 50 mm needs 5 segments, so 4 flip-flops.
10. Before the third edge, Q_1 is 0 (it captured $D = 0$ at the second edge), so Q_2 becomes 0. (At the second edge Q_2 captured $Q_1 = 1$; at the third it captures $Q_1 = 0$.)

Chapter 12

Counters, Clock Dividers and State Machines

With flip-flops and timing understood, we can build circuits that step through sequences: counters, circuits that make slower clocks from faster ones, and finite state machines, which are the "brains" of almost every control circuit. Candidates report questions on all three: clock dividers with particular duty cycles, counting flip-flops in a counter, the minimum flip-flops for a traffic-light controller, and sequence detectors.

12.1 Counters

A **counter** is a register that adds 1 on every clock edge. An n-bit counter steps through $0, 1, 2, \ldots, 2^n - 1$ and then wraps back to 0.

Synchronous counter. A register plus an adder: $Q^+ = Q + 1$ (Figure 12.1). All flip-flops share one clock and change together. This is the standard design.

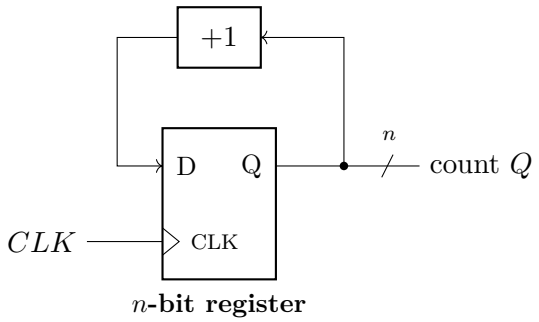


Figure 12.1: A synchronous counter: a register whose input is its own output plus 1. On every clock edge it loads $Q + 1$.

Ripple counter. A chain of toggle flip-flops in which each flip-flop's output clocks the next (Figure 12.2). The first flip-flop toggles on every clock edge, so its output is a square wave at half

the clock frequency. The second toggles on every edge of the first, so it runs at a quarter of the clock frequency, and so on. Ripple counters are small, but each stage adds a clock-to-Q delay, so the bits do not change at the same instant. Reading the count while it ripples gives wrong values briefly.

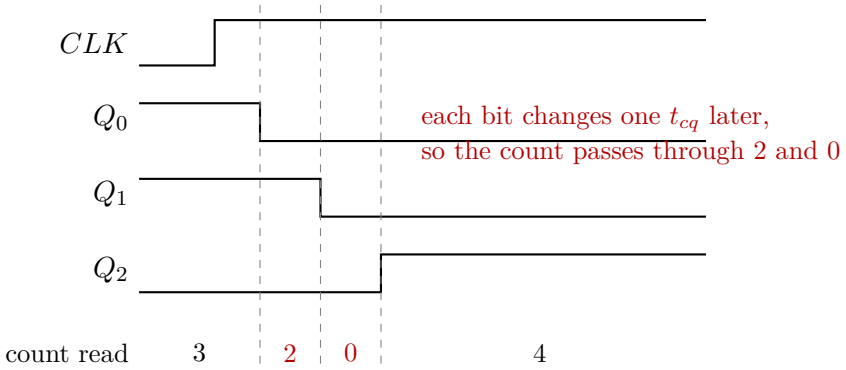


Figure 12.2: A 3-bit ripple counter going from 3 (011) to 4 (100). Each flip-flop is clocked by the one before it, so the bits change one after another, and for a moment the count reads 2, then 0 (red), before it settles at 4.

The big idea

Each bit of a binary counter runs at half the frequency of the bit below it. Bit k (counting from bit 0) is a square wave at $f_{clk}/2^{k+1}$. The most significant bit of an n-bit counter has frequency $f_{clk}/2^n$ and period $2^n \cdot T_{clk}$.

Worked example: how many flip-flops in the counter?

An engineer measures a binary counter: the clock period is 2 ns, and the counter's most significant output has a period of 128 ns. How many flip-flops does the counter have?
The MSB period is 2^n times the clock period: $2^n = 128/2 = 64$, so $n = 6$ flip-flops.

Seen on the assessment

A reported question gives two measured times, $A = 2$ ns and $B = 128$ ns, and asks for the number of flip-flops in a binary counter. With A as the clock period and B as the period of the top bit, the answer is $\log_2(128/2) = 6$. Use the same reasoning for any variant: the frequency ratio between the clock and the top bit is 2^n.

Mod-N counters. A counter that counts 0 to $N-1$ and then wraps to 0 is a **mod-N counter**. It needs $\lceil \log_2 N \rceil$ flip-flops: a decimal digit counter (mod 10) needs 4. It is built as a normal counter with a comparator that resets it to 0 when it reaches $N-1$.

Counters with enable. Often a counter should only count when an enable input is 1: $Q^+ = EN \; ? \; Q+1 : Q$.

Worked example: a counter with an enable input

A counter starts at 0 and increments on each rising clock edge *only if* input $X = 1$ at that edge. The clock is 8 kHz; X is a 2 kHz square wave. Assume X is 1 at clock edges 1 and 2, 0 at edges 3 and 4, 1 at edges 5 and 6, and so on.
The clock is 4 times faster than X, so each period of X spans 4 clock edges: 2 with $X = 1$ and 2 with $X = 0$. After edge 7, the counter has incremented at edges 1, 2, 5 and 6: the count is 4. After edge 10, add edges 9 and 10: the count is 6.

The answer depends on how the two waveforms line up, so read the question's timing diagram carefully: which edges see $X = 1$ is the whole problem.

12.2 Clock dividers

A **clock divider** makes a slower clock from a faster one. It is one of the most commonly reported interview topics.

Divide by 2

Connect a D flip-flop's $\overline{Q}$ to its own D (or use a T flip-flop with $T = 1$). Each rising edge flips the output, so the output completes one full cycle every two clock cycles: half the frequency, with a perfect 50% duty cycle.

Worked example: a toggle flip-flop

A 25 kHz clock drives a T flip-flop with $T = 1$ and its asynchronous set and clear inputs tied inactive. The output is a 12.5 kHz square wave. A second such stage, clocked by the first, gives 6.25 kHz.

A chain of n such stages divides by 2^n.

Divide by an even number

To divide by $N = 2M$ with a 50% duty cycle, count clock edges from 0 to $M - 1$ and toggle the output each time the count wraps. For divide-by-6, toggle every 3 cycles: the output is high for 3 cycles and low for 3.

Divide by an odd number with a 50% duty cycle

Odd division is harder, because N cycles cannot be split into two equal whole numbers of cycles. The trick is to use *both* clock edges. For divide-by-3:

1. Build a mod-3 counter on the rising edge (states 0, 1, 2).
2. Make a signal P that is 1 during state 0: high for 1 cycle out of 3 (a 33% duty cycle).
3. Make N, a copy of P delayed by half a cycle, by sampling P with a flip-flop on the *falling* clock edge.

4. Output P OR N. It is high for $1 + \frac{1}{2} = 1.5$ cycles out of 3: exactly 50%.

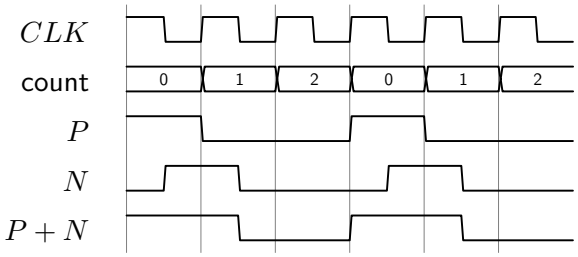


Figure 12.3: Divide-by-3 with a 50% duty cycle. P comes from rising edges (gridlines); N is P delayed to the next falling edge; their OR is high for 1.5 of every 3 clock cycles.

The same method divides by any odd N: make P high for $(N-1)/2$ cycles out of N, delay it half a cycle, and OR. For divide-by-5, P is high for 2 cycles; the OR is high for 2.5 cycles out of 5.

Divide by 2 with a 25% duty cycle

Take the divide-by-2 output Q (high for one full clock period, low for the next) and combine it with the clock itself. Inside each period in which Q is high, the clock is high for half and low for half. ANDing Q with the clock, or with the inverted clock, keeps only one half: a pulse one half-period wide every two periods, which is a 25% duty cycle at half the frequency.

Which version? Q changes just *after* each rising clock edge (by t_{cq}). With $Q \cdot CLK$, at the rising edge where Q falls, the clock rises a moment before Q drops, producing a brief glitch. With $Q \cdot \overline{CLK}$, the inverted clock falls at that edge before Q changes, so no glitch can occur. The glitch-free answer is

$$\text{OUT} = Q \cdot \overline{CLK}.$$

Figure 12.4 shows both versions.

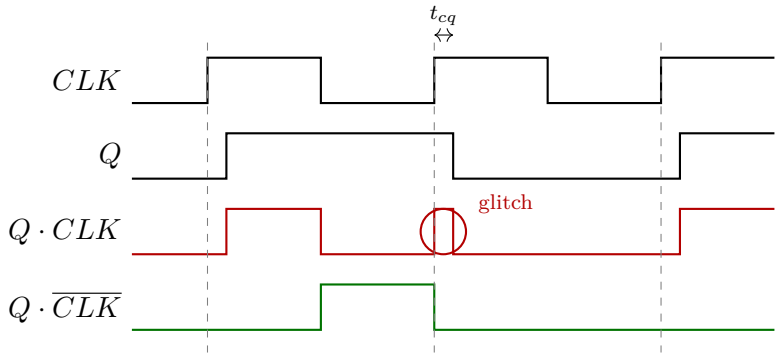


Figure 12.4: Divide by 2 with a 25% duty cycle. Q changes t_{cq} after each rising edge. $Q \cdot CLK$ (red) glitches at the edge where Q falls, because the clock rises first; $Q \cdot \overline{CLK}$ (green) is clean.

Seen on the assessment

Reported clock-divider questions include divide-by-2 with a 25% duty cycle and divide-by-3 with a 50% duty cycle ("bonus for 50% duty cycle"), followed by many follow-up questions

about the design. Be ready to draw the waveforms, explain why odd division needs the falling edge, and point out glitch risks when gating a clock with logic.

A caution about logic on clocks. Combining clocks with ordinary gates can create glitches, and a glitch on a clock is catastrophic: flip-flops see it as an extra edge. Real designs use special glitch-free clock-gating cells and clock muxes, and keep clock logic to a minimum.

12.3 Finite state machines

A **finite state machine** (FSM) is a circuit that remembers which of a fixed set of **states** it is in, and moves between states according to its inputs. Traffic lights, vending machines, communication protocols, and the control unit of every processor are FSMs.

Every synchronous FSM has the same three parts (Figure 12.5):

- a **state register**: flip-flops holding the current state;
- **next-state logic**: combinational logic computing the next state from the current state and inputs;
- **output logic**: combinational logic computing the outputs.

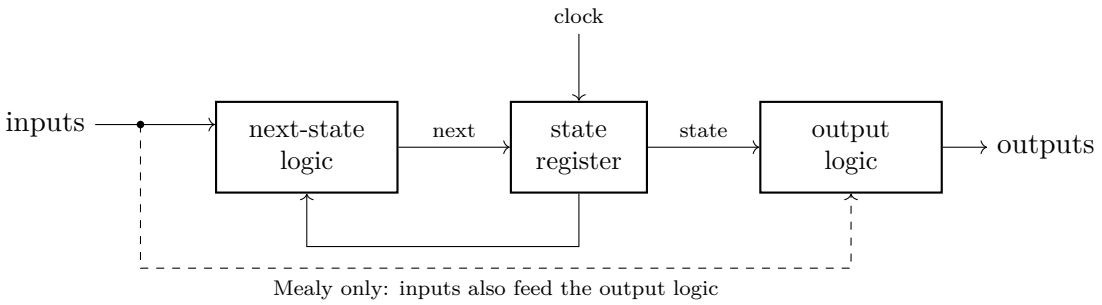


Figure 12.5: Structure of a finite state machine.

Moore versus Mealy.

- In a **Moore machine**, the outputs depend only on the current state. Outputs are stable for a whole cycle and glitch-free, but they react to an input one cycle later.
- In a **Mealy machine**, the outputs depend on the current state *and* the current inputs. Mealy machines react within the same cycle and often need fewer states, but their outputs can glitch when the inputs do.

State encoding. The states must be given binary codes:

- **Binary**: number the states $0, 1, 2, \ldots$. Fewest flip-flops: $\lceil \log_2(\text{number of states}) \rceil$.

- **One-hot**: one flip-flop per state, exactly one of them 1. More flip-flops, but the next-state logic is simple and fast. Popular in FPGAs.
- **Gray**: consecutive states differ by one bit, which reduces switching power and glitches.

12.4 Example: how many flip-flops for a traffic light?

Worked example: the minimum flip-flops for a traffic-light FSM

A traffic light stays green for 30 s, yellow for 5 s and red for 25 s, then repeats. The controller's clock has a 1 s period, and the FSM must count the seconds itself (no separate timer). What is the minimum number of flip-flops?
Every second of the 60-second cycle must be a distinct state, because the machine must know how long it has been in each color. That is $30 + 5 + 25 = 60$ states. Minimum flip-flops: $\lceil \log_2 60 \rceil = 6$ (since $2^5 = 32 < 60 \leq 64 = 2^6$).

A common alternative design uses a 3-state color FSM (2 flip-flops) plus a separate countdown timer big enough for 30 (5 flip-flops): 7 flip-flops in total. It is often cleaner, but it is not the minimum. Read whether the question asks for the minimum.

Seen on the assessment

A reported question asks for the minimum number of flip-flops for a traffic-light FSM, given the green, yellow and red durations and a clock period. The method: total cycles in one full loop = (sum of the durations) / (clock period); minimum flip-flops = $\lceil \log_2(\text{total cycles}) \rceil$.

12.5 Example: a sequence detector

A **sequence detector** watches a serial input, one bit per clock cycle, and outputs 1 when a particular pattern has just arrived. With **overlap**, the end of one match may also be the start of the next.

Worked example: detecting 1011 with overlap (Mealy)

Each state records how much of the pattern 1011 has been seen so far:

- S0: nothing useful yet.
- S1: seen "1".
- S2: seen "10".
- S3: seen "101".

The transitions, written as input/output:

State	Input 0		Input 1	
S0	S0 / 0		S1 / 0	
S1 (“1”)	S2 / 0		S1 / 0	(“11”: the last 1 may start a match)
S2 (“10”)	S0 / 0	(“100”: start over)	S3 / 0	
S3 (“101”)	S2 / 0	(“1010” ends in “10”)	S1 / **1**	(match; last 1 restarts)

The tricky entries are where a mismatch still leaves part of a pattern. From S3 on input 1, the machine has seen 1011: it outputs 1, and the final 1 is the start of a possible new match, so it goes to S1 (that is the overlap). From S3 on input 0, the last two bits “10” are a valid start, so it goes to S2, not S0.
Check with the input 1011011: states S0 → S1 → S2 → S3 → S1 (output 1) → S2 → S3 → S1 (output 1). Two matches, ending at bits 4 and 7, as expected.

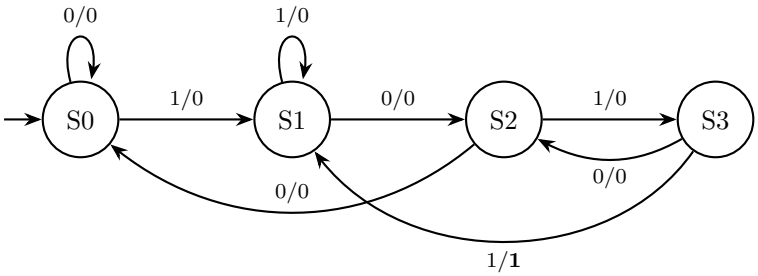


Figure 12.6: Mealy state diagram for detecting 1011 with overlap. Labels are input/output.

A **Moore** version needs one more state, S4 (“just matched”), whose output is 1. From S4, input 1 goes to S1 and input 0 goes to S2 (since “10110” ends in “10”). Moore machines typically need more states than Mealy machines for the same job, because the output must be attached to a state.

The general recipe for sequence detectors. For a pattern of length L, the Mealy machine has L states (one per prefix length, 0 to $L-1$). For each state and input, the next state is the longest suffix of (seen-so-far + new bit) that is also a prefix of the pattern. Doing this carefully is what makes overlapping detection correct.

12.6 Other useful counters

- **Ring counter** (Figure 12.7): a shift register with the last output fed back to the first, holding a single 1. With N flip-flops it has N states (one-hot).
- **Johnson counter**: a shift register with the *inverted* last output fed back. With N flip-flops it cycles through $2N$ states, and only one bit changes per step.
- **Linear-feedback shift register** (LFSR): a shift register whose input is the XOR of certain bits. With the right taps, N flip-flops step through all $2^N - 1$ nonzero states in a pseudo-random order. LFSRs generate test patterns, scramble data and compute checksums (CRCs).

- **Gray-code counter**: counts in Gray code, so only one bit changes per step; used for pointers that cross clock domains (Chapter 19).

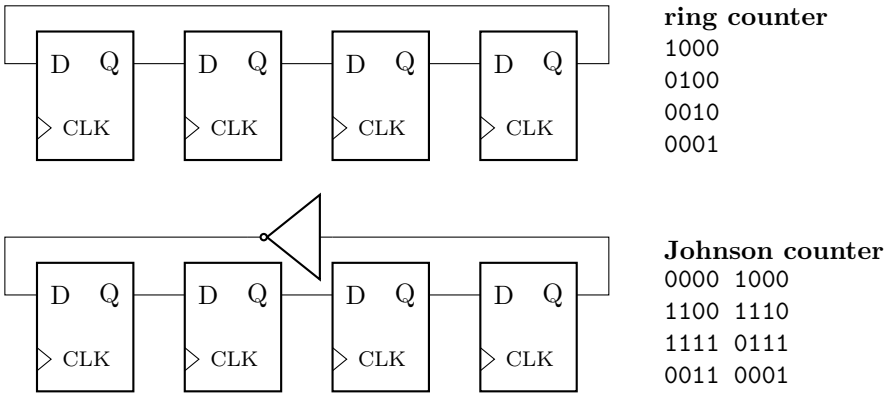


Figure 12.7: Ring counter (top): the last output feeds back to the first, so a single 1 circulates through $N = 4$ states. Johnson counter (bottom): the feedback is inverted, so it passes through $2N = 8$ states, changing one bit per step. Clock wires are omitted.

12.7 Summary

- A binary counter's bit k runs at $f_{clk}/2^{k+1}$; the MSB of n bits has period $2^n T_{clk}$.
- A mod-N counter needs $\lceil \log_2 N \rceil$ flip-flops.
- Divide by 2: toggle flip-flop. Divide by 2^n: n stages. Even N: toggle every $N/2$ cycles.
- Odd N with 50% duty: a rising-edge signal high for $(N-1)/2$ cycles, ORed with a copy delayed half a cycle by a falling-edge flip-flop.
- Divide by 2 with 25% duty: $Q \cdot \overline{CLK}$ (glitch-free).
- FSM: state register + next-state logic + output logic. Moore outputs depend on state only; Mealy on state and inputs.
- Minimum flip-flops $= \lceil \log_2(\text{states}) \rceil$; one-hot uses one per state.
- Overlapping sequence detectors: after a match or mismatch, go to the state of the longest suffix that is still a prefix.

12.8 Exercises

1. A 3-bit ripple counter is clocked at 16 MHz. What is the frequency of its most significant bit?
2. How many flip-flops are needed for a mod-10 counter? A mod-100 counter?
3. Design a divide-by-4 from a 100 MHz clock. What is the output frequency and how many flip-flops are needed?
4. Describe a divide-by-6 circuit with a 50% duty cycle.

5. Describe a divide-by-5 circuit with a 50% duty cycle.
6. An FSM has 12 states. How many flip-flops does it need with binary encoding? With one-hot encoding?
7. What is the difference between a Moore and a Mealy machine? Which usually needs more states?
8. How many states does a Mealy machine need to detect the overlapping sequence 110?
9. A traffic light is green for 20 s, yellow for 4 s and red for 24 s, with a 2 s clock period. What is the minimum number of flip-flops?
10. Why is $Q \cdot \overline{CLK}$ preferred over $Q \cdot CLK$ for a 25% duty-cycle divide-by-2?
11. How many states does a 4-flip-flop Johnson counter have? A 4-flip-flop maximal LFSR?

Answers

1. $16/2^3 = 2$ MHz.
2. 4 ($2^4 = 16 \geq 10$). 7 ($2^7 = 128 \geq 100$).
3. Two toggle stages: 25 MHz, 2 flip-flops.
4. A mod-3 counter on the rising edge; toggle the output each time the counter wraps (every 3 cycles). High for 3 cycles, low for 3.
5. A mod-5 counter on the rising edge; make P high for 2 of the 5 states; delay P by half a cycle with a falling-edge flip-flop to get N; output P OR N, which is high for 2.5 of every 5 cycles.
6. Binary: 4 ($2^4 = 16 \geq 12$). One-hot: 12.
7. Moore outputs depend only on the state; Mealy outputs depend on the state and the current inputs. Moore usually needs more states.
8. 3 (prefixes of length 0, 1 and 2).
9. $(20 + 4 + 24)/2 = 24$ cycles, so $\lceil \log_2 24 \rceil = 5$.
10. Q changes t_{cq} after each rising edge. With $Q \cdot CLK$, when Q falls the clock has already risen, giving a short glitch. With $\overline{CLK}$, that input has already fallen before Q changes, so the AND output stays low.
11. Johnson: $2 \times 4 = 8$ states. LFSR: $2^4 - 1 = 15$ states.

Part V

Speed, Power and the Real World

Chapter 13

Delay: Why Circuits Take Time

Chapter 11 used gate delays as given numbers. This chapter explains where they come from and how designers control them. One idea runs through all of it: a transistor behaves like a resistor, whatever it drives behaves like a capacitor, and delay is the time to charge that capacitor through that resistor.

13.1 The RC model of a gate

When a gate's output switches from 0 to 1, its PMOS pull-up network charges the output node from 0 to V_{DD}. Everything connected to that node is capacitance (Chapter 2):

- the input (gate) capacitance of every gate it drives, called its **fan-out**;
- the capacitance of the wire;
- the gate's own drain (diffusion) capacitance, its **parasitic** capacitance.

The pull-up network acts like a resistor, R, so the output rises like an RC charging curve (Figure 13.1). It reaches 50% after about $0.69\,RC$ (Section 2.4):

$$\boxed{t_{pd} \approx 0.69\,R\,C_{load}}$$

The same applies to a falling output through the pull-down network.

Worked example: an inverter driving four others

A unit inverter has an effective resistance of $10\,\text{k}\Omega$, an input capacitance of 1 fF and a parasitic output capacitance of 1 fF. It drives four identical inverters (a **fan-out of 4**).
$C_{load} = 4 \times 1 + 1 = 5\,\text{fF}$. Delay $\approx 0.69 \times 10^{4} \times 5 \times 10^{-15} \approx 35\,\text{ps}$.

The delay of an inverter driving four copies of itself, the **FO4 delay**, is a standard yardstick for how fast a manufacturing process is. Designers often describe a clock period as "so many FO4 delays," which stays meaningful across processes.

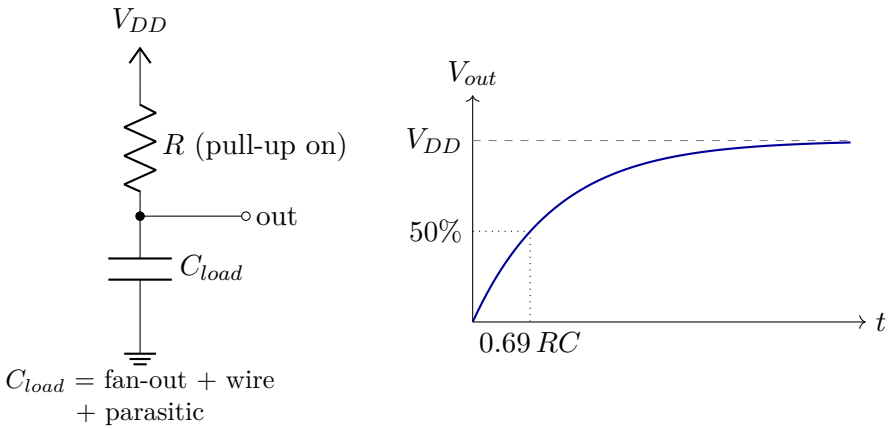


Figure 13.1: The RC model of a gate. While the output rises, the conducting pull-up network acts as a resistor R charging everything on the output node, C_{load}. The output crosses 50% after about $0.69\,RC$.

13.2 Rise, fall and propagation

Precise definitions, all measured on waveforms (Figure 13.2):

- t_{pLH}, t_{pHL}: from the input crossing 50% to the output crossing 50%, for a rising or falling output. The propagation delay is often quoted as their average.
- **Rise time** and **fall time**: how long the output takes to go from 10% to 90% of its swing (or the reverse). For an RC curve that is $2.2\,RC$.
- The **slew** or transition time of a signal is its rise or fall time. A slowly rising input makes the next gate slower and wastes power, because both of its transistors are partly on for longer.

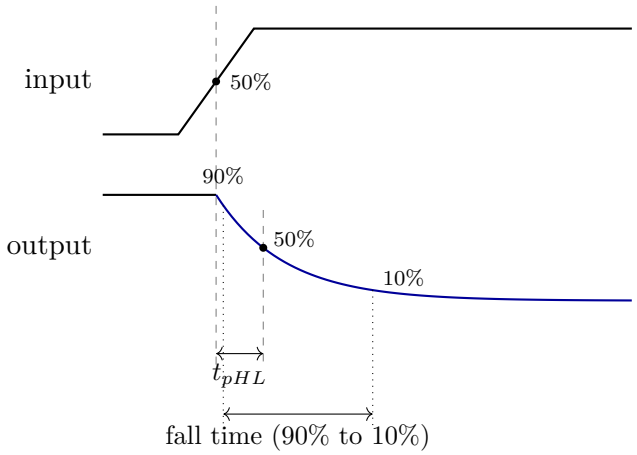


Figure 13.2: Measuring an inverter's delay. The propagation delay runs from the input crossing 50% to the output crossing 50%. The fall time runs from 90% to 10% of the output swing.

13.3 How sizing changes delay

From Chapter 5, a transistor's current is proportional to its width W, so its resistance is proportional to $1/W$. Its gate capacitance is proportional to W. That gives the basic tension of sizing:

The big idea

Making a gate k times wider makes it k times stronger (its own delay falls), but it presents k times more input capacitance to whatever drives it (the previous gate's delay rises). Making gates bigger is not free speed; it moves delay backward.

13.4 Driving a big load: buffer chains

Suppose a small gate must drive a huge capacitance: a long bus, a clock net, an output pin. Driving it directly is very slow, and making the small gate huge just loads *its* driver. The solution is a chain of inverters, each bigger than the last by a constant factor, a **tapered buffer**.

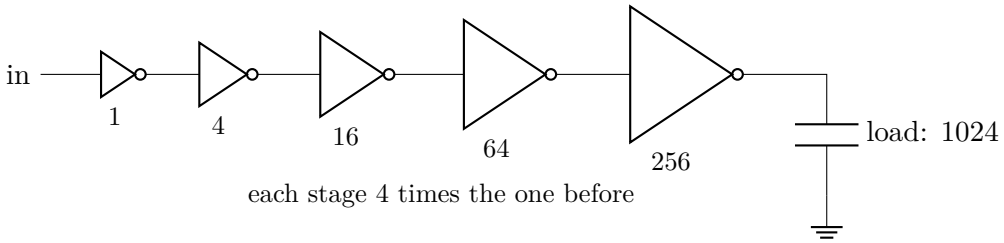


Figure 13.3: A tapered buffer: five inverters, each four times larger than the one before (sizes in units of the first), driving a load 1024 times the first inverter's input capacitance. Every stage sees a fan-out of 4.

If the load is F times the input capacitance of the first inverter and you use N stages, each stage should be $F^{1/N}$ times bigger than the one before. Analysis (and practice) shows the best ratio per stage is around **4**, so

$$N \approx \log_4 F.$$

Worked example: sizing a buffer chain

A unit inverter must drive a load 1024 times its own input capacitance. With a ratio of 4 per stage: $\log_4 1024 = 5$ stages, sized 1, 4, 16, 64 and 256, with the last one driving the 1024 load. Each stage sees a fan-out of 4, so each has roughly an FO4 delay, and the total is about 5 FO4.

13.5 Logical effort

Logical effort is a method for estimating and optimizing the delay of a path of different gates. It writes the delay of each gate, in units of a basic inverter delay, as

$$d = g\,h + p$$

- g, the **logical effort**: how much worse the gate is at driving current than an inverter with the same input capacitance. It depends only on the gate type.
- h, the **electrical effort** or fan-out: C_{out}/C_{in}.
- p, the **parasitic delay**: the gate's delay from its own internal capacitance.

Gate	Logical effort g	Parasitic delay p
Inverter	1	1
NAND2	4/3	2
NAND3	5/3	3
NOR2	5/3	2
NOR3	7/3	3

(These values assume a PMOS twice as wide as an NMOS for equal strength.) NOR has a higher logical effort than NAND, which is the quantitative version of "NAND is better than NOR" (Chapter 8).

For a path of N gates, multiply the logical efforts into G, take the overall electrical effort $H = C_{out,\text{path}}/C_{in,\text{path}}$, and form the **path effort** $F = G\,H$ (ignoring branching). The fastest design gives every stage the same effort, $f = F^{1/N}$, and the minimum delay is

$$D = N\,F^{1/N} + P,$$

where P is the sum of the parasitic delays.

Worked example: a three-gate path

A path is inverter, NAND2, inverter. The final load is 48 times the first inverter's input capacitance ($H = 48$).
$G = 1 \times \frac{4}{3} \times 1 = \frac{4}{3}$. $F = GH = \frac{4}{3} \times 48 = 64$. Best stage effort $f = 64^{1/3} = 4$. $P = 1+2+1 = 4$. Minimum delay $D = 3 \times 4 + 4 = 16$ inverter delay units. Each stage is then sized so that its $g\,h = 4$.

13.6 Elmore delay: delays through networks

Often a driver charges several capacitances through a chain or tree of resistances (a wire with many segments, or a stack of transistors). The **Elmore delay** is a simple estimate of the time constant to a given node:

The big idea

Elmore delay: for each resistor on the path from the source to the node of interest, multiply its resistance by *all* the capacitance downstream of it, then add up the products:

$$\tau_{Elmore} = \sum_i R_i \cdot C_{\text{downstream of } R_i}.$$

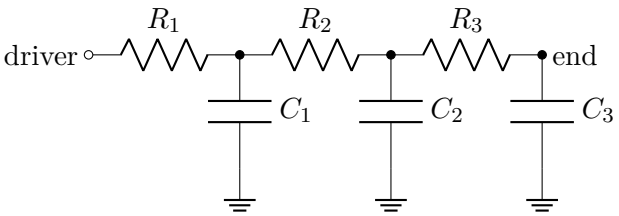


Figure 13.4: An RC ladder, the model of a wire divided into three segments.

Worked example: Elmore delay of a ladder

In Figure 13.4, every R and every C is equal. To the end node:

$$\tau = R_1(C_1 + C_2 + C_3) + R_2(C_2 + C_3) + R_3C_3 = 3RC + 2RC + RC = 6RC.$$

With $R = 100\ \Omega$ and $C = 10\,\text{fF}$, $\tau = 6 \times 10^{-12} = 6\,\text{ps}$.

13.7 Wires

A wire's resistance and capacitance are both spread along its length. Split a wire of length L into n segments, and the Elmore delay grows like $1 + 2 + \cdots + n \approx n^2/2$. Since both R and C are proportional to L:

$$t_{wire} \propto R_{wire}\, C_{wire} \propto L^2.$$

(For a uniformly distributed RC wire, the 50% delay is about $0.38\, R_{wire}C_{wire}$.)

The big idea

An unbuffered wire's delay grows with the *square* of its length. Doubling the length quadruples the delay. The fix is to break a long wire into segments with **repeaters** (buffers) between them: each short segment has a small delay, and the total grows only linearly with length.

Worked example: repeaters

An unbuffered wire has a delay of 400 ps. Insert one repeater in the middle. Each half has a quarter of the original delay (half the length, squared): 100 ps each, 200 ps for both, plus the repeater's own delay (say 20 ps): 220 ps total, almost half. More repeaters help further, up to the point where the repeaters' own delays dominate.

As transistors have shrunk, they have become much faster, but thin wires have become *more* resistive. In modern chips, long-wire delay often matters more than gate delay, which is why chip architects work hard to keep communicating blocks close together.

13.8 Other effects on delay

- **Supply voltage.** A gate's delay is roughly $C\,V_{DD}/I$, and drive current grows with $(V_{DD} - V_t)$ raised to a power between 1 and 2. Raising V_{DD} speeds gates up; lowering it slows them,

dramatically so as V_{DD} approaches V_t.

- **Temperature and process.** Hot transistors are usually slower (lower mobility), and manufacturing variation makes some chips faster than others (Chapter 15).
- **Input ordering in stacks.** In a series stack, the transistor nearest the output must discharge the most internal capacitance. Connecting the *latest-arriving* input to the transistor nearest the output makes the gate respond fastest to it, because the other transistors have already turned on and discharged the internal nodes.
- **Miller effect.** Capacitance between a gate's input and output appears doubled when they swing in opposite directions (as they do in an inverter), adding delay. The same effect makes coupling between neighboring wires worse when they switch in opposite directions (Chapter 16).

13.9 Summary

- Gate delay $\approx 0.69\,R\,C_{load}$; the load is fan-out input capacitance plus wire plus parasitic capacitance.
- FO4 delay is the standard yardstick for process speed.
- Wider gates are stronger but load their drivers more.
- Drive big loads with a tapered chain, about 4 times larger per stage: $N \approx \log_4(C_{load}/C_{in})$.
- Logical effort: $d = gh + p$; NAND2 has $g = 4/3$, NOR2 5/3. Best path delay $D = NF^{1/N} + P$.
- Elmore delay: sum of each resistance times all downstream capacitance.
- Unbuffered wire delay grows with L^2; repeaters make it linear.
- Higher V_{DD} is faster; put the latest-arriving input nearest the output.

13.10 Exercises

1. A gate with a $5\,\text{k}\Omega$ effective resistance drives 20 fF. Estimate its delay.
2. A gate's fan-out doubles. Roughly what happens to its delay, assuming the fan-out capacitance dominates?
3. A unit inverter must drive 64 times its input capacitance. How many stages should a tapered buffer chain have, and what sizes?
4. What are the logical efforts of an inverter, a NAND2 and a NOR2? Why is the NOR2's larger?
5. An RC ladder has $R_1 = R_2 = 1\,\text{k}\Omega$ and $C_1 = C_2 = 10\,\text{fF}$ (the same shape as Figure 13.4 with two segments). Find the Elmore delay to the end.
6. An unbuffered wire's length triples. By what factor does its delay change?
7. Why should the latest-arriving input of a NAND gate drive the transistor nearest the output?
8. Does raising the supply voltage make a gate faster or slower? Why?

Answers

1. $0.69 \times 5000 \times 20 \times 10^{-15} \approx 69\,\text{ps}$.
2. It roughly doubles.
3. $\log_4 64 = 3$ stages, sized 1, 4 and 16 (the last drives the 64 load).
4. 1, 4/3 and 5/3. NOR2 has two PMOS in series; to match an inverter's strength they must be very wide, giving more input capacitance for the same drive.
5. $R_1(C_1 + C_2) + R_2C_2 = 1000 \times 20 \times 10^{-15} + 1000 \times 10 \times 10^{-15} = 30\,\text{ps}$.
6. $3^2 = 9$ times.
7. By the time the last input arrives, the other transistors are already on and the internal nodes have been discharged, so only the transistor nearest the output has to switch, and the output responds fastest.
8. Faster: more voltage gives more drive current (more overdrive above V_t), so capacitances charge and discharge more quickly. The cost is more power.

Chapter 14

Power: Where the Energy Goes

Power limits modern chips more than anything else. A GPU could be faster if it could burn more power, but it would melt (Chapter 3), and a phone could be faster but its battery would die. This chapter explains exactly where a chip's power goes, derives the formula behind the assessment question "if frequency and voltage double, what happens to dynamic power?", and surveys the techniques engineers use to save power.

14.1 Three kinds of power

1. **Dynamic power**: spent charging and discharging capacitance every time a signal switches. Usually the largest part when a chip is busy.
2. **Short-circuit power**: during each transition, both the pull-up and pull-down are briefly on together, and current flows straight from the supply to ground.
3. **Leakage power** (static power): transistors that are "off" still leak a little current, all the time, even when nothing switches.

14.2 Dynamic power

Consider an inverter driving a load capacitance C (Figure 14.1).

Charging (output 0 $\rightarrow$ 1). The PMOS connects the capacitor to V_{DD}. Charge $Q = CV_{DD}$ flows out of the supply at voltage V_{DD}, so the supply delivers energy $QV_{DD} = CV_{DD}^2$. Of that, $\frac{1}{2}CV_{DD}^2$ ends up stored in the capacitor (Chapter 2); the other half is turned into heat in the PMOS's resistance.

Discharging (output 1 $\rightarrow$ 0). The NMOS connects the capacitor to ground, and the stored $\frac{1}{2}CV_{DD}^2$ is turned into heat in the NMOS.

So every full cycle of charge and discharge consumes CV_{DD}^2 from the supply. If the node does this f times per second, the power is $CV_{DD}^2 f$. Most nodes do not switch every cycle, so we multiply by an **activity factor** α: the fraction of clock cycles in which the node makes a 0-to-1 transition.

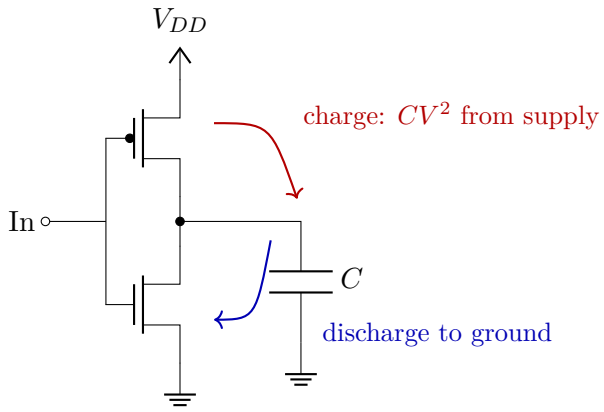


Figure 14.1: Each 0-to-1 output transition draws charge from the supply; each 1-to-0 transition dumps it to ground.

$$P_{dynamic} = \alpha\, C\, V_{DD}^2\, f$$

- α: activity factor. A clock rises once every cycle, so $\alpha = 1$. Typical data nodes have α around 0.1 to 0.25.
- C: the total capacitance that switches.
- V_{DD}: supply voltage, and it is *squared*.
- f: clock frequency.

The big idea

Dynamic power is proportional to the *square* of the voltage and only linearly to frequency and capacitance. Lowering the voltage is the most powerful way to save power.

Worked example: a chip's dynamic power

A block has 1 nF of total switching capacitance, an average activity factor of 0.2, a 1.0 V supply and a 1 GHz clock.

$$P = 0.2 \times 10^{-9} \times 1.0^2 \times 10^{9} = 0.2 \text{ W}.$$

14.3 Scaling questions

Assessment questions change one or more of the variables and ask for the new power. Since $P \propto \alpha C V^2 f$, multiply the factors, remembering to square the voltage factor.

Change	New dynamic power	Working
$f \times 2$	$\times 2$	linear in f
$V \times 2$	$\times 4$	2^2
$f \times 2$ **and** $V \times 2$	$\times 8$	2×2^2
$V \times 0.5$	$\times 0.25$	0.5^2
V up 10%, f up 20%	$\times 1.452$ (+45%)	$1.1^2 \times 1.2$
$(f, 2V) \to (4f, 3V)$	$\times 9$	$4 \times (3/2)^2$
$V \times 0.9$, $f \times 0.9$	$\times 0.729$	$0.9^2 \times 0.9$
$C \times 2$ (twice the logic)	$\times 2$	linear in C

Seen on the assessment

"If the clock frequency doubles and the supply voltage doubles, what is the dynamic power?" was an Ignite assessment question. Answer: **8 times** the original (2 for frequency times $2^2 = 4$ for voltage). Reported variants include voltage up 10% with frequency up 20% ($\times 1.45$) and a change from $(f, 2V)$ to $(4f, 3V)$ ($\times 9$; note that the voltage factor is $3V/2V = 1.5$, squared to 2.25).

Common trap

Watch for the starting point. In "$(f, 2V) \to (4f, 3V)$" the voltage does not triple; it goes from $2V$ to $3V$, a factor of 1.5. Always divide new by old for each variable before multiplying.

14.4 Power versus energy

Power is energy per second. **Energy** is what drains a battery. The energy for one operation is

$$E_{op} = \alpha C V_{DD}^2,$$

with no f in it. So:

- Running the same work at double the frequency (same voltage) *doubles the power* but uses *the same energy* per operation; the work just finishes sooner.
- Lowering the voltage reduces *both* power and energy per operation.

This is why "slow down to save energy" only works if the voltage comes down too.

14.5 Short-circuit power

During an input transition, there is a moment when the input is between V_{tn} and $V_{DD} - |V_{tp}|$ and both transistors are on. Current flows straight from V_{DD} to ground. The slower the input transition, the longer this lasts, which is one reason designers keep signal transitions sharp. In a well-designed chip, short-circuit power is usually 10% or less of dynamic power.

14.6 Leakage power

Even with no switching at all, current trickles through transistors that are supposed to be off. Leakage power is $P_{leak} = I_{leak}\, V_{DD}$. The main sources:

- **Subthreshold leakage**: an "off" transistor still conducts a little, and this current grows *exponentially* as V_t decreases (Chapter 5) and as temperature increases (Chapter 4). It is usually the largest leakage source.
- **Gate leakage**: electrons tunnel through extremely thin gate oxide. High-k gate materials were introduced largely to reduce it.
- **Junction leakage**: small reverse current through the source/drain diodes.

Two transistors that are both off and in series leak much less than one (the **stack effect**), because the node between them rises slightly and turns both off more firmly.

As transistors shrank, thresholds had to fall to keep them fast, and leakage grew from negligible to a large fraction of total power. Leakage also feeds on heat: a hotter chip leaks more and gets hotter still (Chapter 3).

14.7 Saving power

Technique	Reduces	How
Clock gating	dynamic	Stop the clock to idle registers. The clock network has $\alpha = 1$ and is a big part of dynamic power, so this is the most common technique.
Power gating	leakage (and dynamic)	Cut off the supply to an idle block with a large "sleep" transistor (a header PMOS or footer NMOS).
DVFS	dynamic and leakage	Dynamic voltage and frequency scaling: lower V and f together when full speed is not needed.
Multi-V_t cells	leakage	High-V_t (low-leakage) transistors on paths that are not timing-critical.
Multiple voltage domains	dynamic	Run slower blocks at a lower V_{DD}; level shifters translate signals between domains.
Body biasing	leakage or delay	Reverse body bias raises V_t (less leakage); forward bias lowers it (faster).
Reducing activity	dynamic	Avoid useless toggling: operand isolation, Gray-coded counters, removing glitches.
Parallelism	dynamic	Two slower units at a lower voltage can beat one fast unit (below).

Clock gating done safely. Simply ANDing a clock with an enable signal can produce glitches if the enable changes while the clock is high (Figure 14.3). The standard **integrated clock gating** (ICG) cell (Figure 14.2) captures the enable in a latch that is transparent only while the clock is

low, then ANDs the latched enable with the clock. The enable can only change while the clock is low, so the gated clock is always clean.

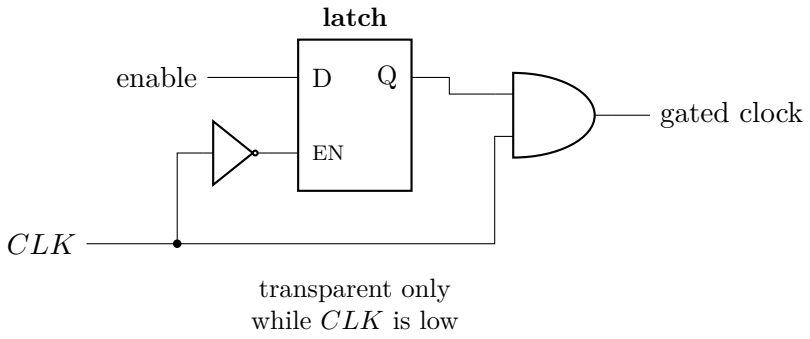


Figure 14.2: An integrated clock-gating cell. The latch passes the enable only while the clock is low, so the AND gate's enable input cannot change while the clock is high, and the gated clock has no glitches.

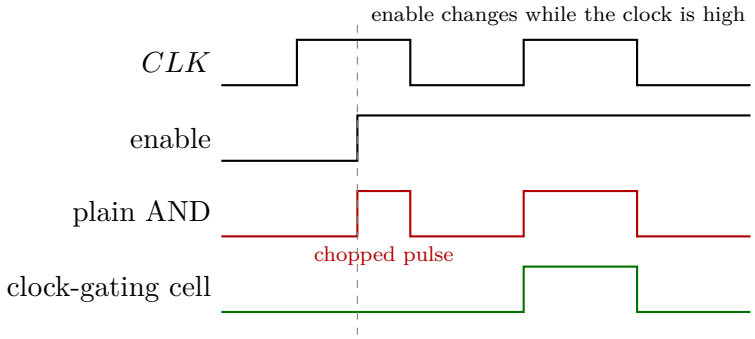


Figure 14.3: Why clock gating needs the special cell. A plain AND passes a chopped, short pulse (red) when the enable changes while the clock is high, and flip-flops may see it as an edge. The clock-gating cell waits for the clock to be low, so its first pulse is a whole one (green).

Power gating trade-offs. A power-gated block loses its state, so registers that must survive need special **retention** flip-flops (or the state is saved elsewhere). Outputs of a powered-off block float, so **isolation cells** clamp them to safe values. Waking up takes time and causes a surge of current as the block's capacitance recharges.

Worked example: parallelism saves power

One processing unit runs at frequency f and voltage V. Replace it with two identical units, each running at $f/2$ (same total throughput), which lets the voltage drop to $0.7V$. Ignoring overhead, the capacitance doubles:

$$\frac{P_{new}}{P_{old}} = \frac{2C \times (0.7V)^2 \times f/2}{CV^2 f} = 0.49.$$

Same work, about half the power. This is a big part of why GPUs, with thousands of moderately clocked cores, are so energy efficient for parallel work.

Worked example: how much power is left after power gating?

In an idle mode, a design power-gates most of its logic. The numbers (with assumptions stated):

- Standard cells: 40 W dynamic and 10 W leakage when active. 10% of the cells must stay on (always-on logic); the other 90% are gated with 80% efficiency (gating removes 80% of their leakage).
- RAMs: 20 W dynamic and 6 W leakage. All are gated with 90% efficiency.
- Gated blocks do no switching, so their dynamic power becomes zero; the always-on cells keep their share of dynamic power.

Standard-cell leakage left: $10 \times [0.1 + 0.9 \times (1 - 0.8)] = 10 \times 0.28 = 2.8$ W. RAM leakage left: $6 \times (1 - 0.9) = 0.6$ W. Dynamic left: $0.1 \times 40 = 4$ W from the always-on cells; RAMs 0 W. Total in idle: $2.8 + 0.6 + 4 = 7.4$ W, down from 76 W.

Seen on the assessment

A reported question gives power-gating efficiencies for standard cells (80%) and RAMs (90%), with 10% of cells always on, and asks for the remaining dynamic and leakage power. The exact answer depends on the numbers given, but the method is the one above: split each block into always-on and gated parts, remove the stated fraction of leakage from the gated part, and remove the gated part's dynamic power.

14.8 The bigger picture: Dennard scaling and dark silicon

For decades, shrinking transistors brought a free lunch called **Dennard scaling**: smaller transistors could run at lower voltage, so power per unit area stayed constant while chips got faster. Around 2005 this broke down. Voltage could no longer drop much, because lower thresholds would make leakage explode. Power density started rising with each generation, clock frequencies stopped climbing (most processors have stayed around 3 to 5 GHz since), and the industry turned to parallelism: multiple cores, and massively parallel GPUs.

Today a chip often cannot run all of its transistors at full speed at once without exceeding its power and cooling budget; some must be idle or slowed at any moment. This is called **dark silicon**. A chip's **thermal design power** (TDP) is the sustained power its cooling system is designed to remove.

14.9 Summary

- Dynamic power = $\alpha C V_{DD}^2 f$: each charge-discharge cycle costs CV_{DD}^2; half is lost charging and half discharging.
- Scaling: multiply the factor changes, squaring the voltage factor. $f \times 2$ and $V \times 2$ gives $\times 8$.
- Energy per operation = $\alpha C V_{DD}^2$, independent of f. Only lowering voltage saves energy per operation.

- Short-circuit power comes from both transistors being on during transitions; sharp edges minimize it.
- Leakage grows exponentially as V_t falls and temperature rises.
- Clock gating cuts dynamic power; power gating cuts leakage; DVFS cuts both.
- Parallel units at lower voltage can do the same work for less power.
- The end of Dennard scaling ended rising clock speeds and pushed the industry toward parallelism.

14.10 Exercises

1. A block has $\alpha = 0.2$, $C = 1\,\text{nF}$, $V_{DD} = 1.0\,\text{V}$ and $f = 1\,\text{GHz}$. Find its dynamic power.
2. Clock frequency and supply voltage both double. By what factor does dynamic power change?
3. Supply voltage rises 10% and frequency rises 20%. What is the percentage change in dynamic power?
4. A chip moves from frequency f at voltage $2V$ to frequency $4f$ at voltage $3V$. By what factor does dynamic power change?
5. Voltage and frequency are both reduced by 10%. What fraction of the original dynamic power remains?
6. A processor runs the same program twice as fast at the same voltage. What happens to its power, and to the energy the program uses?
7. Which kind of power does clock gating reduce? Which does power gating reduce?
8. Does leakage power rise or fall with temperature?
9. What is the activity factor of a clock net?
10. Compare one core at frequency f and $1.0\,\text{V}$ with two cores at $f/2$ and $0.6\,\text{V}$ each. What is the power ratio, ignoring overhead?
11. Why can't a designer simply AND a clock with an enable signal to gate it?

Answers

1. $0.2 \times 10^{-9} \times 1 \times 10^{9} = 0.2\,\text{W}$.
2. $2 \times 2^2 = 8$.
3. $1.1^2 \times 1.2 = 1.452$, an increase of about 45%.
4. $4 \times (3/2)^2 = 9$.
5. $0.9^2 \times 0.9 = 0.729$, about 73%.
6. Power doubles; energy for the program stays the same (energy per operation does not depend on frequency).
7. Clock gating: dynamic power. Power gating: leakage power (and the gated block's dynamic power).

8. Rise (exponentially).
9. 1.
10. $2 \times 0.6^2 \times 0.5 = 0.36$: about 36% of the original power.
11. If the enable changes while the clock is high, the AND output can produce a shortened pulse or glitch, which flip-flops would treat as a clock edge. A latch-based clock-gating cell allows the enable to change only while the clock is low.

Chapter 15

Process, Voltage and Temperature

No two chips are identical, and no chip runs in identical conditions all its life. Transistors come out of the factory a little faster or slower; the supply voltage sags under load; the temperature swings from a cold start to full load. A design must work across all of it. Engineers summarize these three sources of variation as **PVT**: process, voltage and temperature. This chapter explains each, finishes the temperature question from the assessment, and shows how designers decide which conditions to check.

15.1 Process variation

Manufacturing a chip involves printing features tens of nanometers wide and implanting dopant atoms by the thousands (Chapter 17). Nothing can be done perfectly at that scale, so transistor properties vary:

- **Global variation**: whole wafers or whole chips come out faster or slower than average (for example, a slightly thinner gate oxide everywhere on one wafer).
- **Local variation**: neighboring transistors on the same chip differ. With only a few dozen dopant atoms in a tiny transistor's channel, the random count alone changes its threshold voltage (**random dopant fluctuation**). The roughness of printed edges changes channel lengths.

Designers describe global variation with **process corners**, named by the speed of the NMOS and PMOS transistors:

Corner	Meaning
TT	typical NMOS, typical PMOS
FF	fast NMOS, fast PMOS
SS	slow NMOS, slow PMOS
FS	fast NMOS, slow PMOS
SF	slow NMOS, fast PMOS

The skewed corners (FS and SF) matter for circuits that depend on a balance between pull-up

and pull-down, such as an inverter's switching threshold or an SRAM cell's stability (Chapter 18). Local variation is handled statistically, with **Monte Carlo** simulation (running many simulations with randomly varied transistors) and with timing margins called on-chip variation (OCV) derates.

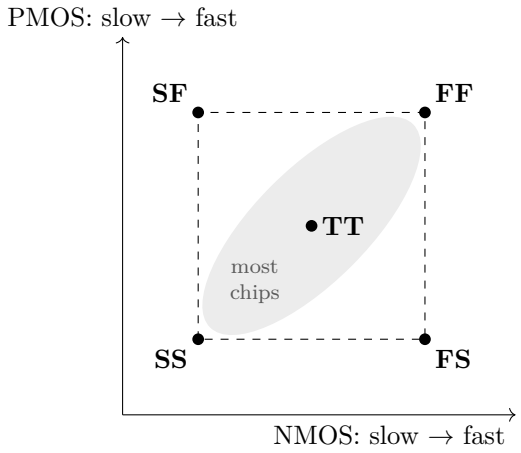


Figure 15.1: Process corners. Most chips land near typical (TT), and whole-wafer effects tend to make NMOS and PMOS fast or slow together (the shaded band). Sign-off checks the extremes: SS, FF, and the skewed corners FS and SF.

A practical consequence is **binning**: chips from the same wafer are tested and sorted by how fast they run, and faster chips are sold as higher-end products.

15.2 Voltage variation

The supply voltage a transistor actually sees is lower than the regulator's output and changes over time (Chapters 1 and 2):

- **Regulator tolerance**: the supply is specified as, say, $0.80\,\mathrm{V} \pm 3\%$.
- **Static IR drop**: steady current through the resistance of the power grid lowers the voltage, more so far from the power pins.
- **Dynamic droop**: sudden increases in current cause $L\,di/dt$ dips until decoupling capacitors and the regulator respond.

Lower voltage means less overdrive, so transistors are slower. Power-delivery design aims to keep the total drop within a budget, often a few percent.

15.3 Temperature

Chips must work across a wide range of temperatures, from a cold start in an unheated room (or a car in winter) to full load under a heat sink. Typical junction-temperature ranges are about 0 to 100 °C for consumer products, and −40 to 125 °C or more for automotive parts. Temperature changes several things at once:

Quantity	As temperature rises	Effect
Carrier mobility μ	decreases	transistors slower
Threshold voltage V_t	decreases	transistors faster
Metal wire resistance	increases	wires slower
Subthreshold leakage	increases exponentially	more power, more heat
Electromigration	accelerates	shorter wire lifetime

Why each happens

Mobility falls. Hot atoms vibrate more, so carriers collide with them more often (phonon scattering) and drift more slowly in the same electric field (Section 4.6).

Wire resistance rises. Metal electrons are scattered by the same vibrations. Copper's resistance increases by about 0.39% per °C.

Threshold voltage falls. Heat gives carriers more energy, so a smaller gate voltage is enough to form the channel. A typical rate is on the order of 1 mV per °C.

Seen on the assessment

The assessment asked how CMOS carrier mobility and wire resistance change as temperature increases. **Mobility decreases** and **wire resistance increases**; both make the circuit slower when hot. A complete answer also knows the exception below.

Worked example: a hot wire

A copper wire has a resistance of 100 Ω at 25 °C. What is its resistance at 85 °C?

$$R = 100 \times [1 + 0.0039 \times (85 - 25)] = 100 \times 1.234 \approx 123\,\Omega.$$

A 60 °C rise adds about 23% to the wire's resistance, and to its RC delay.

15.4 Temperature inversion

For a transistor, heat has two opposite effects: lower mobility (slower) and lower threshold (faster). Which wins depends on the supply voltage (Figure 15.2). Drive current is roughly

$$I \propto \mu(T)\,\big(V_{DD} - V_t(T)\big)^{\alpha}, \qquad \alpha \approx 1 \text{ to } 1.3.$$

Worked example: which effect wins? (illustrative numbers)

Suppose heating a chip from 25 °C to 100 °C lowers mobility by 25% and lowers V_t from 0.35 V to 0.30 V. Take $\alpha = 1.3$.

At $V_{DD} = 1.0$ V: overdrive goes from 0.65 to 0.70 V, up 7.7%; raised to 1.3, about +10%. Combined with the 25% mobility loss: $0.75 \times 1.10 \approx 0.83$. Current *falls*; the chip is slower when hot.
At $V_{DD} = 0.5$ V: overdrive goes from 0.15 to 0.20 V, up 33%; raised to 1.3, about +45%. Combined: $0.75 \times 1.45 \approx 1.09$. Current *rises*; the chip is *faster* when hot.

At high supply voltages the overdrive is large, so a small change in V_t hardly matters and the mobility loss dominates: hotter is slower. At low supply voltages, common in modern power-saving chips, the overdrive is small, so the V_t drop is a large fraction of it and wins: hotter is faster, and the worst-case speed is at the *coldest* temperature. This is **temperature inversion**.

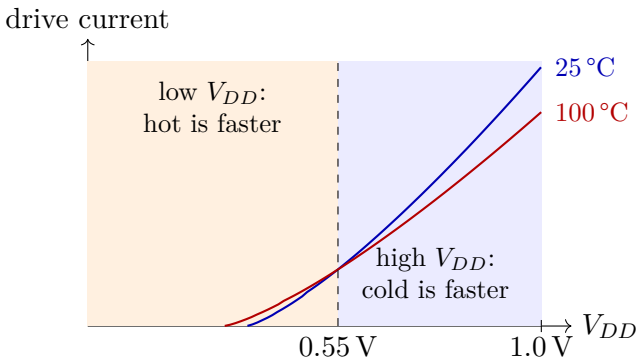


Figure 15.2: Temperature inversion, with the numbers of the worked example. Heat lowers V_t, so the hot curve starts earlier, but it also lowers mobility, so it rises more slowly. Below about 0.55 V the hot transistor is stronger; above it, the cold one is.

Common trap

True or false: "A chip is always slowest at its highest temperature." False for modern low-voltage designs, because of temperature inversion. Wires, however, are always slower when hot.

15.5 Corners for sign-off

Since no single condition is worst for everything, designers check a set of **PVT corners**:

Check	Worst-case corner
Setup (paths too slow)	slow process (SS), low voltage, and the temperature that makes cells slowest (hot at high voltage; cold at low voltage, because of inversion)
Hold (paths too fast)	fast process (FF), high voltage, and the temperature that makes cells fastest
Leakage power	fast process (FF), high voltage, high temperature
Electromigration	high temperature, high current

15.6 Aging

Chips also slow down slowly over their lifetimes. Mechanisms such as NBTI (in PMOS) and hot-carrier injection gradually raise threshold voltages (Chapter 17). Designers add an **aging guardband**: timing margin so that the chip still meets its frequency after years of use.

15.7 Living with variation

Modern chips measure their own conditions and adapt:

- On-die **thermal sensors** trigger throttling before the chip overheats (Chapter 3).
- **Ring oscillators** or replica critical paths measure how fast this particular chip is right now.
- **Adaptive voltage scaling** gives a fast chip a lower voltage (saving power) and a slow chip a higher one, so both meet the same frequency.
- **DVFS** picks voltage and frequency based on workload (Chapter 14).

15.8 Summary

- Process: chips vary; corners TT, FF, SS, FS, SF; local variation handled statistically.
- Voltage: regulator tolerance, IR drop and $L\,di/dt$ droop lower the supply; lower voltage is slower.
- Temperature: mobility down, V_t down, wire resistance up, leakage up.
- At high V_{DD} hot is slower; at low V_{DD} hot can be faster (temperature inversion). Wires are always slower when hot.
- Setup is signed off at the slow corner, hold at the fast corner, leakage at fast, high-voltage, hot.

15.9 Exercises

1. As temperature increases, what happens to carrier mobility, threshold voltage, wire resistance and leakage?
2. A copper wire is $100\,\Omega$ at $25\,^{\circ}\mathrm{C}$. Estimate its resistance at $85\,^{\circ}\mathrm{C}$.
3. True or false: every chip is slowest at its maximum temperature.
4. At which corner is setup checked? Hold? Worst-case leakage?
5. A $0.80\,\mathrm{V}$ supply suffers 5% IR drop. What voltage do the transistors see, and are they faster or slower?
6. What does the FS corner mean? Name a circuit property it might affect.
7. Why is Monte Carlo simulation used?

Answers

1. Mobility decreases; threshold voltage decreases; wire resistance increases; leakage increases (exponentially).
2. About $100 \times (1 + 0.0039 \times 60) \approx 123\,\Omega$.
3. False. At low supply voltage, temperature inversion makes cold the slow case.
4. Setup: slow process, low voltage (and the slowest temperature). Hold: fast process, high voltage (and the fastest temperature). Leakage: fast process, high voltage, high temperature.
5. 0.76 V. Slower, because there is less overdrive.
6. Fast NMOS, slow PMOS. It skews any balance between pull-up and pull-down, for example moving an inverter's switching threshold down, or affecting SRAM read and write margins.
7. To estimate the effect of random local variation, by simulating many randomly varied copies of a circuit and looking at the spread of results.

Part VI

Building Real Chips

Chapter 16

Wires, Crosstalk and Signals Between Chips

Transistors get the attention, but a modern chip contains tens of kilometers of wire, stacked in a dozen or more layers. Wires set much of a chip's speed and power, and they interfere with each other. This chapter covers on-chip wiring, crosstalk, how signals travel between chips, and the bandwidth calculations that come up in assessment questions.

16.1 The metal stack

Above the transistors sits a stack of metal layers separated by insulating material (Figure 16.1). Vertical connections between layers are called **vias**.

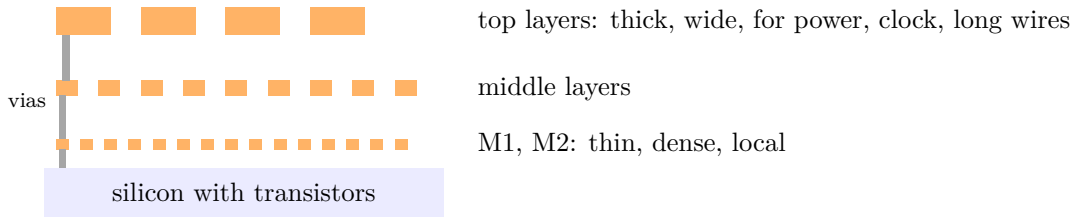


Figure 16.1: Wiring layers above the transistors (not to scale). Lower layers are thin and closely packed; upper layers are thick and widely spaced.

- The **lowest layers** are thin and densely packed, for short connections between nearby transistors.
- The **highest layers** are thick and wide. They carry power, ground, the clock, and long signals that cross the chip, because thick wires have much lower resistance.
- Modern chips use **copper** wires (lower resistivity than aluminum, and more resistant to electromigration), embedded in **low-k** insulators that reduce capacitance between wires.

16.2 Wire resistance

A wire's resistance is $R = \rho L/(WT)$, with length L, width W and thickness T. Since all wires on one layer have the same thickness, designers use the layer's **sheet resistance** $R_\square = \rho/T$, in ohms per square. A wire's resistance is then its sheet resistance times the number of *squares* it contains, length divided by width:

$$R = R_\square \cdot \frac{L}{W}.$$

(A square of any size has the same resistance: a bigger square is longer, but proportionally wider.)

Worked example: resistance of a thin wire

A wire on a lower layer is 1 mm long and 50 nm wide, with a sheet resistance of $0.2\,\Omega/\square$. Squares: $10^{-3}/(50 \times 10^{-9}) = 20{,}000$. Resistance: $0.2 \times 20{,}000 = 4\,\text{k}\Omega$, comparable to a transistor's resistance. Long thin wires are slow.

At these tiny dimensions, copper's effective resistivity is also higher than in bulk copper, because electrons scatter off the wire's surfaces and grain boundaries, and part of the cross-section is taken up by a barrier liner. Wire resistance is one of the hardest problems in advanced chips.

16.3 Wire capacitance and crosstalk

A wire has capacitance to the layers above and below (effectively to ground) and to its *neighbors* on the same layer. In dense wiring, neighbor capacitance, called **coupling capacitance**, is often the largest part. Coupling causes **crosstalk**: a switching wire (the **aggressor**) disturbs a nearby wire (the **victim**), as in Figure 16.2.

Noise on a quiet victim. If the victim is supposed to hold steady, a switching aggressor pushes charge onto it through the coupling capacitance C_c. If the victim's driver did not fight back at all, the victim's voltage would jump by

$$\Delta V \approx V_{DD}\,\frac{C_c}{C_c + C_g},$$

where C_g is the victim's capacitance to ground. A real driver pulls it back, so the result is a glitch. A large enough glitch can flip a latch or a dynamic node.

Worked example: size of a crosstalk glitch

$C_c = 10\,\text{fF}$, $C_g = 30\,\text{fF}$, $V_{DD} = 0.8\,\text{V}$. Worst case: $\Delta V \approx 0.8 \times 10/40 = 0.2\,\text{V}$, a quarter of the supply.

Delay change on a switching victim. If both wires switch at once:

- In **opposite** directions, the voltage across C_c changes by $2V_{DD}$, so it acts like $2C_c$ (the Miller effect, Chapter 13): the victim is *slower*.
- In the **same** direction, the voltage across C_c barely changes, so it acts like almost nothing: the victim is *faster*.

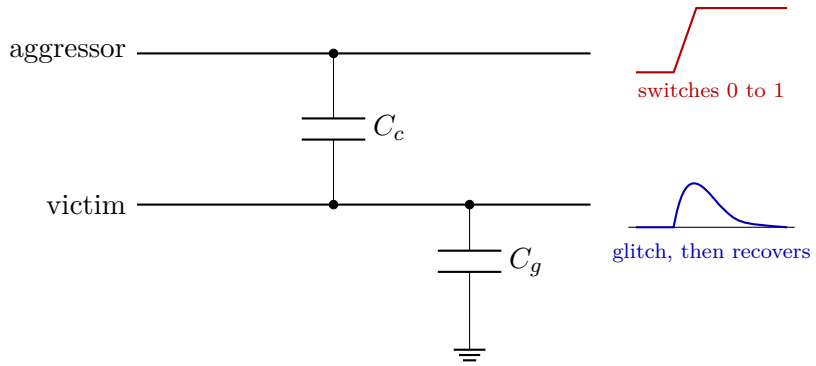


Figure 16.2: Crosstalk. When the aggressor switches, charge couples through C_c onto the quiet victim and makes a glitch; the victim's driver then pulls it back. The bigger C_c is compared with C_g, the bigger the glitch.

Switching windows. Crosstalk only matters if the aggressor switches while the victim is vulnerable. Each net has a **switching window**: the interval between the earliest and latest times it can change, found by adding up the shortest and longest delays along the paths that drive it. Two coupled nets can interact only where their windows overlap.

Worked example: do these nets interact?

Net A can switch anywhere from 1.0 ns to 1.6 ns after the clock edge. Net B, which runs next to it, can switch from 1.4 ns to 2.2 ns. Their windows overlap from 1.4 ns to 1.6 ns, so crosstalk is possible only in that interval. If B's window were 1.8 to 2.2 ns, there would be no overlap and no timing crosstalk.

Seen on the assessment

A reported question gives two adjacent nets driven by gates with known delays and asks during what window crosstalk might occur. Compute each net's earliest and latest switching times from its gate delays, then intersect the two intervals.

Fixes for crosstalk. Increase the spacing between wires; put a grounded **shield** wire between sensitive wires; route neighbors on different layers or perpendicular to each other; make the victim's driver stronger; or insert buffers to shorten the coupled length.

16.4 Signals between chips

Off the chip, signals travel through package pins, circuit-board traces and connectors. At modern speeds these are transmission lines (Section 2.9): signals travel as waves, reflections must be prevented by matching impedances and terminating lines, and losses in the board material smear fast edges.

Differential signaling. Most fast links send each signal on *two* wires carrying opposite voltages, and the receiver looks only at the *difference*. Noise that hits both wires equally (common-mode

noise) cancels out. Differential links can therefore use small voltage swings, which saves power and allows very high speeds. PCIe, USB, HDMI, Ethernet and NVIDIA's NVLink all use differential pairs.

SerDes. Instead of many slow parallel wires that must stay perfectly aligned in time, high-speed links use a few very fast serial lanes. A **serializer/deserializer** (SerDes) converts wide parallel data into a fast bit stream and back. The receiver recovers the clock from the data's transitions, so the data is **encoded** to guarantee enough transitions: 8b/10b encoding (8 data bits sent as 10, 80% efficient) or 128b/130b (about 98.5% efficient).

Eye diagrams. Engineers check link quality by overlaying many bit periods of the received signal. A clean link shows a wide-open "eye" between the 0 and 1 levels; noise, jitter and reflections close it.

16.5 Bandwidth calculations

Bandwidth questions are mostly careful unit-keeping:

$$\text{bandwidth (bytes/s)} = \frac{\text{bus width (bits)} \times \text{transfers per second}}{8} \times \text{efficiency}.$$

- **DDR** (double data rate) transfers data on both the rising and falling clock edges: two transfers per clock cycle.
- Link speeds are often given in **GT/s** (giga-transfers per second), which for a single lane equals Gb/s before encoding overhead.
- Divide bits by 8 for bytes. Apply encoding efficiency and protocol overhead (packet headers) if the question includes them.

Worked example: DDR bandwidth

An 8-bit DDR interface has a 10 GHz clock. Transfers per second: 2×10^{10}. Bandwidth: $8 \times 2 \times 10^{10}/8 = 2 \times 10^{10}$ bytes/s $= 20$ GB/s.
If instead "10 GHz" were the data rate (10 GT/s, already counting both edges), the answer would be 10 GB/s. Read carefully which one a question means.

Worked example: memory and link bandwidth

HBM: one stack has a 1024-bit interface at 6.4 Gb/s per pin: $1024 \times 6.4/8 \approx 819$ GB/s.
PCIe: a 16-lane link at 16 GT/s per lane with 128b/130b encoding: $16 \times 16 \times (128/130)/8 \approx 31.5$ GB/s in each direction.
Protocol overhead: if every 256-byte payload travels with 24 bytes of headers, only $256/280 \approx 91\%$ of the link carries data, so the payload rate is 91% of the raw rate.

16.6 Packets and address alignment

Links such as PCIe move data in **packets** with a maximum payload size. Memory systems also prefer requests that do not cross certain **aligned** boundaries (addresses that are multiples of 64 bytes, for example). A large read is therefore split into several reply packets, and the first packet may be short so that later packets start on aligned boundaries.

Worked example: how many reply packets?

A device reads 202 bytes starting at address `0x31121133`. Replies carry at most 64 bytes and must not cross a 64-byte-aligned boundary.
The address's offset within its 64-byte block is its low 6 bits: `0x33` = 51. The first packet can carry only up to the next boundary: $64 - 51 = 13$ bytes. Remaining: $202 - 13 = 189$ bytes, which takes $\lceil 189/64 \rceil = 3$ more packets (64, 64 and 61 bytes). Total: **4 packets**.
Here the simple estimate $\lceil 202/64 \rceil = 4$ happens to agree, but not always. A 128-byte read starting at offset 32 needs *three* packets (32, 64, 32), not two.

Seen on the assessment

A reported question asked for the number of completion packets for a 202-byte read at address `0x31121133` with a 64-byte maximum payload. The method above, with alignment, gives 4.

16.7 Signal integrity on the board

A board trace carrying a fast signal is a transmission line (Section 2.9), and **signal integrity** is the work of getting a clean edge from one chip to another.

Controlled impedance and reflections. A wave on a line sees the line's **characteristic impedance** Z_0, set by the trace's width, its height above the ground plane and the board material, not by its length. Boards are built to a target, typically $50\,\Omega$ for a single-ended trace and $100\,\Omega$ across a differential pair. Wherever the impedance changes (a connector, a via, a load that does not match), part of the wave reflects. The fraction reflected is the reflection coefficient

$$\Gamma = \frac{Z_L - Z_0}{Z_L + Z_0}.$$

A matched termination ($Z_L = Z_0$) gives $\Gamma = 0$; an open end gives $\Gamma = +1$ (the whole wave returns); a short gives $\Gamma = -1$.

Worked example: a mismatched load

A $50\,\Omega$ trace ends in a $75\,\Omega$ load. $\Gamma = (75 - 50)/(75 + 50) = 0.2$, so 20% of each edge's voltage reflects back toward the driver, where it can reflect again and appear as ringing.

Losses and equalization. Copper resistance (worse at high frequency, where current crowds to the surface) and the board's dielectric absorb more of the high-frequency content than the

low. A sharp edge arrives rounded, and each bit's energy leaks into the next bits (**inter-symbol interference**). Links fight this with **equalization**: the transmitter boosts each transition (**pre-emphasis** or de-emphasis), and the receiver applies continuous-time linear equalization (CTLE, a high-frequency boost) and a decision-feedback equalizer (DFE), which subtracts the expected leftover of bits already received.

Via stubs and crosstalk. A via that goes through the whole board but is used only between two upper layers leaves an unused stub hanging below. The stub is a short open-ended line that resonates, notching out frequencies near where it is a quarter wavelength long; fast boards **back-drill** stubs away. Neighboring traces couple like on-chip wires, so fast traces are spaced apart, routed over a solid ground plane, and never cross a split in it, because the return current flows in the plane right under the trace (Chapter 1 explains why).

Recovering the clock. A SerDes receiver has no separate clock wire. A **clock and data recovery** (CDR) circuit, a kind of PLL, locks onto the transitions in the data and places the sampling point in the middle of each bit, the center of the eye. That is why the data must be encoded to guarantee transitions. The **eye diagram** measures what is left: its height is the voltage margin and its width the timing margin, and a closing eye means loss, reflections, crosstalk or jitter are eating the margin.

Power integrity. The supply network has to hold its voltage while the chip's current jumps. Its design target is an impedance: if the current can step by ΔI and the supply may move by at most ΔV, the network's impedance must stay below

$$Z_{\text{target}} = \frac{\Delta V}{\Delta I}$$

at every frequency up to the fastest current change. A 0.8 V rail with a 3% tolerance (24 mV) and a 60 A step needs $Z \leq 0.4\,\text{m}\Omega$. Voltage regulators handle slow changes, bulk and ceramic capacitors on the board middle frequencies, and capacitors in the package and on the die the fastest ones (the decoupling capacitors of Chapter 2).

16.8 Networking: Ethernet, IP and UDP

Network hardware, from switches to SmartNICs to trading FPGAs, handles data as **packets** wrapped in layers of headers. Each layer adds a header in front and is handled by its own piece of logic. In a hardware datapath the bytes arrive a word at a time, and the design must find each field by its byte offset.

The layers. From the outside in:

- **Ethernet** (layer 2) moves a frame between two directly connected devices. It is identified by 48-bit **MAC addresses**.
- **IP** (layer 3) carries a packet across networks, from a 32-bit IPv4 source address to a destination address.

- **UDP** or **TCP** (layer 4) delivers the data to a numbered **port** on the destination machine.
- The application's own message is inside, for example a market-data update saying a price changed.

Header	Bytes	Main fields, in order
Ethernet	14	destination MAC (6), source MAC (6), EtherType (2; `0x0800` means IPv4)
IPv4	20	version and header length, total length, protocol (17 = UDP, 6 = TCP), header checksum, source IP (4), destination IP (4)
UDP	8	source port (2), destination port (2), length (2), checksum (2)

On the wire, the frame is preceded by a 7-byte preamble and a 1-byte start-of-frame delimiter, and followed by a 4-byte **frame check sequence** (FCS), a CRC-32 over the frame that lets the receiver drop corrupted frames. Between frames the line stays idle for at least 12 byte times (the inter-packet gap). The smallest legal frame is 64 bytes including the FCS, so short messages are padded.

Worked example: where is the UDP payload?

With no VLAN tag and no IP options, the UDP payload starts at byte $14 + 20 + 8 = 42$ of the frame (counting from the destination MAC). On a datapath that delivers 8 bytes per clock, bytes 0–7 arrive in the first cycle, so byte 42 is in the sixth word (bytes 40–47), at byte position 2 within it. A parser that knows this can start decoding the message in that cycle, before the rest of the frame has even arrived; this is **cut-through** processing, the basis of low-latency designs.

Byte order. Network headers are **big-endian** ("network byte order"): the most significant byte of a multi-byte field comes first. EtherType `0x0800` arrives as `0x08` then `0x00`. A little-endian CPU must swap bytes to read it (Section 20.10); hardware just wires the bytes in the right order.

UDP versus TCP. UDP sends independent datagrams with no connection, no acknowledgment and no retransmission: a lost packet is simply lost. That makes it simple and fast, and it supports **multicast**, one sender to many receivers, which is how exchanges publish market data. TCP provides a reliable, ordered byte stream: it numbers bytes, acknowledges them, retransmits losses and controls its sending rate. All that state makes TCP far harder to build in hardware, so it appears in FPGA designs only as dedicated TCP-offload engines, used for example to send orders to an exchange.

Rates and latency. A 10 Gb/s Ethernet link moves 1.25 bytes per nanosecond. It is usually handled 64 bits at a time at 156.25 MHz, the clock of the standard 64b/66b-encoded 10G interface; 25 Gb/s uses the same width at about 390 MHz. Just receiving a 64-byte frame takes $64 \times 8/10 = 51.2$ ns at 10 Gb/s, so a design that waits for the whole frame and its CRC before acting has already lost that time. Low-latency designs start acting on the early fields and discard the result if the CRC later fails.

Remember

Ethernet 14 + IPv4 20 + UDP 8 = 42 bytes of headers, all big-endian. UDP: no connection, no retransmission, multicast; TCP: reliable stream with much more state. At 10 Gb/s a byte takes 0.8 ns.

16.9 Summary

- Lower metal layers are thin and dense; upper layers are thick, for power, clock and long wires.
- $R = R_{\square} \cdot L/W$; very thin wires are surprisingly resistive.
- Coupling capacitance causes crosstalk: glitches on quiet victims ($\Delta V \approx V_{DD} C_c/(C_c + C_g)$) and delay changes on switching ones (opposite directions slower, same direction faster).
- Crosstalk needs overlapping switching windows.
- Fast chip-to-chip links are differential, serial (SerDes) and encoded.
- Bandwidth = width × transfer rate /8× efficiency; DDR doubles transfers per clock.
- Reads are split into packets at the maximum payload size and at aligned boundaries.
- Board traces have a characteristic impedance (50 Ω single-ended, 100 Ω differential). Mismatches reflect $\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$. Equalization undoes frequency-dependent loss; CDR recovers the clock; $Z_{\text{target}} = \Delta V/\Delta I$.
- Packets: Ethernet (14 bytes) + IPv4 (20) + UDP (8) = 42 header bytes, big-endian. UDP is connectionless and supports multicast; TCP is a reliable stream. At 10 Gb/s one byte takes 0.8 ns.

16.10 Exercises

1. A wire is 200 μm long and 0.1 μm wide, on a layer with $R_{\square} = 0.1\,\Omega/\square$. What is its resistance?
2. $C_c = 5$ fF, $C_g = 15$ fF, $V_{DD} = 0.8$ V. Estimate the worst-case crosstalk glitch.
3. Net A switches between 0.2 and 0.5 ns; net B between 0.6 and 0.9 ns. Can they cause timing crosstalk?
4. How does a neighbor switching in the opposite direction affect a victim's delay? In the same direction?
5. A 32-bit DDR bus runs at a 1.6 GHz clock. What is its bandwidth?
6. A PCIe link has 8 lanes at 8 GT/s with 128b/130b encoding. What is its bandwidth per direction?
7. A device reads 100 bytes starting at address `0x1030`, with 64-byte maximum payloads that cannot cross 64-byte boundaries. How many packets?
8. Why does differential signaling resist noise?
9. A 50 Ω line drives a 150 Ω load. What fraction of the voltage reflects?

10. A 1.0 V rail may move 2% when the current steps by 40 A. What is the target impedance?
11. With no VLAN tag and no IP options, at what byte offset does a UDP payload start in an Ethernet frame?
12. Why do exchanges publish market data over UDP multicast rather than TCP?

Answers

1. 2000 squares $\times\, 0.1 = 200\,\Omega$.
2. $0.8 \times 5/20 = 0.2\,\text{V}$.
3. No; their windows do not overlap.
4. Opposite: slower (the coupling acts like double the capacitance). Same: faster.
5. $32 \times 3.2 \times 10^9/8 = 12.8\,\text{GB/s}$.
6. $8 \times 8 \times (128/130)/8 \approx 7.9\,\text{GB/s}$.
7. Offset of `0x1030` in its 64-byte block is `0x30` $= 48$, so the first packet carries 16 bytes; the remaining 84 bytes need 2 more (64 and 20). Total 3 packets.
8. Noise usually couples equally onto both wires of the pair, and the receiver only looks at the difference between them, so the common noise cancels.
9. $\Gamma = (150 - 50)/(150 + 50) = 0.5$.
10. $20\,\text{mV}/40\,\text{A} = 0.5\,\text{m}\Omega$.
11. $14 + 20 + 8 = 42$.
12. One multicast packet reaches every subscriber at once with no per-receiver connection, acknowledgment or retransmission delay; TCP would need a separate connection per receiver and stalls to retransmit lost data.

Chapter 17

Manufacturing and Reliability

A design that is logically perfect can still fail: it might be damaged while it is being manufactured, wear out after a year in the field, or be destroyed by a spark of static electricity. This chapter explains how chips are made and the physical rules designers follow to keep them alive, including the antenna rule from the assessment.

17.1 From sand to wafer

Chips start as extremely pure silicon, grown into a single crystal cylinder (an ingot) and sliced into thin, polished discs called **wafers**, today mostly 300 mm across. Hundreds of chips (**dies**) are built side by side on each wafer.

Manufacturing has two halves:

- the **front end of line** (FEOL) builds the transistors in and on the silicon;
- the **back end of line** (BEOL) builds the stack of metal wiring layers above them, one layer at a time from the bottom up (Chapter 16).

17.2 Photolithography: printing the patterns

Every layer's pattern is printed using light, much like developing a photograph (Figure 17.1):

1. Coat the wafer with a light-sensitive film, **photoresist**.
2. Shine light through a **mask** (or reticle) carrying the layer's pattern, so only some of the resist is exposed.
3. **Develop** the resist, washing away the exposed (or unexposed) parts.
4. **Etch** the material not protected by resist, or implant dopants into it.
5. Strip the remaining resist, and repeat for the next layer.

A modern chip needs dozens of masks and hundreds of process steps. Features are now far smaller than the wavelength of the light used to print them (193 nm deep ultraviolet, or 13.5 nm extreme

ultraviolet, EUV), which requires tricks such as **optical proximity correction** (deliberately distorting mask shapes so they print correctly) and **multi-patterning** (printing one layer with several masks).

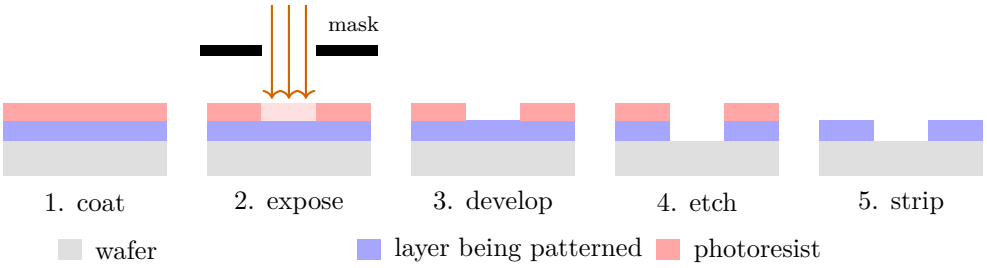


Figure 17.1: Photolithography in five steps. Light through the mask changes the exposed photoresist, which is then washed away; the etch removes the layer only where the resist is gone, copying the mask's pattern into the layer.

17.3 Other process steps

- **Deposition**: growing or depositing thin films of insulator or metal, sometimes one atomic layer at a time.
- **Etching**: removing material, usually with a reactive **plasma** (an electrically charged gas). Plasma etching is why the antenna rule exists.
- **Ion implantation**: firing dopant atoms into the silicon to make n-type and p-type regions, followed by **annealing** (heating) to repair the crystal.
- **Chemical–mechanical polishing** (CMP): grinding each layer perfectly flat before the next. Polishing works evenly only if the density of metal is roughly uniform, so tools add **dummy fill**: small unconnected metal shapes in empty areas.

17.4 Test, dicing and packaging

Each die on the finished wafer is tested with fine probes. The wafer is cut (diced) into individual dies, and good dies are placed in a **package** that connects them to the circuit board, spreads heat and protects them. High-performance chips are mounted face down (**flip-chip**) on thousands of tiny solder bumps.

Advanced packaging now matters as much as transistors. Large GPUs place the processor die and stacks of high-bandwidth memory (HBM) side by side on a silicon **interposer** full of fine wires (2.5D packaging), and some products stack dies directly on top of each other (3D). Splitting a design into several smaller dies, **chiplets**, connected in one package, helps yield, as the next section shows.

17.5 Yield

Tiny defects (a dust particle, a flaw in a film) land randomly across a wafer (Figure 17.2). A die with a defect in a critical spot does not work. If defects land at an average density D per unit area,

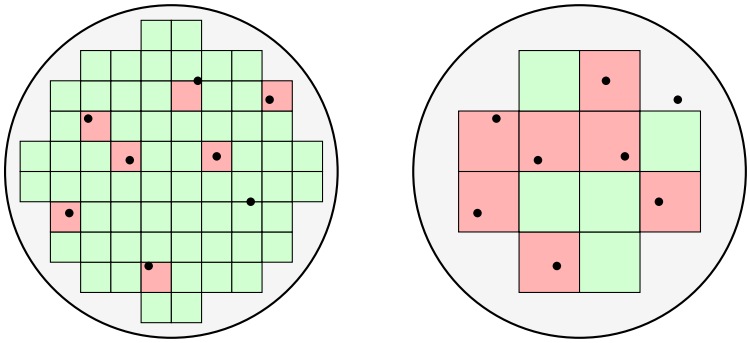
the fraction of dies of area A that escape them all is approximately

$$Y = e^{-DA}.$$

Worked example: yield and die size

With $D = 0.1$ defects per cm^2: a 1 cm^2 die has $Y = e^{-0.1} \approx 90\%$. An 8 cm^2 die has $Y = e^{-0.8} \approx 45\%$.

Large dies are doubly expensive: fewer fit on a wafer, and a smaller fraction of them work. Designers fight back with **redundancy** (spare memory rows and columns that can replace defective ones) and by **disabling** defective units (a GPU with one broken processing cluster can be sold as a lower-tier product with that cluster turned off).



small dies: 61 of 68 good (90%) large dies: 5 of 12 good (42%)

Figure 17.2: The same eight defects (dots) on two wafers. With small dies, each defect ruins only a small die and most dies work; with dies of four times the area, the same defects ruin a much larger fraction. Failed dies are red.

17.6 The design flow

The path from an idea to a manufacturable layout (Figure 17.3):

1. Specification and architecture.
2. RTL design in a hardware description language, and functional verification (Chapter 19).
3. **Synthesis**: tools turn RTL into a netlist of standard cells (gates and flip-flops from a library).
4. **Floorplanning**: placing large blocks, memories, I/O and the power grid.
5. **Placement** of the standard cells.
6. **Clock tree synthesis**.
7. **Routing**: connecting everything with metal.
8. **Sign-off**: static timing analysis, power and IR-drop analysis, and the physical checks below.
9. **Tape-out**: sending the final layout (a GDSII or OASIS file) to the factory.

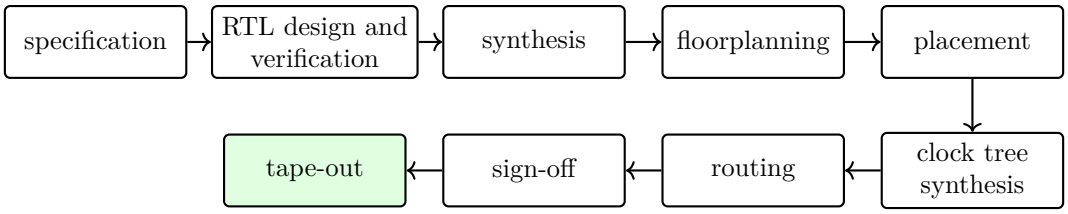


Figure 17.3: The design flow, from an idea to a layout the factory can build.

Key physical checks:

- **DRC** (design rule check): every shape obeys the factory's geometric rules: minimum widths, spacings, enclosures, densities.
- **LVS** (layout versus schematic): the transistors and connections in the layout match the intended netlist.
- **Antenna**, **electromigration** and **ERC** (electrical rule) checks, described next.

17.7 The antenna effect

During manufacturing, metal layers are patterned by plasma etching. The plasma is full of charged particles, and any metal wire exposed to it collects charge, like an antenna collecting a radio signal. The longer (larger in area) the wire, the more charge it collects.

Normally that charge would drain away harmlessly through a transistor's source or drain diffusion, which connects to the silicon. But consider a wire, being built on a lower metal layer, whose only connection so far is to the *gate* of a transistor (Figure 17.4). The connection to the driving gate's output, which would provide a diffusion path, may only be completed later, on a higher metal layer that does not exist yet. The collected charge has nowhere to go except *through the thin gate oxide*. Enough charge forced through the oxide damages it: it shifts the transistor's threshold voltage, makes the oxide leaky, or breaks it down completely.

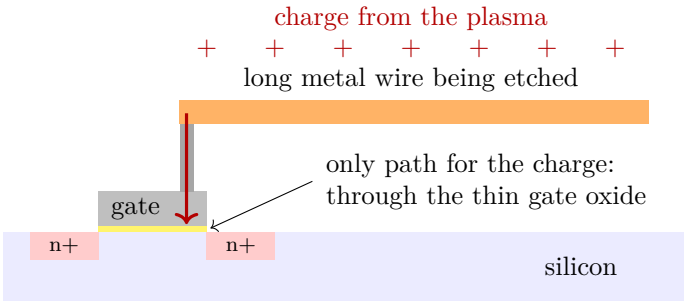


Figure 17.4: The antenna effect: a long wire connected only to a gate funnels plasma charge through the gate oxide.

The antenna rule. The danger depends on how much charge is collected compared with how much oxide area it is spread over. So the rule limits the **antenna ratio**:

$$\text{antenna ratio} = \frac{\text{area of metal connected to the gate (during that etch)}}{\text{area of the gate oxide it connects to}}.$$

The factory gives a maximum ratio for each layer. Notice what each side means:

- More metal (a longer wire) collects more charge: **worse**.
- More gate area spreads the charge over more oxide: **better**.

Seen on the assessment

The assessment asked, true or false: "For the antenna rule, as long as the metal wire is safe, a small connected transistor is good." This is **false**. The wire is not "safe" or "unsafe" on its own; what matters is the *ratio* of metal area to gate area. A *smaller* gate means a *larger* ratio, so a small connected transistor makes an antenna violation *more* likely. A larger gate (or several gates) on the same wire is safer.

Fixing antenna violations.

- **Antenna diode**: add a small diode from the wire to the substrate near the gate. During etching it gives the charge a safe path to the silicon. In normal operation the diode is reverse biased, so it only adds a little leakage and capacitance (Chapter 4).
- **Metal jumper (layer hopping)**: break the long wire and route part of it on a higher layer. While the lower layer is etched, the gate sees only a short piece of metal; by the time the higher layer is etched, the connection to the driver's diffusion is complete and the charge can drain.
- **Buffer insertion**: insert a buffer near the vulnerable gate, splitting the long wire into two nets, each with a smaller ratio.
- **Larger gate area**: a bigger receiving transistor lowers the ratio (at a cost in power and loading).

17.8 Electromigration

Electrons flowing through a wire collide with its metal atoms and, very slowly, push them along in the direction of electron flow. Over months and years this **electromigration** (EM) leaves **voids** where atoms have left (which can break the wire open) and **hillocks** where they pile up (which can short to a neighbor).

The median time to failure follows **Black's equation**:

$$\text{MTTF} = A\, J^{-n}\, e^{E_a/kT},$$

where J is the current density, n is typically around 2, E_a is an activation energy, and T is absolute temperature. Two lessons: higher current density shortens life sharply, and higher temperature shortens it *exponentially*.

Wires that carry steady current in one direction (power and ground) are most at risk; signal wires, whose current flows back and forth, are less so. Designers prevent EM failures by limiting current density: wider wires, more parallel wires, and multiple vias at every connection. Copper's better EM resistance was one reason it replaced aluminum.

17.9 Latch-up

A CMOS chip accidentally contains a parasitic structure that can short the supply to ground. Follow the layers from V_{DD} to ground through a PMOS and a nearby NMOS: the PMOS's p+ source, the n-well, the p-type substrate, and the NMOS's n+ source. That is p-n-p-n, a four-layer device called a **thyristor** (or SCR), which behaves like two bipolar transistors feeding each other.

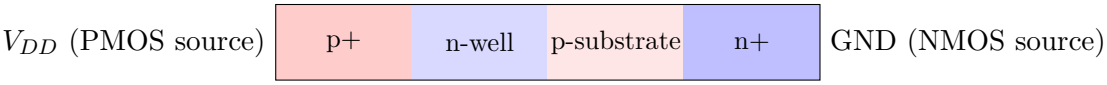


Figure 17.5: The parasitic p-n-p-n path inside CMOS. Once triggered, it can latch into a low-resistance state between the supply and ground.

Normally this structure is off. But a disturbance that injects current into the well or substrate, such as a voltage spike on an input pin, an ESD event or a radiation strike, can turn both parasitic transistors on. Each then keeps the other on: the structure *latches*, conducting heavily from V_{DD} to ground until power is removed, often destroying the chip.

Prevention. Keep the well and substrate firmly tied to their supplies with frequent **well taps** and **substrate taps** (low resistance there keeps the parasitic transistors from turning on); surround sensitive or I/O transistors with **guard rings**; keep NMOS and PMOS devices apart; or use processes that isolate transistors completely (such as silicon-on-insulator).

17.10 Electrostatic discharge

Walking across a carpet can charge your body to thousands of volts. Touching a chip's pin discharges that through the chip in nanoseconds. Gate oxides break down at only a few volts, so an unprotected input would be destroyed instantly. Every I/O pad therefore has **ESD protection**: diodes from the pad to V_{DD} and to ground, plus clamp circuits between the supplies, which steer the discharge current around the core circuitry. ESD structures add capacitance to every pin, which is a challenge for very high-speed I/O.

17.11 Aging and soft errors

Wear-out. Transistors degrade slowly under voltage and heat:

- **NBTI** (negative bias temperature instability): PMOS thresholds rise over time while the transistor is on.
- **HCI** (hot-carrier injection): fast-moving carriers get trapped in the gate oxide, degrading the transistor.

- **TDDB** (time-dependent dielectric breakdown): the gate oxide eventually wears out under a strong electric field.

Designs include an **aging guardband** in their timing so they still work after years.

Soft errors. A neutron from cosmic rays, or an alpha particle from trace radioactive impurities, can deposit enough charge to flip a bit in an SRAM cell or a flip-flop without damaging anything. These **soft errors** are handled with parity and error-correcting codes (Chapter 18).

17.12 Testing manufactured chips

Factory testing looks for manufacturing defects, not design bugs. The standard model is the **stuck-at fault**: a node permanently stuck at 0 or 1. Scan chains (Chapter 10 and Figure 17.6) let testers load every flip-flop with a pattern, run one clock cycle, and read every result back out. Software (**ATPG**, automatic test pattern generation) finds a small set of patterns that detect nearly all possible stuck-at faults. Memories have built-in self-test (**BIST**) circuits that run test patterns on themselves.

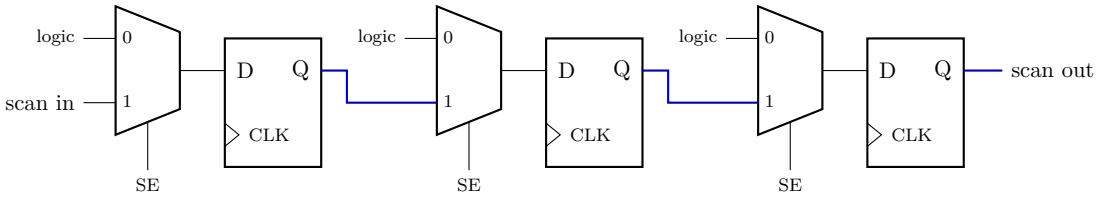


Figure 17.6: A scan chain. Each flip-flop has a mux in front. In normal operation (SE = 0) it captures its logic input; in test mode (SE = 1) the flip-flops form one long shift register (blue), so a tester can shift a pattern in and the results out. Clock wires are omitted.

JTAG and boundary scan. IEEE 1149.1, known as **JTAG**, defines a small test port found on nearly every complex chip: a test clock (TCK), a mode select (TMS), serial data in and out (TDI, TDO) and an optional reset. TMS steers a 16-state controller (the TAP controller) that selects which internal register TDI shifts into. The best-known register is the **boundary-scan** chain: a scan cell beside every I/O pin, linked into one shift register, so a tester can set and read every pin of every chip on an assembled board without touching it with probes. That finds open and shorted solder joints between chips, the board-level equivalent of stuck-at faults. The same port is how FPGAs are programmed during development and how debuggers connect to processors.

17.13 Summary

- Chips are printed layer by layer with photolithography; transistors first (FEOL), then wires (BEOL).
- Yield falls exponentially with die area ($Y = e^{-DA}$), motivating redundancy and chiplets.
- DRC checks shapes; LVS checks that the layout matches the netlist.

- Antenna effect: plasma charge on a wire connected only to a gate flows through the gate oxide. The rule limits metal area / gate area; a small gate is *worse*. Fixes: diodes, layer jumpers, buffers, larger gates.
- Electromigration: current density and heat move metal atoms; MTTF $\propto J^{-n}e^{E_a/kT}$.
- Latch-up: a parasitic p-n-p-n thyristor shorts V_{DD} to ground; prevented by taps and guard rings.
- ESD protection diodes guard every pin.
- Aging (NBTI, HCI, TDDB) needs guardbands; soft errors need ECC.
- JTAG boundary scan shifts patterns through a cell at every pin to test the connections between chips on a board; the same port programs FPGAs and connects debuggers.

17.14 Exercises

1. True or false: for the antenna rule, as long as the metal wire is safe, a small connected transistor is good. Explain.
2. Give two ways to fix an antenna violation.
3. With a defect density of 0.2 per cm^2, what is the yield of a 2 cm^2 die?
4. Does electromigration get better or worse at higher temperature?
5. What causes latch-up, and name two ways to prevent it.
6. What is the difference between DRC and LVS?
7. Why do layout tools add dummy metal fill?
8. What protects a chip's input pins from static electricity?
9. Why does doubling a die's area more than double its cost per working die?

Answers

1. False. The risk depends on the ratio of metal area to gate area. A smaller gate gives a larger ratio, so the charge collected by the wire is concentrated on less oxide: more likely to cause damage.
2. Any two of: an antenna diode near the gate; breaking the wire and jumping to a higher metal layer; inserting a buffer near the gate; increasing the gate area.
3. $e^{-0.4} \approx 67\%$.
4. Worse; lifetime falls exponentially as temperature rises.
5. A parasitic p-n-p-n structure (two coupled parasitic bipolar transistors) between the PMOS and NMOS can be triggered by injected current and then holds a low-resistance path from V_{DD} to ground. Prevent it with frequent well and substrate taps, guard rings, spacing, or isolated processes such as SOI.
6. DRC checks that layout shapes obey the factory's geometric rules. LVS checks that the circuit extracted from the layout matches the intended schematic or netlist.

7. To make the metal density uniform so chemical-mechanical polishing produces flat layers.
8. ESD protection: diodes from each pad to the supply and ground, plus supply clamps, that divert the discharge current.
9. Fewer dies fit on each wafer, and the fraction of working dies falls (yield e^{-DA} drops as A grows), so both effects raise the cost of each good die.

Chapter 18

Memory Circuits

A large part of every GPU's silicon is memory: register files, caches, shared memory. Off the chip sit gigabytes of DRAM. Flip-flops (Chapter 10) are far too big to store that much, so memories use specialized, tiny storage cells packed into dense arrays. This chapter explains the two workhorse cells, SRAM and DRAM, and the circuits around them.

18.1 Kinds of memory

	Flip-flop	SRAM	DRAM	Flash
Cell	20+ transistors	6 transistors	1 transistor, 1 capacitor	1 special transistor
Speed	fastest	very fast	medium	slow (writes very slow)
Density	lowest	low	high	highest
Needs refresh	no	no	yes	no
Keeps data without power	no	no	no	yes
Typical use	pipeline registers	caches, register files	main and GPU memory	SSDs, storage

Memory that loses its contents without power is **volatile** (SRAM, DRAM); memory that keeps them is **nonvolatile** (flash).

18.2 How a memory array is organized

Memory cells are arranged in a grid (Figure 18.1):

- Each **row** shares a horizontal **wordline**. Turning a wordline on connects every cell in that row to its column.
- Each **column** shares a vertical **bitline** (in SRAM, a pair: BL and $\overline{BL}$), which carries data in and out.

- A **row decoder** turns the row part of the address into one active wordline (Chapter 9).
- **Sense amplifiers** at the bottom of the columns detect the small signals the cells put on the bitlines; a **column multiplexer** picks which columns are the requested word.

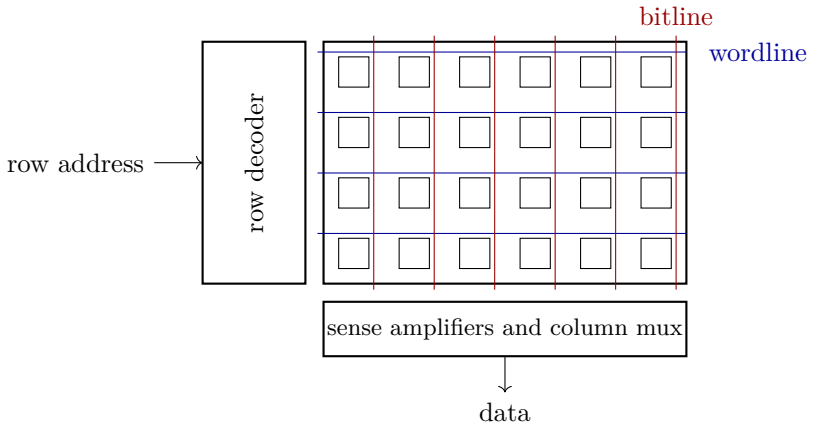


Figure 18.1: A memory array: wordlines select a row; bitlines carry data to the sense amplifiers.

Worked example: address bits

A 256 Kb (2^{18} bits) SRAM array has 512 rows. Each row then holds $2^{18}/2^9 = 512$ bits. If a word is 32 bits, each row holds 16 words. The address needs $\log_2 512 = 9$ row bits and $\log_2 16 = 4$ column bits: 13 bits to choose one of $2^{13} = 8192$ words.

18.3 The 6T SRAM cell

An SRAM (static RAM) cell is a bistable loop of two cross-coupled inverters (four transistors) plus two NMOS **access transistors** that connect the loop's two nodes to the bitlines when the wordline is high (Figure 18.2).

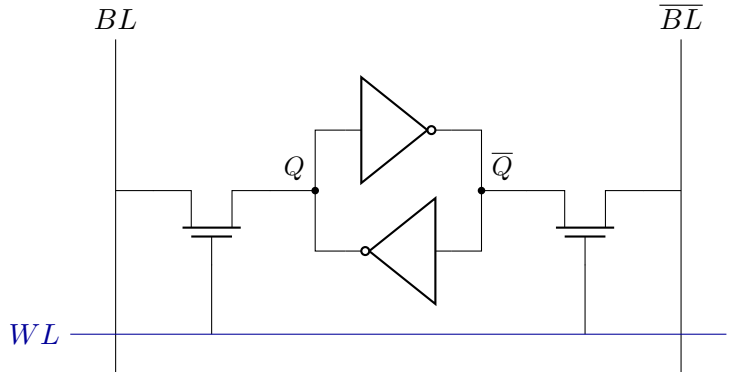


Figure 18.2: A 6T SRAM cell: two cross-coupled inverters store Q and $\overline{Q}$; two access transistors connect them to the bitlines when the wordline WL is high.

Holding. With the wordline off, the access transistors are off and the loop holds its value indefinitely, as long as power is on. No refresh is needed, which is what "static" means.

Reading. Both bitlines are first **precharged** to V_{DD}. Then the wordline turns on. Suppose $Q = 0$. Node Q's side now has a path from the precharged bitline BL, through the access transistor and the inverter's pull-down NMOS, to ground, so BL starts to discharge. The other bitline stays high. A sense amplifier detects which bitline is falling, long before it has fallen far.

Read stability. During that read, node Q (supposed to be 0) is caught in a tug-of-war: the access transistor pulls it up toward the precharged bitline, while the pull-down NMOS pulls it to ground. It is a voltage divider (Chapter 1), so Q rises a little. If it rises above the other inverter's switching threshold, the cell flips and the read destroys the data (a **read upset**). To prevent this, **the pull-down NMOS must be stronger than the access NMOS**. Their strength ratio is the **cell ratio**, typically 1.5 to 2.5.

Writing. To write $Q = 0$, drive BL to 0 and $\overline{BL}$ to V_{DD}, then turn on the wordline. The access transistor on the Q side must pull Q down *against* the pull-up PMOS that is holding it at 1. So **the access NMOS must be stronger than the pull-up PMOS** (the **pull-up ratio**). Conveniently, PMOS transistors are naturally weaker.

The big idea

SRAM cell sizing is a balance: pull-down NMOS > access NMOS > pull-up PMOS. A strong access transistor makes writing easy but reading risky; a weak one makes reading safe but writing hard.

The cell's robustness is measured by its **static noise margin** (SNM), read from a plot of the two inverters' transfer curves (the "butterfly curve"). Process variation (Chapter 15) makes some cells much weaker than others, which sets the lowest voltage a memory can run at, its V_{min}. Some designs use an **8T cell** with a separate read port that never disturbs the storage nodes, removing the read-stability problem at the cost of area.

18.4 Sense amplifiers

A bitline connects to hundreds of cells, so it has a large capacitance, while each cell's transistors are tiny. Waiting for a cell to swing a bitline all the way to 0 would be very slow. Instead, a **sense amplifier** detects a small difference between BL and $\overline{BL}$, often around 100 mV, and amplifies it to a full logic level. A common design is itself a pair of cross-coupled inverters: released when the bitlines are slightly apart, it quickly regenerates the difference into a full 0 and 1.

18.5 DRAM

A **DRAM** (dynamic RAM) cell is one transistor and one capacitor (1T1C, Figure 18.3). The bit is stored as charge on the capacitor: charged means 1, empty means 0. This is far smaller than an SRAM cell, which is why DRAM provides gigabytes cheaply.

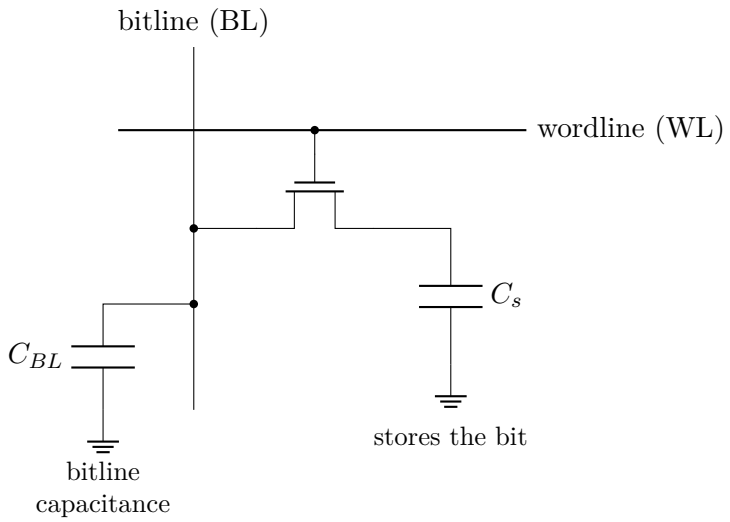


Figure 18.3: A DRAM cell: one transistor and one capacitor. Raising the wordline connects the cell's capacitor C_s to the bitline, whose own capacitance C_{BL} is several times larger.

Reading by charge sharing. The bitline is precharged to $V_{DD}/2$. Turning on the wordline connects the cell capacitor C_s to the much larger bitline capacitance C_{BL}. Charge redistributes, and the bitline moves slightly up (if the cell held a 1) or down (if it held a 0):

$$\Delta V = \left(V_{cell} - \frac{V_{DD}}{2} \right) \frac{C_s}{C_s + C_{BL}}.$$

Worked example: how big is a DRAM read signal?

$C_s = 20\,\text{fF}$, $C_{BL} = 80\,\text{fF}$, $V_{DD} = 1.2\,\text{V}$, and the cell holds a 1 (1.2 V). The bitline starts at 0.6 V.

$$\Delta V = (1.2 - 0.6) \times \frac{20}{20 + 80} = 0.6 \times 0.2 = 0.12\ \text{V}.$$

Only 120 mV, which a sense amplifier must detect.

Destructive reads. Charge sharing disturbs the cell's own charge, so every read destroys the stored value. The sense amplifier, once it has decided, drives the bitline to a full level, which writes the value back into the cell.

Refresh. The capacitor slowly leaks its charge. Every cell must be read and rewritten periodically, typically every 64 ms, which is called **refresh**. Refresh costs power and briefly blocks access. That periodic refresh is what "dynamic" means.

DRAM organization. A DRAM chip is divided into **banks**. Accessing data first opens a whole row into a **row buffer** (slow), after which columns from that open row can be read quickly. Accesses that hit an already open row are much faster than those that need a new row. Graphics memory

(GDDR) and high-bandwidth memory (HBM, stacks of DRAM dies next to the GPU) are DRAM organized for very high bandwidth (Chapter 16).

18.6 Flash memory

A **flash** cell (Figure 18.4) is a transistor with an extra, electrically isolated gate (a floating gate or a charge-trapping layer) under its control gate. Electrons forced onto that layer by tunneling at high voltage stay there for years without power and shift the transistor's threshold voltage. Reading measures the threshold. Flash is nonvolatile and very dense (cells can store 2, 3 or 4 bits by distinguishing several threshold levels), but writing is slow, it must be erased in large blocks, and each cell survives only a limited number of write cycles. It is used for storage (SSDs), not as working memory.

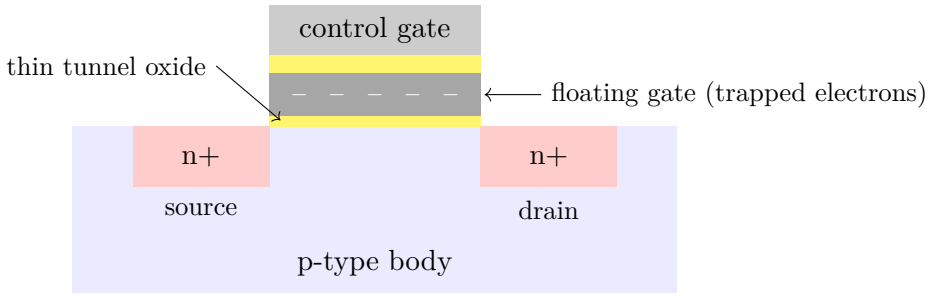


Figure 18.4: A flash cell. Electrons pushed through the thin tunnel oxide onto the isolated floating gate stay there for years without power, and they raise the transistor's threshold voltage, which is how the bit is read.

18.7 Register files, CAMs and ROMs

- A **register file** is a small, very fast SRAM with many **ports**, so several registers can be read and written in the same cycle. Each extra port adds wordlines, bitlines and access transistors to every cell, so ports are expensive. GPUs have very large register files to hold the state of thousands of threads (Chapter 21).
- A **content-addressable memory** (CAM) works backwards: you give it a value and it reports which entries contain it, comparing all entries in parallel. CAMs are used in translation lookaside buffers and network routers. They are fast but power-hungry.
- A **read-only memory** (ROM) holds fixed contents set at manufacture.

18.8 Protecting stored data: ECC

Soft errors (Chapter 17) can flip stored bits. A single **parity** bit (the XOR of all data bits) detects any single-bit error but cannot say which bit flipped. **Error-correcting codes** add more check bits. The common SECDED code (single-error correct, double-error detect) protects 64 data bits with 8 check bits, which is why server and GPU memory often has 72-bit-wide words. Memories may also **scrub**: periodically read, correct and rewrite every word so errors do not accumulate.

18.9 Summary

- Memory arrays: wordlines select rows, bitlines carry data, decoders select rows, sense amps detect small signals.
- 6T SRAM: two cross-coupled inverters plus two access NMOS. Static: no refresh.
- Read stability needs pull-down > access; writability needs access > pull-up.
- Sense amplifiers turn 100 mV bitline differences into full logic levels.
- DRAM: 1T1C, read by charge sharing ($\Delta V = (V_{cell} - V_{DD}/2)\, C_s/(C_s + C_{BL})$), destructive reads, refresh every 64 ms.
- Flash: nonvolatile, dense, slow to write, limited endurance.
- Parity detects single errors; SECDED ECC (64 + 8 bits) corrects them.

18.10 Exercises

1. How many transistors are in a standard SRAM cell? What are they?
2. For read stability, which must be stronger: the pull-down NMOS or the access NMOS? Why?
3. For writability, which must be stronger: the access NMOS or the pull-up PMOS?
4. A DRAM cell of 30 fF holding a 1 (1.0 V) is read onto a 120 fF bitline precharged to 0.5 V. Find the bitline change.
5. Why does DRAM need refresh? Why does SRAM not?
6. Is a DRAM read destructive? An SRAM read?
7. A 256 Kb array has 512 rows and 32-bit words. How many row and column address bits are needed?
8. How many check bits does SECDED use for 64 data bits?
9. Why are sense amplifiers needed instead of waiting for full bitline swings?
10. Which memory keeps its data with power off: SRAM, DRAM or flash?

Answers

1. Six: two PMOS pull-ups and two NMOS pull-downs forming two cross-coupled inverters, plus two NMOS access transistors.
2. The pull-down. During a read, the 0 node is a voltage divider between them; a stronger pull-down keeps the node low enough that the cell does not flip.
3. The access NMOS, so it can overpower the PMOS holding the node at 1.
4. $(1.0 - 0.5) \times 30/150 = 0.1$ V.
5. The DRAM capacitor leaks its charge over time. SRAM's cross-coupled inverters actively hold the value from the supply.

6. DRAM: yes (charge sharing disturbs the cell; it is written back). SRAM: no (if correctly sized).
7. $2^{18}/512 = 512$ bits per row, so 16 words per row: 9 row bits and 4 column bits.
8. 8.
9. Bitlines have large capacitance and cells are tiny, so a full swing would take far too long; sensing a $100\,\text{mV}$ difference is much faster.
10. Flash.

Part VII

Digital Systems

Chapter 19

Describing and Verifying Hardware: RTL, Verilog, Clock Domains and FIFOs

Nobody draws a billion transistors by hand. Digital designers describe hardware in a **hardware description language** (HDL), most often Verilog or SystemVerilog, at the **register-transfer level** (RTL): registers, and the logic that computes their next values. Tools then turn that description into gates (synthesis) and the gates into layout. This chapter covers the HDL ideas that hardware questions test, two design problems that come up constantly (crossing between clock domains, and sizing FIFOs), and finally how designs are verified before they are built.

19.1 An HDL describes hardware, not steps

Verilog looks like a programming language, but it is not one. A program is a list of steps executed one after another. An HDL describes *circuits that all exist at once*, every gate working in parallel all the time. A line of Verilog does not "run"; it describes a piece of hardware that is always there.

19.2 Verilog in one page

```
module mux2 (
  input  wire       s,
  input  wire [7:0] a, b,     // 8-bit buses, bit 7 is the MSB
  output wire [7:0] y
);
  assign y = s ? b : a;       // continuous assignment: combinational logic
endmodule
```

- A `module` is a block of hardware with `input` and `output` ports. Modules are instantiated inside other modules to build hierarchy.
- `[7:0]` declares an 8-bit bus. `{a, b}` concatenates buses; `a[3:0]` selects bits.

- `assign` describes combinational logic that is always active.
- An `always @(*)` block also describes combinational logic, using `if` and `case`.
- An `always @(posedge clk)` block describes flip-flops: everything assigned inside updates on the rising clock edge.

```
module counter4 (
  input  wire       clk, rst_n, en,
  output reg  [3:0] count
);
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n)      count <= 4'd0;          // asynchronous, active-low reset
    else if (en)     count <= count + 4'd1;  // count when enabled
  end
endmodule
```

(In Verilog, a signal assigned in an `always` block is declared `reg`; that does not necessarily mean it becomes a register. SystemVerilog replaces both `wire` and `reg` with `logic`. It also adds `always_ff` and `always_comb`, which state the designer's intent, so tools can warn when the code does not really describe flip-flops or pure combinational logic, for example when an `always_comb` block would infer a latch.)

19.3 Blocking and nonblocking assignments

Verilog has two assignment operators inside `always` blocks, and confusing them is the most famous Verilog bug.

- **Blocking `=`**: the assignment happens immediately, and the next statement sees the new value, like a programming language.
- **Nonblocking `<=`**: the right-hand sides are all evaluated first, using the old values, and all the left-hand sides are updated together at the end of the time step, like flip-flops all capturing on the same clock edge.

Worked example: swapping two registers

Start with `a = 1`, `b = 2`.
Nonblocking:

```
always @(posedge clk) begin
  a <= b;
  b <= a;
end
```

Both right sides use the old values: after the first edge `a = 2, b = 1`; after the second `a = 1, b = 2`. The registers swap every cycle, which is what two cross-connected flip-flops do (Figure 19.1).
Blocking:

```
always @(posedge clk) begin
  a = b;    // a becomes 2 immediately
  b = a;    // reads the NEW a, so b becomes 2
end
```

After the first edge `a = 2, b = 2`, and they stay 2 forever. No swap.

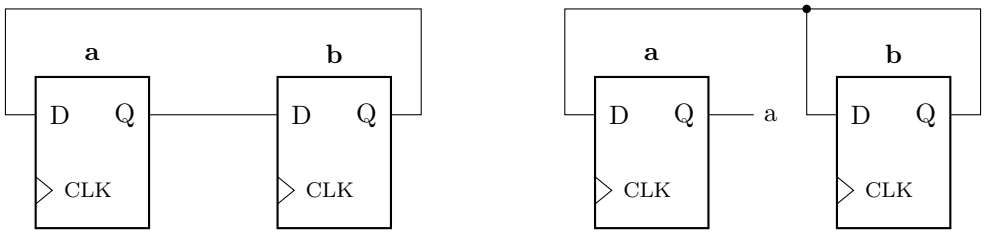


Figure 19.1: The hardware each version describes (clocks omitted). Nonblocking (left): two cross-connected flip-flops that swap. Blocking (right): the second line reads the new a, which is just b, so both flip-flops load b.

Worked example: what does `$display` print?

```
always @(posedge clk) begin
  a <= b;
  b <= a;
  $display("a=%0d b=%0d", a, b);
end
```

With `a = 1, b = 2` initially, the first edge prints `a=1 b=2`, not the swapped values: nonblocking updates happen at the *end* of the time step, after `$display` has already run. The second edge prints `a=2 b=1`.

Seen on the assessment

A reported question shows an `always` block with blocking and nonblocking assignments and asks what values of `a` and `b` are displayed in the first and second cycles. Work through it as above: blocking updates are seen by the following statements immediately; nonblocking updates are not visible until the time step ends.

Remember

Use `<=` (nonblocking) in clocked `always @(posedge clk)` blocks. Use `=` (blocking) in combinational `always @(*)` blocks. Never mix them in one block.

19.4 Accidental latches

In a combinational `always @(*)` block, every output must be assigned a value in *every* path through the code. If some path leaves an output unassigned, the output must "remember" its old value, and synthesis creates a latch, usually unintended (Figure 19.2).

```
always @(*) begin
  if (sel) y = a;        // no else: when sel is 0, y must hold its value
end                      //   -> a latch is inferred
```

The fix: add the `else`, give a `default` in every `case`, or assign a default value at the top of the block.

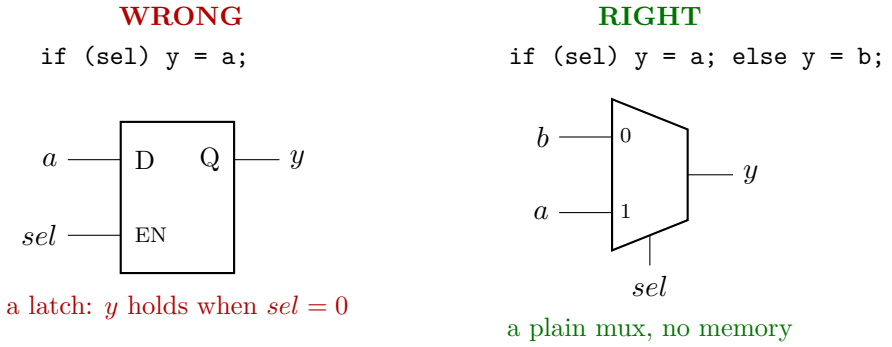


Figure 19.2: What synthesis builds. With no `else` (left), y must remember its value when *sel* is 0, so a latch is inferred. Assigning y on every path (right) gives plain combinational logic, here a 2:1 mux.

19.5 Resets

Synchronous reset. The reset is just another input to the logic in front of the flip-flops: `if (rst) q <= 0;` inside `always @(posedge clk)`. It takes effect at the next clock edge, so it needs a running clock, but it never causes timing surprises.

Asynchronous reset. The reset is in the sensitivity list (`always @(posedge clk or negedge rst_n)`) and drives the flip-flops' clear pins directly, so it works with no clock. The danger is *releasing* it: if reset deasserts close to a clock edge, some flip-flops may leave reset one cycle earlier than others, or go metastable. The standard solution is a **reset synchronizer**: assert reset asynchronously, but release it through two flip-flops clocked by the destination clock ("asynchronous assert, synchronous deassert").

Filtering glitches on a reset line. A noisy reset input should not reset the chip because of a brief glitch. One simple filter compares the reset with a delayed copy of itself (Figure 19.3): for an active-low reset, the filtered reset is asserted only when both the input and its delayed copy are low,

$$\text{rst_n}_{\text{filtered}} = \text{rst_n OR rst_n}_{\text{delayed}}.$$

A low glitch shorter than the delay has already ended by the time its delayed copy arrives, so the two are never low together and the OR output stays high.

Worked example: how many buffers?

Glitches up to 25 ns must be filtered. Each buffer in the delay chain delays the signal by 2 ns. The chain delay must exceed the glitch width: $k \times 2 > 25$, so $k = 13$ buffers (26 ns). In this circuit the OR gate's own delay applies to both inputs equally, so it does not change the count; if a question's circuit puts a gate delay in only one of the two paths, add it to that path's delay.

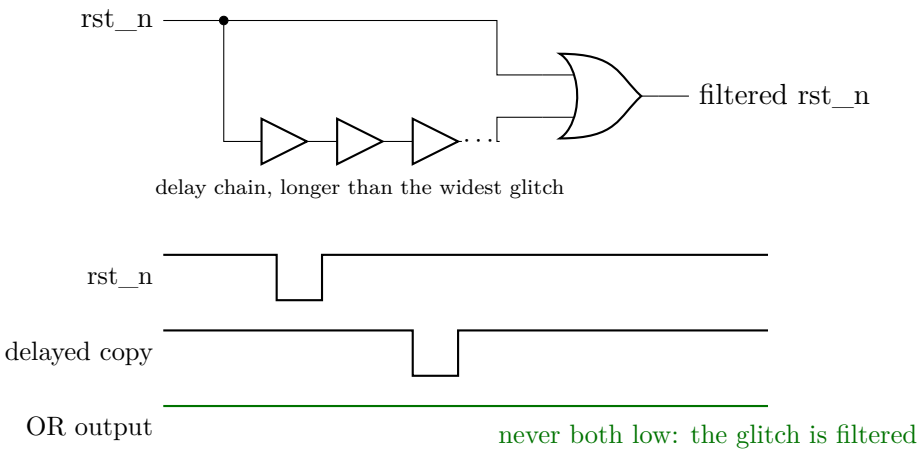


Figure 19.3: A reset glitch filter for an active-low reset. A short low glitch and its delayed copy never overlap, so the OR output stays high. A real reset, longer than the delay, makes both inputs low together and gets through.

Seen on the assessment

A reported question gives a glitch width $N = 25$ ns, a buffer delay of 2 ns and an OR-gate delay of 4 ns, and asks how many buffers are needed to filter glitches of that width or less. The principle is the one above: the delayed path must be longer than the widest glitch to be filtered, counting every delay that lies only on the delayed path.

19.6 Clock domain crossing

Large chips have many clocks: one for the GPU cores, one for memory, one for each I/O interface. A group of flip-flops on one clock is a **clock domain**. A signal passing from one domain to another is a **clock domain crossing** (CDC). The receiving domain's clock has no fixed relation to the sender's, so the signal can change at any moment relative to the receiving clock edge, which violates setup or hold and causes metastability (Section 10.9).

One bit: the two-flop synchronizer. Pass the signal through two flip-flops in a row on the receiving clock (Figure 19.4). The first may go metastable, but it has a full clock period to settle before the second samples it. The probability that it is still unsettled is exponentially small.

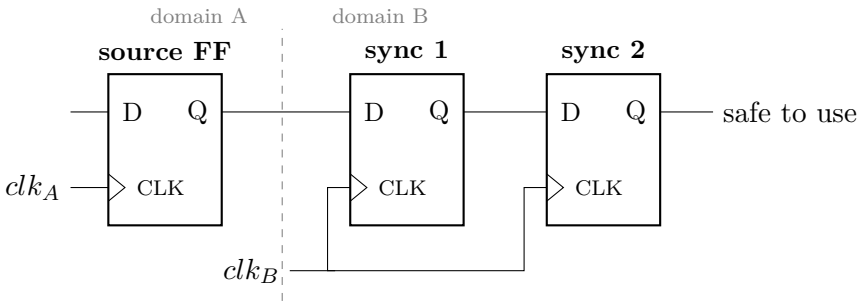


Figure 19.4: A two-flop synchronizer. The signal leaves domain A from a flip-flop (never from raw logic) and is re-timed by two flip-flops on clock B.

Rules that go with it:

- The signal must come straight from a flip-flop in the source domain, not from combinational logic, which could glitch.
- A short pulse from a fast domain may fall entirely between the slow domain's clock edges and be missed. Stretch it, or convert it into a level that toggles (a "toggle synchronizer").

Many bits: never synchronize each bit separately. If a multi-bit value changes, its bits pass through separate synchronizers with slightly different delays, and the receiver can see a mixture of old and new bits, a value that never existed (Figure 19.5). Safe methods:

- **Gray code**: if the value is a counter, encode it in Gray code (Section 7.7). Only one bit changes per step, so the receiver sees either the old value or the new one, never garbage.
- **Handshake**: hold the data stable, send a single "valid" bit through a synchronizer, and wait for an acknowledgment to come back the same way before changing the data.
- **Asynchronous FIFO**: the general solution for streams of data, described next.

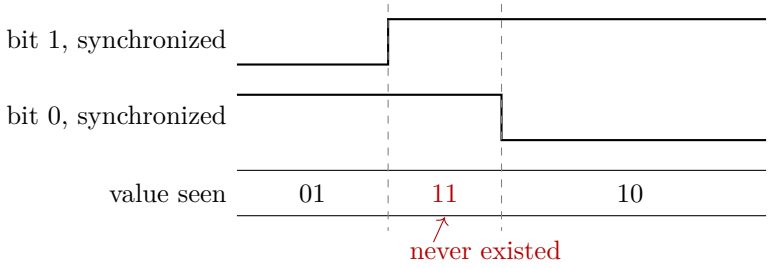


Figure 19.5: Synchronizing each bit separately. The value changes from 01 to 10, but the two bits come through their synchronizers one cycle apart, so for one cycle the receiver sees 11, a value the sender never had. Gray code avoids this by changing only one bit at a time.

19.7 FIFOs

A **FIFO** (first in, first out) is a queue: data is written at one end and read out in the same order at the other. It decouples a producer and a consumer that run at different moments or rates.

Structure. A FIFO is a small memory used as a circular buffer (Figure 19.6), plus a **write pointer** (where the next word goes) and a **read pointer** (where the next word comes from). Both advance and wrap around. The FIFO is **empty** when the pointers are equal and **full** when the write pointer has gone all the way around and caught up with the read pointer. To tell these two cases apart, the pointers get one extra bit: equal including the extra bit means empty; equal except for the extra bit means full.

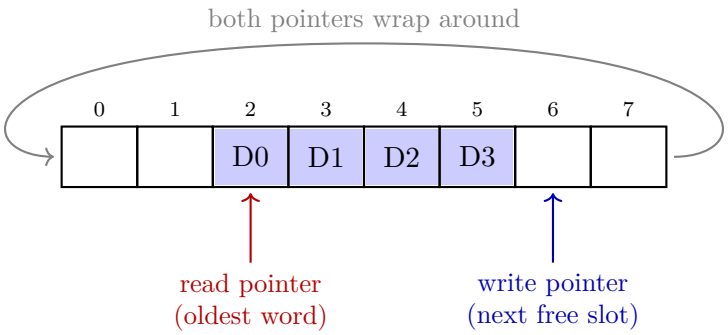


Figure 19.6: A FIFO as a circular buffer of 8 words. Words are written at the write pointer and read at the read pointer; both advance and wrap around. Here it holds four words, D0 to D3.

Asynchronous FIFOs. When the writer and reader use different clocks, each side must compare its own pointer with the other side's pointer, which crosses a clock domain. The pointers are converted to Gray code and passed through two-flop synchronizers. Because only one bit changes at a time, the synchronized pointer is always a valid (if slightly old) value, and a slightly old value can only make the FIFO look a little fuller or emptier than it is, which is safe.

19.8 How deep must a FIFO be?

The big idea

A FIFO must hold whatever the writer produces faster than the reader can consume. During a burst of B words, the writer takes time B/r_w (where r_w is the write rate in words per second), and in that time the reader removes $r_r \cdot B/r_w$ words. The rest must wait in the FIFO:

$$\text{depth} \geq B - B\,\frac{r_r}{r_w} = B\left(1 - \frac{r_r}{r_w}\right).$$

Worked example: a simple burst

The writer runs at 100 MHz and writes one word per cycle in bursts of 120 words. The reader runs at 50 MHz and reads one word per cycle. Burst time: $120/100\,\text{MHz} = 1.2\,\mu\text{s}$. Words read in that time: $1.2\,\mu\text{s} \times 50\,\text{MHz} = 60$. Depth: $120 - 60 = 60$ words.

Worked example: idle cycles

The writer runs at 200 MHz and writes a burst of 100 words, one per cycle. The reader runs at 100 MHz but can only accept one word every 2 cycles, an effective rate of 50 million words per second. Burst time: $100/200\,\text{MHz} = 0.5\,\mu\text{s}$. Words read: $0.5 \times 50 = 25$. Depth: $100 - 25 = 75$ words.

Real questions add details: idle cycles on the write side (use the effective rate), a worst case where two bursts arrive back to back, or a few extra entries for synchronizer latency in an asynchronous FIFO. Always convert everything to *words per second* first. If the reader is at least as fast as the writer, a FIFO only needs to cover latency, not rate mismatch.

Seen on the assessment

FIFOs are reported on the Ignite assessment, and FIFO depth calculations (module A writing to module B at a different frequency) on other NVIDIA hardware tests. Be ready to explain what a FIFO is, how full and empty are detected, why asynchronous FIFOs use Gray-coded pointers, and how to size one.

19.9 Handshakes and arbiters

Valid/ready. Within one clock domain, blocks usually exchange data with a **valid/ready** handshake: the sender raises `valid` when it has data; the receiver raises `ready` when it can take it; a transfer happens on each clock edge where both are high (Figure 19.7). This lets either side pause the other (**backpressure**).

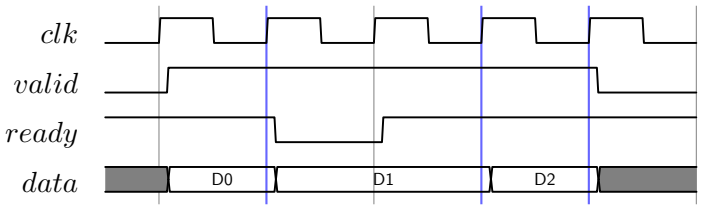


Figure 19.7: A valid/ready handshake. A word moves on each rising edge where both valid and ready are high (blue lines): D0, then D1, then D2. At the edge in between, ready is low, so D1 waits: the receiver is applying backpressure.

Arbiters. When several requesters share one resource (a memory port, a bus), an **arbiter** grants it to one at a time. A **fixed-priority** arbiter always prefers the lowest-numbered requester, which is simple but can starve the others. A **round-robin** arbiter rotates priority after each grant, so every requester is eventually served.

19.10 Verification basics

Writing RTL is half the job. The other half, usually the larger half, is **verification**: showing that the RTL does what the specification says, in every situation it will meet, *before* the chip is manufactured. A bug found in simulation costs an afternoon. A bug found in silicon can cost a new set of manufacturing masks (millions of dollars at advanced nodes) and months of delay, or a product recall. That is why large chip teams often have more verification engineers than designers, and why "GPU Verification and Design" is one of the Ignite internship areas.

The big idea

Verification asks three questions, again and again: **Did I drive the design with enough different inputs?** (stimulus) **Did I check every output for the right answer?** (checking) **How do I know I have tested enough?** (coverage)

The verification plan

Before writing any test, a verification engineer reads the specification and writes a **verification plan**: a list of every feature and every rule the design must obey, how each will be tested, and what "tested enough" means for it. For a FIFO the plan would include: write then read returns the same data in order; `full` rises exactly when the FIFO holds `DEPTH` words; writing while full is ignored or flagged; reading while empty is ignored or flagged; simultaneous read and write work at every fill level; reset empties it.

Testbenches

A **testbench** is a piece of HDL (or software) that surrounds the **design under test** (DUT), drives its inputs and watches its outputs. The testbench never becomes hardware, so it can use every language feature: loops, random numbers, files, classes.

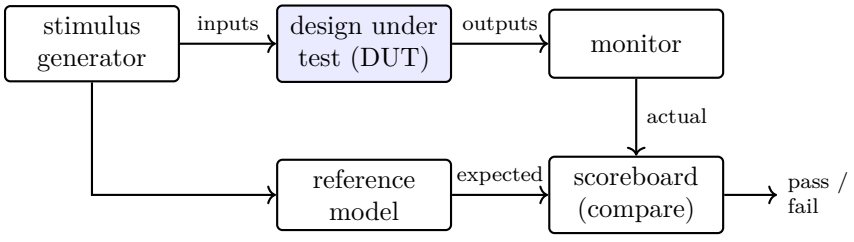


Figure 19.8: A self-checking testbench. The same stimulus goes to the design and to a reference model; a scoreboard compares what the design did with what it should have done.

The most important rule: a testbench must **check** results automatically. Looking at waveforms by eye works for ten cycles, not for ten million. A **self-checking testbench** computes the expected answer itself, from a **reference model** (also called a **golden model**): a simple, obviously correct description of what the design should do, often written in SystemVerilog, C or Python. The reference model does not need to be fast or synthesizable, only right.

```
1 module tb_adder8;
```

```
  logic [7:0] a, b;
  logic [8:0] sum;
  int errors = 0;

  adder8 dut (.a(a), .b(b), .sum(sum));   // the design under test

  task check(input logic [7:0] x, y);
    a = x; b = y;
    #1;                                   // let the logic settle
    if (sum !== x + y) begin              // reference model: the + operator
      errors++;
      $display("FAIL: %0d + %0d gave %0d", x, y, sum);
    end
  endtask

  initial begin
    check(8'd0,   8'd0);                  // directed corner cases first
    check(8'd255, 8'd1);
    check(8'd255, 8'd255);
    repeat (10000) check($urandom, $urandom);   // then random inputs
    $display("%0d errors", errors);
    $finish;
  end
endmodule
```

The comparison uses `!==`, which also catches `X` (unknown) and `Z` values on the output; `!=` would quietly treat them as "not different."

Directed, random and constrained-random tests

- A **directed test** applies inputs chosen by hand to exercise one feature or corner case: fill the FIFO exactly to full, add $255 + 1$. Directed tests are easy to understand, but they only find the bugs someone thought of.
- **Random** stimulus finds the bugs nobody thought of, but purely random inputs are often illegal (a packet length of zero, a misaligned address) or miss rare situations.
- **Constrained-random** stimulus is random within rules that keep it legal and steer it toward interesting cases. SystemVerilog builds this into the language:

```
class packet;
  rand bit [7:0]  len;
  rand bit [31:0] addr;
  constraint legal { len inside {[1:64]};        // 1 to 64 bytes
                     addr[1:0] == 2'b00; }        // word-aligned
endclass
```

Each run uses a **seed** for its random number generator, and the seed is printed, so any failing run can be repeated exactly. Teams run **regressions**: thousands of tests with different seeds every night, so that a change that breaks something is caught the next morning.

Assertions

An **assertion** states a rule that must always hold and reports an error the moment it is broken, at the place where the bug happens, not hundreds of cycles later when a wrong value finally reaches an output. SystemVerilog has two kinds:

- An **immediate assertion** checks a condition at one moment, like an `if` that reports an error: `assert (count <= DEPTH);`
- A **concurrent assertion** (SystemVerilog Assertions, SVA) checks a property over clock cycles:

```
// every request is granted within 1 to 3 cycles
assert property (@(posedge clk) disable iff (!rst_n) req |-> ##[1:3] gnt);

// the FIFO is never written while it is full
assert property (@(posedge clk) disable iff (!rst_n) !(wr_en && full));
```

Read `|->` as "implies, starting this cycle" and `##[1:3]` as "1 to 3 cycles later." Assertions are written once and then checked in every test, and formal tools (below) can try to prove them.

Coverage: how do you know you are done?

Passing tests prove nothing about the situations the tests never reached. **Coverage** measures what was reached.

- **Code coverage** is collected automatically by the simulator: which lines ran (line coverage), which branches of each `if` and `case` were taken (branch coverage), whether every signal toggled both ways (toggle coverage), and which FSM states and transitions occurred.
- **Functional coverage** is written by the engineer from the verification plan: did the scenarios that matter actually happen? Was the FIFO ever full? Was it written and read in the same cycle while full? Did every opcode run with a negative operand?

```
covergroup fifo_cg @(posedge clk);
  level: coverpoint count { bins empty  = {0};
                            bins middle = {[1:DEPTH-1]};
                            bins full   = {DEPTH}; }
  ops:   cross wr_en, rd_en;       // all four write/read combinations
endgroup
```

Common trap

100% code coverage does not mean the design is correct. It means every line *ran*, not that its results were *checked*, and not that every combination of conditions occurred. A line can execute with the wrong answer in a test that has no checker. Coverage measures the stimulus; only checkers and assertions find bugs.

Verification is "done" (**coverage closure**) when the plan's coverage goals are met, every test passes across many seeds, and the rate of newly found bugs has dropped to near zero.

Formal verification

Simulation checks the input sequences you happen to try. **Formal verification** uses mathematical tools to prove that a property holds for *every* possible input sequence (usually up to a bounded number of cycles), or to produce a counterexample: a short, exact input sequence that breaks it. It shines on control logic such as arbiters, FIFOs, handshakes and cache-coherence rules, where rare interleavings hide bugs. **Equivalence checking** is a related formal technique that proves two versions of a design behave identically, for example the RTL and the gate-level netlist produced by synthesis.

UVM and the bigger picture

The **Universal Verification Methodology** (UVM) is a standard SystemVerilog class library for building large, reusable testbenches. Its pieces map onto Figure 19.8: a **sequencer** produces transactions (such as "write 4 bytes to address 0x40"), a **driver** turns them into pin wiggles, a **monitor** turns pin activity back into transactions, and these three form an **agent** for one interface. A **scoreboard** compares observed transactions with the reference model. An agent written once for a bus can be reused on every block that uses that bus.

Verification also happens at several levels. Each block is tested on its own (unit level), then blocks together (subsystem), then the whole chip. Gate-level simulation checks the synthesized netlist. Because full-chip simulation is slow, companies also map the design onto **emulators** and FPGA prototypes, which run thousands of times faster, so real software and drivers can run on the chip before it exists. Arithmetic units are a classic verification target: Chapter 22 describes the corner cases they are checked against.

Remember

A test without a checker tests nothing. Start every testbench with a reference model and an automatic comparison, add constrained-random stimulus and assertions, and measure functional coverage against a written plan.

19.11 SystemVerilog for design

Most new RTL is written in SystemVerilog, which is Verilog plus features that make intent explicit and catch mistakes at compile time. Interviewers for FPGA and ASIC roles expect you to read and write these constructs.

One type, three kinds of block. `logic` replaces both `wire` and `reg`: it can be driven by an `assign` or by a procedural block, but only from one place. The generic `always` is split three ways:

- `always_ff @(posedge clk)` for flip-flops. Tools complain if the block does not describe edge-triggered storage.
- `always_comb` for combinational logic. It needs no sensitivity list, runs once at time zero so outputs start correct, and tools warn if it would infer a latch.
- `always_latch` for the rare latch you really want.

Named states with enum. A state machine's states should have names, not magic numbers:

```
typedef enum logic [1:0] {IDLE, LOAD, RUN, DONE} state_t;
state_t state, next;

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) state <= IDLE;
  else        state <= next;

always_comb begin
  next = state;                 // default: stay, so no latch
  unique case (state)
    IDLE: if (start)    next = LOAD;
    LOAD:               next = RUN;
    RUN:  if (count==0) next = DONE;
    DONE:               next = IDLE;
  endcase
end
```

Waveform viewers show `RUN` instead of `2'b10`, and assigning a plain number to `state` is a compile error. The default assignment `next = state` at the top of the block is the standard way to avoid accidental latches (Section 19.4).

unique and priority. `unique case` tells the tools that exactly one branch matches, so the branches may be built as parallel logic (a mux) rather than a priority chain; simulation warns if no branch or more than one matches. `priority case` (or `priority if`) says the first match wins and at least one must match. Both replace the old, dangerous `full_case`/`parallel_case` synthesis comments, which changed the hardware without changing the simulation.

Wildcards and sets. In `casez`, a `?` (or `z`) bit in a case item is a don't-care, which is how priority encoders are usually written: `4'b1???: idx = 3;`. `casex` also treats `x` as don't-care, so an unknown input silently matches a branch and hides bugs; avoid it. The `inside` operator tests set membership: `if (op inside {ADD, SUB, [8:11]})`.

Packed structs. A `typedef struct packed` groups fields into one bit vector, with the first field in the most significant bits:

```
typedef struct packed {
  logic [3:0]  kind;     // bits 39:36
  logic [3:0]  id;       // bits 35:32
  logic [31:0] data;     // bits 31:0
} pkt_t;                 // 40 bits in all
pkt_t p;
assign p = in_bus;       // a struct is just bits
assign is_write = (p.kind == 4'h1);
```

Packed and unpacked arrays. Dimensions *before* the name are packed: `logic [3:0][7:0] w` is one 32-bit vector seen as four bytes, and `w[2]` is bits 23:16. Dimensions *after* the name are

unpacked: `logic [7:0] mem [0:255]` is 256 separate bytes, a memory. Only packed data can be assigned to or compared with a plain vector in one step.

Parameters, generate and $clog2. Reusable blocks are parameterized, and `$clog2` computes the bits needed: a FIFO of depth `DEPTH` needs pointers of `$clog2(DEPTH)` bits ($clog2(16) = 4, $clog2(17) = 5). A `generate for` loop builds repeated hardware, one copy per iteration, at elaboration time:

```
module sync_bus #(parameter int W = 8) (
  input  logic clk, input logic [W-1:0] d, output logic [W-1:0] q);
  for (genvar i = 0; i < W; i++) begin : g_bit
    sync2 u_sync (.clk, .d(d[i]), .q(q[i]));   // W synchronizers
  end
endmodule
```

(That is the right structure for W *independent* bits, such as separate interrupt lines. For a W-bit *value* it is wrong, for the reason in Section 19.6.) A plain `for` loop inside `always_comb` also unrolls into parallel hardware; it describes a circuit, not a sequence of steps over time.

Interfaces and modports. An `interface` bundles the signals of one connection, such as a valid/ready stream, so a module port list says `stream_if.sink in` instead of repeating every wire. A `modport` gives each side its directions: the source drives `valid` and `data` and reads `ready`; the sink does the opposite.

Packages. A `package` holds shared types, parameters and functions (the `pkt_t` above, a state enum, an opcode list), and modules `import` it, so every block agrees on one definition.

Common trap

SystemVerilog makes intent checkable, but it does not change what you build. An `always_comb` block missing an assignment on one path still describes a latch; the tool now warns you about it. Read the warnings.

A pipelined datapath

Pipelining (Section 11.10) is easy to describe in RTL: each stage is a set of registers, and a `valid` bit travels with the data so the stage after knows when it holds real work.

```
// y = a*b + c in two stages: multiply, then add.
always_ff @(posedge clk) begin
  v1 <= in_valid;  prod <= a * b;  c1 <= c;      // stage 1
  v2 <= v1;        y    <= prod + c1;            // stage 2
end
assign out_valid = v2;
```

Two rules: every value used in a later stage must be carried forward in its own register (here `c` becomes `c1`, so it lines up with `prod`), and the latency is the number of register stages, here 2 cycles.

Throughput stays one result per cycle. If the downstream block can stall, each stage also needs a ready signal, and a skid buffer keeps the ready path short.

19.12 Reading VHDL

VHDL is the other standard HDL. It is common in FPGA teams, defense and aerospace, and at several trading firms. It is wordier and more strictly typed than Verilog, but the hardware ideas are the same. You mostly need to read it and map it onto what you already know.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter4 is
  port ( clk, rst, en : in  std_logic;
         q            : out unsigned(3 downto 0) );
end entity;

architecture rtl of counter4 is
  signal cnt : unsigned(3 downto 0);
begin
  process (clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then  cnt <= (others => '0');   -- synchronous reset
      elsif en = '1' then cnt <= cnt + 1;
      end if;
    end if;
  end process;
  q <= cnt;                                        -- concurrent assignment
end architecture;
```

VHDL	Verilog / SystemVerilog
`entity` (ports) + `architecture` (body)	`module`
`std_logic`, `std_logic_vector(7 downto 0)`	`logic`, `logic [7:0]`
`unsigned`, `signed` (from `numeric_std`)	`logic [7:0]`, `logic signed [7:0]`
`process (clk)` with `if rising_edge(clk)`	`always_ff @(posedge clk)`
`process (all)` (VHDL-2008)	`always_comb`
signal assignment `<=`	nonblocking `<=`
variable assignment `:=`	blocking `=`
`y <= a when s = '1' else b;`	`assign y = s ? a : b;`
`(others => '0')`	`'0`
`x"FF"`, `"1010"`, `'1'`	`8'hFF`, `4'b1010`, `1'b1`
`generic`, `generate`	`parameter`, `generate`

Signals and variables. A VHDL `signal` assigned in a process takes its new value only when the process finishes (at the end of the delta cycle), exactly like a nonblocking assignment. A `variable`, declared inside a process, changes immediately, like a blocking assignment. So `a <= b; b <= a;` on signals swaps them, just as in Verilog.

Strong typing. VHDL will not add a `std_logic_vector` to an integer, or assign an 8-bit value to a 4-bit signal; you convert explicitly, for example `unsigned(v)`, `std_logic_vector(u)` and `to_integer(u)`. This catches width and sign bugs that Verilog silently truncates, at the cost of more typing. Use `numeric_std` for arithmetic, not the old non-standard `std_logic_arith` packages.

std_logic has nine values. Besides `'0'` and `'1'` there are `'Z'` (high impedance), `'U'` (uninitialized, what every signal starts as in simulation), `'X'` (conflict), `'-'` (don't care) and weak levels. Seeing `'U'` in a waveform means a register was never reset or a signal is never driven.

19.13 FPGAs: where your RTL lands

A **field-programmable gate array** (FPGA) is a chip of prebuilt logic and wiring whose function is set by configuration memory loaded at power-up. The same RTL you would give an ASIC flow becomes, on an FPGA, a pattern of configuration bits: no masks, no fab, a new design in minutes to hours. The price is speed, power and area: the same logic is typically several times slower and uses far more silicon than in an ASIC, because every connection passes through programmable switches. FPGAs win where volumes are low, designs change often, or latency matters more than cost: prototyping, networking equipment, test gear, and low-latency trading, where a market-data packet can be parsed and acted on in well under a microsecond without ever touching a CPU.

The LUT. The basic logic element is a k-input **look-up table** (LUT), a tiny memory of 2^k bits addressed by the inputs. Since it stores a whole truth table, a LUT implements *any* function of its k inputs in the same time (this is the "any function from a big multiplexer" idea of Section 9.3). Modern FPGAs use 6-input LUTs, which can often be split into two 5-input LUTs sharing inputs. A function of more inputs takes several LUTs in levels, and each level adds delay, so FPGA timing is counted in *LUT levels* rather than gates.

Worked example: LUTs for a wide function

How many 6-input LUTs does a 16-input AND need, and how many levels?
Each LUT can AND six signals. The first level reduces 16 inputs with $\lceil 16/6 \rceil = 3$ LUTs, giving 3 signals; one more LUT combines them. Total **4 LUTs in 2 levels**. A 36-input AND would need $6 + 1 = 7$ LUTs, still in 2 levels.

Logic blocks and carry chains. LUTs are grouped with flip-flops into slices or logic blocks (on AMD/Xilinx parts, a **CLB**). Each LUT output can feed a flip-flop right next to it, so pipelining is nearly free: the registers are already there, unused, beside every LUT. Each block also has a dedicated **carry chain**, fast hard-wired logic for the carry of an adder, so a 32-bit add is much faster than if the carry rippled through LUTs and routing.

Hard blocks. Some functions are so common that the FPGA builds them in fixed silicon:

- **Block RAM** (BRAM): dual-port memories of tens of kilobits each, hundreds of them per chip. They have *synchronous* reads: the address is registered and data appears the cycle after. RTL written with an asynchronous read cannot map to BRAM and is built from LUTs instead (**distributed RAM**), or from flip-flops, which is much bigger.
- **DSP slices**: hard multiply-accumulate units (on recent AMD parts, a 27×18-bit multiplier with a 48-bit accumulator) with optional pipeline registers inside. A `*` in your RTL lands here. Using the internal registers lets it run at full clock speed.
- **Clocking**: PLLs and MMCMs make new clock frequencies and phases, and dedicated global clock networks distribute them with low skew. Clocks must go through these, never through ordinary logic (Section 11.14).
- **I/O and transceivers**: configurable I/O pins for many voltage standards, and multi-gigabit SerDes transceivers for Ethernet, PCIe and similar links (Chapter 16).

The tool flow. AMD's Vivado, Intel's Quartus and the open-source Yosys + nextpnr all follow the same steps:

1. **Synthesis**: RTL becomes a netlist of LUTs, flip-flops, carry chains, BRAMs and DSPs. Check the utilization report and the warnings (inferred latches, removed logic, multi-driven nets).
2. **Placement**: each element is assigned a physical site on the chip.
3. **Routing**: connections are made through the programmable interconnect. On an FPGA, routing is often more than half of each path's delay.
4. **Timing analysis**: static timing checks every path against the constraints (Chapter 11). A design is *not done* until timing is met; a failing design may work on the bench today and fail when warm.
5. **Bitstream generation**: the configuration file loaded into the FPGA, through JTAG during development or from flash memory at power-up.

Constraints are written in SDC-style Tcl (Vivado calls the files XDC): `create_clock` defines each clock, I/O delays describe the outside world, and false-path or multicycle exceptions mark paths that do not need single-cycle timing. Asynchronous clock groups tell the tool not to time CDC paths, which then must be safe by design (Section 19.6).

Closing timing on an FPGA. When a path fails:

- add pipeline stages, cutting the LUT levels between registers (registers are free on an FPGA, so this is the first thing to try);
- reduce fanout: a signal driving thousands of loads, such as a reset or an enable, is slow, so register and duplicate it;
- use the hard blocks with their internal registers (BRAM output register, DSP pipeline registers);
- let the tool retime (move registers across logic), or rewrite the logic so the critical path is shorter;

- as a last resort, floorplan related logic into one region.

Resets and initial values on FPGAs. The configuration process loads a known value into every flip-flop, so on FPGAs `initial` blocks or declaration initializers (`logic [3:0] cnt = 0;`) really do set power-up values, unlike on an ASIC. Vendors advise using resets only where needed, and synchronous ones: a reset on every flip-flop costs routing and makes BRAMs and DSP registers harder to use.

The big idea

Writing RTL for an FPGA is writing for a fixed set of parts. Know what your code becomes: logic becomes LUTs counted in levels, `+` becomes a carry chain, `*` becomes a DSP slice, a memory with a registered read becomes block RAM, and every LUT comes with a free flip-flop for pipelining.

19.14 Summary

- HDLs describe parallel hardware. `assign` and `always @(*)` make combinational logic; `always @(posedge clk)` makes flip-flops.
- Nonblocking `<=` updates all left sides together at the end of the step (use in clocked blocks). Blocking `=` updates immediately (use in combinational blocks).
- Missing assignments in combinational blocks infer latches.
- Asynchronous resets need synchronous release. A delay-and-combine filter rejects reset glitches shorter than the delay.
- Single bits cross clock domains through two-flop synchronizers; multi-bit values need Gray code, handshakes or asynchronous FIFOs.
- FIFO depth $\geq B(1 - r_r/r_w)$, with rates in words per second.
- Verification: a self-checking testbench compares the design against a reference model. Directed tests hit known corner cases; constrained-random tests find the unexpected. Assertions catch rule violations where they happen. Code coverage shows what ran; functional coverage shows which planned scenarios occurred; neither replaces checking. Formal tools prove properties for all inputs.
- SystemVerilog: `logic`, `always_ff`/`always_comb`, `enum` states, `unique case`, packed structs, interfaces and packages make intent checkable. Avoid `casex`.
- VHDL: entity + architecture; signals (`<=`) update at the end of the process like nonblocking assignments, variables (`:=`) immediately; conversions are explicit.
- FPGAs: logic becomes k-input LUTs counted in levels, adders use carry chains, multipliers DSP slices, registered-read memories block RAM. Flow: synthesis, place, route, timing, bitstream. Close timing by pipelining and cutting fanout.

19.15 Exercises

1. With `a = 3` and `b = 7`, what are `a` and `b` after one clock edge of `a = b; b = a;` (blocking)? Of `a <= b; b <= a;`?
2. In the `$display` example of Section 19.3, what is printed at the third clock edge?
3. Will this infer a latch? `always @(*) case (sel) 2'b00: y = a; 2'b01: y = b; endcase` How would you fix it?
4. Why do we use two flip-flops to synchronize a single bit? Why is this not enough for an 8-bit bus?
5. A writer at 150 MHz sends bursts of 300 words; a reader at 100 MHz reads one word per cycle. What FIFO depth is needed?
6. Why are asynchronous FIFO pointers passed between domains in Gray code?
7. Glitches up to 15 ns must be removed from a reset line using 2 ns buffers. How many buffers are needed?
8. Give one advantage of a synchronous reset and one of an asynchronous reset.
9. Write a Verilog module for a 4-bit counter with enable and synchronous active-high reset.
10. What makes a testbench "self-checking"? What is a reference model?
11. Give one strength of directed tests and one of constrained-random tests.
12. What is the difference between an immediate and a concurrent assertion? Write a concurrent assertion that `gnt` is never high unless `req` is high in the same cycle.
13. A block reaches 100% line coverage. Name two kinds of bug that could still be present.
14. What can formal verification show that simulation cannot?
15. Write a SystemVerilog `enum` for a four-state machine and the `always_comb` next-state block skeleton that cannot infer a latch.
16. Why is `casex` discouraged, and what does `casez` do differently?
17. In VHDL, `a` and `b` are signals. What do `a <= b; b <= a;` inside a clocked process do?
18. How many 6-input LUTs, in how many levels, does a 16-input AND need on an FPGA?
19. A memory written with an asynchronous read does not map to block RAM. Why, and how do you fix it?

Answers

1. Blocking: `a = 7`, `b = 7`. Nonblocking: `a = 7`, `b = 3`.
2. The values have swapped back by then: after two edges `a = 1, b = 2`, so the third edge prints `a=1 b=2`.
3. Yes: for `sel` values 10 and 11, `y` is not assigned, so it must hold its value. Add `default: y = 0;` (or assign `y` a default before the `case`).

4. The first flip-flop may go metastable; the second gives it a full cycle to resolve, making failures extremely rare. For a bus, the bits may be captured on different cycles, giving a mixed value that never existed.

5. Burst time = 300/150 MHz = 2 μs; read in that time = 200; depth = 100.

6. Only one bit changes per increment, so a pointer sampled mid-change is either the old or the new value, never a wrong one.

7. $k \times 2 > 15$, so 8 buffers (16 ns).

8. Synchronous: no timing surprises; filtered by the clock. Asynchronous: works without a running clock and acts immediately.

9. For example:

```
module counter4 (input wire clk, rst, en, output reg [3:0] q);
  always @(posedge clk) begin
    if (rst)      q <= 4'd0;
    else if (en)  q <= q + 4'd1;
  end
endmodule
```

10. It computes the expected result itself and compares it with the design's output automatically, reporting any mismatch. A reference model is a simple, trusted description of the correct behaviour (in SystemVerilog, C or Python) that produces those expected results.

11. Directed: hits a specific corner case on purpose and is easy to debug. Constrained-random: explores many legal combinations nobody thought to write, finding unexpected bugs.

12. An immediate assertion checks a condition at one instant, like an `if`; a concurrent assertion checks a property across clock cycles. For example: `assert property (@(posedge clk) gnt |-> req);`

13. Lines that ran but whose results were never checked (a wrong answer nobody compared), and combinations of conditions that never occurred together (for example, a write and a read in the same cycle while full). Line coverage says nothing about either.

14. That a property holds for *every* possible input sequence (up to a bound), or an exact counterexample if it does not. Simulation only checks the sequences that were run.

15. `typedef enum logic [1:0] {A, B, C, D} st_t;` then `always_comb begin next = state; case (state) ... endcase end`. The default assignment at the top covers every path.

16. `casex` treats `x` bits in the case *expression* as don't-cares, so an unknown input silently matches a branch and hides bugs in simulation. `casez` only treats `z`/`?` as don't-care.

17. They swap: signals take their new values when the process finishes, like nonblocking assignments.

18. 3 LUTs in the first level (6 + 6 + 4 inputs), then 1 to combine: 4 LUTs, 2 levels.

19. Block RAM reads are synchronous: the address (or data) is registered. Register the read (`always_ff @(posedge clk) dout <= mem[addr];`) and accept one cycle of read latency.

Chapter 20

Computer Architecture

Computer architecture is the level above circuits: how a processor is organized to run programs quickly. Interviewers for hardware internships often ask about it ("what are the stages of a five-stage pipeline?"), and NVIDIA hardware tests have included cache and memory-timing calculations. This chapter gives the essentials.

20.1 What a processor does

A processor runs a program stored in memory as a list of **instructions**. The set of instructions a processor understands, and how they are encoded, is its **instruction set architecture** (ISA): x86 in most PCs, ARM in phones, RISC-V in many new designs. A processor holds a small number of fast **registers**, and a **program counter** (PC) that points at the next instruction. Each instruction goes through the same cycle:

1. **Fetch** the instruction at the PC from memory.
2. **Decode** it: what operation, which registers.
3. **Execute**: do the arithmetic, compare, or compute an address.
4. Access **memory** if it is a load or store.
5. **Write back** the result to a register, and move the PC to the next instruction (or the branch target).

Instructions come in a few families: arithmetic and logic (`add r1, r2, r3`), memory (`load r1, 8(r2)` and `store`), and control flow (`branch if equal`, `jump`).

20.2 Measuring performance

The big idea

The **processor performance equation**:

$$\text{execution time} = \text{instruction count} \times \text{CPI} \times \text{clock period},$$

where CPI is the average number of clock cycles per instruction. Every architecture idea attacks one of these three factors.

Worked example: run time

A program executes 10^9 instructions with an average CPI of 1.5 on a 2 GHz processor (0.5 ns period). Time $= 10^9 \times 1.5 \times 0.5 \times 10^{-9} = 0.75$ s.

20.3 The five-stage pipeline

A simple processor could do all five steps of one instruction in one long clock cycle. A **pipelined** processor instead splits the steps into stages separated by registers (Section 11.10), and works on five different instructions at once, one in each stage, like an assembly line.

Seen on the assessment

"What are the stages in a five-stage pipelined processor?" is a reported NVIDIA hardware intern interview question. The classic answer: **IF** (instruction fetch), **ID** (instruction decode and register read), **EX** (execute, using the ALU), **MEM** (memory access for loads and stores), **WB** (write back to the register file).

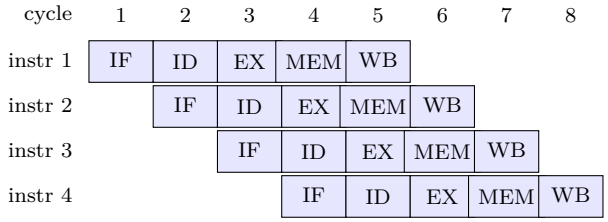


Figure 20.1: A five-stage pipeline. Once full, one instruction finishes every cycle, even though each takes five cycles.

Each instruction still takes five cycles (its latency), but a new one finishes every cycle (the throughput), so the ideal CPI is 1 and the clock can be about five times faster than a single-cycle design.

Worked example: pipeline speedup

A non-pipelined processor runs at 2.5 GHz but needs 5 cycles per instruction (CPI = 5). A pipelined version runs at 2 GHz (pipeline registers add overhead) with CPI = 1.
Non-pipelined: $2.5 \times 10^9/5 = 0.5$ billion instructions per second. Pipelined: $2 \times 10^9/1 = 2$

billion. Throughput speedup: $2/0.5 = 4\times$.

20.4 Hazards

Overlapping instructions causes three kinds of trouble, called **hazards**.

Structural hazards. Two instructions need the same hardware in the same cycle (for example, one memory port for both fetching and loading). Fixed by duplicating hardware, such as separate instruction and data caches.

Data hazards. An instruction needs a result that an earlier instruction has not yet written back:

```
add r1, r2, r3     # r1 is written in WB (cycle 5)
sub r4, r1, r5     # but sub reads r1 in ID (cycle 3)
```

The fix is **forwarding** (or bypassing): send the result straight from the end of the EX stage to the input of the next instruction's EX stage, without waiting for write-back. Forwarding fixes most data hazards with no delay.

One case it cannot fully fix is the **load-use hazard**: a load's data only arrives at the end of MEM, too late for an instruction right behind it that needs it in EX. The pipeline must **stall** (insert a bubble) for one cycle. Compilers reorder instructions to put something useful in that slot.

Control hazards. After a branch, the processor does not know which instruction comes next until the branch is resolved, but the pipeline keeps fetching. If it guessed wrong, the wrongly fetched instructions must be **flushed**, wasting cycles (the **branch penalty**). **Branch prediction** guesses the outcome: simple predictors remember each branch's recent behavior (for example with 2-bit saturating counters), and a branch target buffer remembers where it jumps. Modern predictors are right well over 90% of the time.

20.5 Amdahl's law

Speeding up one part of a task only helps as much as that part matters. If a fraction p of the work is sped up by a factor s:

$$\boxed{\text{overall speedup} = \frac{1}{(1-p) + p/s}}$$

Worked example: parallel speedup

80% of a program can run in parallel on 16 cores. Speedup $= 1/(0.2 + 0.8/16) = 1/0.25 = 4$. Even with infinitely many cores, the speedup can never exceed $1/0.2 = 5$: the serial 20% sets the ceiling.

20.6 The memory hierarchy

Processors are far faster than DRAM. Waiting about 100 ns for every memory access would waste hundreds of clock cycles each time. The solution is a hierarchy of memories, small and fast near the processor, large and slow further away:

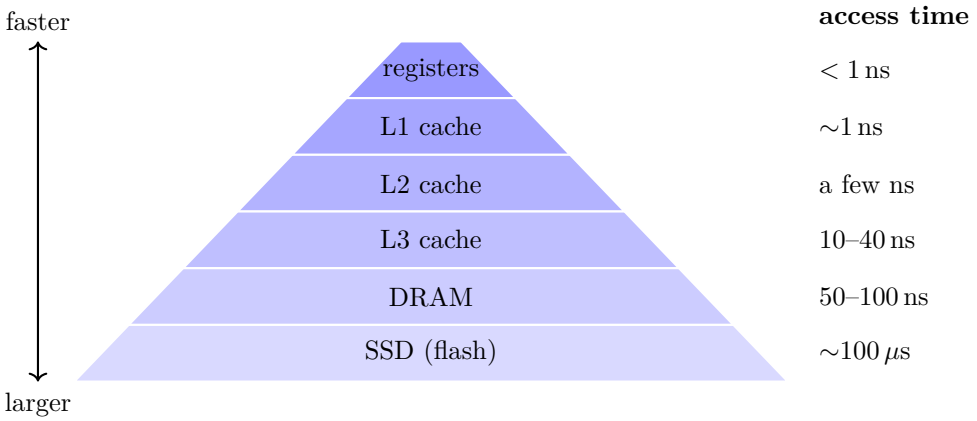


Figure 20.2: The memory hierarchy. Each level down is larger and slower; caches keep the data in use near the top.

Level	Typical size	Typical access time (approximate)
Registers	hundreds of bytes to kilobytes	under 1 ns
L1 cache (SRAM)	tens of KB	about 1 ns
L2 cache	hundreds of KB to a few MB	a few ns
L3 / last-level cache	many MB	10–40 ns
DRAM	gigabytes	50–100 ns
SSD (flash)	terabytes	about 100 µs

This works because programs have **locality**:

- **Temporal locality**: data used recently is likely to be used again soon (loop variables).
- **Spatial locality**: data near recently used data is likely to be used soon (array elements in order).

20.7 How a cache works

A cache stores copies of recently used memory in fixed-size blocks called **lines** (often 64 bytes; fetching a whole line exploits spatial locality). To find whether an address is in the cache, the address is split into three fields (Figure 20.3):



- **Offset** bits $= \log_2$(line size).
- **Index** bits $= \log_2$(number of sets).
- **Tag** bits = address bits − index bits − offset bits. Each line stores its tag so the cache can check whether it holds the right address, plus a **valid** bit and usually a **dirty** bit.

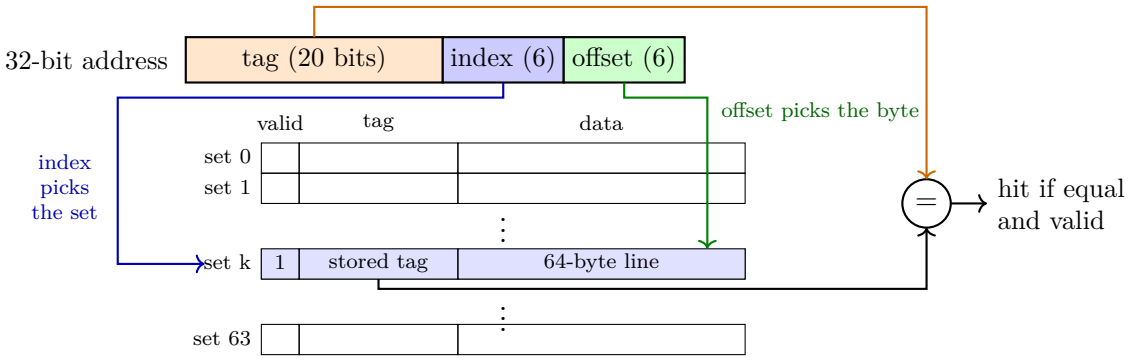


Figure 20.3: Looking up an address in a direct-mapped cache with 64 sets of 64-byte lines. The index picks one set; the tag stored there is compared with the address's tag; on a hit, the offset picks the byte within the line.

Associativity. In a **direct-mapped** cache, each address can go in exactly one line. In an N**-way set-associative** cache, each address maps to one *set* of N lines and can go in any of them. In a **fully associative** cache, it can go anywhere. More associativity means fewer conflicts but more tags to compare in parallel. When a set is full, a **replacement policy** (usually least recently used, LRU, or an approximation) picks a line to evict.

Writes. A **write-through** cache sends every write on to the next level immediately. A **write-back** cache only updates its own copy and marks the line *dirty*, writing it back when it is evicted, which saves bandwidth.

Why misses happen (the three Cs). **Compulsory** misses: the first access to a line. **Capacity** misses: the working data does not fit. **Conflict** misses: too many addresses map to the same set, even though the cache has room elsewhere (eliminated by full associativity).

Worked example: address fields and tag-array size

A 1 MB, 4-way set-associative cache has 64-byte lines and 32-bit addresses. Each line stores its tag plus a valid bit and a dirty bit. How big is the tag array?
Lines: $2^{20}/2^6 = 2^{14} = 16{,}384$. Sets: $16{,}384/4 = 4096 = 2^{12}$. Offset: 6 bits. Index: 12 bits. Tag: $32 - 12 - 6 = 14$ bits. Bits per line: $14 + 1 + 1 = 16$. Tag array: $16{,}384 \times 16 = 262{,}144$ bits = 32 KB.

20.8 Average memory access time

The big idea

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}.$$

With two levels, the L1 miss penalty is itself the AMAT of L2:

$$\text{AMAT} = t_{L1} + m_{L1}\left(t_{L2} + m_{L2}\, t_{mem}\right),$$

where m_{L2} is the *local* miss rate of L2 (the fraction of L2 accesses that miss).

Worked example: a two-level AMAT

L1 hits 80% of the time and L2 hits 40% of the accesses that reach it. Access times are 10 ns for L1, 30 ns for L2 and 100 ns for memory.

$$\text{AMAT} = 10 + 0.2\,(30 + 0.6 \times 100) = 10 + 0.2 \times 90 = 28 \text{ ns}.$$

Common trap

Some questions state each level's time as the *total* time for an access served at that level, not the extra time added on a miss. Then the answer is a weighted average of where accesses are served: $0.8 \times 10 + 0.2 \times 0.4 \times 30 + 0.2 \times 0.6 \times 100 = 8 + 2.4 + 12 = 22.4$ ns. Read the wording to decide which model the question uses, and say which one you assumed.

Worked example: memory stalls and CPI

A processor has a base CPI of 1.0. 30% of instructions are loads or stores, their L1 miss rate is 2%, and a miss costs 200 cycles. Memory stall cycles per instruction: $0.3 \times 0.02 \times 200 = 1.2$. Effective CPI: $1.0 + 1.2 = 2.2$. The memory system more than doubles the run time, which is why caches are worth so much silicon. To compare two memory systems (for example, L3 only versus L2 plus L3), compute the effective CPI for each and take the ratio.

20.9 Virtual memory

Programs use **virtual addresses**; the hardware translates them to **physical addresses** in DRAM (Figure 20.4). Memory is divided into **pages**, typically 4 KB. A per-process **page table** maps virtual page numbers to physical page numbers; the offset within the page (the low 12 bits for 4 KB pages) is unchanged by translation. A **translation lookaside buffer** (TLB), a small cache of recent translations, makes this fast. Accessing a page that is not in memory causes a **page fault**, handled by the operating system.

This is why "are two addresses on the same 4 KB page?" (Section 7.6) is a natural hardware question: same page means same translation.

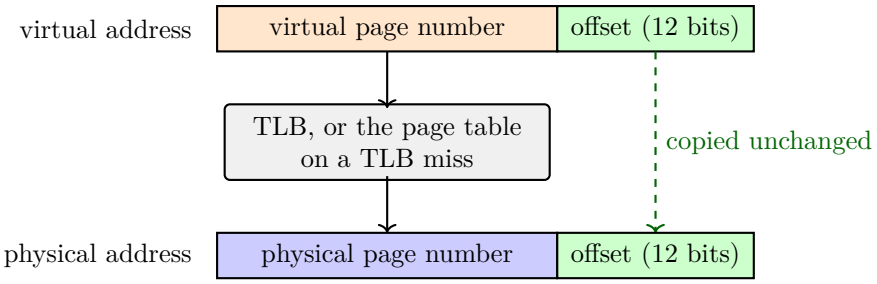


Figure 20.4: Address translation with 4 KB pages. The virtual page number is translated, by the TLB or, on a miss, the page table; the 12-bit offset within the page passes through unchanged.

20.10 Endianness

A 32-bit value occupies 4 bytes in memory. A **little-endian** machine (x86, most ARM and RISC-V systems) stores the least significant byte at the lowest address; a **big-endian** machine stores the most significant byte first. The value `0x12345678` stored at address `0x100`:

Address	`0x100`	`0x101`	`0x102`	`0x103`
Little-endian	`78`	`56`	`34`	`12`
Big-endian	`12`	`34`	`56`	`78`

20.11 Multiple cores

A chip with several cores, each with its own cache, must keep the caches **coherent**: if one core writes a value, other cores must not keep reading a stale copy. Coherence protocols such as MESI track whether each cached line is Modified, Exclusive, Shared or Invalid, and invalidate other copies when one core writes.

How coherence works

Every cached line is in one of four MESI states:

State	**Meaning**	**Matches memory?**
Modified	only this cache has it, and it has been written	no: this copy is the only up-to-date one
Exclusive	only this cache has it, unwritten	yes
Shared	other caches may hold it too; read-only	yes
Invalid	not usable	–

A core may write only a line it holds in M or E. To write a Shared line it first broadcasts an **invalidate**, turning every other copy to I, and moves to M. Exclusive exists so that the common case, reading data no other core has and then writing it, moves from E to M without any bus traffic.

When another core reads a Modified line, the owner supplies the data (or writes it back) and both copies become Shared.

Small systems implement this by **snooping**: every cache watches a shared bus for other cores' requests. Snooping traffic grows with the number of cores, so large chips use a **directory** that records which caches hold each line and sends messages only to them.

Common trap

False sharing: two cores write *different* variables that happen to sit in the same 64-byte line. Neither shares any data, yet every write invalidates the other core's copy, and the line bounces between caches. The fix is to pad or align the variables onto separate lines.

20.12 Out-of-order execution

The five-stage pipeline stalls whenever an instruction waits, for example on a cache miss, even if later instructions are independent and could run. An **out-of-order** (OoO) core lets them go ahead, while still making the program appear to run in order.

Register renaming. Code reuses a few architectural registers, which creates false dependences: if instruction 1 reads `r1` and instruction 2 later writes `r1`, instruction 2 must not finish first (a write-after-read hazard), although no data passes between them. Write-after-write is similar. Renaming maps each new write to a fresh *physical* register, from a pool much larger than the architectural set, so only true read-after-write dependences remain.

The machinery.

1. The front end fetches and decodes several instructions per cycle, predicting branches far ahead.
2. Renamed instructions wait in **reservation stations** (an issue queue) until their operands are ready, then **issue** to any free execution unit, oldest ready first.
3. Results are broadcast to the waiting instructions that need them.
4. The **reorder buffer** (ROB) holds every instruction in program order. An instruction **retires** (commits its result permanently) only when everything before it has retired.

Because results become permanent only at retirement, the core can **speculate**: execute past a predicted branch, and on a misprediction or an exception discard every younger instruction in the ROB. A larger ROB lets the core look further ahead to find independent work and keep more cache misses in flight at once.

Remember

OoO: rename to remove false dependences (WAR, WAW), execute when operands are ready, retire in order from the reorder buffer. In-order retirement is what makes speculation and precise exceptions possible.

20.13 Inside a modern core: prediction, prefetching, SIMD and memory ordering

Superscalar execution. A **superscalar** core fetches, decodes and executes several instructions per cycle, so its ideal CPI is below 1 (equivalently, its IPC, instructions per cycle, is above 1). How many really run together is limited by **instruction-level parallelism** (ILP): dependences, branches and cache misses. Out-of-order execution (Section 20.12) exists to find more of it.

Branch prediction. A wide, deep pipeline fetches dozens of instructions before a branch resolves, so a wrong guess wastes all of them. Predictors learn from history:

- A **2-bit saturating counter** per branch counts up when taken and down when not taken (0–3); predict taken when the counter is 2 or 3. One unusual outcome, such as a loop's final exit, does not flip the prediction, unlike a 1-bit scheme that mispredicts twice per loop.
- A **branch history table** indexes counters by branch address bits. *Global history* predictors (such as gshare, which XORs the address with the recent outcomes of all branches) capture branches that depend on each other. Modern cores combine several predictors and reach well above 95% accuracy.
- The **branch target buffer** (BTB) caches the *target address* of taken branches so fetch can redirect in the same cycle, and a **return address stack** predicts function returns.

Worked example: The cost of mispredictions

A core has a base CPI of 1. 20% of instructions are branches, the predictor is 95% accurate, and a misprediction costs 15 cycles. What is the CPI?
Mispredictions per instruction $= 0.20 \times 0.05 = 0.01$, each costing 15 cycles, so CPI $= 1 + 0.01 \times 15 = 1.15$. Improving accuracy to 98% gives $1 + 0.2 \times 0.02 \times 15 = 1.06$.

Prefetching. A **prefetcher** predicts which cache lines will be needed and fetches them before the program asks, hiding memory latency. A next-line prefetcher fetches the following line; a *stride* prefetcher spots accesses at a constant distance (walking an array of structs) and runs ahead. Prefetching helps regular access patterns and can hurt when it guesses wrong, by wasting bandwidth and evicting useful lines. Pointer chasing (a linked list) defeats it, which is one reason arrays are faster than lists.

SIMD and vectors. A **SIMD** instruction (single instruction, multiple data) applies one operation to a whole vector of values at once: a 256-bit register holds eight 32-bit floats, so one add does eight. x86 has SSE, AVX and AVX-512; Arm has NEON and SVE; RISC-V has the V extension, whose vector length is set by the hardware so the same code runs on narrow and wide implementations. GPUs (Chapter 21) take the same idea much further.

Multithreading. **Simultaneous multithreading** (SMT, sold as hyper-threading) runs two or more threads on one core, sharing its execution units: when one thread stalls on a miss, the other's instructions fill the slots.

Memory ordering. With one core, memory operations appear to happen in program order. With several cores, each with a store buffer and caches, another core may see your writes in a different order. The **memory consistency model** says which orders are allowed.

- **Sequential consistency** (the intuitive model): all cores see one interleaving of everyone's operations, each core's in program order. Easy to reason about, slow to implement.
- **x86 (TSO, total store order)**: a core may read memory before its own earlier stores reach the cache (they wait in the store buffer), but stores become visible to others in order.
- **Arm and RISC-V (weakly ordered)**: loads and stores to different addresses may be reordered freely, unless the program asks otherwise.
- **Memory barriers** (fences: `dmb` on Arm, `fence` on RISC-V, `mfence` on x86) force earlier memory operations to complete before later ones. Acquire and release operations are lighter one-way barriers used to build locks.
- Classic example: core 0 writes `data = 42;` then `flag = 1;`; core 1 waits for `flag == 1` and reads `data`. On a weakly ordered machine, core 1 can see the flag before the data unless core 0 puts a release barrier (or fence) between its stores and core 1 an acquire barrier after reading the flag. Drivers face the same issue with device registers and DMA buffers.

Coherence (Section 20.11 earlier in this chapter) guarantees that all cores agree on the order of writes to *one* address; consistency is about the order of accesses to *different* addresses.

20.14 On-chip buses: AXI

Blocks inside a chip or an FPGA design talk over standard interfaces, most often ARM's AMBA **AXI**. A **manager** (master), such as a CPU or DMA engine, starts transactions; a **subordinate** (slave), such as a memory controller or a register block, answers them. AXI has five independent channels, each a valid/ready handshake (Section 19.9): write address (AW), write data (W), write response (B), read address (AR) and read data (R). A transfer on any channel happens on a clock edge where both `VALID` and `READY` are high. Two rules prevent deadlock: a source must not wait for `READY` before raising `VALID`, and once `VALID` is high it must stay high with stable data until the transfer happens.

A single address can start a **burst** of up to 256 data beats, and transaction IDs let a subordinate return responses out of order. **AXI4-Lite** drops bursts and IDs for simple control registers, and **AXI4-Stream** drops addresses entirely: just `TVALID`, `TREADY`, `TDATA` and `TLAST` marking the end of a packet, the standard way to pass packet data between blocks in FPGA designs.

20.15 Summary

- Execution time = instruction count × CPI × clock period.
- Five-stage pipeline: IF, ID, EX, MEM, WB. Ideal CPI of 1.
- Hazards: structural (duplicate hardware), data (forwarding; load-use needs a stall), control (branch prediction, flush).
- Amdahl: speedup = $1/((1-p)+p/s)$; the serial part sets the limit.

- Caches exploit temporal and spatial locality. Address = tag | index | offset.
- AMAT = hit time + miss rate × miss penalty, applied level by level.
- Virtual memory translates pages (4 KB); the TLB caches translations.
- Little-endian stores the least significant byte first.
- Coherence (MESI): write only in M or E; writing a Shared line first invalidates other copies. Snooping for few cores, directories for many. False sharing bounces a line between cores.
- Out-of-order cores rename registers to remove false dependences, issue instructions when their operands are ready, and retire in order from the reorder buffer.
- AXI: five valid/ready channels (AW, W, B, AR, R); AXI4-Lite for registers, AXI4-Stream for packet data.

20.16 Exercises

1. A program runs 2×10^9 instructions at CPI 1.2 on a 3 GHz processor. How long does it take?
2. Name the five stages of the classic pipeline and say what each does.
3. Why does a load followed immediately by an instruction using its result need a stall, even with forwarding?
4. 90% of a task is parallel. What is the speedup on 10 cores? The maximum on unlimited cores?
5. A 32 KB direct-mapped cache has 64-byte lines and 32-bit addresses. Find the offset, index and tag widths.
6. A cache has a 1 ns hit time, 5% miss rate and 100 ns miss penalty. Find the AMAT.
7. Which kind of miss does full associativity eliminate?
8. Show how `0xA1B2C3D4` is stored at address 0 in a little-endian machine.
9. Base CPI 1.0, 40% loads/stores, 5% miss rate, 100-cycle penalty. What is the effective CPI?
10. Two threads update counters `a` and `b`, which share a cache line, on different cores. Neither reads the other's counter, yet both run slowly. Why, and what is the fix?
11. Which hazards does register renaming remove, and which remain?
12. On an AXI channel, may a source wait for `READY` before asserting `VALID`?

Answers

1. $2 \times 10^9 \times 1.2 / (3 \times 10^9) = 0.8$ s.
2. IF fetches the instruction; ID decodes it and reads registers; EX computes in the ALU; MEM reads or writes data memory; WB writes the result to the register file.
3. The loaded value is only available at the end of MEM, but the next instruction needs it at the start of its EX stage, which happens in the same cycle; it must wait one cycle.
4. $1/(0.1 + 0.09) \approx 5.3$. Maximum $1/0.1 = 10$.

5. Offset 6 bits; lines $= 2^{15}/2^{6} = 512$, so index 9 bits; tag $32 - 9 - 6 = 17$ bits.
6. $1 + 0.05 \times 100 = 6$ ns.
7. Conflict misses.
8. Address 0: `D4`, 1: `C3`, 2: `B2`, 3: `A1`.
9. $1.0 + 0.4 \times 0.05 \times 100 = 3.0$.
10. False sharing: each write invalidates the other core's copy of the line. Put the counters in separate 64-byte lines.
11. It removes write-after-read and write-after-write (false) dependences; read-after-write (true) dependences remain.
12. No. That could deadlock if the destination waits for `VALID`. The source raises `VALID` when it has data and holds it until the transfer.

Chapter 21

GPUs and Parallel Computing

NVIDIA builds GPUs, so it is worth understanding what makes a GPU different from a CPU and why it is so good at graphics and AI. This chapter ties together many earlier ideas: parallelism to save power (Chapter 14), memory bandwidth (Chapter 16), caches (Chapter 20) and number formats (Chapter 7). The Ignite posting lists familiarity with GPU programming (CUDA or OpenCL) as a plus.

21.1 Two philosophies: latency and throughput

A **CPU** is designed to finish *one* thread of instructions as fast as possible. It spends its transistors on large caches, branch prediction, and complex out-of-order logic that finds independent instructions to run early. It has a handful of powerful cores.

A **GPU** is designed to finish a *huge number* of similar, independent tasks at the greatest total rate. It spends its transistors on thousands of simple arithmetic units, and it tolerates slow memory not by avoiding waits but by always having other work ready.

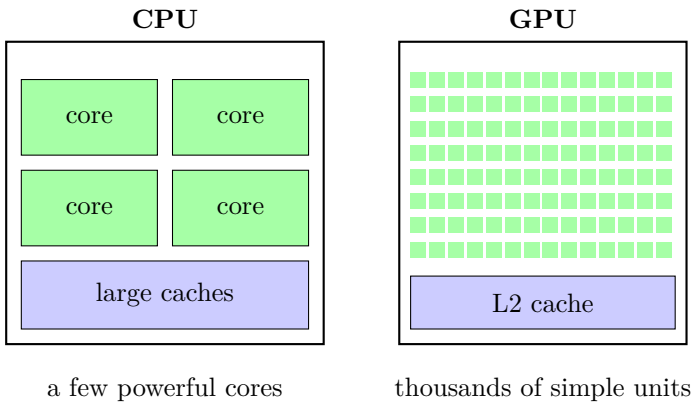


Figure 21.1: Where the transistors go (schematic, not to scale). A CPU spends its area on a few powerful cores and large caches; a GPU spends it on thousands of simple arithmetic units (green).

An everyday picture

A CPU is a few expert chefs who can cook any dish quickly. A GPU is a giant kitchen with thousands of line cooks, each doing the same simple step on a different plate. For one complicated dish, the expert wins. For ten thousand identical burgers, the kitchen wins by a mile.

Graphics and AI are full of identical, independent work: every pixel on the screen, every element of a large matrix multiplication. That is why GPUs excel at both.

21.2 How a GPU is organized

- The chip contains many **streaming multiprocessors** (SMs; Figure 21.2). Each SM has many arithmetic units ("CUDA cores"), special units for matrix math (tensor cores), a very large **register file**, and a fast on-chip SRAM called **shared memory** (configurable together with the L1 cache).
- All SMs share a large **L2 cache**, which connects to high-bandwidth DRAM (HBM or GDDR) through very wide memory interfaces.
- Threads are grouped into **warps** of 32 threads. All 32 threads of a warp execute the same instruction at the same time, each on its own data. NVIDIA calls this **SIMT**: single instruction, multiple threads.

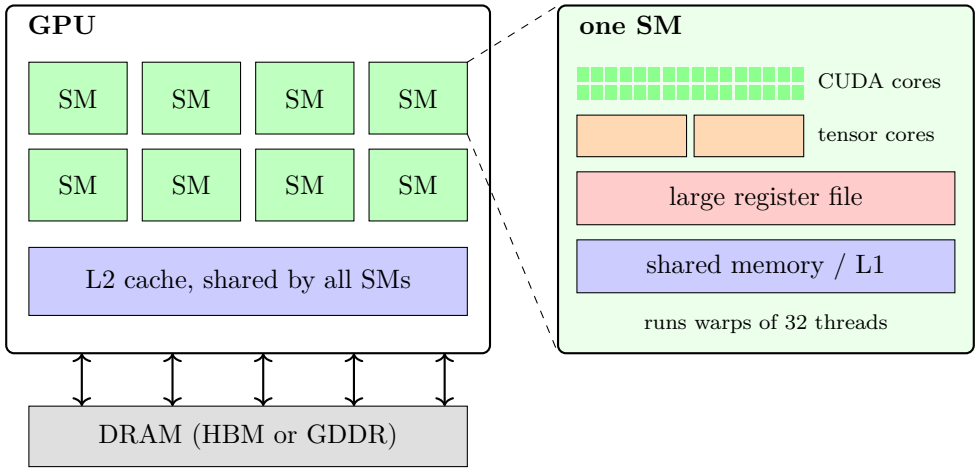


Figure 21.2: How a GPU is organized (schematic). Many streaming multiprocessors (SMs) share an L2 cache and very wide DRAM interfaces. Each SM has its own arithmetic units, tensor cores, register file and shared memory.

21.3 Hiding latency with many threads

A memory access to DRAM takes hundreds of cycles. A CPU tries to avoid that wait with big caches and prediction. A GPU instead keeps many warps resident on each SM, with all of their registers

held in the large register file. When one warp stalls waiting for memory, the warp scheduler simply issues an instruction from another warp that is ready, with no cost to switch. As long as there are enough ready warps, the arithmetic units stay busy. The fraction of the maximum possible warps actually resident on an SM is called **occupancy**.

21.4 Two performance pitfalls

Divergence. The 32 threads of a warp share one instruction stream. If an `if` statement sends some threads one way and others the other way, the warp must execute *both* paths, one after the other, with the inactive threads masked off. In the worst case, half the threads are idle for each path, halving throughput. GPU code is fastest when threads in a warp follow the same path.

Uncoalesced memory access. When the 32 threads of a warp read memory, the hardware combines (**coalesces**) their requests into as few wide memory transactions as possible. If thread i reads element i of an array, the 32 reads fall into a few consecutive lines: perfectly coalesced. If each thread reads an element far from its neighbors' (a large stride), every thread may need its own transaction, wasting most of the bandwidth.

21.5 The CUDA programming model

In CUDA, the programmer writes a **kernel**: a function that one thread runs. The kernel is launched on a **grid** of **thread blocks**, each block containing many threads (up to 1024). Blocks are assigned to SMs; threads within a block can cooperate through shared memory and synchronize with `__syncthreads()`. Each thread finds its own piece of work from its indices.

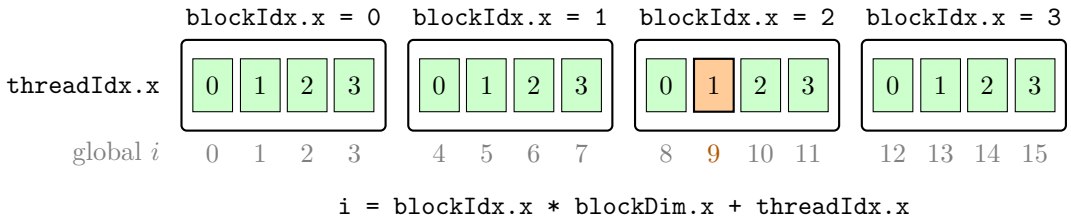


Figure 21.3: A grid of four thread blocks with four threads each (real blocks have up to 1024). The numbers in the boxes are `threadIdx.x`; each thread computes its own global index i. With `blockDim.x` $= 4$, thread 1 of block 2 gets $i = 2 \times 4 + 1 = 9$.

```
// Each thread adds one pair of elements: c[i] = a[i] + b[i]
__global__ void vec_add(const float *a, const float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // global thread index
    if (i < n)                                       // guard the last block
        c[i] = a[i] + b[i];
}

// Launch enough blocks of 256 threads to cover n elements:
// vec_add<<<(n + 255) / 256, 256>>>(a, b, c, n);
```

Consecutive threads touch consecutive elements, so the accesses are coalesced, and there is no divergence except in the final partial block.

21.6 Matrix multiplication and tensor cores

Neural networks spend most of their time multiplying matrices. Multiplying an $M \times K$ matrix by a $K \times N$ matrix takes MNK multiply-adds, which is $2MNK$ floating-point operations (FLOPs).

Worked example: counting FLOPs

Multiplying two 4096×4096 matrices takes $2 \times 4096^3 \approx 1.37 \times 10^{11}$ FLOPs. A GPU sustaining 500 TFLOP/s (5×10^{14} per second) does it in about 0.27 ms.

Tensor cores are hardware units that compute a whole small matrix multiply-and-add ($D = A \times B + C$ on small tiles) in one instruction, often using reduced-precision inputs. They deliver many times the throughput of ordinary arithmetic units for this one operation, which is why they dominate AI performance.

21.7 Number formats

Floating point. A floating-point number is stored as a sign bit s, an exponent e and a fraction (mantissa) m, and represents

$$(-1)^s \times 1.m \times 2^{\,e-\text{bias}}.$$

For 32-bit IEEE floating point (FP32), the exponent has 8 bits with a bias of 127, and the fraction has 23 bits. The exponent sets the *range* (how large or small a number can be); the fraction sets the *precision* (how many significant digits).

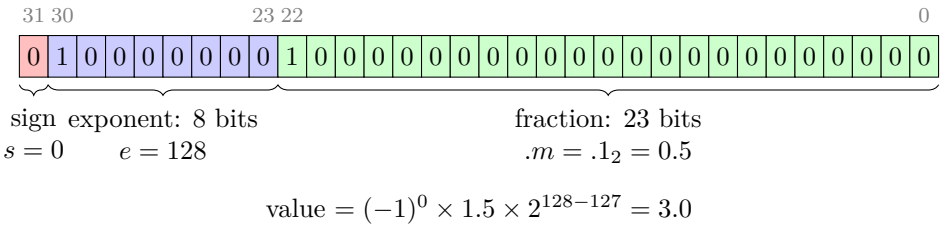


Figure 21.4: The FP32 layout, filled in with `0x40400000` from the worked example: 1 sign bit, 8 exponent bits and 23 fraction bits.

Worked example: decoding an FP32 number

`0x40400000` $= 0\ 10000000\ 10000000000000000000000_2$. Sign 0 (positive). Exponent $128 - 127 = 1$. Fraction $.1_2 = 0.5$, so $1.m = 1.5$. Value: $1.5 \times 2^1 = 3.0$. Likewise `0x3F800000` is 1.0, and `0xC0000000` is -2.0.

Special patterns represent zero, infinity and NaN ("not a number," for results like 0/0).

Format	Sign	Exponent	Fraction	Notes
FP32	1	8	23	standard single precision
FP16	1	5	10	half precision; smaller range
BF16	1	8	7	same range as FP32, less precision
FP8 (E4M3)	1	4	3	more precision, less range
FP8 (E5M2)	1	5	2	more range, less precision
INT8	8-bit two's complement integer			used with a scale factor

Smaller formats move half or a quarter of the bytes and let each multiplier be much smaller, so throughput and energy efficiency rise sharply. AI training and inference tolerate reduced precision well, which is why GPUs keep adding smaller formats. **BF16** keeps FP32's 8-bit exponent, so it covers the same range of values and rarely overflows, at the cost of precision.

21.8 Compute-bound or memory-bound?

Every computation needs both arithmetic and data. Its **arithmetic intensity** is the number of FLOPs it performs per byte moved from memory. A GPU with peak compute P (FLOP/s) and memory bandwidth B (bytes/s) is balanced at an intensity of P/B:

- below that, the program is **memory-bound**: the arithmetic units wait for data;
- above it, the program is **compute-bound**.

Plotting achievable performance against intensity gives the **roofline model**.

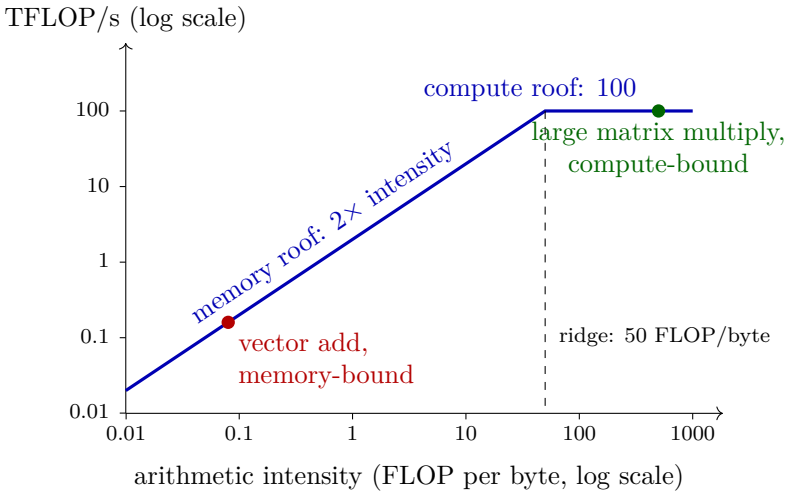


Figure 21.5: The roofline for a GPU with 100 TFLOP/s of compute and 2 TB/s of memory bandwidth. Below 50 FLOP per byte, performance is capped by bandwidth (the slanted roof); above it, by compute.

Worked example: is vector add compute-bound?

A GPU delivers 100 TFLOP/s and 2 TB/s, so it balances at $100 \times 10^{12} / 2 \times 10^{12} = 50$ FLOP per byte. Vector add (`c[i] = a[i] + b[i]` in FP32) does 1 FLOP per 12 bytes (two 4-byte loads and one 4-byte store): about 0.08 FLOP/byte, far below 50. It is heavily memory-bound; only bandwidth matters. A large matrix multiply reuses each loaded value many times, reaching high intensity, so it can use the full compute.

This is why GPUs need enormous memory bandwidth (HBM, Chapter 18) and why so much GPU programming is about reusing data held in registers and shared memory.

21.9 Summary

- CPUs optimize latency for a few threads; GPUs optimize throughput for many.
- GPUs are built from SMs; threads run in warps of 32 under SIMT.
- GPUs hide memory latency by switching among many resident warps.
- Divergence serializes branches within a warp; uncoalesced accesses waste bandwidth.
- CUDA: a kernel runs per thread, over a grid of blocks; index = `blockIdx.x * blockDim.x + threadIdx.x`.
- Matrix multiply = $2MNK$ FLOPs; tensor cores accelerate it.
- Floating point = $(-1)^s \times 1.m \times 2^{e-\text{bias}}$; exponent bits set range, fraction bits set precision. BF16 has FP32's range.
- Arithmetic intensity versus the GPU's FLOP/byte balance decides compute-bound or memory-bound.

21.10 Exercises

1. In one sentence each, what is a CPU optimized for and what is a GPU optimized for?
2. How many threads are in a warp? What does SIMT mean?
3. Half the threads of a warp take the `if` branch and half take the `else`. Roughly what fraction of the warp's peak throughput is achieved during that code?
4. Thread i reads `a[i * 64]` (4-byte floats). Is this coalesced? Why or why not?
5. How many FLOPs does a 1024×1024 by 1024×1024 matrix multiply take?
6. What is the arithmetic intensity of `y[i] = a * x[i] + y[i]` in FP32? Is it compute-bound or memory-bound on a typical GPU?
7. Decode the FP32 bit pattern `0x40400000`.
8. What does BF16 keep from FP32, and what does it give up?
9. How are the 8 bits of FP8 E4M3 divided?

Answers

1. A CPU minimizes the time to finish a single thread (latency); a GPU maximizes the total amount of parallel work finished per second (throughput).
2. 32. One instruction is issued for all threads of a warp, each thread applying it to its own data.
3. About half: the warp runs both paths one after the other, with half the threads idle in each.
4. No. Consecutive threads are 256 bytes apart, so each thread's access falls in a different memory line, requiring many separate transactions.
5. $2 \times 1024^3 \approx 2.15 \times 10^9$.
6. 2 FLOPs (a multiply and an add) per 12 bytes (load x, load y, store y): about 0.17 FLOP/byte. Memory-bound.
7. 3.0 (sign 0, exponent $128 - 127 = 1$, significand 1.5).
8. It keeps FP32's 8-bit exponent, and therefore its range. It gives up precision: 7 fraction bits instead of 23.
9. 1 sign bit, 4 exponent bits, 3 fraction (mantissa) bits.

Chapter 22

Computer Arithmetic: How Hardware Does Math

A GPU is, more than anything, a machine for doing arithmetic: thousands of adders and multipliers working at once. NVIDIA's Ignite posting lists *computer arithmetic* by name as a way to stand out. Chapter 7 showed how numbers are stored as bits, Chapter 9 built an adder, and Chapter 21 introduced floating-point formats. This chapter puts those pieces to work: what happens when a result does not fit, how hardware shifts, adds quickly, multiplies and divides, how fractions are stored as fixed-point numbers, and how a floating-point unit actually adds and multiplies.

The big idea

Every arithmetic circuit is a trade between three things: **speed** (how many gate delays from input to output), **area and power** (how many gates), and **accuracy** (how many bits, and how results are rounded). Faster adders and multipliers spend extra gates; smaller number formats save gates and energy by giving up precision.

22.1 When a result does not fit: carry, overflow and saturation

An n-bit adder always produces an n-bit result, plus a carry out of the top bit. The *same* adder works for unsigned and for two's-complement signed numbers (Section 7.5); only the meaning of "the result did not fit" differs.

Unsigned overflow: the carry. For unsigned numbers, the result is too big exactly when there is a **carry out** of the most significant bit (MSB). In 8 bits, $200 + 100 = 300$, which does not fit in 0–255: the adder produces $300 - 256 = 44$ and a carry of 1.

Signed overflow: the overflow flag. For signed numbers, the carry out means nothing on its own. Signed overflow happens when two numbers of the same sign produce a result of the opposite sign. Hardware detects it with one XOR gate:

$$V = c_{\text{into MSB}} \oplus c_{\text{out of MSB}}.$$

If a carry goes into the sign bit but none comes out (or the reverse), the sign bit has been corrupted.

Worked example: the four flags

Add 0110 0100 (100) and 0011 0010 (50) in 8 bits. The result is 1001 0110.

- As unsigned numbers: $100 + 50 = 150$, which fits in 0–255. There is no carry out, so $C = 0$, and $1001\,0110 = 150$ is correct.
- As signed numbers: $100 + 50 = 150$ does not fit in -128 to 127. The result reads as -106. Bit 6 adds 1, 0 and an incoming carry, so a carry goes into the sign bit, but none comes out of it, so $V = 1$.

Processors record these as status **flags**: C (carry), V (overflow), N (negative: the sign bit) and Z (zero: every bit is 0). Here $C = 0$, $V = 1$, $N = 1$, $Z = 0$.

Wrap or saturate? When a result does not fit, ordinary integer hardware **wraps around**, keeping only the low n bits. That is right for counters and addresses, but disastrous for a pixel brightness or an audio sample: a bright pixel that wraps becomes black. **Saturating arithmetic** instead clamps the result to the largest or smallest representable value.

8-bit operation	Wrapping result	Saturating result
unsigned $200 + 100$	44	255
unsigned $50 - 80$	226	0
signed $100 + 50$	-106	127
signed $-100 - 50$	106	-128

Saturation costs only a little logic: detect overflow (C or V), and if it happened, select the maximum or minimum constant with a multiplexer instead of the adder's output. Digital signal processors, image pipelines and quantized neural-network hardware use it everywhere.

Common trap

The carry flag is not the overflow flag. Adding -1 and -1 in 8 bits ($1111\,1111 + 1111\,1111$) produces a carry out, but the signed result $1111\,1110 = -2$ is perfectly correct, so $V = 0$.

22.2 Shifting

Logical and arithmetic shifts. Shifting left by k multiplies by 2^k (if nothing falls off the top). Shifting right divides by 2^k, and there are two kinds (Section 7.6): a **logical** right shift fills with zeros (for unsigned numbers), and an **arithmetic** right shift copies the sign bit in from the left, so negative numbers stay negative.

Worked example: arithmetic shifts round down

-8 in 8 bits is 1111 1000. Arithmetic right shift by 1: 1111 1100 $= -4$. Correct.
-7 is 1111 1001. Arithmetic right shift by 1: 1111 1100 $= -4$, not -3. An arithmetic shift rounds toward $-\infty$ (it computes $\lfloor -3.5 \rfloor = -4$), while integer division in C rounds toward zero ($-7/2 = -3$). Compilers add a small correction when they replace a signed division by a shift.

The barrel shifter. A processor must shift by any amount in a single cycle. A **barrel shifter** does it with $\log_2 n$ layers of 2:1 multiplexers. For a 32-bit value, the five bits of the shift amount control five layers that shift by 16, 8, 4, 2 and 1, each either shifting or passing the value straight through (Figure 22.1). Any amount from 0 to 31 is a sum of those powers of two. The cost: 5 layers of 32 multiplexers, 160 in all, but only five multiplexer delays.

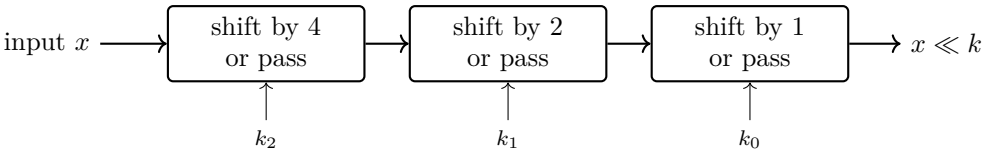


Figure 22.1: An 8-bit barrel shifter. Each layer is a row of 2:1 multiplexers controlled by one bit of the shift amount $k = k_2k_1k_0$. Shifting by 5 (101_2) means: shift by 4, pass, shift by 1.

Multiplying by a constant. Multiplying by a fixed number needs no multiplier at all, only shifts (which are free wiring) and adds:

$$10x = 8x + 2x = (x \ll 3) + (x \ll 1), \qquad 7x = 8x - x = (x \ll 3) - x.$$

The second form uses a subtraction to replace two additions, the same trick Booth recoding uses below.

22.3 Adding faster

Chapter 9 built a ripple-carry adder: simple, but the carry may have to travel through every bit, so a 64-bit add takes 64 full-adder delays. Faster adders break that chain.

Generate and propagate. For bit i, define

$$G_i = A_iB_i \qquad \text{(this bit generates a carry: both bits are 1)},$$
$$P_i = A_i \oplus B_i \qquad \text{(this bit propagates an incoming carry: exactly one bit is 1)}.$$

Then the carry out of bit i is $c_{i+1} = G_i + P_ic_i$: a carry leaves bit i if the bit makes one, or passes one along. Substituting each equation into the next gives every carry directly from the inputs:

$$c_1 = G_0 + P_0c_0$$
$$c_2 = G_1 + P_1G_0 + P_1P_0c_0$$
$$c_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0c_0$$
$$c_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0c_0$$

Each line is just AND gates feeding an OR gate, so all four carries are ready after about two gate delays instead of four full-adder delays. This is a 4-bit **carry-lookahead adder** (CLA). The equations grow quickly, so wide adders apply the same idea in a tree: each 4-bit group computes a group generate and group propagate, and a second level of lookahead combines groups. **Prefix adders** (Kogge–Stone, Brent–Kung) are the systematic version, with delay growing as $\log_2 n$: about 6 levels for 64 bits instead of 64 stages.

Carry-select. Compute the upper half of the sum twice, once assuming its carry-in is 0 and once assuming 1, while the lower half is still adding. When the real carry arrives, a multiplexer picks the right upper half. Speed for area: roughly double the adder hardware for the upper half.

Carry-save: adding three numbers without carrying. A row of independent full adders takes three numbers and produces *two*: a sum vector (each bit's $A \oplus B \oplus C$) and a carry vector (each bit's majority, shifted left one place). No carry travels sideways, so the delay is one full adder no matter how wide the numbers are. This **carry-save adder** (CSA), also called a 3:2 compressor, is the key to fast multipliers.

Worked example: carry-save addition

Add 5, 6 and 7 (101, 110, 111). Sum vector: bitwise XOR of all three = 100 = 4. Carry vector: bitwise majority = 111, shifted left = 1110 = 14. One ordinary addition finishes the job: $4 + 14 = 18 = 5 + 6 + 7$. Reducing three numbers to two cost one full-adder delay; only the final addition needs a carry chain.

22.4 Multiplication

Shift and add. Binary multiplication is the long multiplication taught in school, but easier, because each digit of the multiplier is 0 or 1: each **partial product** is either zero or a shifted copy of the multiplicand.

Worked example: 11×13 in binary

	`1011`	multiplicand = 11
$\times$	`1101`	multiplier = 13
	`1011`	bit 0 is 1: 11
	`0000`	bit 1 is 0: 0
	`1011`	bit 2 is 1: $11 \times 4 = 44$
	`1011`	bit 3 is 1: $11 \times 8 = 88$
	`10001111`	$11 + 44 + 88 = 143$

Each partial-product bit is one AND gate (multiplicand bit AND multiplier bit). Multiplying an n-bit number by an m-bit number gives up to $n + m$ bits: here $4 + 4 = 8$, and indeed $15 \times 15 = 225$ needs all 8.

Array multipliers and trees. An $n \times n$ multiplier makes n partial products from n^2 AND gates, then adds them up. An **array multiplier** adds them row by row in a grid of full adders: regular and easy to lay out, with delay that grows in proportion to n. A **Wallace tree** or **Dadda tree** instead feeds the partial products through layers of carry-save adders, each layer turning every three rows into two. The row count falls by about a third per layer, so n rows shrink to two in about $\log_{1.5} n$ layers, and one fast adder from Section 22.3 produces the product. For 24 rows (an FP32 significand) that is 7 layers.

Booth recoding: fewer partial products. Long runs of 1s are wasteful: $0111_2 = 7$ needs three partial products, but $7 = 8 - 1$ needs only two (a shifted copy and a subtracted copy). **Radix-4 Booth recoding** applies this systematically. It looks at the multiplier in overlapping groups of three bits $(b_{2i+1}, b_{2i}, b_{2i-1})$, with $b_{-1} = 0$, and turns each group into one digit:

$$d_i = -2\,b_{2i+1} + b_{2i} + b_{2i-1} \in \{-2, -1, 0, +1, +2\}.$$

The multiplier then equals $\sum_i d_i\, 4^i$, so an n-bit multiplier needs only about $n/2$ partial products. Each is 0, $\pm M$ or $\pm 2M$ (a shift), and negation is invert-plus-one, so they are cheap to make. Booth recoding also handles two's-complement (signed) multipliers with no special cases.

b_{2i+1}	b_{2i}	b_{2i-1}	digit d_i	partial product
0	0	0	0	0
0	0	1	+1	$+M$
0	1	0	+1	$+M$
0	1	1	+2	$+2M$ (shift left)
1	0	0	−2	$-2M$
1	0	1	−1	$-M$
1	1	0	−1	$-M$
1	1	1	0	0

Worked example: Booth-recoding 7

The multiplier is 0111, so $b_3 b_2 b_1 b_0 = 0111$ and $b_{-1} = 0$. Group 0 is $(b_1, b_0, b_{-1}) = (1, 1, 0)$: $d_0 = -2 + 1 + 0 = -1$. Group 1 is $(b_3, b_2, b_1) = (0, 1, 1)$: $d_1 = 0 + 1 + 1 = +2$. Check: $d_1 \cdot 4 + d_0 = 8 - 1 = 7$. So $7M = 2M \cdot 4 - M$: two partial products instead of three.

Why multipliers dominate AI hardware. An $n \times n$ multiplier needs about n^2 partial-product bits, so its area and energy grow roughly with the *square* of the width. An FP32 multiplier multiplies 24-bit significands; BF16 multiplies 8-bit ones (Chapter 21), so its multiplier is roughly $(24/8)^2 = 9$ times smaller. That, together with moving fewer bytes, is why tensor cores use narrow formats. A **multiply-accumulate** (MAC) unit, which computes $acc \leftarrow acc + a \times b$, is the building block of every tensor core and systolic array.

22.5 Division

Division is the slow operation. Binary long division produces *one quotient bit per step*, and each step depends on the remainder from the previous one, so it cannot be done in parallel like a multiplier's partial products.

Restoring division. Start with a remainder of 0. For each dividend bit, from the most significant down: shift the remainder left and bring in the next bit; if the remainder is now at least the divisor, subtract the divisor and write a quotient bit of 1, otherwise write 0. (The name comes from hardware that subtracts first and adds the divisor back, "restoring," if the result went negative.)

Worked example: $13 \div 3$

Dividend 1101, divisor 3.

Bit brought in	Remainder	≥ 3?	Quotient bit
1	1	no	0
1	$2 \cdot 1 + 1 = 3$	yes, $3 - 3 = 0$	1
0	$2 \cdot 0 + 0 = 0$	no	0
1	$2 \cdot 0 + 1 = 1$	no	0

Quotient $0100 = 4$, remainder 1. Indeed $13 = 4 \times 3 + 1$.

A 32-bit divide takes on the order of 32 steps. Faster dividers (SRT division) produce two or more quotient bits per step, and GPUs usually compute a reciprocal approximation from a small table and refine it with a couple of multiply-add iterations (Newton–Raphson), because multipliers are plentiful and fast.

Dividing by a constant. Compilers and hardware avoid division by a known constant entirely: they multiply by a scaled reciprocal and shift. For any 16-bit unsigned x, $x/3 = (x \times 43691) \gg 17$, because $43691 \approx 2^{17}/3$. A multiply and a shift replace a 16-step division.

22.6 Fixed point: fractions on integer hardware

Integer hardware is small and fast, and many signals, such as audio samples, filter coefficients and neural-network weights, have a known, limited range. **Fixed-point** numbers store fractions in ordinary integers by agreeing on where the binary point is.

Q format

A **Q***m***.***n* number has m integer bits and n fraction bits. Its value is the stored integer divided by 2^n. The **resolution** (the step between neighbouring values) is 2^{-n}. For signed numbers, the sign is counted in m, so signed Q1.7 is 8 bits covering -1 to $1 - 2^{-7}$.

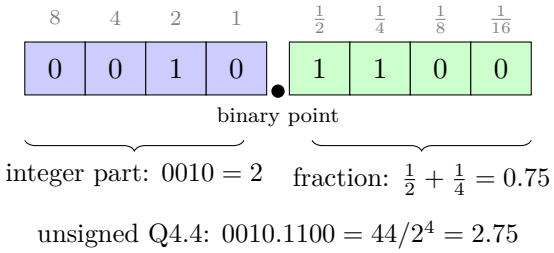


Figure 22.2: A fixed-point number is an integer with an agreed binary point. In unsigned Q4.4, four bits weigh 8, 4, 2 and 1, and four weigh $\frac{1}{2}$ down to $\frac{1}{16}$.

Worked example: reading and writing Q formats

Unsigned Q4.4 pattern 0010 1100: integer part 0010 = 2, fraction part 1100 = 12/16 = 0.75, value 2.75. (Equivalently, $44/2^4 = 2.75$.)
Store 0.3 in signed Q1.7: $0.3 \times 2^7 = 38.4$, which rounds to 38. The stored value is $38/128 = 0.296875$, an error of about 0.003. The worst-case rounding error is half a step, $2^{-8} \approx 0.0039$.

Arithmetic in fixed point.

- **Adding** needs both numbers in the same format, with their binary points lined up: shift one of them first if needed. Then it is a plain integer add (watch for overflow, and consider saturation).
- **Multiplying** two integers multiplies the scale factors too: a Q$a.b$ number times a Q$c.d$ number gives a Q$(a{+}c).(b{+}d)$ product. To return to b fraction bits, shift right by d, ideally rounding rather than truncating.

Worked example: multiplying in Q1.7

0.5 is stored as 64 and 0.25 as 32 (both Q1.7). The integer product is $64 \times 32 = 2048$, a Q2.14 number. Shift right by 7: $2048/128 = 16$, which is $16/128 = 0.125$ in Q1.7. Correct: $0.5 \times 0.25 = 0.125$.

Quantization for AI. Neural-network inference often stores weights and activations as INT8 with a **scale factor** (and sometimes a zero point): real value $\approx s \times (q - z)$, where q is the stored 8-bit integer. The scale is chosen per tensor or per channel so that the 256 available levels cover the values that actually occur. The multiply-adds run on cheap integer hardware with wide (32-bit) accumulators, and the result is rescaled at the end.

22.7 Floating-point arithmetic

Chapter 21 showed how a floating-point number is stored: sign s, exponent e and fraction m, with value $(-1)^s \times 1.m \times 2^{e-127}$ in FP32. Fixed point has a constant step size; floating point keeps a

constant number of *significant bits*, so it covers huge and tiny numbers alike. The cost is that every operation must line up exponents, renormalize and round.

Special values. The smallest and largest exponent fields are reserved:

Exponent field (FP32)	Fraction	Meaning
1 to 254	anything	normal number $1.m \times 2^{e-127}$
0	0	zero (+0 or −0)
0	not 0	subnormal number $0.m \times 2^{-126}$
255	0	$+\infty$ or $-\infty$
255	not 0	NaN (not a number)

Overflow gives $\pm\infty$; $1/0$ gives $+\infty$; $0/0$ and $\infty - \infty$ give NaN. NaN spreads through every later operation, and it is not equal to anything, not even itself.

Subnormal numbers. The smallest normal FP32 number is $2^{-126} \approx 1.2 \times 10^{-38}$. Without subnormals, the next value down would be zero, and $x - y$ could be 0 even when $x \neq y$. **Subnormal** (denormal) numbers drop the hidden leading 1 to fill that gap evenly, down to $2^{-149} \approx 1.4 \times 10^{-45}$. This **gradual underflow** keeps small differences from vanishing. Handling subnormals takes extra hardware or time, so some processors offer **flush-to-zero** modes that replace them with 0 for speed; CUDA has such an option.

Precision. With 23 stored fraction bits plus the hidden 1, FP32 has 24 significant bits. The gap between 1.0 and the next larger FP32 number is $2^{-23} \approx 1.2 \times 10^{-7}$ (**machine epsilon**), about 7 decimal digits of precision. The gap between neighbouring numbers is called a **ULP** (unit in the last place), and it grows with the number's size: between 2^{26} and 2^{27} the FP32 spacing is $2^{26-23} = 8$.

Floating-point addition

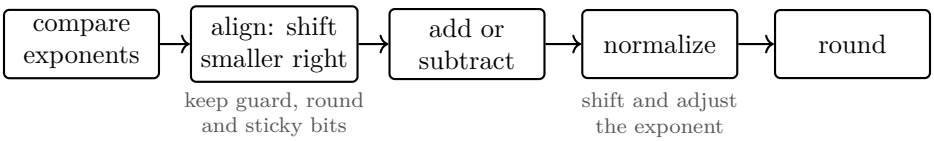


Figure 22.3: The steps of a floating-point adder. Hardware pipelines these steps over several clock cycles.

1. **Compare exponents.** The number with the smaller exponent must be adjusted to match the larger.
2. **Align.** Shift the smaller number's significand right by the exponent difference.
3. **Add or subtract** the significands (subtract if the signs differ).

4. **Normalize.** Shift the result so it again has the form $1.xxx$, adjusting the exponent: one place right if the sum reached 2 or more, possibly many places left if a subtraction cancelled the leading bits.
5. **Round** back to 23 fraction bits, and check for overflow, underflow and special values.

Worked example: $3.0 + 0.5$

$3.0 = 1.1_2 \times 2^1$ and $0.5 = 1.0_2 \times 2^{-1}$. The exponents differ by 2, so shift the smaller significand right by 2: $1.0_2 \times 2^{-1} = 0.01_2 \times 2^1$. Add: $1.10_2 + 0.01_2 = 1.11_2$, so the result is $1.11_2 \times 2^1 = 3.5$. Already normalized; nothing to round.

Guard, round and sticky bits. During alignment, bits shifted off the right end are not simply thrown away. The adder keeps the first two of them (the **guard** and **round** bits) and ORs all the rest into one **sticky** bit ("was anything nonzero shifted out?"). Those three extra bits are exactly enough to round correctly, as if the exact result had been computed with infinite precision.

Floating-point multiplication

Multiplication is simpler than addition because no alignment is needed:

1. **Sign:** XOR of the two signs.
2. **Exponent:** add the exponent fields and subtract one bias: $e = e_1 + e_2 - 127$.
3. **Significand:** multiply the two 24-bit significands (a 24×24 integer multiplier, Section 22.4). The product lies between 1 and 4.
4. **Normalize** (shift right once if the product is 2 or more, and add 1 to the exponent), then **round**.

Worked example: 3.0×2.5

$3.0 = 1.1_2 \times 2^1$ (exponent field 128) and $2.5 = 1.01_2 \times 2^1$ (exponent field 128). Sign: $0 \oplus 0 = 0$. Exponent field: $128 + 128 - 127 = 129$, meaning 2^2. Significands: $1.1_2 \times 1.01_2 = 1.111_2$ (that is, $1.5 \times 1.25 = 1.875$). Result: $1.111_2 \times 2^2 = 7.5$. Correct.

Rounding

The exact result of an operation usually falls between two representable numbers, and the **rounding mode** picks one. IEEE 754 defines four:

- **Round to nearest, ties to even** (the default): pick the nearer neighbour; on an exact tie, pick the one whose last bit is 0. In decimal terms, 2.5 rounds to 2 and 3.5 to 4. Always rounding ties upward would bias long sums upward; ties to even makes the errors cancel on average.
- **Toward zero** (truncation), **toward** $+\infty$, and **toward** $-\infty$: used for special purposes such as interval arithmetic.

Floating-point addition is not associative. Because every operation rounds, the order of operations changes the result.

Worked example: order matters

In FP32, numbers near 10^8 are spaced 8 apart (10^8 lies between 2^{26} and 2^{27}). So 10^8+1 rounds back to 10^8, and

$$(10^8+1)-10^8=0, \qquad \text{but} \qquad (10^8-10^8)+1=1.$$

A GPU that sums a million numbers in parallel adds them in a different order than a CPU loop does, so the two answers can differ in the last bits. Neither is "wrong." This matters when verifying GPU arithmetic: results are compared within a tolerance, not bit for bit, unless the order is fixed.

Fused multiply-add. A **fused multiply-add** (FMA) computes $a\times b+c$ with a *single* rounding at the end, instead of rounding the product and then the sum. It is both faster (one instruction) and more accurate. Modern GPUs and CPUs build their floating-point units around FMA, and a tensor core is essentially a block of multiply-adds.

Mixed precision. AI hardware multiplies in a narrow format (FP16, BF16, FP8) but *accumulates* in FP32. A long sum of small products in a narrow format would lose the small terms, exactly as in the 10^8+1 example; a wide accumulator keeps them.

22.8 How arithmetic hardware is checked

An arithmetic unit has far too many inputs to test exhaustively (a 32-bit adder has 2^{64} input pairs). It is verified by comparing it against a **reference model**, a simple program that computes the right answer (Section 19.10), using millions of random inputs plus the **corner cases** where bugs hide: zero, one, the largest and smallest values, values that overflow, and, for floating point, ±0, $\pm\infty$, NaN, subnormals and results that fall exactly halfway between two representable numbers. Formal tools can prove an adder correct for all inputs; multipliers are hard even for formal tools and need special techniques.

22.9 Summary

- Unsigned overflow is the carry out of the MSB; signed overflow is $V = c_{\text{into MSB}} \oplus c_{\text{out of MSB}}$. Saturating arithmetic clamps instead of wrapping.
- An arithmetic right shift rounds toward $-\infty$. A barrel shifter shifts any amount with $\log_2 n$ layers of multiplexers. Constant multiplies are shifts and adds.
- Carry-lookahead and prefix adders compute carries from $G_i = A_iB_i$ and $P_i = A_i \oplus B_i$ in $O(\log n)$ delay. A carry-save adder turns three numbers into two with one full-adder delay.

- An $n \times m$ multiply gives $n + m$ bits. Partial products are summed with arrays or Wallace/-Dadda trees; radix-4 Booth recoding halves their number. Multiplier area grows roughly with width squared.
- Division produces one quotient bit per step (restoring division); division by a constant becomes a multiply and shift.
- Q$m.n$ fixed point: value = integer $/2^n$, resolution 2^{-n}; Q$a.b\times$ Q$c.d$ gives $b + d$ fraction bits. INT8 quantization uses a scale factor.
- Floating-point add: compare exponents, align, add, normalize, round (with guard, round and sticky bits). Multiply: XOR signs, add exponents minus the bias, multiply significands, normalize, round.
- Special values: ± 0, $\pm\infty$, NaN, and subnormals (gradual underflow). FP32 epsilon is 2^{-23}. Default rounding is round to nearest, ties to even. FP addition is not associative. FMA rounds once.

22.10 Exercises

1. Add 0111 0000 and 0101 0000 in 8 bits. What is the 8-bit result, and what are the carry (C) and overflow (V) flags?
2. What does 8-bit unsigned saturating arithmetic give for $250 + 10$? And for $5 - 9$?
3. What is -9 (8 bits) arithmetically shifted right by 1? How does that differ from $-9/2$ in C?
4. How many layers of 2:1 multiplexers does a 64-bit barrel shifter need?
5. Write c_2 for a carry-lookahead adder in terms of G_0, G_1, P_0, P_1 and c_0.
6. A carry-save adder adds 9, 5 and 3. What are the sum vector and the carry vector? Check that they add to 17.
7. How many bits does the product of a 12-bit and an 8-bit unsigned number need?
8. Booth-recode the 4-bit multiplier 0110 (6) into radix-4 digits.
9. Divide 1110 (14) by 4 with restoring division. What are the quotient and remainder?
10. What value does unsigned Q4.4 0101 1010 represent? What is the resolution of Q4.4?
11. A Q2.6 number is multiplied by a Q3.5 number. How many integer and fraction bits does the full product have?
12. Add 1.5 and 0.375 in floating point, step by step.
13. In FP32, multiply 1.5×2^3 by 1.5×2^{-1}. What are the result's exponent field and value?
14. Name the five kinds of value an FP32 exponent field can indicate. What is $0/0$?
15. Why does IEEE 754 round ties to even instead of always rounding ties up?
16. Give one reason a tensor core uses BF16 or FP8 multipliers but accumulates in FP32.

Answers

1. $112 + 80 = 192$: the result is 1100 0000. No carry out of the MSB, so $C = 0$ (as unsigned, 192 is correct). A carry went into the sign bit but none came out, so $V = 1$ (as signed, the result -64 is wrong).
2. 255 (instead of wrapping to 4), and 0 (instead of wrapping to 252).
3. $-9 = 1111\,0111$; shifted: $1111\,1011 = -5$ (rounds toward $-\infty$). C's $-9/2$ gives -4 (rounds toward zero).
4. $\log_2 64 = 6$ layers (shifts of 32, 16, 8, 4, 2 and 1).
5. $c_2 = G_1 + P_1 G_0 + P_1 P_0 c_0$.
6. $9 = 1001$, $5 = 0101$, $3 = 0011$. Sum vector (XOR) $= 1111 = 15$. Majority $= 0001$, shifted left $= 0010 = 2$. $15 + 2 = 17$.
7. $12 + 8 = 20$ bits.
8. $b_3 b_2 b_1 b_0 = 0110$, $b_{-1} = 0$. Group 0, $(b_1, b_0, b_{-1}) = (1, 0, 0)$: $d_0 = -2$. Group 1, $(b_3, b_2, b_1) = (0, 1, 1)$: $d_1 = +2$. Check: $2 \times 4 - 2 = 6$.
9. Quotient 3 (0011), remainder 2. Step by step: remainders 1, 3, 7 (≥ 4, so subtract: 3, quotient bit 1), then 6 (≥ 4, subtract: 2, quotient bit 1); quotient bits 0011.
10. $0101 = 5$ and $1010/16 = 0.625$, so 5.625. Resolution $2^{-4} = 0.0625$.
11. $2 + 3 = 5$ integer bits and $6 + 5 = 11$ fraction bits (16 bits in all).
12. $1.5 = 1.1_2 \times 2^0$ and $0.375 = 1.1_2 \times 2^{-2}$. Align: $1.1_2 \times 2^{-2} = 0.011_2 \times 2^0$. Add: $1.100_2 + 0.011_2 = 1.111_2$, so $1.111_2 \times 2^0 = 1.875$.
13. Exponent: $3 + (-1) = 2$, field $130 + 126 - 127 = 129$. Significands: $1.5 \times 1.5 = 2.25$, which is ≥ 2, so normalize to 1.125×2^1 and add 1 to the exponent: field 130, value $1.125 \times 2^3 = 9$. Check: $12 \times 0.75 = 9$.
14. Normal numbers, zero, subnormal numbers, infinity and NaN. $0/0$ is NaN.
15. Always rounding ties up biases results upward, and in long computations the bias accumulates. Ties to even rounds up half the time and down half the time, so the errors cancel on average.
16. Narrow multipliers are far smaller and cheaper (area grows roughly with the significand width squared) and move fewer bytes; but a long sum in a narrow format would lose small terms to rounding, so the accumulator stays FP32.

Part VIII

The Assessment

Chapter 23

Programming for Hardware Engineers

Even the hardware track of the assessment includes coding: reported examples are one or two easy problems (a linked-list problem, bit manipulation, an XOR-of-subarrays problem) plus multiple-choice questions asking for the output of short C snippets. Hardware engineers write a lot of C and C++ (drivers, firmware, test code, performance models), so this chapter focuses on C, the classic traps in output questions, linked lists, and bit manipulation.

23.1 What the coding section looks like

Reports describe one or two problems at LeetCode "easy" level (occasionally harder), graded automatically by hidden test cases with partial credit. Languages usually include C, C++, Java and Python, though some teams restrict to C. Strategy:

1. Read the constraints first; they tell you how fast the solution must be.
2. Handle edge cases explicitly: empty input, one element, zero, the largest values.
3. Test with a tiny example by hand before submitting.
4. A correct simple solution beats an unfinished clever one: partial credit is per test case.

23.2 C essentials

Sizes. On a typical 64-bit Linux machine: `char` 1 byte, `short` 2, `int` 4, `long` 8, `long long` 8, pointers 8, `float` 4, `double` 8. For exact widths use `<stdint.h>`: `uint8_t`, `int32_t`, `uint64_t`.

Integer arithmetic. Integer division truncates toward zero: `7/2 == 3` and `-7/2 == -3`; the remainder takes the sign of the dividend: `-7 % 2 == -1`. Unsigned arithmetic wraps around: an `unsigned char` at 255 plus 1 is 0. Signed overflow is undefined behavior in C; do not rely on it.

Pointers. `&x` is the address of `x`; `*p` is the value at address `p`. Adding 1 to a pointer advances it by the size of the thing it points to: if `int *p` holds address 1000, `p + 1` is 1004. An array name used in an expression becomes a pointer to its first element, so `a[i]` means `*(a + i)`. But `sizeof` an array gives the whole array's size, while `sizeof` a pointer gives 8.

23.3 Output-prediction classics

Worked example: a static local variable

```c
int counter(void) {
    static int a = 12;   // initialized ONCE; keeps its value between calls
    a++;
    return a;
}
// printf("%d %d %d", counter(), counter(), counter());  -- see note below
```

Called three times in sequence, `counter()` returns 13, 14, 15. A `static` local is created once, before the first call, and keeps its value; the initializer does not run again. (Without `static`, it would return 13 every time.) Careful: if all three calls appear as arguments of one `printf`, C does not specify the order in which they are evaluated, so write them as separate statements.

Worked example: the macro trap

```c
#define x a+b
int a = 2, b = 2;
printf("%d", x*x);
```

A macro is plain text substitution. `x*x` becomes `a+b*a+b` $= 2 + (2 \times 2) + 2 = 8$, not 16. Always parenthesize macros: `#define x (a+b)` gives 16.

Worked example: shifts and increments

With `int i = 8, j = 2;`: `i << j` is 32 and `i >> j` is 2.
With `int i = 5; int k = i++ * 2;`: post-increment uses the old value, so `k = 10` and then `i = 6`. With `++i` instead, `k = 12`.

Worked example: tracing recursion

```c
int power(int a, int b) {
    if (b == 0) return 1;
    return a * power(a, b - 1);
}
```

`power(2, 5)` $= 2\times$ `power(2, 4)` $= \ldots = 2^5 = 32$. To trace any recursion, write the calls down to the base case, then multiply (or combine) back up.

23.4 Structs, padding and alignment

Processors read memory fastest when each value sits at an address that is a multiple of its size (its **alignment**). The compiler inserts unused **padding** bytes (Figure 23.1) inside structs to keep each member aligned, and pads the end so that arrays of the struct stay aligned.

Worked example: size of a struct on a 64-bit machine

```
struct S { char c; void *p; };          // c at 0, 7 bytes padding, p at 8
struct T { char a; int b; char c; };    // a at 0, pad 3, b at 4, c at 8, pad 3
struct U { int b; char a; char c; };    // b at 0, a at 4, c at 5, pad 2
```

`sizeof(struct S) == 16`, `sizeof(struct T) == 12`, `sizeof(struct U) == 8`. Ordering members from largest to smallest minimizes padding.

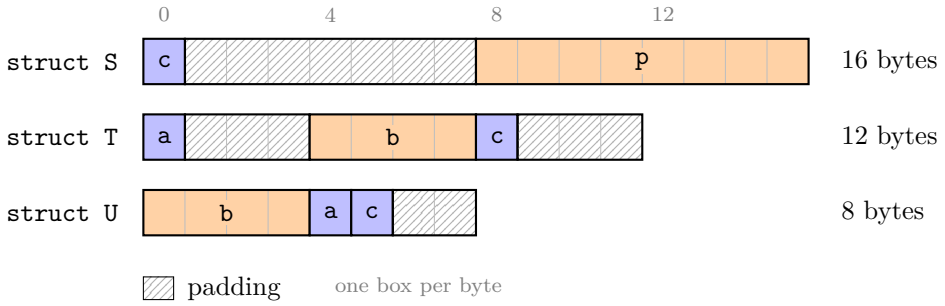


Figure 23.1: The three structs from the worked example, byte by byte on a 64-bit machine. Each member starts at a multiple of its size, so the compiler inserts padding (hatched).

Seen on the assessment

A reported question asks for the output of a C program on a 64-bit machine involving a `sizeof`-style macro and a struct containing a `char` and a `void*`. The answer turns on padding: 1 byte of `char` plus 7 bytes of padding plus an 8-byte pointer gives 16. Static variables, macro expansion, shift operators and recursion are also reported topics.

23.5 Linked lists

A **linked list** is a chain of nodes, each holding a value and a pointer to the next node. The last node's pointer is `NULL`. The list is reached through a pointer to the first node, the **head**.

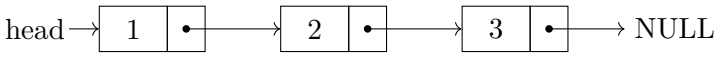


Figure 23.2: A singly linked list $1 \to 2 \to 3$.

```
struct Node { int val; struct Node *next; };
```

Almost every linked-list problem is solved by one of three patterns: walking with one pointer, rewiring links with a `prev`/`cur`/`next` trio, or running a slow and a fast pointer together.

Reversing a list. Walk the list once, flipping each node's pointer to point backward.

```
struct Node *reverse(struct Node *head) {
```

```
    struct Node *prev = NULL, *cur = head;
    while (cur != NULL) {
        struct Node *next = cur->next;  // 1. remember the rest of the list
        cur->next = prev;               // 2. flip this node's link
        prev = cur;                     // 3. move prev forward
        cur = next;                     // 4. move cur forward
    }
    return prev;                        // prev is the new head
}
```

Trace on $1 \to 2 \to 3$: after the first pass, $1 \to$ NULL and `prev` $= 1$; after the second, $2 \to 1$; after the third, $3 \to 2$; `cur` is NULL, and the new head is 3. Time $O(n)$, extra space $O(1)$.

Finding the middle, and detecting a cycle. Move `slow` one step and `fast` two steps at a time. When `fast` reaches the end, `slow` is at the middle. If the list has a cycle, `fast` eventually laps `slow` and they meet (Floyd's algorithm).

```
struct Node *middle(struct Node *head) {
    struct Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

int has_cycle(struct Node *head) {
    struct Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1;
    }
    return 0;
}
```

Deleting nodes, with a dummy head. Removing the first node is a special case unless you put a temporary `dummy` node in front of the head.

```
struct Node *remove_value(struct Node *head, int v) {
    struct Node dummy = { 0, head };
    struct Node *p = &dummy;
    while (p->next != NULL) {
        if (p->next->val == v) {
            struct Node *gone = p->next;
            p->next = gone->next;       // unlink it
            free(gone);
        } else {
            p = p->next;
        }
    }
```

```
    return dummy.next;
}
```

Other classic list problems use the same tools: merging two sorted lists (dummy head, pick the smaller node each step), removing the nth node from the end (move one pointer n steps ahead, then move both), and checking whether a list is a palindrome (find the middle, reverse the second half, compare).

23.6 Bit manipulation problems

Chapter 7 explained the tricks; here they are as code.

```
#include <stdint.h>

// Do two 32-bit addresses fall on the same 4 KB page?
int same_page(uint32_t a, uint32_t b) {
    return (a >> 12) == (b >> 12);
}

// Mask with 1s in bits lo..hi (inclusive). Use 64-bit math so width 32 is safe.
uint32_t make_mask(int lo, int hi) {
    uint64_t width = (uint64_t)(hi - lo + 1);
    return (uint32_t)(((1ULL << width) - 1) << lo);
}

// Count the 1 bits: each step clears the lowest set bit.
int popcount(uint32_t x) {
    int count = 0;
    while (x) { x &= x - 1; count++; }
    return count;
}

// Reverse the order of the 32 bits.
uint32_t reverse_bits(uint32_t x) {
    uint32_t r = 0;
    for (int i = 0; i < 32; i++) { r = (r << 1) | (x & 1); x >>= 1; }
    return r;
}
```

To print a mask as a hex string, use `printf("0x%X", m)` (or `sprintf` into a buffer).

Counting subarrays whose XOR equals k. Let P_j be the XOR of the first j elements. The XOR of elements $i+1$ through j is $P_j \oplus P_i$ (Section 7.6). It equals k exactly when $P_i = P_j \oplus k$. So walk the array once, keep a hash map counting how often each prefix value has appeared, and at each step add the count of $P_j \oplus k$:

```
#include <unordered_map>
#include <vector>

long long count_xor_subarrays(const std::vector<int>& a, int k) {
```

```
    std::unordered_map<int, long long> seen;
    seen[0] = 1;                    // the empty prefix
    int prefix = 0;
    long long count = 0;
    for (int x : a) {
        prefix ^= x;
        auto it = seen.find(prefix ^ k);
        if (it != seen.end()) count += it->second;
        seen[prefix]++;
    }
    return count;
}
```

This runs in $O(n)$ time instead of the $O(n^2)$ of checking every subarray. For `[4, 2, 2, 6, 4]` and $k = 6$ it returns 4.

Seen on the assessment

Reported Ignite coding problems include checking whether two addresses share a 4 KB page, generating a bit mask from bit `lo` to bit `hi` as a hex string, a "bitwise XOR subarray" problem, a greedy problem using a heap, and, in your own assessment, a linked-list problem. Recovering an array from its prefix XORs is another reported pattern: $a_i = P_i \oplus P_{i-1}$.

23.7 Complexity in one paragraph

Big-O notation describes how running time grows with input size n. A rough guide to what fits in about a second: n up to 10^8 for $O(n)$; 10^5 to 10^6 for $O(n \log n)$ (sorting); a few thousand for $O(n^2)$. Hash maps give $O(1)$ average lookup; sorting is $O(n \log n)$; a heap (priority queue) gives $O(\log n)$ insert and remove-min, which suits greedy problems that repeatedly take the best remaining item.

23.8 Hardware-flavored C

Two features appear in firmware and drivers. `volatile` tells the compiler that a variable can change outside the program's control (a memory-mapped hardware register), so every read and write must actually happen. And registers are often manipulated with masks and shifts: to set a 3-bit field at bits 4–6 to value `v`: `reg = (reg & ~(0x7u << 4)) | ((v & 0x7u) << 4);`.

23.9 Summary

- Know C sizes on 64-bit systems, integer division and modulo rules, and pointer arithmetic.
- `static` locals are initialized once and keep their values; macros are text substitution, so parenthesize them; post-increment uses the old value.
- Structs are padded for alignment: `{char; void*}` is 16 bytes on 64-bit.
- Linked lists: one pointer to walk, `prev`/`cur`/`next` to rewire, slow/fast to find the middle or a cycle, a dummy head to simplify deletions.

- Bit problems: shifts and masks; `x & (x-1)`; prefix XOR with a hash map counts XOR subarrays in $O(n)$.

23.10 Exercises

1. With `#define x a+b`, `a = 3` and `b = 1`, what is `x*x`?
2. A function contains `static int n = 0; return ++n;`. What does it return on its first four calls?
3. On a 64-bit machine, what is `sizeof` of `struct { char a; double d; char b; }`? How could you make it smaller?
4. Write a C function that returns the number of nodes in a linked list.
5. Trace `reverse` on the list $1 \to 2 \to 3 \to 4$. What does each node point to at the end?
6. What is `popcount(0xF0F0)`?
7. How many subarrays of `[4, 2, 2, 6, 4]` have XOR equal to 6? List them.
8. What are `-7 / 2` and `-7 % 2` in C?
9. After `int i = 5; int j = i++ * 2;`, what are `i` and `j`?

Answers

1. `a+b*a+b` $= 3 + 3 + 1 = 7$.
2. 1, 2, 3, 4.
3. `a` at 0, 7 bytes padding, `d` at 8, `b` at 16, 7 bytes padding: 24 bytes. Reordering as `{ double d; char a; char b; }` gives 16.
4. One way to write it:

```c
int length(struct Node *head) {
    int n = 0;
    for (struct Node *p = head; p != NULL; p = p->next) n++;
    return n;
}
```

5. The result is $4 \to 3 \to 2 \to 1 \to$ NULL: node 4 is the new head and node 1 points to NULL.
6. 8.
7. Four: `[4, 2]`, `[4, 2, 2, 6, 4]`, `[2, 2, 6]`, `[6]`.
8. -3 and -1.
9. `i = 6`, `j = 10`.

Chapter 24

Aptitude and Puzzles

Some NVIDIA hardware tests mix in general aptitude questions: work rates, travel times, counting, probability and classic puzzles. They test careful reasoning under time pressure more than knowledge. Each has a standard method, and once you know it these become quick points.

24.1 Work and rates

The big idea

Convert everything to a *rate*: the fraction of the job done per unit time. Rates of people working together *add*. Work done = rate $\times$ time.

Worked example: workers who quit

A can finish a job alone in 10 days, B in 15 days, C in 30 days. All three work together for 4 days, then A and B quit. How many more days does C need?
Rates: $\frac{1}{10}+\frac{1}{15}+\frac{1}{30}=\frac{3+2+1}{30}=\frac{1}{5}$ of the job per day. In 4 days: $\frac{4}{5}$ done. Remaining: $\frac{1}{5}$. C alone does $\frac{1}{30}$ per day, so $\frac{1}{5}\div\frac{1}{30}=6$ more days.

24.2 Speed, distance and time

distance = speed $\times$ time. For a boat in a river with still-water speed b and current s: downstream speed is $b+s$, upstream speed is $b-s$.

Worked example: a round trip on a river

A rower moves at 5 km/h in still water; the river flows at 1 km/h. A round trip to a place and back takes 75 minutes. How far is the place?
Downstream 6 km/h, upstream 4 km/h, total time 1.25 h:

$$\frac{d}{6}+\frac{d}{4}=1.25 \;\Rightarrow\; \frac{2d+3d}{12}=1.25 \;\Rightarrow\; d=3 \text{ km}.$$

24.3 Percentages and compound growth

Growing by r per period for n periods multiplies by $(1+r)^n$. Going backward in time divides by it.

Worked example: working backward

A chip's area grows 5% per year and is 2300 units in 2020. What was it in 2018?

$$\frac{2300}{1.05^2} = \frac{2300}{1.1025} \approx 2086 \text{ units.}$$

24.4 Factorials and trailing zeros

Each trailing zero of $n!$ comes from a factor of $10 = 2 \times 5$. Factors of 2 are plentiful, so count factors of 5. Multiples of 25 contribute an extra 5, multiples of 125 another, and so on:

$$\text{zeros} = \left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{n}{25} \right\rfloor + \left\lfloor \frac{n}{125} \right\rfloor + \left\lfloor \frac{n}{625} \right\rfloor + \cdots$$

Worked example: trailing zeros of 2021!

$\lfloor 2021/5 \rfloor = 404$, $\lfloor 2021/25 \rfloor = 80$, $\lfloor 2021/125 \rfloor = 16$, $\lfloor 2021/625 \rfloor = 3$, $\lfloor 2021/3125 \rfloor = 0$. Total: $404 + 80 + 16 + 3 = 503$.

24.5 Probability

- For equally likely outcomes: probability = favorable outcomes / total outcomes.
- Independent events: multiply probabilities.
- "At least one" is usually easiest through the complement: $P(\text{at least one}) = 1 - P(\text{none})$.

Worked example: ants on a polygon

Eight ants sit at the corners of a regular octagon. Each picks a direction (clockwise or counterclockwise) at random and walks along the edges. What is the probability that no two ants collide?

There are $2^8 = 256$ equally likely choices. The ants avoid collisions only if they all walk the same way: all clockwise or all counterclockwise, 2 outcomes. Probability: $2/256 = 1/128$. (For a triangle: $2/8 = 1/4$.)

Worked example: at least one

A test chip has 10 independent blocks, each working with probability 0.95. The probability that *all* work is $0.95^{10} \approx 0.60$, so the probability that at least one fails is about 0.40.

24.6 Counting

- Ordered arrangements of n distinct items: $n!$.
- Ordered choices of k from n: $n!/(n-k)!$.
- Unordered choices of k from n: $\binom{n}{k} = \dfrac{n!}{k!\,(n-k)!}$.

For example, the number of 8-bit values with exactly three 1s is $\binom{8}{3} = 56$.

24.7 Classic puzzles

100 bulbs (or doors). 100 bulbs start off. Person 1 toggles every bulb, person 2 every second bulb, person 3 every third, and so on up to person 100. Which bulbs end up on? Bulb n is toggled once for each divisor of n. Divisors come in pairs (d and n/d), except when n is a perfect square, where one divisor pairs with itself. So only perfect squares have an odd number of toggles and end up on: bulbs 1, 4, 9, …, 100. That is 10 bulbs.

Three switches, one bulb. Three switches outside a closed room control one bulb inside; you may enter once. Turn on switch 1 for a few minutes, turn it off, turn on switch 2, and enter. If the bulb is on, it is switch 2; if it is off but warm, switch 1; if off and cold, switch 3. (Using a second property, heat, gives more information than on/off alone.)

Two eggs, 100 floors. With two eggs, find the highest floor from which an egg survives, minimizing the worst-case number of drops. Drop the first egg from floors spaced so that each new gap is one smaller than the last: 14, 27, 39, …. Since $14 + 13 + \cdots + 1 = 105 \geq 100$, the worst case is 14 drops.

24.8 Binary search

Each comparison in binary search halves the remaining candidates. After N comparisons, at most $M/2^N$ of M elements remain, and finding an element (or proving it is absent) takes about $\lceil \log_2(M+1) \rceil$ comparisons.

Worked example: what is left after 6 comparisons?

$M = 32768 = 2^{15}$ elements. After 6 comparisons, at most $2^{15}/2^6 = 512$ elements remain. Finding any element takes at most 16 comparisons.

24.9 Estimation

Interviewers sometimes ask for rough estimates (“how many transistors switch per second in a GPU?”). Break the quantity into factors you can guess, round to powers of ten, multiply, and state

your assumptions. For instance: 10^{10} transistors $\times$ about 10% active per cycle $\times$ 2×10^9 cycles per second $\approx 2 \times 10^{18}$ switching events per second. Being organized matters more than being exact.

24.10 Exercises

1. A can paint a wall in 6 hours and B in 3 hours. How long do they take together?
2. A train travels 120 km at 60 km/h and returns at 40 km/h. What is its average speed for the whole trip?
3. How many trailing zeros does 100! have?
4. A population of 1000 grows 10% per year. What is it after 3 years?
5. Four ants on the corners of a square each pick a random direction. What is the probability that none collide?
6. How many 5-bit strings contain exactly two 1s?
7. In the 100-bulbs puzzle with 50 bulbs instead, how many end up on?
8. How many comparisons does binary search need, at most, for 1000 sorted elements?

Answers

1. Rates $\frac{1}{6} + \frac{1}{3} = \frac{1}{2}$ per hour: 2 hours.
2. Total distance 240 km; time $2 + 3 = 5$ h; average 48 km/h (not 50: average speed is total distance over total time).
3. $20 + 4 = 24$.
4. $1000 \times 1.1^3 = 1331$.
5. $2/2^4 = 1/8$.
6. $\binom{5}{2} = 10$.
7. Perfect squares up to 50: 1, 4, 9, 16, 25, 36, 49: 7 bulbs.
8. $\lceil \log_2 1001 \rceil = 10$.

Chapter 25

A Full Practice Assessment

This chapter is a practice assessment modeled on what candidates report: hardware multiple-choice and true/false questions, short C output questions, a few aptitude questions, and two coding problems. Take it under real conditions: no notes, a timer, and no checking answers until the end.

Remember

Suggested timing: 60 minutes for Parts A to C, then 30 minutes for Part D. If your assessment uses negative marking, skip questions you cannot narrow down to two options.

25.1 Part A: Hardware (multiple choice and true/false)

1. An NMOS pass transistor with its gate at $V_{DD} = 1.0\,\text{V}$ and $V_{tn} = 0.3\,\text{V}$ passes a logic 1 to a capacitive load. The load settles at:

   (a) 1.0 V (b) 0.3 V (c) 0.7 V (d) 0 V

2. In a static CMOS gate, the network that connects the output to V_{DD} is made of:

   (a) NMOS in series (b) PMOS transistors (c) NMOS in parallel (d) resistors

3. An inverter is built with its NMOS connected to V_{DD} and its PMOS to ground (gates tied together as the input, sources tied as the output). For input 1, the output is approximately:

   (a) 0 V (b) $|V_{tp}|$ (c) V_{DD} (d) $V_{DD} - V_{tn}$

4. The circuit in question 3 behaves as:

   (a) an inverter with full swing (b) an oscillator (c) a non-inverting buffer with degraded levels (d) an output stuck at 0

5. One advantage of a flip-flop over a latch is that a flip-flop:

   (a) uses fewer transistors (b) allows time borrowing (c) has a lower clock load (d) captures its input only at the clock edge, so data cannot race through

6. A 2:1 mux has $S = B$, $I_0 = A$, $I_1 = 1$. Its output is:

   (a) $A + B$ (b) AB (c) $A \oplus B$ (d) $\overline{A}$

7. The minimum number of 2:1 muxes needed to build XOR, if only A, B, 0 and 1 are available, is:

   (a) 1 (b) 2 (c) 3 (d) 4

8. The clock frequency and supply voltage of a chip both double. Dynamic power becomes:

   (a) 2× (b) 4× (c) 16× (d) 8×

9. Supply voltage rises 10% and frequency rises 20%. Dynamic power rises by about:

   (a) 30% (b) 45% (c) 32% (d) 21%

10. As temperature increases, carrier mobility _____ and metal wire resistance _____.

    (a) increases; increases (b) increases; decreases (c) decreases; increases (d) decreases; decreases

11. True or false: for the antenna rule, connecting a long wire to a *smaller* gate reduces the risk of oxide damage.

12. Which of these fixes an antenna violation?

    (a) adding a reverse-biased diode near the gate (b) making the gate smaller (c) making the lower-layer wire longer (d) raising V_{DD}

13. A path has a hold violation. Which fixes it?

    (a) lowering the clock frequency (b) raising the clock frequency (c) removing buffers from the short path (d) adding delay to the short path

14. $t_{cq} = 50\,\text{ps}$, $t_{pd} = 350\,\text{ps}$, $t_{su} = 100\,\text{ps}$, no skew. The maximum clock frequency is:

    (a) 1 GHz (b) 2 GHz (c) 2.5 GHz (d) 2.86 GHz

15. The capture flip-flop's clock arrives later than the launch flip-flop's. This:

    (a) helps setup and hurts hold (b) hurts setup and helps hold (c) helps both (d) affects neither

16. A T flip-flop with $T = 1$ is clocked at 25 kHz. Its output frequency is:

    (a) 50 kHz (b) 25 kHz (c) 12.5 kHz (d) 6.25 kHz

17. A binary counter's clock period is 2 ns and its most significant bit has a period of 128 ns. It has how many flip-flops?

    (a) 5 (b) 6 (c) 7 (d) 64

18. The minimum number of flip-flops for a state machine with 60 states is:

    (a) 5 (b) 60 (c) 6 (d) 7

19. For the same sequence detector, which usually needs more states?

    (a) Mealy (b) Moore (c) they always need the same (d) it depends only on the clock frequency

20. How many transistors are in a static CMOS 3-input NAND?

    (a) 3 (b) 4 (c) 6 (d) 8

21. PMOS transistors are usually drawn wider than NMOS transistors because:

    (a) holes have lower mobility than electrons (b) PMOS always has a higher threshold (c) it is required for ESD protection (d) it reduces leakage

22. NAND gates are generally preferred over NOR gates because:

    (a) NAND uses fewer transistors (b) NOR is not universal (c) NAND has no parasitic capacitance (d) NOR puts slow PMOS transistors in series

23. A $50\,\Omega$ transmission line ends in a $25\,\Omega$ load. The reflection coefficient is:

    (a) $+1/3$ (b) $-1/3$ (c) $-1/2$ (d) 0

24. Which best supplies a chip's power pins during a current step lasting a few nanoseconds?

    (a) a large capacitor far from the chip (b) no capacitor; the regulator responds instantly (c) a small, low-inductance capacitor right at the pins (d) an inductor in series at the pins

25. The main purpose of a heat sink's fins is to:

    (a) store heat (b) increase thermal resistance (c) insulate the chip (d) increase surface area for convection

26. A 300 W chip has $0.2\,^\circ\text{C/W}$ total thermal resistance to $30\,^\circ\text{C}$ air. Its junction temperature is:

    (a) $60\,^\circ\text{C}$ (b) $75\,^\circ\text{C}$ (c) $90\,^\circ\text{C}$ (d) $120\,^\circ\text{C}$

27. True or false: two boards exchanging digital signals must share a ground connection.

28. In a 6T SRAM cell, read stability requires:

    (a) access NMOS stronger than pull-down NMOS (b) pull-up PMOS stronger than access NMOS (c) pull-down NMOS stronger than access NMOS (d) all transistors the same size

29. DRAM needs periodic refresh because:

    (a) the stored charge leaks away (b) reads are too slow (c) neighboring SRAM interferes (d) clock gating requires it

30. A block will be idle for a long time. What reduces its leakage power most?

    (a) clock gating (b) lowering only the frequency (c) power gating (d) adding decoupling capacitors

31. In the classic five-stage pipeline, loads read data memory in which stage?

    (a) IF (b) ID (c) EX (d) MEM

32. A cache has a 1 ns hit time, a 10% miss rate and a 50 ns miss penalty. The AMAT is:

    (a) 5 ns (b) 51 ns (c) 6 ns (d) 50 ns

33. A two-flop synchronizer is appropriate for crossing:

    (a) a 32-bit data bus (b) a single-bit level signal (c) the clock itself (d) a multi-bit counter in binary

34. A writer at 100 MHz sends bursts of 200 words; a reader at 50 MHz reads one word per cycle. The minimum FIFO depth is:

    (a) 100 (b) 50 (c) 150 (d) 200

35. An unbuffered wire's length doubles. Its delay becomes about:

    (a) the same (b) $2\times$ (c) $4\times$ (d) $8\times$

36. A divide-by-3 clock divider with a 50% duty cycle requires:

    (a) only rising clock edges (b) both clock edges (c) a PLL (d) an asynchronous reset

37. Electromigration gets worse with:

    (a) lower temperature (b) lower supply voltage (c) wider wires (d) higher current density and temperature

38. Latch-up is prevented by:

    (a) guard rings and frequent well and substrate taps (b) lowering the clock frequency (c) adding decoupling capacitors (d) using NOR instead of NAND

39. BF16 has the same number of exponent bits as:

    (a) FP16 (b) FP32 (c) INT8 (d) FP8 E4M3

40. In steady state, an ideal CMOS inverter draws:

    (a) only leakage current, nearly zero (b) a large current (c) its saturation current (d) a current that depends on the input

Challenge.

41. A chain of 50 XOR gates: gate 1 has inputs 1 and X; every later gate has inputs (previous gate's output) and X. The output of gate 50 is:

    (a) X (b) $\overline{X}$ (c) 1 (d) 0

42. Identical NMOS transistors, $V_{DD} = 5\,\text{V}$, $V_t = 2\,\text{V}$. An input of $4\,\text{V}$ passes through one NMOS whose gate is at $5\,\text{V}$. The output node reaches:

    (a) $4\,\text{V}$ (b) $3\,\text{V}$ (c) $2\,\text{V}$ (d) $5\,\text{V}$

43. A 1 MB, 4-way set-associative cache has 64-byte lines and 32-bit addresses. The tag is how many bits?

    (a) 12 (b) 14 (c) 16 (d) 20

44. A 202-byte read starts at address `0x31121133`. Replies carry at most 64 bytes and may not cross 64-byte boundaries. How many reply packets?

    (a) 3 (b) 4 (c) 5 (d) 6

45. A design moves from $(f, 2V)$ to $(4f, 3V)$. Dynamic power changes by a factor of:

    (a) 6 (b) 9 (c) 4 (d) 12

46. Net A can switch between 1.0 and 1.6 ns; adjacent net B between 1.4 and 2.2 ns. Crosstalk between them is possible during:

    (a) no time (b) 1.0 to 2.2 ns (c) 1.4 to 1.6 ns (d) 1.6 to 2.2 ns

25.2 Part B: What does this C code print?

Assume a 64-bit machine.

1. What does this print?

```c
#define SQ(x) x*x
printf("%d", SQ(1+2));
```

2. What does this print?

```c
int f(void) { static int a = 12; return a++; }
int main(void) { f(); f(); printf("%d", f()); return 0; }
```

3. What does this print?

```c
int i = 8, j = 2;
printf("%d %d", i << j, i >> j);
```

4. What does this print?

```c
struct S { char c; void *p; };
printf("%zu", sizeof(struct S));
```

5. What does this print?

```c
unsigned char c = 250;
c += 10;
printf("%d", c);
```

25.3 Part C: Aptitude

1. How many trailing zeros does 50! have?
2. Six ants on the corners of a hexagon each pick a random direction along the edges. What is the probability that none collide?
3. One pipe fills a tank in 4 hours and another in 12 hours. How long do both together take?
4. What is the maximum number of comparisons binary search needs on 4096 sorted elements?
5. 1000 bulbs start off; person k toggles every kth bulb, for $k = 1$ to 1000. How many bulbs end up on?

25.4 Part D: Coding

1. **Reverse a linked list.** Given the head of a singly linked list, reverse it and return the new head. Use $O(1)$ extra memory.
2. **XOR subarrays.** Given an integer array and an integer k, return the number of contiguous subarrays whose elements XOR to k. The array can have up to 10^5 elements.

25.5 Answer key

Part A

1. **(c)** NMOS passes a weak 1: $V_{DD} - V_{tn} = 0.7\,\text{V}$.
2. **(b)** The pull-up network is PMOS, which passes a strong 1.

3. **(d)** The top NMOS pulls up but stops one threshold below V_{DD}.
4. **(c)** Input 1 gives a weak 1, input 0 gives a weak 0: non-inverting with degraded levels.
5. **(d)** Edge triggering prevents race-through; (a), (b) and (c) are latch advantages.
6. **(a)** $\overline{B}A + B = A + B$.
7. **(b)** One mux makes $\overline{B}$; one selects B or $\overline{B}$ by A.
8. **(d)** $2 \times 2^2 = 8$.
9. **(b)** $1.1^2 \times 1.2 = 1.452$.
10. **(c)** More phonon scattering in both silicon and metal.
11. **False.** A smaller gate raises the metal-to-gate area ratio, increasing the risk.
12. **(a)** The diode drains plasma charge to the substrate.
13. **(d)** Hold does not depend on the clock period.
14. **(b)** $50 + 350 + 100 = 500\,\text{ps}$, so 2 GHz.
15. **(a)** The data gets extra time before the capture edge, but the hold window closes later.
16. **(c)** A toggle flip-flop divides by 2.
17. **(b)** $128/2 = 64 = 2^6$.
18. **(c)** $\lceil \log_2 60 \rceil = 6$.
19. **(b)** Moore outputs are attached to states, which usually needs an extra state.
20. **(c)** $2n$ for n inputs.
21. **(a)** Lower hole mobility is compensated by width.
22. **(d)** Series PMOS is slow or must be very wide.
23. **(b)** $(25 - 50)/(25 + 50) = -1/3$.
24. **(c)** Fast steps are limited by inductance; small, close capacitors have the least.
25. **(d)** Fins add area for the air to carry heat away.
26. **(c)** $30 + 300 \times 0.2 = 90\,^\circ\text{C}$.
27. **True.** Voltages are only meaningful relative to a shared reference, and currents need a return path.
28. **(c)** Otherwise the 0 node rises during a read and the cell can flip.
29. **(a)** The capacitor leaks.
30. **(c)** Power gating cuts the supply and thus most leakage; clock gating only stops switching.
31. **(d)** MEM.
32. **(c)** $1 + 0.1 \times 50 = 6\,\text{ns}$.
33. **(b)** Buses and binary counters need Gray code, handshakes or FIFOs.
34. **(a)** Burst lasts $2\,\mu\text{s}$; 100 words are read in that time; $200 - 100 = 100$.
35. **(c)** Delay grows with the square of length.

36. **(b)** A falling-edge flip-flop provides the half-cycle delay.
37. **(d)** Black's equation: worse with current density and exponentially with temperature.
38. **(a)** Taps and guard rings keep the parasitic bipolar transistors off.
39. **(b)** BF16 keeps FP32's 8-bit exponent.
40. **(a)** One transistor is always off.

    Challenge

41. **(c)** Odd gates output $\overline{X}$, even gates output 1.
42. **(b)** $\min(4, 5 - 2) = 3\,\text{V}$.
43. **(b)** Offset 6, index 12 (4096 sets), tag $32 - 18 = 14$.
44. **(b)** 13 bytes to the first boundary, then 64, 64 and 61.
45. **(b)** $4 \times (3/2)^2 = 9$.
46. **(c)** The intersection of the two windows.

Part B

1. **5.** `SQ(1+2)` expands to `1+2*1+2`.
2. **14.** The calls return 12, 13, 14 (post-increment returns the old value; the static keeps counting).
3. **32 2.**
4. **16.** One byte, seven bytes of padding, an eight-byte pointer.
5. **4.** $250 + 10 = 260$, which wraps modulo 256 to 4.

Part C

1. **12.** $\lfloor 50/5 \rfloor + \lfloor 50/25 \rfloor = 10 + 2$.
2. **1/32.** $2/2^6$.
3. **3 hours.** Rates $\frac{1}{4} + \frac{1}{12} = \frac{1}{3}$ per hour.
4. **13.** $\lceil \log_2 4097 \rceil = 13$.
5. **31.** Perfect squares up to 1000: 1^2 through $31^2 = 961$.

Part D

1. The iterative `prev`/`cur`/`next` solution of Chapter 23: $O(n)$ time, $O(1)$ space. Test it on an empty list, a single node and two nodes.
2. Prefix XOR with a hash map counting earlier prefixes (Chapter 23): for each prefix P_j, add the number of earlier prefixes equal to $P_j \oplus k$. Initialize the map with $\{0 : 1\}$. $O(n)$ time; use a 64-bit counter because the answer can exceed 2^{31}.

Scoring yourself. If you missed a question, do not just memorize the answer. Go back to the chapter that teaches it, reread the idea behind it, and redo the chapter's exercises. The real assessment will ask a different twist on the same idea.

Appendix A

Formula Sheet

Every formula in the book, grouped by topic, with the chapter that explains it. Knowing *why* each one holds matters more than memorizing it.

Electricity (Chapters 1–3)

Ohm's law	$V = IR$
Power	$P = VI = I^2R = V^2/R$
Wire resistance	$R = \rho L/A = R_{\square} \cdot L/W$
Series / parallel resistors	$R_1 + R_2 \quad / \quad R_1R_2/(R_1 + R_2)$
Voltage divider	$V_{out} = V_{in}\, R_2/(R_1 + R_2)$
Capacitor	$Q = CV, \quad I = C\, dV/dt, \quad E = \frac{1}{2}CV^2$
RC charging	$V(t) = V_s(1 - e^{-t/RC})$; 50% at $0.69RC$; 10–90% in $2.2RC$
Inductor	$V = L\, dI/dt, \quad E = \frac{1}{2}LI^2$
Reactance; RC cutoff	$X_C = 1/(2\pi fC)$, $X_L = 2\pi fL; \quad f_c = 1/(2\pi RC)$
Reflection coefficient	$\Gamma = (Z_L - Z_0)/(Z_L + Z_0)$
Scope rise time	$t_r \approx 0.35/\text{bandwidth}$
Thermal	$\Delta T = P\,\theta; \quad T_j = T_a + P(\theta_{jc} + \theta_{cs} + \theta_{sa})$
Heat carried by a fluid	$P = \dot{m}\, c\, \Delta T$

Devices (Chapters 4–6)

Drift velocity	$v = \mu E; \quad \mu_n \approx 2\text{–}3\,\mu_p$
Diode	$I = I_S(e^{V/V_T} - 1), \quad V_T = kT/q \approx 26\,\text{mV}$
MOSFET saturation	$I_D = \frac{1}{2}\mu C_{ox}(W/L)(V_{GS} - V_t)^2$
MOSFET linear	$I_D = \mu C_{ox}(W/L)\left[(V_{GS} - V_t)V_{DS} - \frac{1}{2}V_{DS}^2\right]$
Pass transistor	NMOS: $V_{out} = \min(V_{in}, V_G - V_{tn})$; NMOS weak $1 = V_{DD} - V_{tn}$; PMOS weak $0 = \lvert V_{tp}\rvert$
Equal-current sizing	$(W/L)_p/(W/L)_n = \mu_n/\mu_p$
Transconductance; CS gain	$g_m = 2I_D/(V_{GS} - V_t); \quad A_v = -g_m R_D$
Op-amps	inverting $-R_f/R_{in}$; non-inverting $1 + R_f/R_g$
ADC step	$V_{ref}/2^N$

Logic (Chapters 7–9)

Bits needed	$\lceil \log_2 N \rceil$ for N values
Two's complement	range -2^{n-1} to $2^{n-1} - 1$; negate = invert $+1$
Gray code	$g = b \oplus (b \gg 1)$
De Morgan	$\overline{A + B} = \overline{A}\,\overline{B}, \quad \overline{AB} = \overline{A} + \overline{B}$
Mux; Shannon	$Y = \overline{S}I_0 + SI_1; \quad F = \overline{S}F\vert_{S=0} + SF\vert_{S=1}$
Full adder	$S = A \oplus B \oplus C_{in}, \quad C_{out} = AB + C_{in}(A \oplus B)$
Number of functions	2^{2^n} for n inputs
Noise margins	$NM_H = V_{OH} - V_{IH}, \quad NM_L = V_{IL} - V_{OL}$

Timing (Chapters 11–13)

Setup	$t_{cq} + t_{pd} + t_{su} \le T + t_{skew}$
Hold	$t_{cq} + t_{cd} \ge t_h + t_{skew}$ (no T!)
Maximum frequency	$f_{max} = 1/(t_{cq} + t_{pd} + t_{su})$
Counter bit k	$f_{clk}/2^{k+1}$; MSB of n bits has period $2^n T$
Retiming flops	segments $= \lceil \text{delay}/(T - \text{overhead}) \rceil$; flops = segments -1
Gate delay	$t_{pd} \approx 0.69\,R\,C_{load}$
Elmore delay	$\sum_i R_i\,C_{\text{downstream},i}$
Logical effort	$d = gh + p$; best path delay $NF^{1/N} + P$; buffer chain $N \approx \log_4 F$
Wire delay	$\propto L^2$ unbuffered

Power and reliability (Chapters 14–17)

Dynamic power	$P = \alpha C V_{DD}^2 f$
Energy per operation	$E = \alpha C V_{DD}^2$
Leakage power	$P = I_{leak} V_{DD}$; I_{leak} grows exponentially as V_t falls or T rises
Copper resistance vs temperature	$R(T) \approx R_0[1 + 0.0039(T - T_0)]$
Crosstalk glitch	$\Delta V \approx V_{DD}\, C_c/(C_c + C_g)$
Yield	$Y = e^{-DA}$
Electromigration (Black)	MTTF $= A\, J^{-n} e^{E_a/kT}$
Antenna ratio	metal area connected to gate / gate area (smaller gate is worse)

Systems (Chapters 18–21)

DRAM charge sharing	$\Delta V = (V_{cell} - V_{DD}/2)\, C_s/(C_s + C_{BL})$
FIFO depth	$B(1 - r_r/r_w)$, rates in words per second
Bandwidth	width × transfers/s /8× efficiency; DDR = 2 transfers per clock
Performance	time = instructions × CPI × period
Amdahl	speedup $= 1/((1 - p) + p/s)$
Cache fields	offset $= \log_2$(line), index $= \log_2$(sets), tag = rest
AMAT	$t_{L1} + m_{L1}(t_{L2} + m_{L2}\, t_{mem})$
Effective CPI	base + (memory accesses per instruction) × miss rate × penalty
Matrix multiply	$2MNK$ FLOPs
Floating point	$(-1)^s \times 1.m \times 2^{e-\text{bias}}$; FP32 bias 127

Computer arithmetic (Chapter 22)

Overflow	unsigned: carry out of the MSB; signed: $V = c_{\text{into MSB}} \oplus c_{\text{out of MSB}}$
Generate, propagate	$G_i = A_i B_i, \quad P_i = A_i \oplus B_i, \quad c_{i+1} = G_i + P_i c_i$
Carry-save adder	3 numbers in, 2 out (sum = XOR, carry = majority shifted left); one full-adder delay
Product width	n bits $\times$ m bits $\rightarrow$ $n + m$ bits
Radix-4 Booth digit	$d_i = -2b_{2i+1} + b_{2i} + b_{2i-1} \in \{-2, \ldots, 2\}$; about $n/2$ partial products
Barrel shifter	$\log_2 n$ layers of n 2:1 multiplexers
Fixed point Q$m.n$	value = integer $/2^n$; resolution 2^{-n}; Q$a.b\times$ Q$c.d \rightarrow$ Q$(a{+}c).(b{+}d)$
FP multiply	sign = $s_1 \oplus s_2$; exponent field = $e_1 + e_2 - 127$; multiply significands
FP add	compare exponents, align, add, normalize, round
FP32 limits	smallest normal 2^{-126}; smallest subnormal 2^{-149}; epsilon $2^{-23} \approx 1.2 \times 10^{-7}$

Appendix B

Glossary

Activity factor (α)
Fraction of clock cycles in which a node makes a 0-to-1 transition. A clock has $\alpha = 1$.

AMAT
Average memory access time: hit time plus miss rate times miss penalty.

Antenna effect
Damage to a gate oxide from charge collected by a long wire during plasma etching. Limited by the antenna ratio (metal area / gate area).

Arbiter
Circuit that grants a shared resource to one of several requesters.

Arithmetic intensity
FLOPs performed per byte moved from memory.

Assertion
A rule, written in the design or testbench, that must always hold (for example, "never write a full FIFO"); the simulator reports the moment it is broken.

Bistable
Having two stable states; the basis of every latch, flip-flop and SRAM cell.

Bitline / wordline
In a memory array, the column wire carrying data / the row wire selecting a row.

Body effect
Increase of a MOSFET's threshold voltage when its source is above its body.

Booth recoding
Rewriting a multiplier as digits in $\{-2, -1, 0, 1, 2\}$ so that an n-bit multiply needs only about $n/2$ partial products.

Carry-save adder
A row of independent full adders that turns three numbers into two (a sum vector and a carry vector) with one full-adder delay; the core of fast multipliers.

Clock domain crossing (CDC)
A signal passing between parts of a chip on unrelated clocks.

Clock gating
Stopping the clock to idle registers to save dynamic power.

Clock skew
Difference in clock arrival time between two flip-flops.

Clock-to-Q (t_{cq})
Delay from a clock edge to a flip-flop's output changing.

CMOS
Complementary MOS: logic using PMOS pull-up and NMOS pull-down networks.

Coalescing
Combining the memory requests of a GPU warp into a few wide transactions.

Constrained-random testing
Verification with random inputs restricted by rules (constraints) that keep them legal and steer them toward interesting cases.

Contamination delay (t_{cd})
Shortest time from an input change until an output starts to change.

Coverage
A measure of how thoroughly tests exercised a design: code coverage (lines, branches, toggles, FSM states) or functional coverage (scenarios from the verification plan).

Critical path
The slowest register-to-register path; it sets the maximum clock frequency.

Crosstalk
Interference between neighboring wires through coupling capacitance.

Decoupling capacitor
Capacitor near a chip's power pins that supplies fast bursts of current.

Dennard scaling
The historical rule that shrinking transistors kept power density constant; it ended around 2005.

Depletion region
Region at a PN junction with no free carriers.

Divergence
Threads in one GPU warp taking different branches, forcing both paths to execute.

Doping
Adding impurity atoms to silicon to create n-type (electrons) or p-type (holes) material.

DRAM
Dynamic RAM: one transistor and one capacitor per bit; needs refresh.

DRC / LVS
Design rule check (layout geometry rules) / layout versus schematic (layout matches netlist).

DVFS
Dynamic voltage and frequency scaling.

ECC
Error-correcting code; SECDED corrects one error and detects two.

Electromigration
Gradual movement of metal atoms by electron current, creating voids and shorts.

Elmore delay
Estimate of RC network delay: sum of each resistance times all downstream capacitance.

ESD
Electrostatic discharge; protection diodes guard every pin.

FIFO
First-in first-out queue between a producer and a consumer.

Fixed point
Storing fractions in integers with an agreed binary point; a Q$m.n$ number is the stored integer divided by 2^n.

Flip-flop
Edge-triggered storage element; samples its input only at a clock edge.

FO4 delay
Delay of an inverter driving four copies of itself; a yardstick of process speed.

Formal verification
Proving with mathematical tools that a property holds for every possible input sequence (within a bound), or finding an exact counterexample.

Forwarding
Passing a result directly between pipeline stages to avoid waiting for write-back.

FSM
Finite state machine: state register plus next-state and output logic. Moore outputs depend on state; Mealy on state and inputs.

Fused multiply-add (FMA)
Computing $a \times b + c$ with a single rounding at the end; faster and more accurate than a separate multiply and add.

Gray code
Binary ordering in which consecutive values differ in exactly one bit.

Ground (GND)
The shared 0 V reference node of a circuit.

Hold time (t_h)
How long a flip-flop's input must stay stable after the clock edge.

Hole
The absence of an electron in a bond, behaving as a positive charge carrier.

IR drop
Voltage lost across the resistance of the power-delivery path.

Latch
Level-sensitive storage element; transparent while enabled.

Latch-up
Parasitic p-n-p-n thyristor turning on and shorting the supply to ground.

Leakage
Current that flows through transistors that are supposed to be off.

Logical effort
How much worse a gate is at driving current than an inverter with the same input capacitance.

Metastability
A flip-flop output hanging between 0 and 1 for an unpredictable time after a setup or hold violation.

Mobility (μ)
How fast carriers drift per unit electric field; decreases with temperature.

MOSFET
Metal–oxide–semiconductor field-effect transistor; a voltage-controlled switch.

Multiplexer (mux)
Selects one of several inputs based on select lines.

NaN
"Not a number": the floating-point result of undefined operations such as $0/0$ or $\infty - \infty$.

Noise margin
How much noise a logic level can tolerate before being misread.

Pass transistor
A transistor used as a switch to pass a signal; NMOS passes a weak 1, PMOS a weak 0.

Pipelining
Splitting logic into stages separated by registers to raise clock frequency and throughput.

Power gating
Cutting the supply to an idle block to remove its leakage.

Propagation delay (t_{pd})
Longest time from an input change until an output is final.

PVT
Process, voltage and temperature: the three sources of variation.

Reference model
A simple, trusted description of correct behaviour that a testbench uses to compute expected results (also called a golden model).

Round to nearest, ties to even
The default IEEE 754 rounding mode: round to the nearer representable value, and on an exact tie to the one whose last bit is 0.

Saturating arithmetic
Arithmetic that clamps a result that does not fit to the largest or smallest representable value instead of wrapping around.

Scoreboard
The part of a testbench that compares what the design produced with what the reference model expected.

Setup time (t_{su})
How long a flip-flop's input must be stable before the clock edge.

SIMT
Single instruction, multiple threads: one instruction executed by all 32 threads of a warp.

Slack
Timing margin of a path; negative slack is a violation.

SRAM
Static RAM: six-transistor cell of cross-coupled inverters; no refresh needed.

Subnormal number
A floating-point value below the smallest normal number, with no hidden leading 1; gives gradual underflow toward zero.

Synchronizer
Two flip-flops in series that bring an asynchronous single bit safely into a clock domain.

Temperature inversion
At low supply voltage, circuits get faster when hot because threshold voltage falls.

Testbench
Non-synthesized code that drives a design under test's inputs and checks its outputs.

Threshold voltage (V_t)
Gate-to-source voltage at which a MOSFET's channel forms.

Transmission gate
Parallel NMOS and PMOS switch that passes both 0 and 1 at full strength.

Transmission line
A wire long enough that signals travel along it as waves and can reflect.

ULP
Unit in the last place: the gap between a floating-point number and its neighbour. It grows with the number's size.

UVM
Universal Verification Methodology: a standard SystemVerilog class library for reusable testbenches (sequencers, drivers, monitors, agents, scoreboards).

Warp
Group of 32 GPU threads that execute together.

Yield
Fraction of manufactured dies that work; falls exponentially with die area.

Appendix C

A Twelve-Month Plan and Resources

The goal: by the time next year's postings open, you can explain every idea in this book from first principles, write and verify Verilog, solve coding problems quickly, and point to finished projects. Recent Ignite windows were short: September 16 to October 18, 2024 (for 2025 internships), October 6 to 19, 2025 (for 2026), and October 5 to 18, 2026 (for 2027). Before applying, check the posting's eligibility rules: Ignite has been open only to first- and second-year undergraduates (see Appendix D).

The weekly routine

About 4 to 6 hours a week of study, plus 4 to 8 hours of building once the projects start. In heavy exam weeks, keep only the study.

1. **Read** one chapter section by section. After each section, close the book and explain its big idea out loud in two sentences.
2. **Do every exercise** before looking at the answer. Mark the ones you missed.
3. **Redo** the missed ones a week later.
4. **Build**: the HDLBits exercises, the companion books' modules, or the current project (Appendix D).

Month by month

"Book 1," "Book 2" and "Book 3" are the companion books listed in Appendix D; projects are numbered as in that appendix.

Month	Study	Build
Nov 2026	Chapters 1–4: electricity, capacitors, heat, semiconductors.	Book 1, chapters 1–5, with HDLBits "Getting Started" and "Verilog Language". Install the free tools.
Dec 2026	Chapters 5–6: reread the strong/weak level section until it is obvious.	Book 1, chapters 6–11; HDLBits through basic sequential circuits.
Jan 2027	Chapters 7–9: build every gate from 2:1 muxes on paper.	Finish Book 1. Project 5, experiments 1–2 (the inverter, the pass transistor, the swapped inverter).
Feb 2027	Chapters 10–12: latches, flip-flops, timing, counters, FSMs.	Book 2, chapters 1–6. Start Project 1 with a single-cycle processor.
Mar 2027	Chapters 13–15: drill power and setup/hold calculations until each takes under a minute.	Book 2 projects (ALU, timer, FIFO). Pipeline Project 1. Project 5, experiments 3–7.
Apr 2027	Chapters 16–18: wires, manufacturing, memory.	Finish Book 2. Verify Project 1, add branch prediction, run it on an FPGA.
May 2027	Chapters 19–22: Verilog, verification, architecture, GPUs and computer arithmetic. Work every floating-point example by hand.	Book 3, chapters 1–11. Project 2: one PE, then the 4×4 array.
Jun 2027	Chapters 23–24; three coding problems a week.	Book 3, chapters 12–19 (learn its methods; skip its networking capstone). Project 2's host interface and BF16 variant. Project 3 if a shuttle is open.
Jul 2027	The practice assessment (Chapter 25) under timed conditions; reread the chapters behind every miss.	Project 4 (CUDA).
Aug 2027	Second pass through every exercise; formula-sheet flash cards (Appendix A); mock interviews with Book 3's drills.	Finish every README and results table. Projects 6–7 only if you are targeting those teams.
Sep 2027	Keep coding practice warm.	Polish the resume; apply as soon as postings open.
Oct 2027	Take the assessment within its window, rested.	Interview prep: gates and flip-flops from muxes, the five-stage pipeline, setup and hold, clock dividers, and a two-minute deep dive on each project.

Projects

Appendix D describes eight projects mapped to the Ignite job description, the minimum set to complete, and how to present them on a resume.

Resources

Books.

- Harris and Harris, *Digital Design and Computer Architecture* (RISC-V edition): the most

beginner-friendly path from gates to a processor, and a guide for Project 1.

- Weste and Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*: the standard reference for transistors, gates, delay, power, memories and reliability (Chapters 5–18 of this book).
- Rabaey, Chandrakasan and Nikolić, *Digital Integrated Circuits*: an alternative to Weste and Harris.
- Patterson and Hennessy, *Computer Organization and Design*: pipelines, caches and memory.
- Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, or Koren, *Computer Arithmetic Algorithms*: multipliers, dividers and floating point.
- Sedra and Smith, *Microelectronic Circuits*: devices and analog circuits in depth.
- Horowitz and Hill, *The Art of Electronics*: practical electronics.

Free online.

- HDLBits: hundreds of small Verilog exercises with an online simulator.
- MIT OpenCourseWare 6.004, *Computation Structures*: digital design and architecture lectures.
- Nand2Tetris: build a computer starting from NAND gates.
- Ben Eater's video series building an 8-bit computer on breadboards.
- NPTEL's free digital VLSI courses.
- NVIDIA's *CUDA C++ Programming Guide*.
- The Tiny Tapeout website, and the OpenLane/OpenROAD documentation with the SkyWater SKY130 PDK.
- LeetCode problems tagged "linked list," "bit manipulation," "hash table" and "heap" (easy, then medium).

Appendix D

Beyond This Book: Gaps and Projects

This book teaches the theory the assessment tests. It is meant to be studied together with three companion books: *Book 1: HDLBits from Zero*, *Book 2: After HDLBits, Step by Step*, and *Book 3: Digital Hardware Foundations to Interview Practice*. This appendix says what the four books cover together, what none of them covers yet, and which projects turn that knowledge into an application that stands out.

D.1 What the four books cover together

Book	What it gives you
Hardware from the Ground Up (this book)	The theory behind the multiple-choice section: electricity, transistors, CMOS gates, timing, power, PVT, wires, manufacturing rules, memory circuits, verification basics, architecture, GPUs, computer arithmetic, C traps, and a practice assessment.
Book 1: HDLBits from Zero	Writing Verilog for the first time: widths, gates, multiplexers, K-maps, adders, registers, counters, state machines and simple testbenches, alongside the HDLBits exercises.
Book 2: After HDLBits, Step by Step	Building and testing small systems in SystemVerilog: an ALU with flags, a timer, a FIFO, a UART transmitter, valid/ready pipelines, basic setup/hold, synchronizers and the FPGA flow.
Book 3: Digital Hardware Foundations to Interview Practice	Interview-level RTL and verification: cocotb, assertions and UVM, timing constraints, asynchronous FIFOs, arbitration, caches, AXI and serial protocols, and interview drills.

Every topic candidates have reported for the Ignite hardware assessment is taught in at least one of the four. The overlap is small: this book explains *why* circuits behave as they do, and the companions train you to *build and verify* digital designs. Reading all four is not the same as mastering them, though, and a few things are still missing.

D.2 What is still missing

1. **Practice under time pressure.** Together the books contain only about fifty assessment-style questions plus chapter exercises, and one reported coding problem was harder than "easy."
   Close it: redo every chapter's exercises under a timer, turn the formula sheet (Appendix A) into flash cards, and solve about three coding problems a week at LeetCode medium level in C or C++, focusing on linked lists, bit manipulation, hash maps and heaps.
2. **Computer arithmetic in depth.** The Ignite posting lists it by name. Chapter 22 covers the core: overflow and saturation, fast adders, carry-save adders, multipliers and Booth recoding, division, fixed point, and floating-point addition, multiplication and rounding. What it does not cover is building and verifying these units at full width.
   Close it: build Project 2's BF16 processing element and verify it against a reference model on millions of random and corner-case inputs. For more depth, read Parhami's *Computer Arithmetic: Algorithms and Hardware Designs* or Koren's *Computer Arithmetic Algorithms* (non-restoring and SRT division, prefix-adder families, rounding hardware).
3. **Hands-on CUDA.** Chapter 21 explains GPUs conceptually; the posting lists CUDA or OpenCL as a way to stand out.
   Close it: work through NVIDIA's *CUDA C++ Programming Guide* while doing Project 4.
4. **Circuit simulation.** All of this book's transistor material is theory. Simulating the circuits yourself is what makes it stick.
   Close it: Project 5.
5. **The physical design flow.** Chapter 17 describes synthesis, placement, routing, timing sign-off, DRC, LVS and antenna checks, but you have not run them. Book 3 itself says physical design needs separate study.
   Close it: Project 3.
6. **CAD and MATLAB.** The posting lists mechanical CAD (Catia, Siemens NX, SolidWorks or Creo) and MATLAB. None of the books teach them, and they matter mainly for the power, thermal and mechanical teams.
   Close it: only if those teams interest you; see Project 7.
7. **Book 3's capstone targets a different industry.** Its capstone is an Ethernet/UDP market-data parser, aimed at FPGA and networking roles.
   Close it: learn its methods, but build Projects 1 and 2 as your capstones instead.
8. **Untested code.** Books 1–3 state that their HDL listings were reviewed but not run in a simulator.
   Close it: type in and simulate every listing yourself. Fixing any that fail is excellent practice.

D.3 Check your eligibility

Ignite accepts only first- and second-year undergraduates. In the 2027 cycle that meant the classes of 2030 and 2029, and the application window ran from October 5 to October 18, 2026. If you are a first-year now, you can apply to Ignite again next fall. If you are a second-year, next fall you will be a junior and should apply to NVIDIA's regular hardware internships instead. Those postings ask

for more: the 2027 ASIC design and verification internships list RTL design, simulation, synthesis, static timing analysis, lint, CDC, and SystemVerilog with UVM or SVA. That makes the projects below even more important.

D.4 What the job description asks for

The 2027 Ignite Hardware Engineering posting requires coursework in hardware basics, basic knowledge of C, C++, Python, Perl or MATLAB, and involvement in clubs, leadership or extracurricular projects. Its "ways to stand out" list, and the teams interns can join, map onto the projects like this:

What the posting asks for	Projects that show it
Computer architecture	1 (and 8)
Computer arithmetic	2, 3
VLSI design and CMOS circuits	5, 3
Verilog, SystemVerilog or VHDL	1, 2, 3
GPU APIs (CUDA or OpenCL)	4
Self-directed learning in AI and parallel computing	2, 4
CAD (Catia, Siemens NX, SolidWorks, Creo)	7
Programming in C, C++, Python, Perl or MATLAB	all (testbenches, scripts, analysis)
Team: GPU Verification and Design; Hardware ASIC	1, 2
Team: ASIC Physical Design and Timing	3, 5
Team: Hardware CPU Infrastructure	1
Team: AI Developer Technology; HW Infrastructure Content/Performance	4
Team: Power Architect	6, 5

D.5 The plan: from transistor to software

Five core projects together cover every layer of the hardware stack, which is exactly the story an NVIDIA interviewer wants to hear: you understand a design from the transistor up to the software that runs on it (Figure D.1).

The big idea

Three finished, verified, documented projects beat eight half-finished ones. The **minimum set** is Projects 1, 2 and 4, plus Project 3 whenever a Tiny Tapeout shuttle is open. Project 5 is small and directly strengthens your assessment knowledge, so do it early.

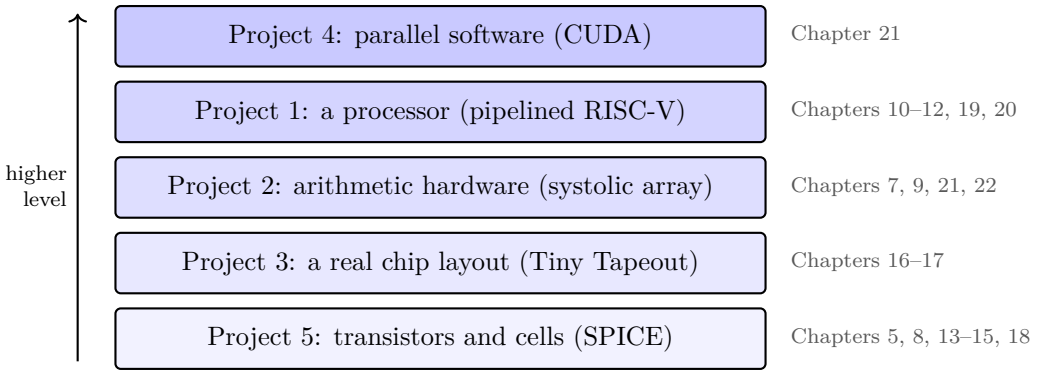


Figure D.1: The five core projects, from the lowest layer (transistors) to the highest (software), with the chapters each one exercises.

All the tools below are free: Icarus Verilog or Verilator, cocotb, Yosys, nextpnr or the free edition of Vivado, OpenLane/OpenROAD, the SkyWater SKY130 process design kit, ngspice, Xschem, KiCad and the CUDA toolkit.

D.6 Project 1: A pipelined RISC-V processor on an FPGA

Goal. A processor you designed, verified, and ran a real program on.

Build it in steps.

1. A single-cycle processor that executes the RV32I base instructions.
2. Split it into the five stages of Chapter 20 (IF, ID, EX, MEM, WB). Add forwarding, the load-use stall and flushing on taken branches.
3. Add a simple branch predictor (2-bit counters and a small branch target buffer) and measure how much it lowers CPI.
4. Add a memory-mapped UART so a C program compiled with the RISC-V GCC toolchain can print text.
5. Run it on an FPGA board. A low-cost Artix-7 board with the free Vivado edition, or a Lattice board with Yosys and nextpnr, both work.

Verify it. Compare every retired instruction against a reference model (Spike, the official RISC-V simulator, or your own Python model) in a cocotb testbench. Run the official RISC-V architecture tests. Add SVA assertions for pipeline rules, and run random instruction sequences.

Measure. CPI on a small benchmark with and without branch prediction, maximum clock frequency from the timing report, and LUT, flip-flop and block-RAM usage.

Shows: computer architecture, SystemVerilog, verification. It also lets you answer "what are the stages of a five-stage pipeline?" with your own design. **Time:** about 10–12 weeks at 6–8 hours a week. **Stretch:** instruction and data caches, the M extension (hardware multiply and divide, which is computer arithmetic), interrupts.

D.7 Project 2: A systolic-array matrix multiplier (a mini tensor core)

Goal. Hardware for the operation that AI runs on, matrix multiplication (Chapter 21). This is the most NVIDIA-specific project in the plan.

A **systolic array** is a grid of small **processing elements** (PEs). Each PE multiplies the two numbers arriving from its neighbors, adds the product to a running sum, and passes the numbers on to the next PE in a steady rhythm, like a heartbeat (hence "systolic"). Rows of one matrix flow in from the left and columns of the other flow in from the top (Figure D.2). An $N \times N$ array performs N^2 multiply-adds every clock cycle.

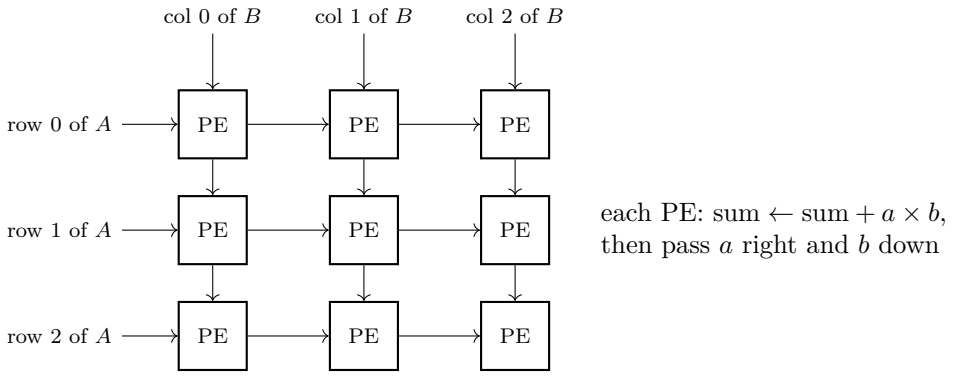


Figure D.2: A 3×3 output-stationary systolic array. Each PE accumulates one element of the result matrix.

Build it in steps.

1. One PE: an INT8 $\times$ INT8 multiply feeding an INT32 accumulator.
2. A 4×4 array with control for load, compute and drain.
3. A host interface: a UART link from a PC, or memory-mapped registers on your Project 1 processor.
4. A BF16 variant of the PE: multiply the significands, add the exponents, normalize, and round to nearest even. This is where computer arithmetic gets real.

Verify it. In cocotb, compare the results bit for bit against NumPy for random matrices and corner cases such as -128×-128 and all-maximum inputs.

Measure. Multiply-adds per cycle, latency for an $N \times N$ product, maximum clock frequency and resources. Compare the achieved rate with the theoretical peak, PEs times clock frequency.

Shows: computer arithmetic, AI and parallel computing, VLSI, SystemVerilog. **Time:** about 6–8 weeks. **Stretch:** an 8×8 array, double buffering, INT4 or FP8 inputs.

D.8 Project 3: Put a design on a real chip with Tiny Tapeout

Goal. Take a design through the full physical design flow, and receive the manufactured chip.

Tiny Tapeout is an educational program that places many small designs side by side on one shared chip, so each person pays for only a small piece (a **tile**). It uses an open-source design flow and an open **process design kit** (PDK): the files that describe a factory's transistors and layout rules. Several shuttles (production runs) were offered during 2026. Dates and prices change, so check the Tiny Tapeout website for the next one.

What to put on it. Something small enough for a tile and its limited pins: a 2×2 version of Project 2, a Booth multiplier, or a compact state machine.

Steps.

1. Write and verify the RTL with cocotb, using the program's project template.
2. Run the automated flow to produce a layout (GDS file).
3. Read every report: area, setup and hold slack, DRC and LVS results, and any antenna fixes. You can now explain the antenna rule (Section 17.7) from your own layout.
4. Submit. When the chip arrives months later, test it and compare it with your simulation.

If no shuttle is open, run OpenLane/OpenROAD on your own machine with the SKY130 PDK all the way to a layout. You will not get a chip, but you will have real physical design reports to discuss.

Shows: VLSI design, ASIC physical design and timing, CMOS. "I taped out a chip" is rare for a first- or second-year student. **Time:** about 4–6 weeks.

D.9 Project 4: Optimizing matrix multiplication in CUDA

Goal. Show that you can make a real NVIDIA GPU fast, and explain why each step helped, using Chapter 21.

Steps.

1. A naive kernel: one thread computes one output element.
2. Shared-memory tiling: each thread block loads tiles of the inputs once and reuses them.
3. Register tiling: each thread computes a small block of outputs, reusing values held in registers.
4. Vectorized loads, then (optionally) tensor cores through CUDA's WMMA interface with FP16 inputs.
5. Compare each version with NVIDIA's cuBLAS library.

Measure. GFLOP/s for each version at matrix sizes from 512 to 8192, as a percentage of cuBLAS. Profile with Nsight Compute (memory throughput, occupancy) and plot each version on a roofline chart. Then compare the GPU's multiply-adds per cycle with your Project 2 array.

Hardware. Any NVIDIA GPU: your own, a campus computing cluster, or a cloud notebook that offers one.

Shows: CUDA, parallel computing, AI. It fits the AI Developer Technology and performance teams especially well. **Time:** about 3–4 weeks.

D.10 Project 5: Characterizing standard cells in SPICE

Goal. See with your own simulations the transistor behavior this book describes. It is the fastest way to make the assessment's theory questions feel obvious.

Use ngspice and Xschem with the SKY130 PDK. Each experiment checks a chapter:

1. An inverter's transfer curve, switching threshold and noise margins at several PMOS/NMOS size ratios (Chapter 8).
2. An NMOS pass transistor's weak 1, and the swapped inverter from the assessment (Chapters 5 and 8).
3. Delay versus fan-out, the FO4 delay, and a tapered buffer chain (Chapter 13).
4. Energy per transition versus supply voltage, to confirm the CV^2 law (Chapter 14).
5. A temperature sweep from -40 to 125 °C at normal and at low supply voltage, looking for temperature inversion (Chapter 15).
6. A ring oscillator's frequency versus voltage and temperature.
7. A 6T SRAM cell's read and write margins, and its butterfly curve (Chapter 18).

Deliverable. A short report with one plot per experiment, each compared with what this book predicts.

Shows: CMOS circuits, VLSI. **Time:** about 2–3 weeks.

D.11 Track-specific options

Project 6: GPU power and thermal characterization (power and infrastructure teams). Log power draw, clock speed and temperature with `nvidia-smi` while running your Project 4 kernels at different sizes (and different clock settings, if your GPU allows it). Separate idle power from the extra power under load, fit the results to the $\alpha CV^2 f$-plus-leakage model of Chapter 14, estimate the energy per FLOP, and estimate the cooling system's thermal resistance from temperature rise versus power (Chapter 3).

Project 7: Mechanical CAD (thermal and mechanical teams). Design a heat sink, fan duct or enclosure in SolidWorks, Onshape or a similar tool. Size the fins and calculate the thermal resistance with Chapter 3, then use the CAD tool's thermal simulation if it has one. Do the analysis in MATLAB or Python.

Project 8: A mini SIMT core (stretch goal). Extend Project 1 into 4 to 8 lanes that execute one instruction stream on different data, with per-thread registers, an active mask for divergent branches, and a scheduler that switches between two warps. Run vector-add and matrix-add kernels and compare throughput with the scalar core. Open-source teaching designs such as tiny-gpu and the Vortex RISC-V GPGPU are worth studying for ideas, but write your own; interviewers will ask about every design decision.

D.12 When to do what

The month-by-month plan in Appendix C fits the projects in alongside the reading. In short: Project 5 in January and March, Project 1 from February to April, Project 2 in May and June, Project 3 whenever a shuttle opens, Project 4 in July, and the track-specific projects in August if you want them.

D.13 Making the projects count

Each project's repository needs:

- ☐ a README with the specification, a block diagram and known limitations;
- ☐ one command that runs all the tests, ideally also run automatically on every push;
- ☐ a results table (clock frequency, resources, CPI, GFLOP/s or whatever the project measures);
- ☐ waveforms or plots that show the design working.

Writing resume bullets. Each bullet says what you built, how you verified it, and a number you measured:

Remember

- *Designed a 5-stage pipelined RV32I processor in SystemVerilog with forwarding and branch prediction; verified against a reference model with cocotb, passing X of Y architecture tests; ran at Z MHz on an Artix-7 FPGA using N LUTs.*
- *Built a 4×4 INT8 systolic-array matrix multiplier, bit-exact against NumPy over X random tests; sustained Y multiply-adds per cycle at Z MHz.*
- *Optimized CUDA matrix multiplication from X to Y GFLOP/s (Z% of cuBLAS) with shared-memory and register tiling; analyzed each version with Nsight Compute and a roofline model.*

Replace X, Y, Z and N with numbers you actually measured. Never write a number you did not measure.

The rest of the resume. The posting requires your expected graduation month and year, and asks for clubs, leadership or extracurricular projects. A hardware-related club (IEEE, robotics, a solar-car or Formula team) or a hackathon project counts.

Be ready to defend every line. Interviewers pick a project and dig: why these widths, what limits the clock frequency, what happens under backpressure, what the hardest bug was. Rehearse a two-minute explanation of each project, using Book 3's project deep-dive drill.

Appendix E

Sources

The job-description details in Appendix D come from NVIDIA's own postings, listed at the end of this appendix. The "Seen on the assessment" boxes in this book draw on two kinds of source: the questions from an actual Ignite hardware assessment that motivated this book, and anonymous candidate reports gathered online in October 2026. The reports are unverified, some describe other NVIDIA hardware roles rather than Ignite, and formats change between years. Glassdoor pages could only be read through search-engine summaries. Treat every reported detail as a hint about the *kind* of question to expect, not as a promise.

From the assessment that motivated this book

A linked-list coding problem; the advantage of a flip-flop over a latch; an OR gate from a 2:1 mux; the inverter with swapped transistors; dynamic power when frequency and voltage double; carrier mobility and wire resistance versus temperature; and a true/false question on the antenna rule.

Candidate reports and preparation sites

- Glassdoor, NVIDIA IGNITE Intern interview questions:
  `https://www.glassdoor.com/Interview/NVIDIA-IGNITE-Intern-Interview-Questions-EI_IE7633.0,6_KO7,20.htm`
- Glassdoor, NVIDIA Hardware Intern interview questions:
  `https://www.glassdoor.com/Interview/NVIDIA-Hardware-Intern-Interview-Questions-EI_IE7633.0,6_KO7,22.htm`
- Glassdoor, NVIDIA interview review (hardware multiple choice on transistors, circuits and heat sinks):
  `https://www.glassdoor.com/Interview/NVIDIA-Interview-E7633-RVW104894219.htm`
- Glassdoor, NVIDIA clock-divider interview question:
  `https://www.glassdoor.com/Interview/One-of-the-questions-is-about-clock-divider-and-the-interviewer-asked-a-lot-different-questions-related-to-divider-design-QTN_418519.htm`
- InterviewFox, NVIDIA Ignite OA on HackerRank (format, coding problems, GND and FIFO topics):
  `https://interviewfox.ai/interview-questions/nvidia-oa-hackerrank-questions/`

- Jointaro, NVIDIA Ignite software engineer interview experience (November 2025): https://www.jointaro.com/interviews/companies/nvidia/experiences/software-engineer-ignite-united-states-november-4-2025-no-offer-neutral-c2955f45/
- PracHub, NVIDIA Ignite Internship 2027 guide: https://prachub.com/resources/nvidia-ignite-internship-2027-eligibility-interview-process-and-preparation
- Nora HQ, NVIDIA Ignite intern interview guide: https://interview.norahq.com/interview-guides/nvidia-ignite-intern-interview-guide-2025
- AnitaB.org job board, NVIDIA 2025 Ignite Internships: Hardware Engineering (eligibility, skills, roles): https://jobs.anitab.org/companies/nvidia/jobs/40510638-nvidia-2025-ignite-internships-hardware-engineering
- VLSI4Freshers, NVIDIA ASIC design test questions (power scaling, FIFO depth, caches, NMOS networks, aptitude): https://www.vlsi4freshers.com/2022/02/nvidia-asic-design-interview-questions.html
- VLSI4Freshers, NVIDIA ASIC verification test questions (counters, XOR chains, reset glitch filtering, Verilog, AMAT, C output): https://www.vlsi4freshers.com/2023/03/nvidia-asic-verification-interview-questions.html
- Snubber, NVIDIA engineering interview questions (practice problems on decoupling, heat sinks, reflections, ADCs): https://snubber.ai/blog/nvidia-interview-questions
- Voltage Learning, NVIDIA internship interview process: https://voltagelearning.com/blog/nvidia-internship-interview-process
- LeakCode, NVIDIA online assessment reports: https://leakcode.dev/company/nvidia/oa/2026
- Lodely, NVIDIA online assessment overview: https://www.lodely.com/companies/nvidia/online-assessment

Job postings and project resources (Appendix D)

- NVIDIA 2027 Ignite Internships: Hardware Engineering (eligibility, requirements, ways to stand out, teams, dates), AnitaB.org copy: https://jobs.anitab.org/companies/nvidia/jobs/95889112-nvidia-2027-ignite-internships-hardware-engineering
- The same posting on NVIDIA's careers site: https://jobs.nvidia.com/careers/job/893397961094
- NVIDIA 2026 Ignite Internships: Hardware Engineering (application window October 6–19, 2025): https://jobs.anitab.org/companies/nvidia/jobs/59992858-nvidia-2026-ignite-internships-hardware-engineering
- NVIDIA 2027 Internships: Hardware ASIC Design: https://jobs.nvidia.com/careers/job/893397025803
- NVIDIA ASIC Design and Verification Intern, SOC, 2027: https://nvidia.wd5.myworkdayjobs.com/en-US/NVIDIAExternalCareerSite/job/ASIC-Design-and-Verification-Intern--SOC---2027_JR2024860

- NVIDIA 2026 Internships: Hardware Physical Design / VLSI:
  `https://jobs.nvidia.com/careers/job/893384237248`
- Extern, NVIDIA Ignite 2027 program guide (application dates):
  `https://www.extern.com/post/nvidia-ignite-program-guide`
- Tiny Tapeout, list of shuttles:
  `https://tinytapeout.com/tags/shuttle/`
- tiny-gpu, an educational GPU in Verilog:
  `https://hellogithub.com/en/repository/adam-maj/tiny-gpu`
- Vortex, an open-source RISC-V GPGPU from Georgia Tech:
  `https://github.com/vortexgpgpu/vortex`